

REPRESENTATION LEARNING AND ALGORITHMS IN HYPERBOLIC SPACES

A DISSERTATION  
SUBMITTED TO THE DEPARTMENT OF ENGINEERING  
AND THE COMMITTEE ON GRADUATE STUDIES  
OF STANFORD UNIVERSITY  
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS  
FOR THE DEGREE OF  
DOCTOR OF PHILOSOPHY

Ines Chami  
June 2021

© 2021 by Ines Chami. All Rights Reserved.

Re-distributed by Stanford University under license with the author.



This work is licensed under a Creative Commons Attribution-Noncommercial 3.0 United States License.

<http://creativecommons.org/licenses/by-nc/3.0/us/>

This dissertation is online at: <http://purl.stanford.edu/bj040rx3340>

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

**Chris Re, Primary Adviser**

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

**Leonidas Guibas**

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

**Jure Leskovec**

Approved for the Stanford University Committee on Graduate Studies.

**Stacey F. Bent, Vice Provost for Graduate Education**

*This signature page was generated electronically upon submission of this dissertation in electronic format. An original signed hard copy of the signature page is on file in University Archives.*

# Abstract

Graph embedding methods aim at learning representations of nodes that preserve graph properties (e.g., graph distances). These embeddings can then be used in downstream applications such as clustering, visualization, nearest neighbor search and classification tasks. Most machine learning algorithms learn embeddings in standard Euclidean spaces. Recent research shows promise for more faithful embeddings by leveraging non-Euclidean geometries, such as hyperbolic or spherical geometries. In particular, trees can be embedded almost perfectly into two-dimensional hyperbolic spaces, while this is not possible in Euclidean spaces of any dimension. In this thesis, we develop machine learning models that operate in hyperbolic spaces.

We start by introducing hyperbolic representation learning methods for hierarchical graphs, including methods for multi-relational graphs or graphs with node features. We demonstrate the benefits of these hyperbolic representations compared to their Euclidean counterparts on a variety of link prediction benchmark datasets.

We then design algorithms that operate on these hyperbolic representations. We first propose a method for hierarchical clustering in hyperbolic space, which can be used to solve any similarity-based hierarchical clustering problem with gradient-descent. We then propose a generalization of Principal Component Analysis for hyperbolic inputs, and show that it yields improved low-dimensional representations and data visualizations compared to previous manifold dimensionality reduction methods.

# Acknowledgments

First, I would like to thank my advisor Chris Ré. Chris has been instrumental in my academic career in more ways than one. As my advisor, Chris helped me get the confidence and gave me the freedom to trust my research instincts and pursue my own ideas, while providing invaluable technical guidance and encouraging me to dive deep in the problems I was interested in. Both in and out of research, Chris always had my back and has been a true life mentor. I sincerely thank him for this.

Next, I would like to thank to my oral examination committee including Jure Leskovec who got me started on the path of Graph Neural Networks—Leonidas Guibas—who gave me the opportunity to co-teach a class on geometry during Fall 2020—and Moses Charikar; their research contributions to the field of graphs, geometry and algorithms are immense and it was a privilege to have them evaluate my work. I give my heartfelt thanks to Margot Gerritsten for chairing my thesis committee, and along with Indira Choudhury and Gianluca Iaccarino, for making the ICME environment so vibrant and creating such an amazing community.

My Ph.D. experience would not have been as memorable and enlightening without my co-authors including Albert Gu, Fred Sala, Adva Wolf, Dat Nguyen, Vaggos Chatziafratis and Rex Ying. I would like to give a special thanks to Albert and Fred for our numerous and illuminating research discussions about hyperbolic geometry, it was a great pleasure to work with you. Further thanks to Bryan Perozzi, Sami Abu-El-Haija, Da-Cheng Juan, Kevin Murphy and Sujith Ravi for two wonderful internships at Google Research.

I also want to acknowledge all of my great labmates from the Hazy Research group including Sen Wu, Braden Hancock, Vincent Chen, Megan Leszczynski, Laurel Orr, Paroma Varma, Avner May, Stephen Bach, Nimit Sohoni, Tri Dao, Dan Fu and the many others I had the chance to interact with. I am honored to have been part of this team; this has made the research process truly fulfilling.

I want to thank my friends at Stanford for making my Ph.D. student life so enjoyable: Daniel Lévy, Kristen Kessel, Annie Marsden, Daniel Byrnes, Christopher Lazarus, Beliz Gunel, Ana Saavedra, Javier Sagastuy, Willie Aboumrad, Daniel Kunin. Thank you for making me feel like I have a family here, but also for being my friends despite all the pranks I regularly pull on you.

Last but not least, I want to thank my parents and my sisters Sophia, Yasmine and Meïssane, for their unconditional love and constant support.

# Contents

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                    | <b>iv</b> |
| <b>Acknowledgments</b>                                             | <b>v</b>  |
| <b>1 Introduction</b>                                              | <b>1</b>  |
| 1.1 Representation Learning . . . . .                              | 1         |
| 1.2 Euclidean Embeddings and the Curse of Dimensionality . . . . . | 5         |
| 1.3 Hyperbolic Embeddings . . . . .                                | 7         |
| 1.3.1 The Poincaré Ball Model of Hyperbolic Space . . . . .        | 8         |
| 1.3.2 Hyperbolic Metrics . . . . .                                 | 10        |
| 1.4 Contributions and Outline . . . . .                            | 13        |
| 1.4.1 Part I: Embeddings . . . . .                                 | 13        |
| 1.4.2 Part II: Algorithms . . . . .                                | 14        |
| <b>I Hyperbolic Graph Representation Learning</b>                  | <b>16</b> |
| <b>2 Graph Representation Learning</b>                             | <b>17</b> |
| 2.1 Introduction . . . . .                                         | 17        |
| 2.2 Preliminaries . . . . .                                        | 19        |
| 2.2.1 Definitions . . . . .                                        | 19        |
| 2.2.2 The Generalized Network Embedding Problem . . . . .          | 21        |
| 2.3 A Taxonomy of GRL Models . . . . .                             | 24        |
| 2.3.1 The GRAPHEDM Framework . . . . .                             | 24        |
| 2.3.2 Taxonomy of Objective Functions . . . . .                    | 26        |
| 2.3.3 Taxonomy of Encoders . . . . .                               | 27        |
| 2.4 Examples . . . . .                                             | 29        |
| 2.4.1 Shallow Embeddings . . . . .                                 | 29        |
| 2.4.2 Graph Neural Networks . . . . .                              | 34        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 2.5      | GRL Applications . . . . .                                     | 37        |
| 2.5.1    | Unsupervised Applications . . . . .                            | 37        |
| 2.5.2    | Supervised Applications . . . . .                              | 39        |
| <b>3</b> | <b>Graph Neural Networks</b>                                   | <b>40</b> |
| 3.1      | Introduction . . . . .                                         | 40        |
| 3.2      | Related Work . . . . .                                         | 42        |
| 3.3      | Background . . . . .                                           | 43        |
| 3.4      | Hyperbolic Graph Convolutional Networks . . . . .              | 43        |
| 3.4.1    | Mapping from Euclidean to Hyperbolic Spaces . . . . .          | 44        |
| 3.4.2    | Feature Transform in Hyperbolic Space . . . . .                | 44        |
| 3.4.3    | Neighborhood Aggregation on the Hyperboloid Manifold . . . . . | 45        |
| 3.4.4    | HGCN Architecture . . . . .                                    | 47        |
| 3.4.5    | Trainable Curvature . . . . .                                  | 47        |
| 3.5      | Experiments . . . . .                                          | 49        |
| 3.5.1    | Datasets and Baselines . . . . .                               | 49        |
| 3.5.2    | Training Details . . . . .                                     | 50        |
| 3.5.3    | Results . . . . .                                              | 52        |
| 3.5.4    | Analysis . . . . .                                             | 53        |
| 3.6      | Conclusion . . . . .                                           | 54        |
| <b>4</b> | <b>Knowledge Graph Embeddings</b>                              | <b>55</b> |
| 4.1      | Introduction . . . . .                                         | 55        |
| 4.2      | Related Work . . . . .                                         | 57        |
| 4.3      | Methodology . . . . .                                          | 58        |
| 4.3.1    | Hierarchies in hyperbolic space . . . . .                      | 58        |
| 4.3.2    | Hyperbolic Isometries . . . . .                                | 59        |
| 4.3.3    | Hyperbolic Attention . . . . .                                 | 62        |
| 4.3.4    | The ATTH model . . . . .                                       | 62        |
| 4.3.5    | Tangent Space Optimization . . . . .                           | 63        |
| 4.4      | Experiments . . . . .                                          | 64        |
| 4.4.1    | Experimental Setup . . . . .                                   | 64        |
| 4.4.2    | Results in Low Dimensions . . . . .                            | 66        |
| 4.4.3    | Hyperbolic Rotations and Reflections . . . . .                 | 67        |
| 4.4.4    | Attention-based Transformations . . . . .                      | 67        |
| 4.4.5    | Results in High Dimensions . . . . .                           | 68        |
| 4.4.6    | Visualizations . . . . .                                       | 69        |
| 4.5      | Conclusion . . . . .                                           | 69        |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| <b>II</b> | <b>Algorithms in Hyperbolic Spaces</b>                             | <b>73</b> |
| <b>5</b>  | <b>Hierarchical Clustering</b>                                     | <b>74</b> |
| 5.1       | Introduction . . . . .                                             | 74        |
| 5.2       | Related Work . . . . .                                             | 76        |
| 5.3       | Background . . . . .                                               | 78        |
| 5.3.1     | Notation . . . . .                                                 | 78        |
| 5.3.2     | HC Discrete Optimization Framework . . . . .                       | 78        |
| 5.4       | Hyperbolic Hierarchical Clustering . . . . .                       | 79        |
| 5.4.1     | Continuous Tree Representation via Hyperbolic Embeddings . . . . . | 80        |
| 5.4.2     | Differentiable Objective Function . . . . .                        | 80        |
| 5.4.3     | Hyperbolic Decoding . . . . .                                      | 82        |
| 5.4.4     | Approximation Ratio Result . . . . .                               | 82        |
| 5.5       | HYPHC Practical Considerations . . . . .                           | 84        |
| 5.6       | Experiments . . . . .                                              | 85        |
| 5.6.1     | Experimental Setup . . . . .                                       | 85        |
| 5.6.2     | Clustering Quality . . . . .                                       | 88        |
| 5.6.3     | Analysis . . . . .                                                 | 89        |
| 5.6.4     | End-to-End Training . . . . .                                      | 90        |
| 5.7       | Conclusion . . . . .                                               | 90        |
| <b>6</b>  | <b>Dimensionality Reduction</b>                                    | <b>92</b> |
| 6.1       | Introduction . . . . .                                             | 92        |
| 6.2       | Related Work . . . . .                                             | 94        |
| 6.3       | Background . . . . .                                               | 95        |
| 6.3.1     | Directions in Hyperbolic space . . . . .                           | 96        |
| 6.3.2     | Geodesic Projections . . . . .                                     | 97        |
| 6.3.3     | Manifold Statistics . . . . .                                      | 97        |
| 6.4       | Generalizing PCA to the Hyperbolic Space . . . . .                 | 98        |
| 6.4.1     | Hyperbolic Flags . . . . .                                         | 98        |
| 6.4.2     | Projections via Horospheres . . . . .                              | 99        |
| 6.4.3     | Intrinsic Variance Objective . . . . .                             | 101       |
| 6.5       | HOROPCA . . . . .                                                  | 102       |
| 6.6       | Experiments . . . . .                                              | 103       |
| 6.6.1     | Experimental Setup . . . . .                                       | 103       |
| 6.6.2     | Dimensionality Reduction . . . . .                                 | 107       |
| 6.6.3     | Hyperbolic Data Whitening . . . . .                                | 109       |
| 6.6.4     | Visualizations . . . . .                                           | 109       |

|                                                      |            |
|------------------------------------------------------|------------|
| 6.7 Conclusion                                       | 110        |
| <b>7 Conclusion</b>                                  | <b>112</b> |
| <b>A Geometry Review</b>                             | <b>114</b> |
| A.1 Topological Manifolds                            | 114        |
| A.1.1 Definitions                                    | 114        |
| A.1.2 Continuity                                     | 116        |
| A.1.3 Topological Manifolds                          | 117        |
| A.2 Smooth Manifolds                                 | 119        |
| A.2.1 Definitions                                    | 119        |
| A.2.2 Smooth Functions                               | 120        |
| A.2.3 Tangent Space                                  | 121        |
| A.3 Riemannian Manifolds                             | 123        |
| A.3.1 Definitions                                    | 123        |
| A.3.2 Geodesics                                      | 124        |
| A.3.3 The Exponential Map                            | 127        |
| A.4 Hyperbolic Geometry                              | 128        |
| A.4.1 Minkowski Spaces                               | 128        |
| A.4.2 The Hyperbolic Space as a Riemannian Manifold  | 130        |
| A.4.3 Hyperbolic Geodesics                           | 132        |
| A.4.4 Connexions with the Poincaré Ball Model        | 133        |
| <b>B Deferred Proofs of Chapter 5</b>                | <b>134</b> |
| B.1 Preliminaries                                    | 134        |
| B.1.1 Notation                                       | 134        |
| B.1.2 Hyperbolic LCA and the Gromov product          | 135        |
| B.2 Proof of Theorem 5.4.1                           | 137        |
| B.2.1 Proof Outline                                  | 137        |
| B.2.2 Proof of Lemma B.2.1                           | 139        |
| B.2.3 Proof of Lemma B.2.2                           | 145        |
| <b>C Deferred Proofs of Chapter 6</b>                | <b>147</b> |
| C.1 Horospherical Projection: Proofs and Discussions | 147        |
| C.1.1 Well-definedness                               | 147        |
| C.1.2 Geometric Properties                           | 149        |
| C.1.3 Computation of Horospherical Projections       | 152        |
| C.2 Geodesic Projection: Distortion Analysis         | 156        |

# List of Tables

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                       |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Summary of the notation used in this chapter. . . . .                                                                                                                                                                                                                                                                                                                                                                                                 | 20 |
| 3.1 | Benchmarks' statistics . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                      | 49 |
| 3.2 | ROC AUC for Link Prediction (LP) and F1 score for Node Classification (NC) tasks.<br>For inductive datasets, we only evaluate inductive methods since shallow methods<br>cannot generalize to unseen nodes/graphs. We report graph hyperbolicity values $\delta$<br>(lower is more hyperbolic). . . . .                                                                                                                                               | 51 |
| 3.3 | ROC AUC for link prediction on AIRPORT and DISEASE datasets. . . . .                                                                                                                                                                                                                                                                                                                                                                                  | 52 |
| 4.1 | Datasets statistics. The lower the metric $\xi_G$ is, the more tree-like the knowledge graph<br>is. . . . .                                                                                                                                                                                                                                                                                                                                           | 64 |
| 4.2 | Link prediction results for low-dimensional embeddings ( $d = 32$ ) in the filtered setting.<br>Best score in <b>bold</b> and best published <u>underlined</u> . Hyperbolic isometries significantly<br>outperform Euclidean baselines on WN18RR and YAGO3-10, both of which exhibit<br>hierarchical structures. . . . .                                                                                                                              | 65 |
| 4.3 | Comparison of H@10 for WN18RR relations. Higher $\text{Khs}_G$ and lower $\xi_G$ means more<br>hierarchical. . . . .                                                                                                                                                                                                                                                                                                                                  | 66 |
| 4.4 | Comparison of geometric transformations on a subset of YAGO3-10 relations. . . . .                                                                                                                                                                                                                                                                                                                                                                    | 68 |
| 4.5 | Link prediction results for high-dimensional embeddings (best for $d \in \{200, 400, 500\}$ )<br>in the filtered setting. DistMult, ConvE and ComplEx results are taken from [64].<br>Best score in <b>bold</b> and best published <u>underlined</u> . ATTE and ATTH have similar<br>performance in the high-dimensional setting, performing competitively with or better<br>than state-of-the-art methods on WN18RR, FB15k-237 and YAGO3-10. . . . . | 69 |
| 4.6 | Best hyperparameters in low- and high-dimensional settings. NA negative samples<br>indicates that the full cross-entropy loss is used, without negative sampling. . . . .                                                                                                                                                                                                                                                                             | 72 |
| 5.1 | Clustering quality results measured in discrete Dasgupta's cost (DC). Best score in<br>bold, second best underlined. Dashes indicate that the method could not scale to large<br>datasets. . . . .                                                                                                                                                                                                                                                    | 86 |

|     |                                                                                                                                                                                                                                                                |     |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.2 | Clustering quality results measured in dendrogram purity (DP). Best score in bold, second best underlined. gHHC scores are directly taken from [123]. . . . .                                                                                                  | 87  |
| 5.3 | Speed/quality analysis for the exact and greedy decoding. . . . .                                                                                                                                                                                              | 89  |
| 5.4 | Classification accuracy for different training strategies. . . . .                                                                                                                                                                                             | 90  |
| 6.1 | Analogies of components and their corresponding coordinates, in both Euclidean and hyperbolic space. . . . .                                                                                                                                                   | 96  |
| 6.2 | Average distortion for embeddings learned with different methods. Sarkar refers to combinatorial embeddings learned with Sarkar’s construction [150, 152] while PyTorch refers to embeddings learned using the PyTorch code from [81]. . . . .                 | 104 |
| 6.3 | Dimensionality reduction results on 10-dimensional hyperbolic embeddings reduced to two dimensions. Results are averaged over 5 runs for non-deterministic methods. Best in <b>bold</b> and second best <u>underlined</u> . . . . .                            | 106 |
| 6.4 | Dimensionality reduction results on 10-dimensional combinatorial embeddings (Sarkar’s construction) reduced to two dimensions. Results are averaged over 5 runs for non-deterministic methods. Best in <b>bold</b> and second best <u>underlined</u> . . . . . | 107 |
| 6.5 | Data whitening experiments. We report classification accuracy averaged over 5 embedding configurations. Best in <b>bold</b> and second best <u>underlined</u> . . . . .                                                                                        | 109 |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                     |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Overview of Machine Learning pipelines for graph-structured data. First, embedding models are used to learn graph representations. Then, these representations are used as inputs in downstream applications such as classification or data clustering. . . .                                                                                                                                                                       | 2  |
| 1.2 | Machine Learning pipelines and relation between manifold learning and graph representation learning. We note that this categorization is not an exhaustive view of the field. For instance, a common learning setup is when the input consists of both points in a Euclidean space and a graph, i.e. data defined on a discretized graph domain. We will discuss this setting in Chapter 2. . . . .                                 | 3  |
| 1.3 | An illustration of Euclidean vs. non-Euclidean graphs. . . . .                                                                                                                                                                                                                                                                                                                                                                      | 4  |
| 1.4 | Simple graph metric on four nodes. . . . .                                                                                                                                                                                                                                                                                                                                                                                          | 6  |
| 1.5 | From left to right: a surface of negative curvature, a surface of zero (Gaussian) curvature, and a surface of positive curvature. Curvature essentially measures how manifolds deviate from the flat Euclidean space. . . . .                                                                                                                                                                                                       | 7  |
| 1.6 | Hyperbolic geodesics in the Poincaré disk and illustration of exponential volume growth property. . . . .                                                                                                                                                                                                                                                                                                                           | 8  |
| 1.7 | Illustration of the notion of $\delta$ -slim triangles. . . . .                                                                                                                                                                                                                                                                                                                                                                     | 10 |
| 2.1 | Illustration of the GRAPHEDM framework. Based on the supervision available, methods will use some or all of the branches. In particular, unsupervised methods do not leverage label decoding for training and only optimize the similarity or dissimilarity decoder (lower branch). On the other hand, semi-supervised and supervised methods leverage the additional supervision to learn models' parameters (upper branch). . . . | 24 |
| 2.2 | Taxonomy of graph representation learning methods. Based on what information is used in the encoder network, we categorize graph embedding approaches into four categories: shallow embeddings, graph auto-encoders, graph-based regularization and graph neural networks. . . . .                                                                                                                                                  | 28 |
| 2.3 | Shallow embedding methods. The encoder is a simple embedding look-up and the graph structure is only used in the loss function. . . . .                                                                                                                                                                                                                                                                                             | 30 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                              |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.4 | Supervised graph neural networks (GNNs). Rather than leveraging the graph structure to act as a regularizer, GNNs leverage the graph structure in the encoder to propagate information across neighbouring nodes and learn structural representations. Labels are then decoded and compared to ground truth labels (e.g. via the cross-entropy loss).                                                                                        | 34 |
| 3.1 | (a) Poincaré disk geodesics (shortest path) connecting $\mathbf{x}$ and $\mathbf{y}$ for different curvatures. As curvature $(-1/K)$ decreases, the distance between $\mathbf{x}$ and $\mathbf{y}$ increases, and the geodesics lines get closer to the origin. Center: Hyperbolic distance vs curvature. (b) Hyperbolic distance vs curvature. (c) Poincaré geodesic lines.                                                                 | 41 |
| 3.2 | HGCN neighborhood aggregation (Eq. 3.9) first maps messages/embeddings to the tangent space, performs the aggregation in the tangent space, and then maps back to the hyperbolic space.                                                                                                                                                                                                                                                      | 44 |
| 3.3 | Visualization of embeddings for LP on DISEASE and NC on CORA (visualization on the Poincaré disk for HYPHC. (a) GCN embeddings in first and last layers for DISEASE LP hardly capture hierarchy (depth indicated by color). (b) In contrast, HYPHC reserves node hierarchies. (c) On CORA NC, HGCN leads to better class separation (indicated by different colors).                                                                         | 46 |
| 3.4 | Decreasing curvature $(-1/K)$ improves link prediction performance on DISEASE.                                                                                                                                                                                                                                                                                                                                                               | 52 |
| 3.5 | Attention: Euclidean GAT (left), HGCN (right). Each graph represents a 2-hop neighborhood of the DISEASE-M dataset.                                                                                                                                                                                                                                                                                                                          | 53 |
| 4.1 | A toy example showing how KGs can simultaneously exhibit hierarchies and logical patterns.                                                                                                                                                                                                                                                                                                                                                   | 56 |
| 4.2 | The curvature estimate $\xi_G$ and the Krackhardt hierarchy score $\text{Khs}_G$ for several simple graphs. The top-left graph is the most hierarchical, while the bottom-right graph is the least hierarchical.                                                                                                                                                                                                                             | 60 |
| 4.3 | Euclidean (left) and hyperbolic (right) isometries. In hyperbolic space, the distance between start and end points after applying rotations or reflections is much larger than the Euclidean distance; it approaches the sum of the distances between the points and the origin, giving more "room" to separate embeddings. This is similar to trees, where the shortest path between two points goes through their nearest common ancestor. | 61 |
| 4.4 | (a): ROTH offers improved performance in low dimensions; in high dimensions, fixed curvature degrades performance, while trainable curvature approximately recovers Euclidean space. (b): As the dimension increases, the learned curvature of hierarchical relationships tends to zero.                                                                                                                                                     | 67 |
| 4.5 | Histogram of embeddings norm learned with fixed and trainable curvatures for the hypernym relation in WN18RR.                                                                                                                                                                                                                                                                                                                                | 70 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |     |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 4.6 | Visualizations of the embeddings learned by ROTE and ROTH on a sub-tree of WN18RR for the <i>hypernym</i> relation. In contrast to ROTE, ROTH preserves hierarchies by learning tree-like embeddings. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 71  |
| 5.1 | The tree shown in (a) is embedded into hyperbolic space ( $\mathbb{B}_2$ ) in (b), (c), and (d). In (c) and (d) we show the hyperbolic LCA (in red) and illustrate the relationship between the discrete LCA, which is central to Dasgupta’s cost, and geodesics (shortest paths) in hyperbolic space. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 76  |
| 5.2 | Visualization of HYPHC embeddings and decoded trees during optimization. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 79  |
| 5.3 | Circle inversion used for hyperbolic LCA construction in Lemma 5.4.1. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 82  |
| 5.4 | An illustration of the greedy decoding algorithm. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 84  |
| 5.5 | Normalized cost vs number of triplets sampled. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 88  |
| 6.1 | Given datapoints (black dots), Euclidean and horospherical projections preserve distance information across different subspaces (black lines) pointing towards the same direction or point at infinity, while geodesic projections do not. (a): Distances between points, and therefore the explained variance, are invariant to translations along orthogonal directions (red lines). (b): Geodesic projections do not preserve this property: distances between green projections are not the same as distances between blue projections. (c): Horospherical projections project data by sliding it along the complementary direction defined by horospheres (red circles) and there exist an isometric mapping between the blue and green projections. . . . . | 93  |
| 6.2 | Hyperbolic geodesics (black lines) going through an ideal point (in red), and horospheres (red circles) centered at that same ideal point. The hyperbolic lengths of geodesic segments between two horospheres are equal. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 98  |
| 6.3 | $x', y'$ are horospherical (green) projections of $x, y$ . Proposition 6.4.1 shows $d_{\mathbb{H}}(x', y') = d_{\mathbb{H}}(x, y)$ . The distance between the two geodesic (blue) projections is smaller. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 100 |
| A.1 | A coordinate chart $(U, x)$ on a manifold $M$ . Source: [71] . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 117 |
| A.2 | Illustration of four charts covering the circle $\mathbb{S}^1$ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 118 |
| A.3 | Transition function between two coordinate charts on a smooth manifold. Source: <a href="https://en.wikipedia.org/wiki/Atlas_(topology)">https://en.wikipedia.org/wiki/Atlas_(topology)</a> . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 119 |
| A.4 | Illustration of the exponential map on a Riemannian manifold. Source: [71]. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 126 |
| A.5 | Illustration of the hyperboloid (top) and its connection to the Poincaré disk (bottom). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 129 |
| B.1 | Tree transformations used to satisfy the leaf and the binary conditions in Proposition B.1.1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 137 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |     |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| B.2 | (a): Leaves embeddings and the origin in $\mathbb{B}_2$ . (b): Binary tree that is a quasi-isometry of the leaves embeddings and the origin (Proposition B.1.1) and corresponding discrete binary tree (not rooted) in (c). (d): Rooted binary tree obtained by decoding hyperbolic leaves embeddings. Observe that the quasi-isometric tree in (c) and the decoded tree in (d) are LCA equivalent with respect to 0. (e): Figure used in induction proof in Lemma B.2.3. . . . . | 141 |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|

# Chapter 1

## Introduction

### 1.1 Representation Learning

In the past decade, Machine Learning (ML) and Deep Learning methods have revolutionized many research areas including object detection [144], the game of Go [157], medical imaging [68], and protein folding [156]. In most scenarios, these methods follow a rather simple two-step paradigm. First they automatically extract meaningful patterns from data, without the need for manual feature engineering. This is the so-called *representation learning* step [23]. Second, they use these representations in downstream applications which can be *supervised* (e.g. classification) or *unsupervised* (e.g. clustering, visualization, nearest-neighbor search). This is the so-called *downstream task*.<sup>1</sup> A good data representation should be expressive enough that it preserves meaningful features found in the original data, but simple enough that it makes the downstream task easier. For instance, having low-dimensional representations of high-dimensional datasets can help overcome many issues caused by the curse of dimensionality such as overfitting.

When the input data lies in a high-dimensional Euclidean space, it is common to assume it sits in an intrinsically low-dimensional manifold. This is the standard *manifold assumption* problem. *Manifold learning* methods seek to recover this intrinsically low-dimensional manifold. This is usually done by first building a discrete approximation of the manifold using a graph in which nearby points in the ambient Euclidean space are connected by an edge. Because manifolds are locally Euclidean (see Appendix A.1.3), graph distances provide a good proxy for both local and global manifold distances. The second step is to “flatten” this graph representation by learning a non-linear mapping from nodes in the graph to points in a low-dimensional Euclidean space, while preserving graph distances as best as possible. These representations are usually easier to work with than the original high-dimensional representations, and can then be used in downstream applications (Fig. 1.1).

---

<sup>1</sup>For supervised tasks, these two steps are often combined into one learning algorithm, which learns both representations and decision rules on top of these representations.

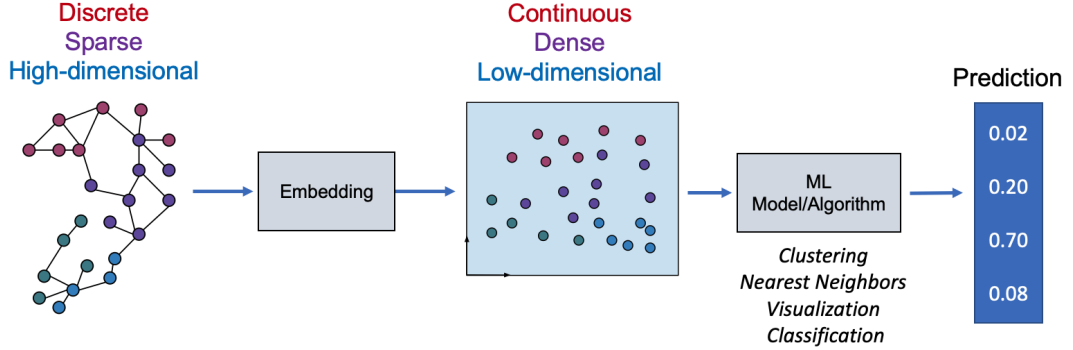


Figure 1.1: Overview of Machine Learning pipelines for graph-structured data. First, embedding models are used to learn graph representations. Then, these representations are used as inputs in downstream applications such as classification or data clustering.

In the early 2000s, non-linear<sup>2</sup> dimensionality reduction methods were extremely popular to solve the manifold learning problem. For instance, Laplacian Eigenmaps (LE) [20] use spectral techniques to compute embeddings, and IsoMap [169] use a combination of the Floyd–Warshall algorithm and the classical Multi-dimensional scaling algorithm to preserve global graph geodesics. While these methods have had critical impact on important applications such as RNA sequencing [126], they do not scale to large datasets. For instance, IsoMAP requires computing all pairs of shortest paths which takes quadratic time. A perhaps more important limitation of non-linear dimensionality reduction methods is their inability to compute embeddings for new datapoints because the mappings from node to embeddings are non-parametric.

In more recent years, graph embedding grew into its own area of research, due to the widespread prevalence of graph-structured data in many applications such as computational chemistry [76], biology [162], recommendation systems [106], semi-supervised learning [75]. Many ML and deep learning models have been proposed for the problem of graph embedding, and these methods are often referred to using the umbrella term of *Graph Representation Learning* (GRL) (see Chapter 2). In contrast with manifold learning applications where the graph structure is latent and inferred from high-dimensional data, graphs in GRL applications can be directly read from the data (e.g., user links in a social network). Because they leverage the expressiveness of deep neural networks, GRL models often yield more expressive, scalable and generalizable embeddings than classical methods.

In most GRL models [79, 102, 142], embeddings are learned in low-dimensional Euclidean spaces, in which the vector-space structure makes computations extremely easy (e.g., straight-line geodesics).

<sup>2</sup>The non-linearity term here comes to contrast with Euclidean dimensionality reduction methods (such as Principal Component Analysis) which rely on linear projections.

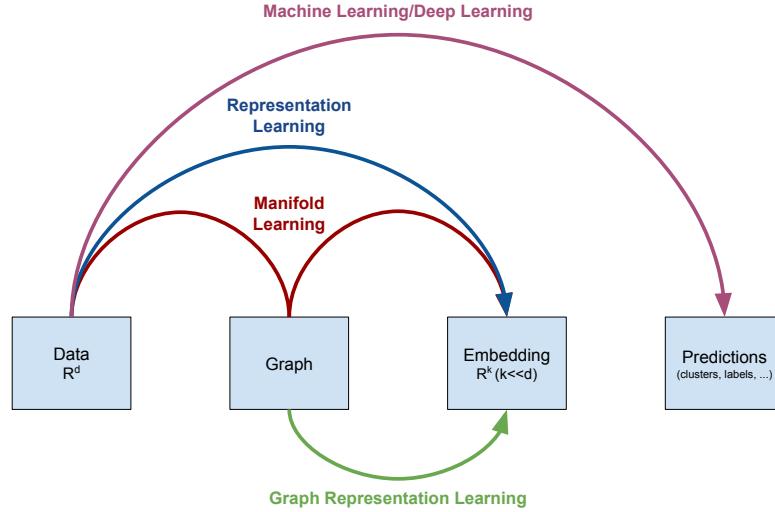


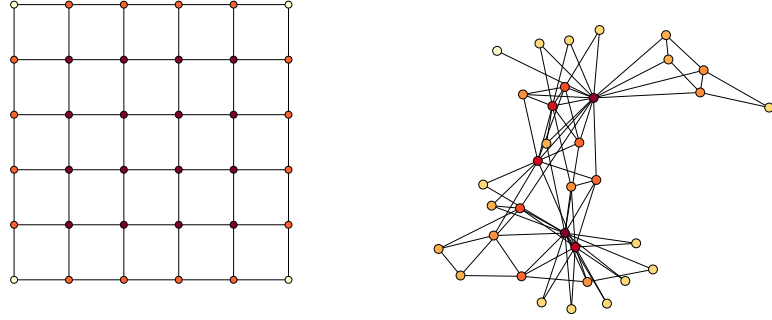
Figure 1.2: Machine Learning pipelines and relation between manifold learning and graph representation learning. We note that this categorization is not an exhaustive view of the field. For instance, a common learning setup is when the input consists of both points in a Euclidean space and a graph, i.e. data defined on a discretized graph domain. We will discuss this setting in Chapter 2.

However, while manifolds (or their discrete approximation via graphs) are locally Euclidean, it is not always possible to isometrically “flatten” all manifolds in low-dimensional Euclidean spaces. This is the case for curved manifolds such as hyperbolic or spherical manifolds, and learning Euclidean embeddings for graphs sampled from these manifolds will incur high distortion. In fact, many studies in network science [108, 139] showed that real-world graphs often exhibit non-Euclidean structures that cannot be modeled with Euclidean geometry without significant losses of information.

Our goal in this thesis is to develop models for graph-structured data that exhibits non-Euclidean structures (Fig. 1.3) by using alternative embedding geometries.<sup>3</sup> In particular, we focus on graphs with hierarchical and tree-like structures which arise in many core areas of computer science including computer vision [188], biology [8, 151], phylogenetics [25], natural language processing [112, 168] and internet [130, 133]. These hierarchical graphs can be represented with much better fidelity in *hyperbolic spaces* [152]. Intuitively, this is because hyperbolic spaces can be thought of as continuous versions of trees, where geodesics (generalization of shortest paths on manifolds) resemble shortest paths in discrete trees. Therefore, embedding hierarchical graphs in curved spaces that best match the geometry of the data yields significantly better-quality embeddings.

Leveraging hyperbolic spaces in ML applications comes with important challenges. First, in the representation learning step, one needs to define objective functions in terms of geodesics which are no longer simple straight lines, optimize embeddings with Riemannian optimization, and generalize

<sup>3</sup>If we think of a graph as the discrete analogue of a smooth manifolds, then a non-Euclidean graph can be thought of as the analog of a non-Euclidean manifold, i.e. which is not globally isomorphic to the Euclidean space.



(a) Grid (Euclidean).

(b) Arbitrary graph (Non-Euclidean).

Figure 1.3: An illustration of Euclidean vs. non-Euclidean graphs.

neural network operations to these curved spaces. Second, the usual ML algorithms for downstream applications (e.g. nearest neighbor, clustering) often rely on basic vector operations of Euclidean spaces and cannot be easily generalized to the hyperbolic space.

Our contributions in this thesis are twofold: First, we propose GRL models to learn graph representations in hyperbolic spaces, and generalize popular GRL models to the hyperbolic space, namely graph convolution networks, and knowledge graph embedding models. Second, we generalize fundamental downstream ML algorithms to visualize and extract information from data, namely clustering and dimensionality reduction, to the case where input datapoints come from a hyperbolic space. We structure the remainder of this introduction as follows:

- In Section 1.2, we review standard results of Euclidean embeddings and discuss some limitations that arise in high dimensions due to the curse of dimensionality.
- In Section 1.3, we provide more details on the properties of hyperbolic spaces and motivate their use to embed tree-like and hierarchical graphs. We will see that there are direct connections between tree metric spaces and hyperbolic spaces, leading to low-distortion embeddings of trees in hyperbolic spaces.
- In Section 1.4 we provide an outline for this dissertation and summarize our contributions in more details.

## 1.2 Euclidean Embeddings and the Curse of Dimensionality

The goal in GRL is to seek mappings from nodes in graphs to points in a continuous space (e.g. Euclidean) which preserve the graph information. Such mappings are called *embeddings*, a terminology that was borrowed from differential geometry and the theory of metric spaces. Formally, an embedding between metric spaces is defined as follows.

**Definition 1.2.1** (Embedding). *Let  $(X, d_X)$  and  $(Y, d_Y)$  denote two metric spaces. A mapping  $f : X \rightarrow Y$  is called an embedding.*

Given a complicated metric space  $(X, d_X)$  (e.g. metric derived from biological DNA sequences), the metric embedding problem is to find a simpler metric description (e.g. tree metric or Euclidean metric) via an embedding of  $(X, d_X)$ . This embedding should preserve distances in the original metric space as best as possible. It is not always possible to find an isometric (distance-preserving) embedding, and the quality of embeddings can be measured via the *worst-case distortion*:

$$D_{wc}(f) = \max_{x, x' \in X} \frac{d_Y(f(x), f(x'))}{d_X(x, x')} \cdot \max_{x, x' \in X} \frac{d_X(x, x')}{d_Y(f(x), f(x'))}.$$

Another commonly used metric is the average distortion:

$$D_{avg}(f) = \frac{1}{\binom{|X|}{2}} \left( \max_{x, x' \in X} \frac{|d_Y(f(x), f(x')) - d_X(x, x')|}{d_X(x, x')} \right).$$

The graph embedding problem can then be thought of as a metric embedding problem, where the input metric space  $X$  is simply a graph metric, and the output metric space is a simpler metric space such as the Euclidean space in small dimensions.

This metric embedding problem has been extensively studied in theoretical computer science, one motivation being the design of algorithms in simplified metric spaces. For instance, many problems (e.g. nearest neighbors, sparsest cuts, max cuts) deal with distance measures between items, and designing efficient algorithms for any arbitrary metric can be challenging.

In contrast, if the metric comes from a simplified geometric space such as the Euclidean space, efficient algorithms are easier to design. Examples of problems that can be solved efficiently using low-dimensional Euclidean embeddings include nearest neighbor search. For instance, k-d trees are space-partitioning data structures based on Euclidean coordinates and that can be leveraged to efficiently find nearest neighbors in low-dimensions.

Therefore, a natural question is: *can we embed any arbitrary metric in the Euclidean space with low-distortion?* Bourgain's fundamental result says the following.

**Theorem 1.2.1** (Bourgain, 1985). *Every metric space on  $n$  points embeds in  $(\mathbb{R}^{O(\log^2 n)}, \|\cdot\|_p)$  with distortion  $O(\log n)$ .*

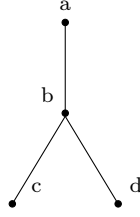


Figure 1.4: Simple graph metric on four nodes.

The embedding in Bourgain’s theorem is far from being isometric (distance-preserving) for large inputs. We may therefore ask a follow-up question: *can we always decrease the distortion of Euclidean embeddings by increasing the number of dimensions?* The answer is no and we’ll see an example next.

**Example 1.2.1.** Consider for instance a simple graph metric on four nodes in Fig. 1.4,  $(G, d_G)$ , and assume each edge has length one. It is not possible to embed this metric isometrically into the Euclidean space  $(\mathbb{R}^d, \|\cdot\|_2)$  of any dimension. We can prove this claim by contradiction: assume that there exist an isometric embedding  $f : G \rightarrow \mathbb{R}^d$ . We get:  $\|f(a) - f(d)\|_2 = \|f(a) - f(b)\|_2 + \|f(b) - f(d)\|_2$ . Therefore,  $(a, b, d)$  are colinear in  $\mathbb{R}^d$ . Similarly, we get that  $(a, b, c)$  are colinear in  $\mathbb{R}^d$  which implies that  $\|f(c) - f(d)\|_2 = 0$ . This contradicts the fact that  $d_G(c, d) = 2$ , which completes the proof.

**Remark 1.2.1.** This result in fact holds for arbitrary tree metrics, i.e. the Euclidean space of any dimension cannot embed (nontrivial) trees with low distortion [150, 152].

**Remark 1.2.2.** The example shown in Fig. 1.4 is a specific case of a tree metric, or more generally a 0-hyperbolic metric in the sense of Gromov [78]. We will see in Section 1.3 how this property ties nicely to the geometry of the hyperbolic space, which can be used to represent trees with arbitrarily low distortion in just two dimensions [152].

Therefore, it is not always possible to find low-distortion embeddings of graphs by increasing the dimensionality of the embedding space. Furthermore, for situations where this may be possible, increasing the dimensionality too much may cause issues due to the *curse of dimensionality*.

For instance, in the specific case of high-dimensional Euclidean spaces, the data becomes very sparse and distances concentrate to the same value. This causes many issues in downstream applications such as nearest neighbor search, clustering, or ML prediction tasks. For instance, the performance of k-d tree-based nearest neighbor search algorithms significantly degrades in high dimensions due to the curse of dimensionality. In the context of ML, working with high-dimensional may cause optimization issues and overfitting, thus the need to find low-dimensional representations. This therefore motivate the need for low-distortion *and* low-dimensional representations.

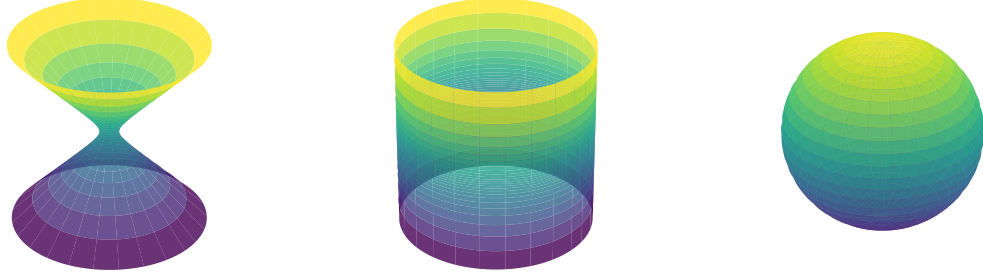


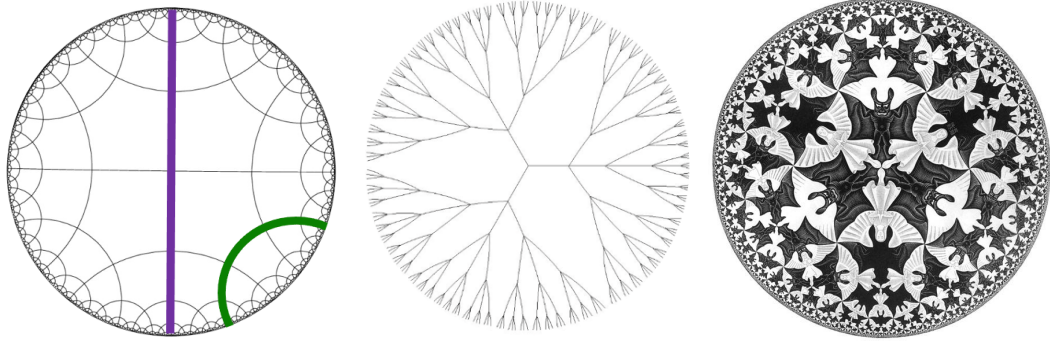
Figure 1.5: From left to right: a surface of negative curvature, a surface of zero (Gaussian) curvature, and a surface of positive curvature. Curvature essentially measures how manifolds deviate from the flat Euclidean space.

### 1.3 Hyperbolic Embeddings

We saw that Euclidean embeddings may not always yield low-distortion for certain graph structures such as trees. In particular, if we think of a graph as the discrete approximation of some smooth manifold, then this graph will isometrically embed in the Euclidean space only if the underlying manifold is intrinsically Euclidean. However, graphs are complex relational structure that may not always have a Euclidean geometry, and it is therefore natural to ask the following: *can we identify topological structures in graphs that correspond to certain underlying geometries, and can we use these geometries to learn better embeddings of graphs?* The answer is yes, specifically for the case of cycle and tree-like graphs, which have nice correspondences with spherical and hyperbolic geometries.

In this dissertation, our focus will be on the latter, and we will study different representation learning models and downstream algorithms that operate in hyperbolic spaces. In contrast with Euclidean spaces which have zero curvature, and spherical spaces which have constant positive curvature, hyperbolic spaces have constant negative curvature (Fig. 1.5). This induces very tight connections with tree-like graphs, making them ideal candidates to embed hierarchical data. We will discuss these connections in Section 1.3.2 but first start by giving a brief review of the Poincaré model of hyperbolic space in Section 1.3.1.

We assume familiarity with basic concepts of differential and Riemannian geometry, including notions like metric tensors, tangent spaces and exponential maps. For the reader that is not familiar with these concepts, we provide a brief review of Riemannian geometry in Appendix A, and we refer to standard texts [36, 113] for a more in-depth treatment.



(a) Poincaré geodesics. (b) Volume growth in trees. (c) Hyperbolic volume growth.  
 Source: <https://mathcs.clarku.edu/~djoyce/poincare> Source: [https://infovis-wiki.net/wiki/Hyperbolic\\_trees](https://infovis-wiki.net/wiki/Hyperbolic_trees) Source: <https://www.wikiart.org/en/m-c-escher/>

Figure 1.6: Hyperbolic geodesics in the Poincaré disk and illustration of exponential volume growth property.

### 1.3.1 The Poincaré Ball Model of Hyperbolic Space

There exist five equivalent models of hyperbolic spaces. Among these, the Poincaré ball model represents the infinite hyperbolic space in a finite ball, and is therefore very useful for visualizations. The hyperboloid (or Lorentz) model represents the space using the upper sheet of a two-sheeted hyperboloid, and is useful for computation due to relatively simple formulas for geodesics and similarities with spherical geometry. Here, we briefly review the Poincaré model of hyperbolic space and refer to Appendix A.4.4 and standard texts [33, 145] for more details on the foundations of hyperbolic geometry.

Consider the unit ball in  $d$  dimensions:  $\mathbb{B}^d := \{p \in \mathbb{R}^d : \|p\|_2^2 < 1\}$ . Endowed with the family of inner products:

$$g_p = \left( \frac{2}{1 - \|p\|_2^2} \right)^2 I_d,$$

the pair  $(\mathbb{B}^d, g)$  forms a Riemannian manifold of constant negative curvature  $-1$ , where curvature measures the deviation from flat Euclidean geometry.

**Distance function and geodesics** The induced distance between two points  $(x, y)$  in  $\mathbb{B}^d$  can be computed as:

$$d_{\mathbb{B}}(x, y) = \operatorname{arcosh} \left( 1 + 2 \frac{\|x - y\|_2^2}{(1 - \|x\|_2^2)(1 - \|y\|_2^2)} \right).$$

If  $y = o$ , the origin of the hyperbolic space, the distance function has the simple expression:  $d_o(x) := d(o, x) = 2 \tanh^{-1}(\|x\|_2)$ .

This distance function induces two types of geodesics (which are shortest paths in the hyperbolic

space): straight lines that go through the origin, and segments of circles perpendicular to the boundary of the ball (Fig. 1.6a).

Further, we can see from the distance formula that (Euclidean) norm-preserving transformations such as reflections, rotations centered at the origin and inversions are isometries of the hyperbolic space. We will extensively use these isometries throughout our work, especially inversions that map a reference point to the origin of the space.

**Geodesic submanifolds** A submanifold  $S \subset \mathbb{B}^d$  is called *(totally) geodesic* if for every  $x, y \in S$ , the geodesic line connecting  $x$  and  $y$  belongs to  $S$ . This generalizes the notion of affine subspaces in Euclidean spaces. In the Poincaré model, geodesic submanifolds are represented by linear subspaces going through the origin and spherical caps perpendicular to the boundary of the unit ball.

**Exponential and logarithmic maps** For each point  $x$  in a Riemannian manifold, the tangent space at  $x$ ,  $T_x$  is a  $d$ -dimensional vector space containing all possible directions of paths leaving from  $x$ . The tangent space maps to the manifold via the exponential map, and conversely, the logarithmic map maps point on the manifold to the tangent space at a point. In particular, we have closed-form expressions for these maps in the Poincaré at the when the reference point is the origin:

$$\exp_o(v) = \tanh(\sqrt{c}\|v\|) \frac{v}{\sqrt{c}\|v\|}, \quad (1.1)$$

$$\log_o(y) = \operatorname{arctanh}(\sqrt{c}\|y\|) \frac{y}{\sqrt{c}\|y\|}. \quad (1.2)$$

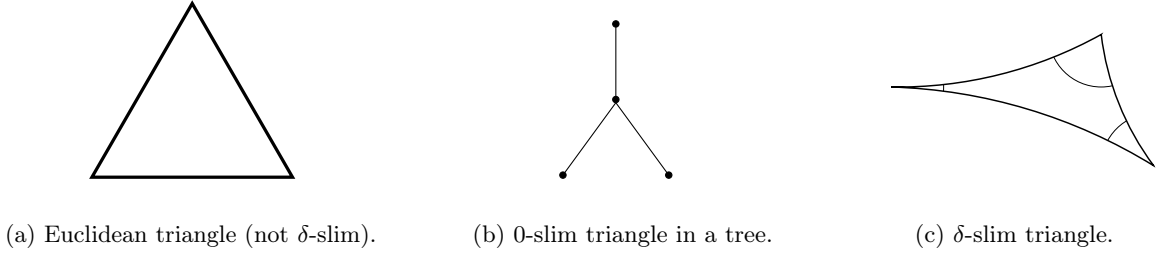
Because the tangent space has a linear structure, we will rely on these maps to leverage some of the tools of Euclidean geometry for hyperbolic methods, such as optimization or neural network operations.

**Gyrovector framework** We now briefly describe a mathematical framework by Ungar [177, 178], which introduces elementary operations such as additions and scalar multiplications, making it particularly useful to describe hyperbolic geometry in analogy with Euclidean geometry.

Vector addition is not well-defined in the hyperbolic space (adding two points in the Poincaré ball might result in a point outside the ball). Instead, Möbius addition  $\oplus$  [177] provides an analogue to Euclidean addition for hyperbolic space. The Möbius addition operation in the Poincaré ball model has the closed-form expression:

$$x \oplus y = \frac{(1 + 2x^T y + \|y\|^2)x + (1 - \|x\|^2)y}{1 + 2x^T y + \|x\|^2\|y\|^2}.$$

In contrast to Euclidean addition, it is neither commutative nor associative. However, it provides an analogue through the lens of parallel transport: given two points  $x, y$  and a tangent vector  $v$  in  $T_x$ , there is a unique tangent vector in  $T_y$  which creates the same angle as  $v$  with the direction of the

Figure 1.7: Illustration of the notion of  $\delta$ -slim triangles.

geodesic (shortest path) connecting  $x$  to  $y$ . This map is the parallel transport  $P_{x \rightarrow y}(\cdot)$ ; Euclidean parallel transport is the standard Euclidean addition. Analogously, the Möbius addition satisfies:  $x \oplus y = \exp_x(P_{o \rightarrow x}(\log_o(y)))$ .

Similarly, the Möbius scalar multiplications provides a way to scale points on a manifold by a scalar, and is defined in the Poincaré ball as:

$$s \otimes x := \tanh(s \tanh^{-1}(\|x\|_2)) \frac{x}{\|x\|_2}.$$

Previous work [73] used this gyrovector framework to design neural networks that operate in hyperbolic spaces. In fact, these operations show up in many of the formulas on the Poincaré ball, significantly simplifying notation. For instance, given two points  $x, y \in \mathbb{B}^d$ , the unique unit-speed geodesic connecting them can be parameterized by:

$$\gamma_{x \rightarrow y}(t) = x \oplus (-x \oplus y) \otimes t.$$

The hyperbolic distance on  $\mathbb{B}^d$  can then also be written as:

$$d_{\mathbb{B}}(x, y) = \frac{2}{\sqrt{c}} \operatorname{arctanh}(\sqrt{c} \| -x \oplus y \|).$$

### 1.3.2 Hyperbolic Metrics

Having introduced some basics of hyperbolic geometry, we now discuss the connections between tree metrics and hyperbolic spaces and motivate the use of hyperbolic spaces to represent hierarchical data.

Before its use in ML, hyperbolic geometry has been extensively studied to understand the geometry of complex networks and theoretical studies showed that many real-world networks exhibit hierarchical and tree-like structures with underlying hyperbolic geometry [108, 139]. But *how can we identify these hierarchical and tree-like structures?* This can be formalized the notion of Gromov delta-hyperbolicity [3, 49, 78] which we discuss next.

**Gromov delta-hyperbolicity** We define the Gromov [78] product which can be used to define  $\delta$ -hyperbolic spaces.

**Definition 1.3.1** (Gromov product). *In any metric space  $(X, d)$ , the Gromov product of points  $x, y \in X$  with respect to a third point  $z \in X$  is:*

$$\langle x, y \rangle_z = \frac{1}{2} (d(x, z) + d(z, y) - d(x, y)).$$

When the base point  $z$  is taken to be the origin or the root of a tree, we shorten this to  $\langle x, y \rangle$  unambiguously.

A key characterization of hyperbolic spaces is the notion of  $\delta$ -hyperbolicity, which essentially measures how close a metric is to a tree metric.

**Definition 1.3.2** ( $\delta$ -hyperbolicity, four-point condition). *A metric space  $(X, d)$  is  $\delta$ -hyperbolic if there exists  $\delta \geq 0$  such that for all  $w, x, y, z$  in  $X$ :*

$$\langle x, y \rangle_z \geq \min\{\langle x, w \rangle_z, \langle w, y \rangle_z\} - \delta.$$

Up to changing  $\delta$  by a constant multiple, there are many equivalent variations of the notion of  $\delta$ -hyperbolicity. In particular, one intuitive interpretation of  $\delta$ -hyperbolic spaces is using the notion of  $\delta$ -slim triangles, which says that any triangle in a  $\delta$ -hyperbolic space has distance from any side to the other two less than  $\delta$ .

**Example 1.3.1.** The Euclidean space  $\mathbb{R}^d$  is not  $\delta$ -hyperbolic and Euclidean triangles are not  $\delta$ -slim. For instance the midpoint of a large isosceles triangle might be far from the other two sides (Fig. 1.7a).

**Example 1.3.2.** Metric trees are 0-hyperbolic (Fig. 1.7b).

Therefore, metric spaces with smaller  $\delta$  will be close to a tree metric (Fig. 1.7c). This is the case for hyperbolic spaces which are  $\delta$ -hyperbolic in the sense of Gromov.

**Example 1.3.3.** The hyperbolic space is log 3-hyperbolic.

**Connections between trees and hyperbolic spaces** Now let's take another look at Example 1.2.1. We proved that a small tree on four nodes cannot be embedded into the Euclidean space of any dimension (in fact this generally holds for any nontrivial tree). Looking at the notion of delta-hyperbolicity, we see that hyperbolic spaces are much more similar to tree metric spaces, due to the delta-hyperbolic condition. In particular, the behaviour of geodesics in  $\delta$ -slim triangles resembles that of shortest paths in trees: the distance between two points in a slim triangle approaches the sum of distances to the center of the triangle, similar to trees where the shortest path between two nodes in a triangle is the sum of distances to the center or root of the tree.

More generally, there are important properties such as quasi-isometries between  $\delta$ -hyperbolic spaces and tree metric spaces. First, any finite set of points in a  $\delta$ -hyperbolic space can be embedded in a tree metric with bounded distortion:

**Proposition 1.3.1** (Tree-likeness of hyperbolic space). *There is a constant  $C_n = \delta \cdot O(n)$  such that for any set of  $n$  points  $x_1, \dots, x_n$  in a  $\delta$ -hyperbolic space  $(M, d_M)$  can be embedded via  $f : M \rightarrow T$  into a tree metric  $(T, d_T)$  such that:*

$$\forall i, j : d_M(x_i, x_j) \leq d_T(f(x_i), f(x_j)) \leq d_M(x_i, x_j) + C_n.$$

Conversely, Sarkar [152] derived a construction to embed trees in hyperbolic spaces with arbitrarily low-distortion in just two-dimensions.

**Proposition 1.3.2** (Sarkar). *Any unit-weight tree  $T$  can be embedded into  $(\mathbb{B}_2, d_{\mathbb{B}})$  with scale  $\zeta = O(1/\varepsilon)$  and worst-case distortion at most  $1 + \varepsilon$ , i.e.*

$$\zeta d_T(i, j) \leq d_{\mathbb{B}}(z_i, z_j) \leq \zeta(1 + \varepsilon) d_T(i, j).$$

These nice quasi-isometries between hyperbolic spaces and tree metrics spaces motivate the use of hyperbolic geometry to embed tree-like graphs with low-distortion. Further, because we have mapping from and to trees, these hyperbolic spaces can be thought as some continuous version of trees. We will leverage these properties in Chapter 5 to continuously search the discrete space of discrete binary trees with gradient-based optimization.

**Exponential volume growth** Finally, another intuitive property of hyperbolic spaces which illustrates their connections to trees is the exponential volume growth. Thinking again of trees as the discrete approximation of some underlying hyperbolic manifold, we can define a notion of volume in trees which as the number of nodes contained within some bounded distance (the radius) to the root of the tree. This volume grows exponentially as we increase the radius (Fig. 1.6b).

In Euclidean space, the volume of balls grows polynomially as a function of the radius, making it less suitable to embed exponentially growing graphs like trees. More concretely, because the growth is not as fast as that of the embedded graph, the embedding space quickly becomes too crowded (e.g. geodesics intersect each other and cause high distortion).

In contrast,  $\delta$ -hyperbolic spaces, and in particular hyperbolic spaces have exponential growth, which yields more room to fit complex hierarchies and makes these spaces ideal candidates to embed trees while accommodating for their exponential volume growth (Fig. 1.6c).

## 1.4 Contributions and Outline

We now move provide an overview of our key contributions. As mentioned in Section 1.1, ML generally follow a two step paradigm: *representation learning* and *downstream tasks* such as classification, clustering or data visualization.

Our contribution in this dissertation follows this paradigm. First, we propose GRL models that learn representations for hierarchical graphs in hyperbolic spaces (Section 1.4.1). Second, we design algorithms that work with hyperbolic inputs for important downstream applications: hierarchical clustering and dimensionality reduction (Section 1.4.2).

### 1.4.1 Part I: Embeddings

A core aspect of GRL is the underlying embedding space geometry. Graphs are discrete, high-dimensional, non-Euclidean structures, and there is no straightforward way to encode this information into low-dimensional Euclidean spaces while preserving the graph topology. We discussed the limitations of Euclidean embeddings in Section 1.2 and motivated the use of hyperbolic geometry to embed tree-like graphs in Section 1.3. Many real-world graphs such as knowledge graphs, biological protein-protein interactions networks have hierarchies and it is therefore important to design GRL models with the right inductive biases for these hierarchical graphs.

Our first contribution in this dissertation is the development of hyperbolic graph embedding models. In particular, we propose two embedding models for applications that deal with multi-relational graphs and graphs that come with additional node feature information.

- Chapter 2 is based on the 2019 review paper “Machine Learning on Graphs: A Model and Comprehensive Taxonomy” [39]. In this chapter, we describe the Graph Representation Learning (GRL) problem in more details, and provide a technical review of different GRL algorithms that will be used throughout this dissertation. We provide a comprehensive framework, GRAPHEDM, for GRL and discuss key notions such as inductive and transductive learning, supervised and unsupervised GRL, deep and shallow encoders, and we discuss the role of node features in GRL. The GRAPHEDM framework connects many areas of GRL, including traditional manifold learning methods and more recent deep neural networks for graph-structured data. To illustrate its generality, we fit over thirty existing methods into this framework, including Poincaré embeddings [136] and Graph Convolutional Networks (GCNs) [102].
- Chapter 3 is based on the NeurIPS 2019 paper “Hyperbolic Graph Convolutional Neural Networks” [38]. In this chapter, we generalize neural network operations such as neighborhood aggregation to hyperbolic spaces and propose Hyperbolic Graph Convolutional Neural Network (HGCN), the first hyperbolic GCN that leverages both the expressiveness of GCNs and hyperbolic geometry to learn node representations for hierarchical graphs. In contrast with existing shallow

hyperbolic GRL methods [136, 150], HGCN leverages node features making it inductive and applicable to unseen test data. Our experiments show that our method leads to significant improvements in terms of expressiveness and performance compared to Euclidean analogs on graphs which exhibit different extents of hierarchical structure. On the semi-supervised node classification tasks, this approach improves the state-of-the-art on the Pubmed citation benchmark.

- Chapter 4 is based on the ACL 2020 paper “Low-dimensional Hyperbolic Knowledge Graph Embeddings” [41]. In this work, we focus on learning hyperbolic representations for multi-relational graphs. We specifically focus on Knowledge Graphs (KGs), which often exhibit hierarchical and logical patterns that must be preserved in the embedding space. We introduce a class of hyperbolic KG embedding models that simultaneously capture hierarchical and logical patterns. Our approach combines hyperbolic isometries with attention to model complex relational patterns. Experimental results on standard KG benchmarks show that our method improves over previous methods by up to 6.1% in mean reciprocal rank, improving the state-of-the-art.

#### 1.4.2 Part II: Algorithms

GRL methods learn graph representations in a continuous embedding space. These representations can then be used as features for downstream prediction tasks, or for data visualization and analysis.

When representations come from non-Euclidean spaces such as the hyperbolic space, crucial ML algorithms such as clustering, nearest neighbor search and dimensionality reduction do not easily generalize to the hyperbolic case since they rely on fundamental axioms of Euclidean geometry (e.g. shortest paths between points are straight lines). This motivates the need for algorithms that operate in hyperbolic spaces, which is our second main contribution.

- Chapter 5 is based on the NeurIPS 2020 paper “From Trees to Continuous Embeddings and Back: Hyperbolic Hierarchical Clustering” [40]. In this work, we propose an algorithm to hierarchically cluster data in hyperbolic spaces with provable guarantees. These clusters can be visualized to understand properties of hyperbolic embeddings, or to detect communities in networks. In particular, our decoding algorithm can be used in conjunction with an encoding algorithm to continuously search the discrete space of binary trees with gradient-based optimization for similarity-based Hierarchical Clustering (HC). This work is the first continuous relaxation of Dasgupta’s discrete optimization for HC [62] with provable quality guarantees. Experimentally, we compare our approach to agglomerative heuristics on a variety of HC benchmarks and find that even approximate solutions found with gradient descent have superior clustering quality.
- Chapter 6 is based on the ICML 2021 paper “Hyperbolic Dimensionality Reduction via Horospherical Projections” [42]. In this work, we study the problem of dimensionality reduction in hyperbolic spaces, which is important for data analysis and visualization. We propose HoroPCA, a generalization to the hyperbolic space of the standard Euclidean Principal Component Analysis (PCA)

In contrast with previous methods, our formulation of hyperbolic PCA is the direct analogue of Euclidean PCA and preserves core PCA properties such as location-invariance. By focusing on the core problem of extracting principal directions, HoroPCA theoretically better preserves information in the original data such as distances, compared to previous generalizations of PCA. Empirically, we validate that HoroPCA outperforms existing dimensionality reduction methods, significantly reducing error in distance preservation and yielding much more interpretable data visualizations.

## Part I

# Hyperbolic Graph Representation Learning

## Chapter 2

# Graph Representation Learning

### 2.1 Introduction

Learning data representations has been a challenging and active research area in recent years, and many successful models have been developed for certain kinds of structured data, including image and text data. In many settings, data is not nearly as regular: complex relational structures commonly arise, and extracting information from that structure is key to understanding how objects interact with each other. In particular, graphs are universal data structures to represent relational data, and their prevalence in many important applications has led to the development of Graph Representation Learning (GRL) methods. These methods map nodes in a graph to low-dimensional representations (embeddings) in a continuous space, called embedding space, while preserving structural information in the embedding space (e.g. nodes that are close in the graph should be mapped to points that are close in the embedding space).

Broadly speaking, GRL can be divided into two classes of learning problems, *unsupervised* and *supervised* (or semi-supervised) GRL. The first family aims at learning low-dimensional Euclidean representations that preserve the structure of an input graph. The second family also learns low-dimensional Euclidean representations but for a specific downstream prediction task such as node or graph classification. Different from the unsupervised setting where inputs are usually graph structures, inputs in supervised settings are usually composed of different signals defined on graphs, commonly known as *node features*. Additionally, the underlying discrete graph domain can be fixed, which is the *transductive* learning setting (e.g. predicting user properties in a large social network), but can also vary in the *inductive* learning setting (e.g. predicting molecules attribute where each molecule is a graph). Finally, while most supervised and unsupervised methods learn representations in Euclidean vector spaces, there recently has been interest for *non-Euclidean representation learning*, which aims at learning non-Euclidean embedding spaces such as hyperbolic or spherical spaces. The main motivations for this body of work is to use a continuous embedding space that resembles the

underlying discrete structure of the input data it tries to embed (e.g. the hyperbolic space is a continuous version of trees [152]).

Given the impressive pace at which the field of GRL is growing, we believe it is important to summarize and describe all methods in one unified and comprehensible framework. The goal of this chapter is to provide a unified view of representation learning methods for graph-structured data, to better understand the different ways to leverage graph structure in deep learning models.

In this work, we extend the encoder-decoder framework proposed by Hamilton et al. [85] and introduce a general framework, the Graph Encoder Decoder Model (GRAPHEDM), which groups existing work into four major categories: (i) shallow embedding methods, (ii) auto-encoding methods, (iii) graph regularization methods, and (iv) graph neural networks (GNNs). This allows us to analyze and compare a variety of GNNs, ranging in construction from methods operating in the Graph Fourier<sup>1</sup> domain to methods applying self-attention as a neighborhood aggregation function [180]. We hope that this unified formalization of recent work would help the reader gain insights into the various learning methods on graphs to reason about similarities, differences, and point out potential extensions and limitations. That said, our contributions are threefold:

- We introduce a general framework (Section 2.3.1), GRAPHEDM, to describe a broad range of supervised and unsupervised methods that operate on graph-structured data, namely shallow embedding methods, graph regularization methods, graph auto-encoding methods and graph neural networks.
- We provide a general taxonomy (Section 2.3.2 and Section 2.3.3) unifying these different lines of work from the same perspective. In particular, this taxonomy encapsulates over thirty existing GRL methods. Describing these methods within a comprehensive taxonomy gives insight to exactly how these methods differ.
- We release an open-source library for GRL which includes state-of-the-art GRL methods and important graph applications, including node classification and link prediction. Our implementation is publicly available at <https://github.com/google/gcnv-survey-paper>.

**Organization of this chapter** We first review basic graph definitions and clearly state the problem setting for GRL (Section 2.2). In particular, we define and discuss the differences between important concepts in GRL, including the role of node features in GRL and how they relate to supervised GRL, the distinctions between inductive and transductive learning, and the differences between supervised and unsupervised embeddings. We then introduce GRAPHEDM (Section 2.3) a general framework to describe both supervised and unsupervised GRL methods, with or without the presence of node features, which can be applied in both inductive and transductive learning settings. Based on GRAPHEDM, we introduce a general taxonomy of GRL methods (Fig. 2.2) which encapsulates over thirty recent GRL models. We then use the GRAPHEDM framework and

---

<sup>1</sup>As defined by the eigenspace of the graph Laplacian.

the corresponding taxonomy to describe popular GRL models (Section 2.4) that we will refer to throughout this dissertation. Finally, we survey important graph applications in ML in Section 2.5.

## 2.2 Preliminaries

Here we introduce the notation used throughout this chapter (see Table 2.1 for a summary), and the generalized network embedding problem which graph representation learning methods aim to solve.

### 2.2.1 Definitions

**Definition 2.2.1.** (*Graph*). A graph  $G$  given as a pair:  $G = (V, E)$ , comprises a set of vertices (or nodes)  $V = \{v_1, \dots, v_{|V|}\}$  connected by edges  $E = \{e_1, \dots, e_{|E|}\}$ , where each edge  $e_k$  is a pair  $(v_i, v_j)$  with  $v_i, v_j \in V$ . A graph is weighted if there exist a weight function:  $w : (v_i, v_j) \rightarrow w_{ij}$  that assigns weight  $w_{ij}$  to edge connecting nodes  $v_i, v_j \in V$ . Otherwise, we say that the graph is unweighted. A graph is undirected if  $(v_i, v_j) \in E$  implies  $(v_j, v_i) \in E$ , i.e. the relationships are symmetric, and directed if the existence of edge  $(v_i, v_j) \in E$  does not necessarily imply  $(v_j, v_i) \in E$ . Finally, a graph can be homogeneous if nodes refer to one type of entity and edges to one relationship. It can be heterogeneous if it contains different types of nodes and edges.

For instance, social networks are homogeneous graphs that can be undirected (e.g. to encode symmetric relations like friendship) or directed (e.g. to encode the relation following); weighted (e.g. co-activities) or unweighted.

**Definition 2.2.2.** (*Path*). A path  $P$  is a sequence of edges  $(u_{i_1}, u_{i_2}), (u_{i_2}, u_{i_3}), \dots, (u_{i_k}, u_{i_{k+1}})$  of length  $k$ . A path is called simple if all  $u_{i_j}$  are distinct from each other. Otherwise, if a path visits a node more than once, it is said to contain a cycle.

**Definition 2.2.3.** (*Distance*). Given two nodes  $(u, v)$  in a graph  $G$ , we define the distance from  $u$  to  $v$ , denoted  $d_G(u, v)$ , to be the length of the shortest path from  $u$  to  $v$ , or  $\infty$  if there exist no path from  $u$  to  $v$ .

The graph distance between two nodes is the analog of geodesic lengths on manifolds.

**Definition 2.2.4.** (*Vertex degree*). The degree,  $\deg(v_i)$ , of a vertex  $v_i$  in an unweighted graph is the number of edges incident to it. Similarly, the degree of a vertex  $v_i$  in a weighted graph is the sum of incident edges weights. The degree matrix  $D$  of a graph with vertex set  $V$  is the  $|V| \times |V|$  diagonal matrix such that  $D_{ii} = \deg(v_i)$ .

**Definition 2.2.5.** (*Adjacency matrix*). A finite graph  $G = (V, E)$  can be represented as a square  $|V| \times |V|$  adjacency matrix, where the elements of the matrix indicate whether pairs of nodes are adjacent or not. The adjacency matrix is binary for unweighted graph,  $A \in \{0, 1\}^{|V| \times |V|}$ , and non-binary for weighted graphs  $W \in \mathbb{R}^{|V| \times |V|}$ . Undirected graphs have symmetric adjacency matrices, in

|                   | Notation                                              | Meaning                                                                         |
|-------------------|-------------------------------------------------------|---------------------------------------------------------------------------------|
| Abbreviations     | GRL                                                   | Graph Representation Learning                                                   |
|                   | GRAPHEDM                                              | Graph Encoder Decoder Model                                                     |
|                   | GNN                                                   | Graph Neural Network                                                            |
| Graph notation    | $G = (V, E)$                                          | Graph with vertices (nodes) $V$ and edges $E$                                   |
|                   | $v_i \in V$                                           | Graph vertex                                                                    |
|                   | $d_G(\cdot, \cdot)$                                   | Graph distance (length of shortest path)                                        |
|                   | $\deg(\cdot)$                                         | Node degree                                                                     |
|                   | $D \in \mathbb{R}^{ V  \times  V }$                   | Diagonal degree matrix                                                          |
|                   | $W \in \mathbb{R}^{ V  \times  V }$                   | Graph weighted adjacency matrix                                                 |
|                   | $\widetilde{W} \in \mathbb{R}^{ V  \times  V }$       | Symmetric normalized adjacency matrix ( $\widetilde{W} = D^{-1/2} W D^{-1/2}$ ) |
|                   | $A \in \{0, 1\}^{ V  \times  V }$                     | Graph unweighted weighted adjacency matrix                                      |
|                   | $L \in \mathbb{R}^{ V  \times  V }$                   | Graph unnormalized Laplacian matrix ( $L = D - W$ )                             |
|                   | $\tilde{L} \in \mathbb{R}^{ V  \times  V }$           | Graph normalized Laplacian matrix ( $\tilde{L} = I - D^{-1/2} W D^{-1/2}$ )     |
|                   | $L^{\text{rw}} \in \mathbb{R}^{ V  \times  V }$       | Random walk normalized Laplacian ( $L^{\text{rw}} = I - D^{-1} W$ )             |
| GRAPHEDM notation | $d_0$                                                 | Input feature dimension                                                         |
|                   | $X \in \mathbb{R}^{ V  \times d_0}$                   | Node feature matrix                                                             |
|                   | $d$                                                   | Final embedding dimension                                                       |
|                   | $Z \in \mathbb{R}^{ V  \times d}$                     | Node embedding matrix                                                           |
|                   | $d_\ell$                                              | Intermediate hidden embedding dimension at layer $\ell$                         |
|                   | $H^\ell \in \mathbb{R}^{ V  \times d_\ell}$           | Hidden representation at layer $\ell$                                           |
|                   | $\mathcal{Y}$                                         | Label space                                                                     |
|                   | $y^S \in \mathbb{R}^{ V  \times  \mathcal{Y} }$       | Graph ( $S = G$ ) or node ( $S = N$ ) ground truth labels                       |
|                   | $\hat{y}^S \in \mathbb{R}^{ V  \times  \mathcal{Y} }$ | Predicted labels                                                                |
|                   | $s(W) \in \mathbb{R}^{ V  \times  V }$                | Target similarity or dissimilarity matrix in graph regularization               |
|                   | $\widehat{W} \in \mathbb{R}^{ V  \times  V }$         | Predicted similarity or dissimilarity matrix                                    |
|                   | $\text{ENC}(\cdot; \Theta^E)$                         | Encoder network with parameters $\Theta^E$                                      |
|                   | $\text{DEC}(\cdot; \Theta^D)$                         | Graph decoder network with parameters $\Theta^D$                                |
|                   | $\text{DEC}(\cdot; \Theta^S)$                         | Label decoder network with parameters $\Theta^S$                                |
|                   | $\mathcal{L}_{\text{SUP}}^S(y^S, \hat{y}^S; \Theta)$  | Supervised loss                                                                 |
|                   | $\mathcal{L}_{G, \text{REG}}(W, \widehat{W}; \Theta)$ | Graph regularization loss                                                       |
|                   | $\mathcal{L}_{\text{REG}}(\Theta)$                    | Parameters' regularization loss                                                 |
|                   | $d_1(\cdot, \cdot)$                                   | Matrix distance used for to compute the graph regularization loss               |
|                   | $d_2(\cdot, \cdot)$                                   | Embedding distance for distance-based decoders                                  |
|                   | $\ \cdot\ _p$                                         | $p$ -norm                                                                       |
|                   | $\ \cdot\ _F$                                         | Frobenius norm                                                                  |

Table 2.1: Summary of the notation used in this chapter.

which case,  $\widetilde{W}$  denotes symmetrically-normalized adjacency matrix:  $\widetilde{W} = D^{-1/2}WD^{-1/2}$ , where  $D$  is the degree matrix.

**Definition 2.2.6.** (*Laplacian*). The unnormalized Laplacian of an undirected graph is the  $|V| \times |V|$  matrix  $L = D - W$ . The symmetric normalized Laplacian is  $\widetilde{L} = I - D^{-1/2}WD^{-1/2}$ . The random walk normalized Laplacian is the matrix  $L^{\text{rw}} = I - D^{-1}W$ .

The name random walk comes from the fact that  $D^{-1}W$  is a stochastic transition matrix that can be interpreted as the transition probability matrix of a random walk on the graph. The graph Laplacian is a key operator on graphs and can be interpreted as the analogue of the continuous Laplace-Beltrami operator on manifolds. Its eigenspace capture important properties about a graph (e.g. cut information often used for spectral graph clustering) but can also serve as a basis for smooth functions defined on the graph for semi-supervised learning [21]. The graph Laplacian is also closely related to the heat equation on graphs as it is the generator of diffusion processes on graphs and can be used to derive algorithms for semi-supervised learning on graphs [202].

**Definition 2.2.7.** (*First order proximity*). The first order proximity between two nodes  $v_i$  and  $v_j$  is a local similarity measure indicated by the edge weight  $w_{ij}$ . In other words, the first-order proximity captures the strength of an edge between node  $v_i$  and node  $v_j$  (should it exist).

**Definition 2.2.8.** (*Second-order proximity*). The second order proximity between two nodes  $v_i$  and  $v_j$  is measures the similarity of their neighborhood structures. Two nodes in a network will have a high second-order proximity if they tend to share many neighbors.

Note that there exist higher-order measures of proximity between nodes such as Katz Index, Adamic Adar or Rooted PageRank [116]. These notions of node proximity are particularly important in network embedding as many algorithms are optimized to preserve some order of node proximity in the graph.

## 2.2.2 The Generalized Network Embedding Problem

*Network embedding* is the task that aims at learning a mapping function from a discrete graph to a continuous domain. Formally, given a graph  $G = (V, E)$  with weighted adjacency matrix  $W \in \mathbb{R}^{|V| \times |V|}$ , the goal is to learn low-dimensional vector representations  $\{Z_i\}_{i \in V}$  (embeddings) for nodes in the graph  $\{v_i\}_{i \in V}$ , such that important graph properties (e.g. local or global structure) are preserved in the embedding space. For instance, if two nodes have similar connections in the original graph, their learned vector representations should be close. Let  $Z \in \mathbb{R}^{|V| \times d}$  denote the node<sup>2</sup> embedding matrix. In practice, we often want low-dimensional embeddings ( $d \ll |V|$ ) for efficient

<sup>2</sup>Although we present the model taxonomy via embedding nodes yielding  $Z \in \mathbb{R}^{|V| \times d}$ , it can also be extended for models that embed an entire graph i.e. with  $Z \in \mathbb{R}^d$  as a  $d$ -dimensional vector for the whole graph (e.g. [6, 67]), or embed graph edges  $Z \in \mathbb{R}^{|V| \times |V| \times d}$  as a (potentially sparse) 3D matrix with  $Z_{u,v} \in \mathbb{R}^d$  representing the embedding of edge  $(u, v)$ .

representations. That is, network embedding can be viewed as a dimensionality reduction technique for graph structured data, where the input data is defined on a non-Euclidean, high-dimensional, discrete domain.

### 2.2.2.1 Node features in network embedding

**Definition 2.2.9.** (*Vertex and edge fields*). A vertex field is a function defined on vertices  $f : V \rightarrow \mathbb{R}$ . Vertex fields can be viewed as analogs of scalar fields on manifolds.

Graphs may have node attributes (e.g. gender or age in social networks; article contents for citation networks) which can be represented as multiple vertex fields, commonly referred to as *node features*, which we denote  $X \in \mathbb{R}^{|V| \times d_0}$ , where  $d_0$  is the input feature dimension. These features are to contrast with input datapoints in manifold learning, which usually indicate the position of nodes on a intrinsically low-dimensional manifold embedding in a high-dimensional space. Here, node features and edges come from different data sources, and proximity in the graph (or manifold) does not necessarily correlate proximity in feature space.

However, both sources of data may contain valuable information and for some tasks and some network embedding algorithms leverage this information by learning mappings:

$$W, X \rightarrow Z.$$

In other scenarios, node features might be unavailable or not useful for a given task: network embedding can be *featureless*. That is, the goal is to learn graph representations via mappings:

$$W \rightarrow Z.$$

Note that depending on whether node features are used or not in the embedding algorithm, the learned representation could capture different aspects about the graph. If nodes features are being used, embeddings could capture both *structural* and *semantic* graph information. On the other hand, if node features are not being used, embeddings will only preserve topological structure of the graph.

### 2.2.2.2 Transductive and inductive network embedding

Historically, a popular way of categorizing a network embedding method has been by whether the model can generalize to unseen data instances – methods are referred to as operating in either a *transductive* or *inductive* setting [193]. While we do not use this concept for constructing our taxonomy, we include a brief discussion here for completeness.

In transductive settings, it assumed that all nodes in the graph are observed in training (typically the nodes all come from one fixed graph). These methods are used to infer information about or between observed nodes in the graph (e.g. predicting labels for all nodes, given a partial labeling).

For instance, if a transductive method is used to embed the nodes of a social network, it can be used to suggest new edges (e.g. friendships) between the nodes of the graph. One major limitation of models learned in transductive settings is that they fail to generalize to new nodes (e.g. evolving graphs) or new graph instances.

On the other hand, in inductive settings, models are expected to generalize to new nodes, edges, or graphs that were not observed during training. Formally, given training graphs  $(G_1, \dots, G_k)$ , the goal is to learn a mapping to continuous representations that can generalize to unseen test graphs  $(G_{k+1}, \dots, G_{k+l})$ . For instance, inductive learning can be used to embed molecular graphs, each representing a molecule structure [76], generalizing to new graphs and showing error margins within chemical accuracy on many quantum properties. Embedding dynamic or temporally evolving graphs is also another inductive graph embedding problem.

There is a strong connection between inductive graph embedding and *node features* (Section 2.2.2.1) as the latter are usually necessary for most inductive graph representation learning algorithms. More concretely, node features can be leveraged to learn embeddings with parametric mappings and instead of directly optimizing the embeddings, one can optimize the mapping’s parameters. The learned mapping can then be applied to any node (even those that were not present at training time). On the other hand, when node features are not available, the first mapping from nodes to embeddings is usually a one-hot encoding which fails to generalize to new graphs where the canonical node ordering is not available.

Finally, we note that this categorization of graph embedding methods is at best an incomplete lens for viewing the landscape. While some models are inherently better suited to different tasks in practice, recent theoretical results [161] show that models previously assumed to be capable of only one setting (e.g. only transductive) can be used in both.

### 2.2.2.3 Unsupervised and supervised network embedding

Network embedding can be *unsupervised* in the sense that the only information available is the graph structure (and possibly node features) or *supervised*, if additional information such as node or graph labels is provided. In unsupervised network embedding, the goal is to learn embeddings that preserve the graph structure and this is usually achieved by optimizing some reconstruction loss, which measures how well the learned embeddings can approximate the original graph.

In supervised network embedding, the goal is to learn embeddings for a specific purpose such as predicting node or graph attributes, and models are optimized for a specific task such as graph classification or node classification. Looking back at the two-step ML pipeline diagram in Fig. 1.1, this is like solving both the embedding and downstream step together using a supervised loss objective.

We will use these different levels of supervision to build our taxonomy in Section 2.3.2 and will discuss differences between supervised and unsupervised methods in more details in Section 2.3.

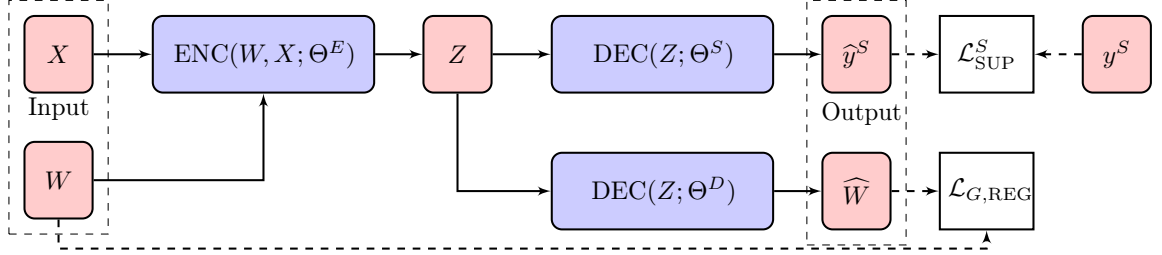


Figure 2.1: Illustration of the GRAPHEDM framework. Based on the supervision available, methods will use some or all of the branches. In particular, unsupervised methods do not leverage label decoding for training and only optimize the similarity or dissimilarity decoder (lower branch). On the other hand, semi-supervised and supervised methods leverage the additional supervision to learn models’ parameters (upper branch).

## 2.3 A Taxonomy of GRL Models

We first describe our proposed framework, GRAPHEDM, a general framework for GRL (Section 2.3.1). In particular, GRAPHEDM is general enough that it can be used to succinctly describe over thirty GRL methods (both unsupervised and supervised). We use GRAPHEDM to introduce a comprehensive taxonomy in Section 2.3.2 and Section 2.3.3, which summarizes exiting works with shared notations and simple block diagrams, making it easier to understand similarities and differences between GRL methods.

### 2.3.1 The GRAPHEDM Framework

The GRAPHEDM framework builds on top of the work of Hamilton et al. [85], which describes unsupervised network embedding methods from an encoder-decoder perspective. Cruz et al. [58] also recently proposed a modular encoder-based framework to describe and compare unsupervised graph embedding methods. Different from these unsupervised frameworks, we provide a more general framework which additionally encapsulates supervised graph embedding methods, including ones utilizing the graph as a regularizer (e.g. [203]), and graph neural networks such as ones based on message passing [76, 153] or graph convolutions [30, 102].

**Input** The input in GRAPHEDM is an undirected weighted graph  $G = (V, E)$ , with adjacency matrix  $W \in \mathbb{R}^{|V| \times |V|}$ , and optional node features  $X \in \mathbb{R}^{|V| \times d_0}$ . In (semi-)supervised settings, we assume that we are given training target labels for nodes (denoted  $N$ ), edges (denoted  $E$ ), and/or for the entire graph (denoted  $G$ ). We denote the supervision signal as  $S \in \{N, E, G\}$ , as presented below.

**Model** The GRAPHEDM framework can be decomposed as follows:

- **Graph encoder network**  $\text{ENC}_{\Theta^E} : \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d_0} \rightarrow \mathbb{R}^{|V| \times d}$ , parameterized by  $\Theta^E$ , which combines the graph structure with node features (or not) to produce node embedding matrix  $Z \in \mathbb{R}^{|V| \times d}$  as:

$$Z = \text{ENC}(W, X; \Theta^E).$$

As we shall see next, this node embedding matrix might capture different graph properties depending on the supervision used for training.

- **Graph decoder network**  $\text{DEC}_{\Theta^D} : \mathbb{R}^{|V| \times d} \rightarrow \mathbb{R}^{|V| \times |V|}$ , parameterized by  $\Theta^D$ , which uses the node embeddings  $Z$  to compute similarity or dissimilarity scores for all node pairs, producing a matrix  $\widehat{W} \in \mathbb{R}^{|V| \times |V|}$  as:

$$\widehat{W} = \text{DEC}(Z; \Theta^D).$$

- **Classification network**  $\text{DEC}_{\Theta^S} : \mathbb{R}^{|V| \times d} \rightarrow \mathbb{R}^{|V| \times |\mathcal{Y}|}$ , where  $\mathcal{Y}$  is the label space. This network is used in (semi-)supervised settings and parameterized by  $\Theta^S$ . The output is a distribution over the labels  $\hat{y}^S$ , using node embeddings, as:

$$\hat{y}^S = \text{DEC}(Z; \Theta^S).$$

Our GRAPHEDM framework is general (see Fig. 2.1 for an illustration). Specific choices of the aforementioned (encoder and decoder) networks allows GRAPHEDM to realize specific graph embedding methods. Before presenting the taxonomy and showing realizations of various methods using our framework, we briefly discuss an application perspective.

**Output** The GRAPHEDM model can return a reconstructed graph similarity or dissimilarity matrix  $\widehat{W}$  (often used to train *unsupervised* embedding algorithms), as well as a output labels  $\hat{y}^S$  for *supervised* applications. The label output space  $\mathcal{Y}$  varies depending on the supervised application.

- **Node-level supervision**, with  $\hat{y}^N \in \mathcal{Y}^{|V|}$ , where  $\mathcal{Y}$  represents the node label space. If  $\mathcal{Y}$  is categorical, then this is also known as (semi-)supervised node classification (Section 2.5.2.1), in which case the label decoder network produces labels for each node in the graph. If the embedding dimensions  $d$  is such that  $d = |\mathcal{Y}|$ , then the label decoder network can be just a simple softmax activation across the rows of  $Z$ , producing a distribution over labels for each node. Additionally, the graph decoder network might also be used in supervised node-classification tasks, as it can be used to regularize embeddings (e.g. neighbor nodes should have nearby embeddings, regardless of node labels).
- **Edge-level supervision**, with  $\hat{y}^E \in \mathcal{Y}^{|V| \times |V|}$ , where  $\mathcal{Y}$  represents the edge label space. For example,  $\mathcal{Y}$  can be multinomial in knowledge graphs (for describing the types of relationships between two entities), setting  $\mathcal{Y} = \{0, 1\}^{\#(\text{relation types})}$ . It is common to have  $\#(\text{relation types}) =$

1, and this is known as *link prediction*, where edge relations are binary. In this review, when  $\hat{y}^E = \{0, 1\}^{|V| \times |V|}$  (i.e.  $\mathcal{Y} = \{0, 1\}$ ), then rather than naming the output of the decoder as  $\hat{y}^E$ , we instead follow the nomenclature and position link prediction as an *unsupervised* task. Then in lieu of  $\hat{y}^E$  we utilize  $\widehat{W}$ , the output of the graph decoder network (which is learned to reconstruct a target similarity or dissimilarity matrix) to rank potential edges.

- **Graph-level supervision**, with  $\hat{y}^G \in \mathcal{Y}$ , where  $\mathcal{Y}$  is the graph label space. In the graph classification task (Section 2.5.2.2), the label decoder network converts node embeddings into a single graph labels, using *graph pooling* via the graph edges captured by  $W$ . More concretely, the graph pooling operation is similar to pooling in standard CNNs, where the goal is to downsample local feature representations to capture higher-level information. However, unlike images, graphs don't have a regular grid structure and it is hard to define a pooling pattern which could be applied to every node in the graph. A possible way of doing so is via graph coarsening, which groups similar nodes into clusters to produce smaller graphs [63]. There exist other pooling methods on graphs such as DiffPool [195] or SortPooling [200] which creates an ordering of nodes based on their structural roles in the graph. Details about graph pooling operators is outside the scope of this work and we refer the reader to recent surveys [190] for a more in-depth treatment.

### 2.3.2 Taxonomy of Objective Functions

We now focus our attention on the optimization of models that can be described in the GRAPHEDM framework by describing the loss functions used for training. Let  $\Theta = \{\Theta^E, \Theta^D, \Theta^S\}$  denote all model parameters. GRAPHEDM models can be optimized using a combination of the following loss terms:

- **Supervised loss** term,  $\mathcal{L}_{\text{SUP}}^S$ , which compares the predicted labels  $\hat{y}^S$  to the ground truth labels  $y^S$ . This term depends on the task the model is being trained for. For instance, in semi-supervised node classification tasks ( $S = N$ ), the graph vertices are split into labelled and unlabelled nodes ( $V = V_L \cup V_U$ ), and the supervised loss is computed for each labelled node in the graph:

$$\mathcal{L}_{\text{SUP}}^N(y^N, \hat{y}^N; \Theta) = \sum_{i|v_i \in V_L} \ell(y_i^N, \hat{y}_i^N; \Theta),$$

where  $\ell(\cdot)$  is the loss function used for classification (e.g. cross-entropy). Similarly for graph classification tasks ( $S = G$ ), the supervised loss is computed at the graph-level and can be summed across multiple training graphs:

$$\mathcal{L}_{\text{SUP}}^G(y^G, \hat{y}^G; \Theta) = \ell(y^G, \hat{y}^G; \Theta).$$

- **Graph regularization loss** term,  $\mathcal{L}_{G, \text{REG}}$ , which leverages the graph structure to impose regularization constraints on the model parameters. This loss term acts as a smoothing term and

measures the distance between the decoded similarity or dissimilarity matrix  $\widehat{W}$ , and a target similarity or dissimilarity matrix  $s(W)$ , which might capture higher-order proximities than the adjacency matrix itself:

$$\mathcal{L}_{G,\text{REG}}(W, \widehat{W}; \Theta) = d_1(s(W), \widehat{W}), \quad (2.1)$$

where  $d_1(\cdot, \cdot)$  is a distance or dissimilarity function. Examples for such regularization are constraining neighboring nodes to share similar embeddings, in terms of their distance in L2 norm. We will cover more examples of regularization functions in Section 2.4.

- **Weight regularization loss** term,  $\mathcal{L}_{\text{REG}}$ , e.g. for representing prior, on trainable model parameters for reducing overfitting. The most common regularization is L2 regularization (assumes a standard Gaussian prior):

$$\mathcal{L}_{\text{REG}}(\Theta) = \sum_{\theta \in \Theta} \|\theta\|_2^2.$$

Finally, models realizable by GRAPHEDM framework are trained by minimizing the total loss  $\mathcal{L}$  defined as:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{SUP}}^S(y^S, \hat{y}^S; \Theta) + \beta \mathcal{L}_{G,\text{REG}}(W, \widehat{W}; \Theta) + \gamma \mathcal{L}_{\text{REG}}(\Theta), \quad (2.2)$$

where  $\alpha$ ,  $\beta$  and  $\gamma$  are hyper-parameters, that can be tuned or set to zero. Note that graph embedding methods can be trained in a *supervised* ( $\alpha \neq 0$ ) or *unsupervised* ( $\alpha = 0$ ) fashion. Supervised graph embedding approaches leverage an additional source of information to learn embeddings such as node or graph labels. On the other hand, unsupervised network embedding approaches rely on the graph structure only to learn node embeddings.

A common approach to solve supervised embedding problems is to first learn embeddings with an unsupervised method and then train a supervised model on the learned embeddings. However, as pointed by Weston et al. [187] and others, using a two-step learning algorithm might lead to sub-optimal performances for the supervised task, and in general, supervised methods outperform two-step approaches.

### 2.3.3 Taxonomy of Encoders

Having introduced all the building blocks of the GRAPHEDM framework, we now introduce our graph embedding taxonomy. While most methods we describe next fall under the GRAPHEDM framework, they will significantly differ based on the encoder used to produce the node embeddings, and the loss function used to learn model parameters. We divide graph embedding models into four main categories:

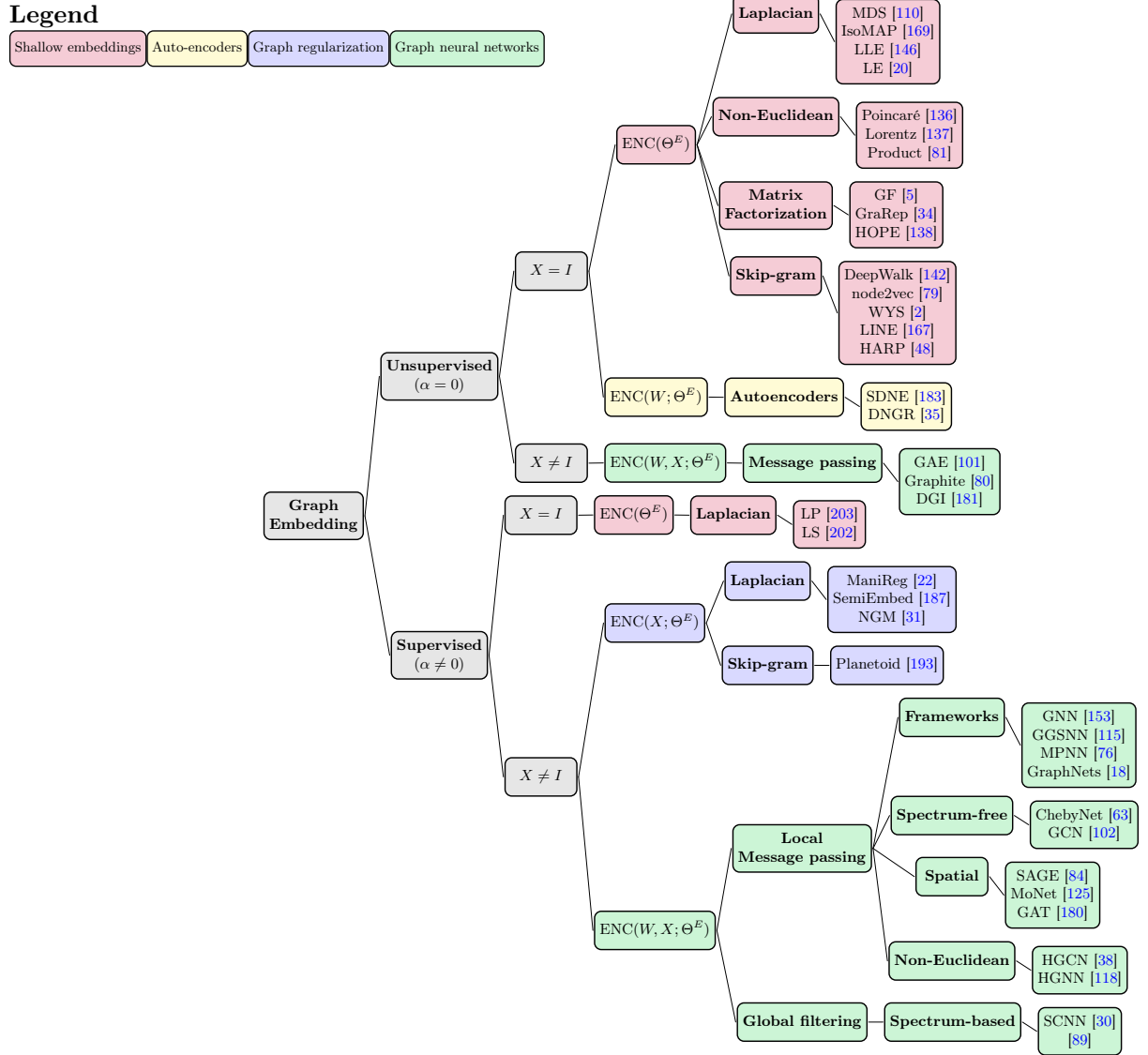


Figure 2.2: Taxonomy of graph representation learning methods. Based on what information is used in the encoder network, we categorize graph embedding approaches into four categories: shallow embeddings, graph auto-encoders, graph-based regularization and graph neural networks.

- **Shallow embedding methods**, where the encoder function is a simple embedding lookup. That is, the parameters of the model  $\Theta^E$  are directly used as node embeddings:

$$\begin{aligned} Z &= \text{ENC}(\Theta^E) \\ &= \Theta^E \in \mathbb{R}^{|V| \times d}. \end{aligned}$$

Note that shallow embedding methods rely on an embedding lookup and are therefore *transductive*, i.e. they generally cannot be directly applied in *inductive* settings where the graph structure is not fixed.

- **Graph regularization methods**, where the encoder network ignores the graph structure and only uses node features as input:

$$Z = \text{ENC}(X; \Theta^E).$$

As its name suggests, graph regularization methods leverage the graph structure through the graph regularization loss term in Eq. (2.2) ( $\beta \neq 0$ ) to regularize node embeddings.

- **Graph auto-encoding methods**, where the encoder is a function of the graph structure only:

$$Z = \text{ENC}(W; \Theta^E).$$

- **Neighborhood aggregation methods**, including graph convolutional methods, where both the node features and the graph structure are used in the encoder network. Neighborhood aggregation methods use the graph structure to propagate information across nodes and learn embeddings that encode structural properties about the graph:

$$Z = \text{ENC}(W, X; \Theta^E).$$

## 2.4 Examples

In this section follows, we use the GRAPHEDM framework to describe important GRL models, which we will refer to throughout this dissertation. We do not review all methods presented in the taxonomy in Fig. 2.2 and refer to the extended version of this chapter [39] for a detailed review of these methods.

### 2.4.1 Shallow Embeddings

Shallow embedding methods are transductive graph embedding methods where the encoder function is a simple embedding lookup. More concretely, each node  $v_i \in V$  has a corresponding low-dimensional

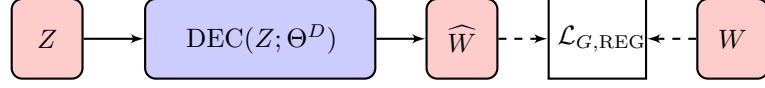


Figure 2.3: Shallow embedding methods. The encoder is a simple embedding look-up and the graph structure is only used in the loss function.

learnable embedding vector  $Z_i \in \mathbb{R}^d$  and the shallow encoder function is simply:

$$\begin{aligned} Z &= \text{ENC}(\Theta^E) \\ &= \Theta^E \in \mathbb{R}^{|V| \times d}. \end{aligned}$$

Embeddings of nodes can be learned such that the structure of the data in the embedding space corresponds to the underlying graph structure. At a high level, this is similar to dimensionality reduction methods such as PCA, except that the input data might not have a linear structure. In particular, methods used for non-linear dimensionality reduction often start by building a discrete graph from the data (to approximate the manifold) and can be applied to graph embedding problems. Here, we analyze two major types of shallow graph embedding methods, namely *distance-based* and *outer product-based* methods. In particular, we focused on unsupervised embedding methods (i.e. that only use the graph structure to learn embeddings) but note that there exists supervised extensions of these methods such as Label Propagation [203], Label Spreading [202].

**Distance-based methods** These methods optimize embeddings such that points that are close in the graph (as measured by their graph distances for instance) stay as close as possible in the embedding space using a predefined distance function. Formally, the decoder network computes pairwise distance for some distance function  $d_2(\cdot, \cdot)$ , which can lead to Euclidean (Section 2.4.1.1) or non-Euclidean (Section 2.4.1.2) embeddings:

$$\begin{aligned} \widehat{W} &= \text{DEC}(Z; \Theta^D) \\ \text{with } \widehat{W}_{ij} &= d_2(Z_i, Z_j) \end{aligned}$$

**Outer product-based methods** These methods on the other hand rely on pairwise dot-products to compute node similarities and the decoder network can be written as:

$$\begin{aligned} \widehat{W} &= \text{DEC}(Z; \Theta^D) \\ &= ZZ^\top. \end{aligned}$$

Embeddings are then learned by minimizing the graph regularization loss:  $\mathcal{L}_{G, \text{REG}}(W, \widehat{W}; \Theta) = d_1(s(W), \widehat{W})$ . Note that for distance-based methods, the function  $s(\cdot)$  measures dissimilarity or distances between nodes (higher values mean less similar pairs of nodes), while in outer-product

methods, it measures some notion of similarity in the graph (higher values mean more similar pairs).

#### 2.4.1.1 Distance-based: Euclidean methods

Most distance-based methods optimize Euclidean embeddings by minimizing Euclidean distances between similar nodes. Among these, we find linear embedding methods such as PCA or MDS, which learn low-dimensional linear projection subspaces, or nonlinear methods such as Laplacian eigenmaps, IsoMAP and Local linear embedding. Note that all these methods have originally been introduced for the manifold learning problem, where the graph is constructed from high-dimensional data, but the techniques used to embed graphs can also be applied in the GRL setting.

**Multi-Dimensional Scaling** (MDS) [110] refers to a set of embedding techniques used to map objects to positions while preserving the distances between these objects. In particular, metric MDS (mMDS) [57] minimizes the regularization loss in Eq. (2.1) with  $s(W)$  set to some distance matrix measuring the dissimilarity between objects (e.g. Euclidean distance between points in a high-dimensional space):

$$d_1(s(W), \widehat{W}) = \left( \frac{\sum_{ij} (s(W)_{ij} - \widehat{W}_{ij})^2}{\sum_{ij} s(W)_{ij}^2} \right)^{1/2}$$

$$\widehat{W}_{ij} = d_2(Z_i, Z_j) = \|Z_i - Z_j\|_2.$$

That is, mMDS finds an embedding configuration where distances in the low-dimensional embedding space are preserved by minimizing a residual sum of squares called the *stress* cost function. Note that if the dissimilarities are computed from Euclidean distances of a higher-dimensional representation, then mMDS is equivalent to the PCA dimensionality reduction method. Finally, there exist variants of this algorithm such as non-metric MDS, when the dissimilarity matrix  $s(W)$  is not a distance matrix, or classical MDS (cMDS) which can be solved in closed form using a low-rank decomposition of the gram matrix.

**Isometric Mapping** (IsoMap) [169] is an algorithm for non-linear dimensionality reduction which estimates the intrinsic geometry of a data lying on a manifold. This method is similar to MDS, except for a different choice of the distance matrix. IsoMap approximates manifold distances (in contrast with straight-line Euclidean geodesics) by first constructing a discrete neighborhood graph  $G$ , and then using the graph distances (length of shortest paths computed using Dijkstra's algorithm for example) to approximate the manifold geodesic distances:

$$s(W)_{ij} = d_G(v_i, v_j).$$

IsoMAP then uses the cMDS algorithm to compute representations that preserve these graph geodesic distances. Different from cMDS, IsoMAP works for distances that do not necessarily come from a Euclidean metric space (e.g. data defined on a Riemannian manifold). It is however computationally expensive due to the computation of all pairs of shortest path lengths in the neighborhood graph.

#### 2.4.1.2 Distance-based: non-Euclidean methods

The distance-based methods described so far assumed embeddings are learned in a Euclidean space. Graphs are non-Euclidean discrete data structures, and as discussed in Section 1.3, Euclidean geometry may not always be the best choice to match the structure of input graphs.

In particular, many network studies showed that real-world networks have a underlying hyperbolic geometry. Kleinberg [103] proposed a greedy algorithm for geometric rooting, which maps nodes in sensor networks to coordinates on a hyperbolic plane via spanning trees, and then performs greedy geographic routing. Hyperbolic geometry has also been used to study the structural properties of complex networks (networks with non-trivial topological features used to model real-world systems). Krioukov et al. [108] developed a geometric framework to construct scale-free networks (a family of complex networks with power-law degree distributions), and conversely showed that any scale-free graph with some metric structure has an underlying hyperbolic geometry. Papadopoulos et al. [139] introduced the Popularity-Similarity (PS) framework to model the evolution and growth of complex networks. In this model, new nodes are likely to be connected to popular nodes (modelled by their radial coordinates in hyperbolic space) as well as similar nodes (modelled by the angular coordinates). This framework has further been used to map nodes in graphs to hyperbolic coordinates, by maximising the likelihood that the network is produced by the PS model [140]. Further works extend non-linear dimensionality reduction techniques such as LLE [20] to efficiently map graphs to hyperbolic coordinates [7, 128].

In more recent years, Nickel and Kiela [136] proposed a shallow graph embedding model that learned representations of hierarchical graphs in hyperbolic spaces instead of conventional Euclidean space using gradient descent and Riemannian optimization [26]. We describe this model using GRAPHEDM next.

**Poincaré embeddings** Nickel and Kiela [136] learn embeddings of hierarchical graphs such as lexical databases (e.g. WordNet) in the Poincaré model hyperbolic space. Using our notations, this approach learns hyperbolic embeddings via the Poincaré distance function:

$$\begin{aligned} d_2(Z_i, Z_j) &= d_{\mathbb{B}}(Z_i, Z_j) \\ &= \operatorname{arcosh} \left( 1 + 2 \frac{\|Z_i - Z_j\|_2^2}{(1 - \|Z_i\|_2^2)(1 - \|Z_j\|_2^2)} \right). \end{aligned}$$

Embeddings are then learned by minimizing distances between connected nodes while maximizing

distances between disconnected nodes:

$$d_1(W, \widehat{W}) = - \sum_{ij} W_{ij} \log \frac{e^{-\widehat{W}_{ij}}}{\sum_{k|W_{ik}=0} e^{-\widehat{W}_{ik}}} = - \sum_{ij} W_{ij} \log \text{Softmax}_{k|W_{ik}=0}(-\widehat{W}_{ij}),$$

where the denominator is approximated using negative sampling. Note that since the hyperbolic space has a manifold structure, embeddings need to be optimized using Riemannian optimization techniques [26] to ensure that they remain on the manifold.

Other variants of these methods have been proposed. In particular, Nickel and Kiela [137] explore a different model of hyperbolic space, namely the Lorentz model (also known as the hyperboloid model), and show that it provides better numerical stability than the Poincaré model. Another line of work extends non-Euclidean embeddings to mixed-curvature product spaces [81], which provide more flexibility for other types of graphs (e.g. ring of trees). Finally, Chamberlain et al. [37] extend Poincaré embeddings to incorporate skip-gram losses using hyperbolic inner products.

Note that all of these hyperbolic GRL methods learn representations using a shallow encoder and do not leverage node features. Our main contribution in Chapter 3 will be to extend these methods to the supervised setting where node features are available.

### 2.4.1.3 Outer product-based: Matrix factorization methods

Matrix factorization approaches learn embeddings that lead to a low rank representation of some similarity matrix  $s(W)$ , where  $s : \mathbb{R}^{|V| \times |V|} \rightarrow \mathbb{R}^{|V| \times |V|}$  is a transformation of the weighted adjacency matrix, and many methods set it to the identity, i.e.  $s(W) = W$ . Other transformations include the Laplacian matrix or more complex similarities derived from proximity measures such as the Katz Index, Common Neighbours or Adamic Adar. The decoder function in matrix factorization methods is a simple outer product:

$$\widehat{W} = \text{DEC}(Z; \Theta^D) = ZZ^\top. \quad (2.3)$$

Matrix factorization methods learn embeddings by minimizing the regularization loss in Eq. (2.1) with:

$$\mathcal{L}_{G,\text{REG}}(W, \widehat{W}; \Theta) = \|s(W) - \widehat{W}\|_F^2. \quad (2.4)$$

That is,  $d_1(\cdot, \cdot)$  in Eq. (2.1) is the Frobenius norm between the reconstructed matrix and the target similarity matrix. By minimizing the regularization loss, graph factorization methods learn low-rank representations that preserve structural information as defined by the similarity matrix  $s(W)$  and we now review important matrix factorization methods.

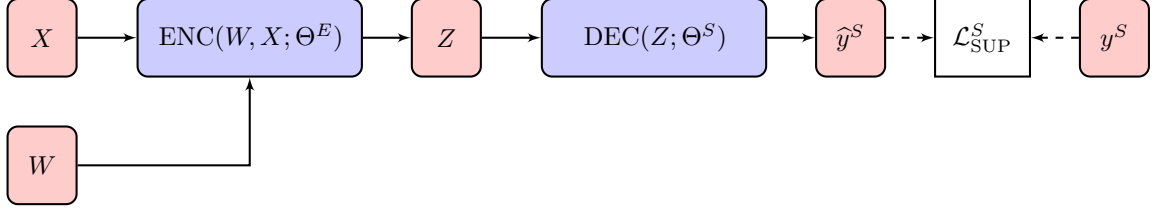


Figure 2.4: Supervised graph neural networks (GNNs). Rather than leveraging the graph structure to act as a regularizer, GNNs leverage the graph structure in the encoder to propagate information across neighbouring nodes and learn structural representations. Labels are then decoded and compared to ground truth labels (e.g. via the cross-entropy loss).

**Graph factorization** (GF) [5] learns a low-rank factorization for the adjacency matrix by minimizing graph regularization loss in Eq. (2.1) using:

$$d_1(W, \widehat{W}) = \sum_{(v_i, v_j) \in E} (W_{ij} - \widehat{W}_{ij})^2.$$

Note that if  $A$  is the binary adjacency matrix, that is  $A_{ij} = 1$  *iff*  $(v_i, v_j) \in E$  and  $A_{ij} = 0$  otherwise, then we can express the graph regularization loss in terms of Frobenius norm:

$$\mathcal{L}_{G, \text{REG}}(W, \widehat{W}; \Theta) = \|A \cdot (W - \widehat{W})\|_F^2,$$

where  $\cdot$  is the element-wise matrix multiplication operator. Therefore, GF also learns a low-rank factorization of the adjacency matrix  $W$  measured in Frobenius norm. Note that the sum is only over existing edges in the graph, which reduces the computational complexity of this method from  $O(|V|^2)$  to  $O(|E|)$ .

## 2.4.2 Graph Neural Networks

The methods we described so far are shallow in the sense that the mapping from nodes to embeddings is simply a one-hot encoding and therefore non-parametric. Consequently, these methods can only learn nodes for embeddings that are seen during training and fail to generalize to new nodes. This may be a significant downside, especially when learning with dynamic graphs that change over time.

In some scenarios, we may have additional node features as input as discussed in Section 2.2.2.1. The presence of node features suggests the possibility to learn smooth parametric mappings from this feature space to embeddings, for instance using feed-forward neural networks, whose weights can then be learned using gradient descent. However, we also want models that preserve the graph structure in the embedding space. A possibility is to use a graph regularization loss term to encourage neighboring nodes in the graph to be mapped to points that are close in the embedding space. We call these methods graph regularization methods and examples include Manifold Regularization [22] and

Planetoid [192].

Another option is to inject relational inductive biases (constraints such as weight sharing) within neural network architectures to facilitate learning. However, most neural network architectures have been restricted to simple relational structures such as data that is discretized on Euclidean domains (e.g. sequential data, or grid data) (Fig. 1.3). For instance, sequential data, such as text or videos, can be modelled via recurrent neural networks, which can capture sequential information, yielding efficient representations as measured on machine translation and speech recognition tasks. Convolutional neural networks (CNNs), which parameterize neural networks according to structural priors such as shift-invariance, and have achieved unprecedented performance in pattern recognition tasks such as image classification or speech recognition.

For graph-structured data, it is challenging to define networks with strong structural priors, as structures can be arbitrary, and can vary significantly across different graphs and even across different nodes within the same graph. In particular, operations like Euclidean convolutions heavily rely on geometric priors (such as translation invariance) and cannot be directly applied on irregular graph domains (e.g. translations may not even be defined on arbitrary graphs). For instance in images, each pixel has the same neighborhood structure, allowing to apply the same filter weights at multiple locations in the image. However in graphs, one can't define an ordering of node since each node might have a different neighborhood structure (Fig. 1.3).

*Graph Neural Networks* (GNNs) tackle these challenges and form a family of GRL methods that inject relational inductive biases in the design of neural networks. GNNs learn differentiable mappings over discrete graph topological structures and the encoder function can be expressed using GRAPHEDM as (Fig. 2.4):

$$Z = \text{ENC}(X, W; \Theta^E).$$

**Graph Neural Networks** The first formulation of deep learning methods for graph-structured data dates back to the graph neural network model of Gori et al. [77]. This formulation views the supervised graph embedding problem as an information diffusion mechanism, where nodes send information to their neighbors until some stable equilibrium state is reached. More concretely, given randomly initialized node embeddings  $Z^0$ , the following message passing recursion is applied until convergence:

$$Z_i^{t+1} = \sum_{j|(v_i, v_j) \in E} f(X_i, X_j, Z_j^t; \Theta^E),$$

where  $f(\cdot)$  is a multi-layer perceptron (MLP) constrained to be a contraction mapping.<sup>3</sup> The intuition between these message passing steps is that at every step, each node updates its representation by summing messages from all its neighbors. In some sense, this message passing step smoothens the embeddings until it reaches an equilibrium where nodes that are similar in the graph have similar

---

<sup>3</sup>This ensures that the recursion converges.

embeddings.

After convergence ( $t = T$ ), the node embeddings  $Z^T$  are used to predict the final output such as node or graph labels:

$$\hat{y}^S = \text{DEC}(X, Z^T; \Theta^S).$$

The decoder function in GNNs does not need to fulfill any constraint and can be any MLP. This process is repeated several times and the GNN parameters  $\Theta^E$  and  $\Theta^D$  are learned with backpropagation.

**Message Passing Neural Networks** In more recent years, Gilmer et al. [76] provided a framework for graph neural networks, encapsulating many recent GNN models. In contrast with the original GNN model which runs for an indefinite number of iterations, MPNN provide an abstraction for modern graph neural networks, which consist of multi-layer neural networks with a *fixed* number of layers. Further, rather than just use randomly initialized embeddings, MPNNs interleaves message passing steps with feature transformation steps to learn smooth mappings from features to embeddings. At every layer  $\ell$ , message functions  $f^\ell(\cdot)$  compute messages using neighbors' hidden representations, which are then passed to aggregation functions  $g^\ell(\cdot)$ :

$$\begin{aligned} m_i^{\ell+1} &= \sum_{j|(v_i, v_j) \in E} f^\ell(H_i^\ell, H_j^\ell) \\ H_i^{\ell+1} &= g^\ell(H_i^\ell, m_i^{\ell+1}). \end{aligned} \tag{2.5}$$

After  $\ell$  layers of message passing, nodes' hidden representations encode messages from  $\ell$ -hop neighborhoods.

**Graph Convolutional Neural Networks** The GCN model is an extremely popular GNN proposed by [102]. It was originally motivated as a simplification of spectral convolution neural networks [30, 89], which learn convolution filters in the spectral domain of the graph Laplacian. We can describe the GCN model using simple message passing updates, similar to the MPNN framework. If  $(W^\ell, \mathbf{b}^\ell)$  are weights and bias parameters for layer  $\ell$ , and  $\sigma(\cdot)$  is a non-linear activation function, the general GCN message passing rule at layer  $\ell$  for node  $i$  can be written in matrix form as:

$$H^{\ell+1} = \sigma(\tilde{L}H^\ell W^\ell), \tag{2.6}$$

where  $\tilde{L} = (D + I)^{-1/2}(W + I)(D + I)^{-1/2}$  is the re-normalized Laplacian matrix. This normalization ensures that the system is well conditioned (eigenvalues in  $[-1, 1]$ ) to avoid numerical instabilities and facilitate optimization.

We can draw some analogies between GCNs and standard Euclidean spatial convolutions. In standard CNNs, each node (pixel) in a filter interacts with its neighbors in a small rectangular patch around the center pixel. Since images have a Euclidean grid-like structure, it is possible to

assign an ordering of nodes in this neighborhood and learn one weight parameter per neighbour node. Similarly, GCNs aggregate information around local neighborhoods (k-hop). However, because there is no ordering of nodes in a neighborhood, GCNs use the same filter weight for all nodes in a neighborhood. This makes the application of filters possible across different neighborhood that share different structures.

## 2.5 GRL Applications

GRL methods can be applied to a wide range of applications, which can be unsupervised or supervised. In unsupervised applications, the goal is to learn embeddings that preserve the graph structure and unsupervised supervised embedding methods such as shallow methods described in Section 2.4.1 can be applied.

On the other hand, in supervised applications, node embeddings are optimized for some specific task, such as classifying nodes or graphs. In this setting, supervised embedding methods such as graph neural networks described in Section 2.4.2 can be applied. We review common unsupervised and supervised graph applications next.

### 2.5.1 Unsupervised Applications

#### 2.5.1.1 Graph reconstruction

The most standard unsupervised graph application is graph reconstruction. In this setting, the goal is to learn mapping functions (which can be parametric or not) that map nodes to dense distributed representations which preserve graph properties such as node similarity. Graph reconstruction doesn't require any supervision and models can be trained by minimizing a reconstruction error, which is the error in recovering the original graph from learned embeddings. At a high level, graph reconstruction is similar to dimensionality reduction in the sense that the main goal is summarize some input data into a low-dimensional embedding. Instead of compressing high dimensional vectors into low-dimensional ones as standard dimensionality reduction methods (e.g. PCA) do, the goal of graph reconstruction models is to compress data defined on graphs into low-dimensional vectors.

#### 2.5.1.2 Link prediction

Link prediction is the task of predicting links in a graph. In other words, the goal in link prediction tasks is to predict missing or unobserved links (e.g. links that may appear in the future for dynamic and temporal networks). Link prediction can also help identifying spurious link and remove them. It is a major application of graph learning models in industry, and common example of applications include predicting friendships in social networks or predicting user-product interactions in recommendation systems.

A common approach for training link prediction models is to mask some edges in the graph (positive and negative edges), train a model with the remaining edges and then test it on the masked set of edges. Note that link prediction is different from graph reconstruction. In link prediction, we aim at predicting links that are not observed in the original graph while in graph reconstruction, we only want to compute embeddings that preserve the graph structure through reconstruction error minimization.

Finally, while link prediction has similarities with supervised tasks in the sense that we have labels for edges (positive, negative, unobserved), we group it under the unsupervised class of applications since edge labels are usually not used during training, but only used to measure the predictive quality of embeddings.

We will see examples of link prediction problems in Chapter 4 and Chapter 3.

### 2.5.1.3 Clustering

Clustering is particularly useful for discovering communities and has many real-world applications. For instance, clusters exist in biological networks (e.g. as groups of proteins with similar properties), or in social networks (e.g. as groups of people with similar interests).

Note that unsupervised methods introduced in this survey can be used to solve clustering problems: one can run a clustering algorithm (e.g. k-means) on embeddings that are output by an encoder. Further, clustering can be joined with the learning algorithm while learning a shallow [148] or Graph Convolution [50, 51] embedding model.

We will discuss a hyperbolic clustering application in Chapter 5.

### 2.5.1.4 Visualization

There are many off-the-shelf tools for mapping graph nodes onto two-dimensional manifolds for the purpose of visualization. Visualizations allow network scientists to qualitatively understand graph properties, understand relationships between nodes or visualize node clusters. Among the popular tools are methods based on *Force-Directed Layouts*, with various web-app Javascript implementations.

Unsupervised graph embedding methods are also used for visualization purposes: by first training an encoder-decoder model (corresponding to a shallow embedding or graph convolution network), and then mapping every node representation onto a two-dimensional space using, t-distributed stochastic neighbor embeddings (t-SNE) [120] or PCA [95]. Such a process (embedding  $\rightarrow$  dimensionality reduction) is commonly used to qualitatively evaluate the performance of graph learning algorithms. If nodes have attributes, one can use these attributes to color the nodes on 2D visualization plots. Good embedding algorithms embed nodes that have similar attributes nearby in the embedding space, as demonstrated in visualizations of various methods [2, 102, 142]. Finally, beyond mapping every node to a 2D coordinate, methods which map every graph to a representation [6] can similarly be projected into two dimensions to visualize and qualitatively analyze graph-level properties.

We will discuss a visualization application for hyperbolic embeddings in Chapter 5.

## 2.5.2 Supervised Applications

### 2.5.2.1 Node classification

Node classification is an important supervised graph application, where the goal is to learn node representations that can accurately predict node labels. For instance, node labels could be scientific topics in citation networks, or gender and other attributes in social networks.

Since labelling large graphs can be time-consuming and expensive, semi-supervised node classification is a particularly common application. In semi-supervised settings, only a small fraction of nodes is labelled and the goal is to leverage links between nodes to predict attributes of unlabelled nodes. This setting is transductive since there is only one partially labelled fixed graph. It is also possible to do inductive node classification, which corresponds to the task of classifying nodes in multiple graphs.

Note that node features can significantly boost the performance on node classification tasks if these are descriptive for the target label. Indeed, recent methods such as GCN [102] or GraphSAGE [84] have achieved state-of-the-art performance on multiple node classification benchmarks due to their ability to combine structural information and semantics coming from features. On the other hand, other methods such as random walks on graphs fail to leverage feature information and therefore achieve lower performance on these tasks.

We will discuss a node classification application in Chapter 3.

### 2.5.2.2 Graph classification

Graph classification is a supervised application where the goal is to predict graph labels. Graph classification problems are inductive and a common example is classifying chemical compounds (e.g. predicting toxicity).

Graph classification is a particularly challenging task because it requires some notion of pooling, in order to aggregate node-level information into graph-level information. As discussed earlier, generalizing this notion of pooling to arbitrary graphs is non trivial because of the lack of regularity in the graph structure making graph pooling an active research area. In addition to the supervised methods discussed above, a number of unsupervised methods for learning graph-level representations have been proposed [6, 175, 176].

## Chapter 3

# Graph Neural Networks

### 3.1 Introduction

As discussed in the previous chapter, Graph Convolutional Neural Networks (GCNs) are state-of-the-art models for representation learning in graphs, where nodes of the graph are embedded into points in Euclidean space [84, 102, 180, 191]. However, many real-world graphs, such as protein interaction networks and social networks, often exhibit scale-free or hierarchical structure [54, 204] and Euclidean embeddings, used by existing GCNs, have a high distortion when embedding such graphs [49, 143]. In particular, scale-free graphs have tree-like structure and in such graphs the graph volume, defined as the number of nodes within some radius to a center node, grows exponentially as a function of radius. However, the volume of balls in Euclidean space only grows polynomially with respect to the radius, which leads to high distortion embeddings.

We saw earlier that hyperbolic geometry offers an exciting alternative as it enables embeddings with much smaller distortion when embedding scale-free and hierarchical graphs (see Section 1.3 in Chapter 1). However, current hyperbolic embedding techniques only account for the graph structure and do not leverage rich node features. For instance, Poincaré embeddings [136] capture the hyperbolic properties of real graphs by learning shallow embeddings with hyperbolic distance metric and Riemannian optimization. Compared to deep alternatives such as GCNs, shallow embeddings do not take into account features of nodes, lack scalability, and lack inductive capability. Furthermore, in practice, optimization in hyperbolic space is challenging.

While extending GCNs to hyperbolic geometry has the potential to lead to more faithful embeddings and accurate models, it also poses many hard challenges: (1) Input node features are usually Euclidean, and it is not clear how to optimally use them as inputs to hyperbolic neural networks; (2) It is not clear how to perform set aggregation, a key step in message passing, in hyperbolic space; And (3) one needs to choose hyperbolic spaces with the right curvature at every layer of the GCN.

Here we solve the above challenges and propose *Hyperbolic Graph Convolutional Networks*

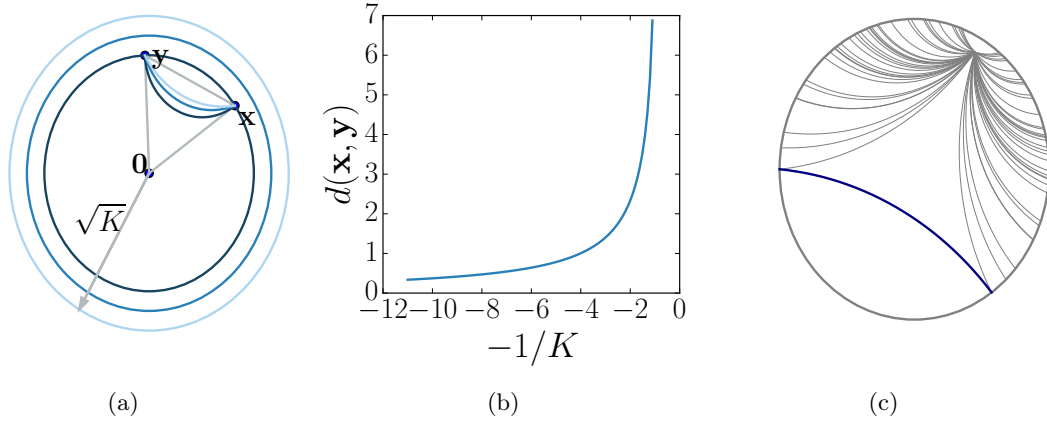


Figure 3.1: (a) Poincaré disk geodesics (shortest path) connecting  $x$  and  $y$  for different curvatures. As curvature ( $-1/K$ ) decreases, the distance between  $x$  and  $y$  increases, and the geodesics lines get closer to the origin. Center: Hyperbolic distance vs curvature. (b) Hyperbolic distance vs curvature. (c) Poincaré geodesic lines.

(HGCN)<sup>1</sup>, a class of graph representation learning models that combines the expressiveness of GCNs and hyperbolic geometry to learn improved representations for real-world hierarchical and scale-free graphs in inductive settings: (1) We derive the core operations of GCNs in the hyperboloid model of hyperbolic space to transform input features which lie in Euclidean space into hyperbolic embeddings; (2) We introduce a hyperbolic attention-based aggregation scheme that captures hierarchical structure of networks; (3) At different layers of HGCN we apply feature transformations in hyperbolic spaces of different trainable curvatures to learn low-distortion hyperbolic embeddings.

The transformation between different hyperbolic spaces at different layers allows HGCN to find the best geometry of hidden layers to achieve low distortion and high separation of class labels. Our approach jointly trains the weights for hyperbolic graph convolution operators, layer-wise curvatures and hyperbolic attention to learn inductive embeddings that reflect hierarchies in graphs.

Compared to Euclidean GCNs, HGCN offers improved expressiveness for hierarchical graph data. We demonstrate the efficacy of HGCN in link prediction and node classification tasks on a wide range of open graph datasets which exhibit different extent of hierarchical structure. Experiments show that HGCN significantly outperforms Euclidean-based state-of-the-art graph neural networks on scale-free graphs and reduces error from 11.5% up to 47.5% on node classification tasks and from 28.2% up to 63.1% on link prediction tasks. Furthermore, HGCN achieves new state-of-the-art results on the standard PUBMED benchmark. Finally, we analyze the notion of hierarchy learned by HGCN and show how the embedding geometry transforms from Euclidean features to hyperbolic embeddings.

<sup>1</sup>Project website with code and data: <http://snap.stanford.edu/hgcn>.

## 3.2 Related Work

The problem of graph representation learning belongs to the field of geometric deep learning. There exist two major types of approaches: transductive shallow embeddings and inductive GCNs which we described in Chapter 2. Here, we briefly review these concepts and related works.

**Transductive, shallow embeddings** The first type of approach attempts to optimize node embeddings as parameters by minimizing a reconstruction error. In other words, the mapping from nodes in a graph to embeddings is an embedding look-up. Examples include matrix factorization [20, 110] and random walk methods [79, 142]. Shallow embedding methods have also been developed in hyperbolic geometry [136, 137] for reconstructing trees [152] and graphs [37, 81, 103], or embedding text [171]. However, shallow (Euclidean and hyperbolic) embedding methods have three major downsides: (1) They fail to leverage rich node feature information, which can be crucial in tasks such as node classification. (2) These methods are transductive, and therefore cannot be used for inference on unseen graphs. And, (3) they scale poorly as the number of model parameters grows linearly with the number of nodes.

**(Euclidean) Graph Neural Networks** Instead of learning shallow embeddings, an alternative approach is to learn a mapping from input graph structure as well as node features to embeddings, parameterized by neural networks [84, 102, 115, 180, 191, 194]. While various Graph Neural Network architectures resolve the disadvantages of shallow embeddings, they generally embed nodes into a Euclidean space, which leads to a large distortion when embedding real-world graphs with scale-free or hierarchical structure. Our work builds on Graph Neural Networks and extends them to hyperbolic geometry.

**Hyperbolic neural networks** Hyperbolic geometry has been applied to neural networks, to problems of computer vision or natural language processing [65, 83, 99, 168]. More recently, hyperbolic neural networks [73] were proposed, where core neural network operations are in hyperbolic space. In contrast to previous work, we derive the core neural network operations in a more stable model of hyperbolic space, and propose new operations for set aggregation, which enables HGCN to perform graph convolutions with attention in hyperbolic space with trainable curvature.

A similar approach to ours is the Hyperbolic Graph Neural Network model [118]. The main difference with our work is how our HGCN defines the architecture for neighborhood aggregation and uses a learnable curvature. Additionally, while [118] demonstrates strong performance on graph classification tasks and provides an elegant extension to dynamic graph embeddings, we focus on link prediction and node classification.

### 3.3 Background

**Problem setting** Without loss of generality we describe graph representation learning on a single graph. Let  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  be a graph with vertex set  $\mathcal{V}$  and edge set  $\mathcal{E}$ , and let  $(\mathbf{x}_i^{0,E})_{i \in \mathcal{V}}$  be  $d$ -dimensional input node features where the superscript indicates the layer depth. We use the superscript  $E$  to indicate that node features lie in a Euclidean space and use  $H$  to denote hyperbolic features. The goal in graph representation learning is to learn a mapping  $f$  which maps nodes to embedding vectors:

$$f : (\mathcal{V}, \mathcal{E}, (\mathbf{x}_i^{0,E})_{i \in \mathcal{V}}) \rightarrow Z \in \mathbb{R}^{|\mathcal{V}| \times d'},$$

where  $d' \ll |\mathcal{V}|$ . These embeddings should capture both structural and semantic information and can then be used as input for downstream tasks such as node classification and link prediction.

**Graph Convolutional Networks (GCNs)** Let  $\mathcal{N}(i) = \{j : (i, j) \in \mathcal{E}\}$  denote a set of neighbors of  $i \in \mathcal{V}$ ,  $(W^\ell, \mathbf{b}^\ell)$  be weights and bias parameters for layer  $\ell$ , and  $\sigma(\cdot)$  be a non-linear activation function. Similar to the update in message passing neural networks (see Eq. (2.5) in Chapter 2), the general GCN message passing rule at layer  $\ell$  for node  $i$  consists of:

$$\mathbf{h}_i^{\ell,E} = W^\ell \mathbf{x}_i^{\ell-1,E} + \mathbf{b}^\ell \quad (\text{feature transform}) \quad (3.1)$$

$$\mathbf{x}_i^{\ell,E} = \sigma(\mathbf{h}_i^{\ell,E} + \sum_{j \in \mathcal{N}(i)} w_{ij} \mathbf{h}_j^{\ell,E}) \quad (\text{neighborhood aggregation}) \quad (3.2)$$

where aggregation weights  $w_{ij}$  can be computed using different mechanisms [84, 102, 180]. Message passing is then performed for multiple layers to propagate messages over network neighborhoods. Unlike shallow methods, GCNs leverage node features and can be applied to unseen nodes/graphs in inductive settings.

### 3.4 Hyperbolic Graph Convolutional Networks

Here we introduce HGCN, a generalization of inductive GCNs in hyperbolic geometry that benefits from the expressiveness of both graph neural networks and hyperbolic embeddings. First, since input features are often Euclidean, we derive a mapping from Euclidean features to hyperbolic space. Next, we derive two components of graph convolution: The analogs of Euclidean feature transformation and feature aggregation (Equations 3.1, 3.2) in the hyperboloid model. Finally, we introduce the HGCN algorithm with trainable curvature.

**Notation** We start by introducing some notation. In this work, we use the hyperboloid model of hyperbolic space (see Appendix A.4.2) for a description of this model. We denote  $\mathbb{H}^{d,K}$  as the

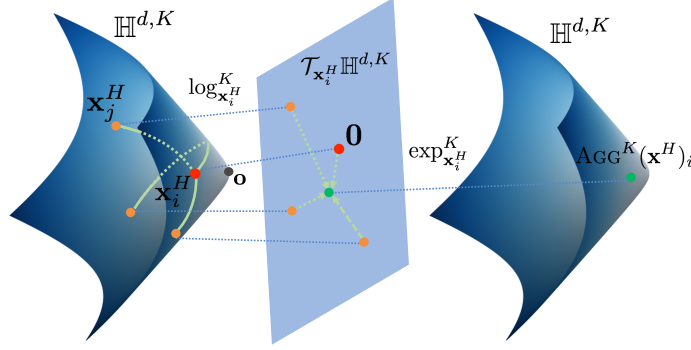


Figure 3.2: HGCN neighborhood aggregation (Eq. 3.9) first maps messages/embeddings to the tangent space, performs the aggregation in the tangent space, and then maps back to the hyperbolic space.

hyperboloid manifold in  $d$  dimensions with constant negative **curvature**  $-1/K$  ( $K > 0$ ), and  $T_{\mathbf{x}} \mathbb{H}^{d,K}$  the (Euclidean) **tangent space** centered at point  $\mathbf{x} \in \mathbb{H}^{d,K}$ :

$$\mathbb{H}^{d,K} := \{\mathbf{x} \in \mathbb{R}^{d+1} : \langle \mathbf{x}, \mathbf{x} \rangle_L = -K, x_0 > 0\} \quad T_{\mathbf{x}} \mathbb{H}^{d,K} := \{\mathbf{v} \in \mathbb{R}^{d+1} : \langle \mathbf{v}, \mathbf{x} \rangle_L = 0\}, \quad (3.3)$$

where  $\langle \cdot, \cdot \rangle_L : \mathbb{R}^{d+1} \times \mathbb{R}^{d+1} \rightarrow \mathbb{R}$  is the Minkowski inner product:  $\langle \mathbf{x}, \mathbf{y} \rangle_L := -x_0 y_0 + x_1 y_1 + \dots + x_d y_d$ .

### 3.4.1 Mapping from Euclidean to Hyperbolic Spaces

HGCN first maps input features to the hyperboloid manifold via the exp map. Let  $\mathbf{x}^{0,E} \in \mathbb{R}^d$  denote input Euclidean features. For instance, these features could be produced by pre-trained Euclidean neural networks. Let  $\mathbf{o} := \{\sqrt{K}, 0, \dots, 0\} \in \mathbb{H}^{d,K}$  denote the north pole (origin) in  $\mathbb{H}^{d,K}$ , which we use as a reference point to perform tangent space operations. We have  $\langle (0, \mathbf{x}^{0,E}), \mathbf{o} \rangle_L = 0$ . Therefore, we interpret  $(0, \mathbf{x}^{0,E})$  as a point in  $T_{\mathbf{o}} \mathbb{H}^{d,K}$  and use the hyperbolic exponential map to map it to  $\mathbb{H}^{d,K}$  with:

$$\mathbf{x}^{0,H} = \exp_{\mathbf{o}}^K((0, \mathbf{x}^{0,E})) = \left( \sqrt{K} \cosh\left(\frac{\|\mathbf{x}^{0,E}\|_2}{\sqrt{K}}\right), \sqrt{K} \sinh\left(\frac{\|\mathbf{x}^{0,E}\|_2}{\sqrt{K}}\right) \frac{\mathbf{x}^{0,E}}{\|\mathbf{x}^{0,E}\|_2} \right). \quad (3.4)$$

We refer to Appendix A.3 for a review of exponential and logarithmic maps in Riemannian manifolds.

### 3.4.2 Feature Transform in Hyperbolic Space

The feature transform in Equation 3.1 is used in GCN to map the embedding space of one layer to the next layer embedding space and capture large neighborhood structures. We now want to learn transformations of points on the hyperboloid manifold. However, there is no notion of vector space structure in hyperbolic space. We build upon Hyperbolic Neural Network (HNN) [73] and

derive transformations in the hyperboloid model. The main idea is to leverage the hyperbolic exponential and logarithmic maps so that we can use the tangent space  $T_{\mathbf{o}}\mathbb{H}^{d,K}$  to perform Euclidean transformations.

**Hyperboloid linear transform** Linear transformation requires multiplication of the embedding vector by a weight matrix, followed by bias translation. To compute matrix vector multiplication, we first use the logarithmic map to project hyperbolic points  $\mathbf{x}^H$  to  $T_{\mathbf{o}}\mathbb{H}^{d,K}$ . Thus the matrix representing the transform is defined on the tangent space, which is Euclidean and isomorphic to  $\mathbb{R}^d$ . We then project the vector in the tangent space back to the manifold using the exponential map. Let  $W$  be a  $d' \times d$  weight matrix. We define the hyperboloid matrix multiplication as:

$$W \otimes^K \mathbf{x}^H := \exp_{\mathbf{o}}^K(W \log_{\mathbf{o}}^K(\mathbf{x}^H)), \quad (3.5)$$

where  $\log_{\mathbf{o}}^K(\cdot)$  is on  $\mathbb{H}^{d,K}$  and  $\exp_{\mathbf{o}}^K(\cdot)$  maps to  $\mathbb{H}^{d',K}$ . In order to perform bias addition, we use a result from the HNN model and define  $\mathbf{b}$  as an Euclidean vector located at  $T_{\mathbf{o}}\mathbb{H}^{d,K}$ . We then parallel transport  $\mathbf{b}$  to the tangent space of the hyperbolic point of interest and map it to the manifold. Parallel transport is a generalization of translation to non-Euclidean geometry, it essentially moves tangent vectors from one tangent space to another without changing it, to allow the comparison of tangent vectors defined in different tangent spaces.

If two points  $\mathbf{x}, \mathbf{y} \in \mathbb{H}^{d,K}$  are connected by a geodesic, then the parallel transport of a tangent vector  $\mathbf{v} \in T_{\mathbf{x}}\mathbb{H}^{d,K}$  to the tangent space  $T_{\mathbf{y}}\mathbb{H}^{d,K}$  is:

$$P_{\mathbf{x} \rightarrow \mathbf{y}}^K(\mathbf{v}) = \mathbf{v} - \frac{\langle \log_{\mathbf{x}}^K(\mathbf{y}), \mathbf{v} \rangle_L}{d_L^K(\mathbf{x}, \mathbf{y})^2} \left( \log_{\mathbf{x}}^K(\mathbf{y}) + \log_{\mathbf{y}}^K(\mathbf{x}) \right). \quad (3.6)$$

The hyperboloid bias addition is then computed as:

$$\mathbf{x}^H \oplus^K \mathbf{b} := \exp_{\mathbf{x}^H}^K(P_{\mathbf{o} \rightarrow \mathbf{x}^H}^K(\mathbf{b})). \quad (3.7)$$

### 3.4.3 Neighborhood Aggregation on the Hyperboloid Manifold

Aggregation (Equation 3.2) is a crucial step in GCNs as it captures neighborhood structures and features. Suppose that  $\mathbf{x}_i$  aggregates information from its neighbors  $(\mathbf{x}_j)_{j \in N(i)}$  with weights  $(w_j)_{j \in N(i)}$ . Mean aggregation in Euclidean GCN computes the weighted average  $\sum_{j \in N(i)} w_j \mathbf{x}_j$ . An analog of mean aggregation in hyperbolic space is the Fréchet mean [72], which, however, has no closed form solution. Instead, we propose to perform aggregation in tangent spaces using hyperbolic attention.

**Attention-based aggregation** Attention in GCNs learns a notion of neighbors' importance and aggregates neighbors' messages according to their importance to the center node. However, attention on Euclidean embeddings does not take into account the hierarchical nature of many real-world

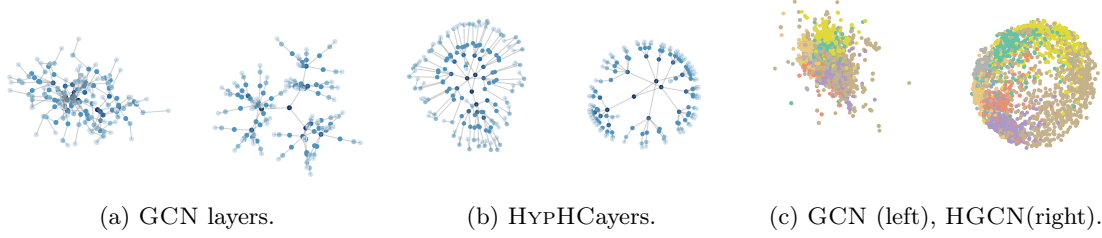


Figure 3.3: Visualization of embeddings for LP on DISEASE and NC on CORA (visualization on the Poincaré disk for HYPHC). (a) GCN embeddings in first and last layers for DISEASE LP hardly capture hierarchy (depth indicated by color). (b) In contrast, HYPHC preserves node hierarchies. (c) On CORA NC, HGCN leads to better class separation (indicated by different colors).

networks. Thus, we further propose hyperbolic attention-based aggregation. Given hyperbolic embeddings  $(\mathbf{x}_i^H, \mathbf{x}_j^H)$ , we first map  $\mathbf{x}_i^H$  and  $\mathbf{x}_j^H$  to the tangent space of the origin to compute attention weights  $w_{ij}$  with concatenation and Euclidean Multi-layer Perceptron (MLP). We then propose a hyperbolic aggregation to average nodes' representations:

$$w_{ij} = \text{SOFTMAX}_{j \in N(i)}(\text{MLP}(\log_{\mathbf{o}}^K(\mathbf{x}_i^H) \parallel \log_{\mathbf{o}}^K(\mathbf{x}_j^H))) \quad (3.8)$$

$$\text{AGG}^K(\mathbf{x}^H)_i = \exp_{\mathbf{x}_i^H}^K \left( \sum_{j \in N(i)} w_{ij} \log_{\mathbf{x}_i^H}^K(\mathbf{x}_j^H) \right). \quad (3.9)$$

Note that our proposed aggregation is directly performed in the tangent space of each center point  $\mathbf{x}_i^H$ , as this is where the Euclidean approximation is best (cf. Figure 3.2). We show in our ablation experiments (cf. Table 3.3) that this local aggregation outperforms aggregation in tangent space at the origin ( $\text{AGG}_{\mathbf{o}}$ ), due to the fact that relative distances have lower distortion in our approach.

**Non-linear activation with different curvatures** Analogous to Euclidean aggregation (Equation 3.2), HGCN uses a non-linear activation function,  $\sigma(\cdot)$  such that  $\sigma(0) = 0$ , to learn non-linear transformations. Given hyperbolic curvatures  $-1/K_{\ell-1}, -1/K_{\ell}$  at layer  $\ell - 1$  and  $\ell$  respectively, we introduce a hyperbolic non-linear activation  $\sigma^{\otimes K_{\ell-1}, K_{\ell}}$  with different curvatures. This step is crucial as it allows us to smoothly vary curvature at each layer. More concretely, HGCN applies the Euclidean non-linear activation in  $T_{\mathbf{o}}\mathbb{H}^{d, K_{\ell-1}}$  and then maps back to  $\mathbb{H}^{d, K_{\ell}}$ :

$$\sigma^{\otimes K_{\ell-1}, K_{\ell}}(\mathbf{x}^H) = \exp_{\mathbf{o}}^{K_{\ell}}(\sigma(\log_{\mathbf{o}}^{K_{\ell-1}}(\mathbf{x}^H))). \quad (3.10)$$

Note that in order to apply the exponential map, points must be located in the tangent space at the north pole. Fortunately, tangent spaces of the north pole are shared across hyperboloid manifolds of the same dimension that have different curvatures, making Equation 3.10 mathematically correct.

### 3.4.4 HGCN Architecture

Having introduced all the building blocks of HGCN, we now summarize the model architecture. Given a graph  $G = (V, E)$  and input Euclidean features  $(\mathbf{x}^{0,E})_{i \in V}$ , the first layer of HGCN maps from Euclidean to hyperbolic space as detailed in Section 3.4.1. HGCN then stacks multiple hyperbolic graph convolution layers. At each layer HGCN transforms and aggregates neighbour's embeddings in the tangent space of the center node and projects the result to a hyperbolic space with different curvature. Hence the message passing in a HGCN layer is:

$$\mathbf{h}_i^{\ell,H} = (W^\ell \otimes^{K_{\ell-1}} \mathbf{x}_i^{\ell-1,H}) \oplus^{K_{\ell-1}} \mathbf{b}^\ell \quad (\text{hyperbolic feature transform}) \quad (3.11)$$

$$\mathbf{y}_i^{\ell,H} = \text{AGG}^{K_{\ell-1}}(\mathbf{h}_i^{\ell,H})_i \quad (\text{attention-based neighborhood aggregation}) \quad (3.12)$$

$$\mathbf{x}_i^{\ell,H} = \sigma^{\otimes^{K_{\ell-1}, K_\ell}}(\mathbf{y}_i^{\ell,H}) \quad (\text{non-linear activation with different curvatures}) \quad (3.13)$$

where  $-1/K_{\ell-1}$  and  $-1/K_\ell$  are the hyperbolic curvatures at layer  $\ell-1$  and  $\ell$  respectively. Hyperbolic embeddings  $(\mathbf{x}^{L,H})_{i \in V}$  at the last layer can then be used to predict node attributes or links.

For link prediction, we use the Fermi-Dirac decoder [108, 136], a generalization of sigmoid, to compute probability scores for edges:

$$p((i, j) \in E | \mathbf{x}_i^{L,H}, \mathbf{x}_j^{L,H}) = \left[ e^{(d_L^{K_L}(\mathbf{x}_i^{L,H}, \mathbf{x}_j^{L,H})^2 - r)/t} + 1 \right]^{-1}, \quad (3.14)$$

where  $d_L^{K_L}(\cdot, \cdot)$  is the hyperbolic distance and  $r$  and  $t$  are hyper-parameters. We then train HGCN by minimizing the cross-entropy loss using negative sampling.

For node classification, we map the output of the last HGCN layer to the tangent space of the origin with the logarithmic map  $\log_{\mathbf{o}}^{K_L}(\cdot)$  and then perform Euclidean multinomial logistic regression. Note that another possibility is to directly classify points on the hyperboloid manifold using the hyperbolic multinomial logistic loss [73]. This method performs similarly to Euclidean classification (cf. [73] for an empirical comparison). Finally, we also add a link prediction regularization objective in node classification tasks, to encourage embeddings at the last layer to preserve the graph structure.

### 3.4.5 Trainable Curvature

We further analyze the effect of trainable curvatures in HGCN. Theorem 3.4.1 shows that assuming infinite precision, for the link prediction task, we can achieve the same performance for varying curvatures with an affine invariant decoder by scaling embeddings.

**Theorem 3.4.1.** *For any hyperbolic curvatures  $-1/K, -1/K' < 0$ , for any node embeddings  $H = \{\mathbf{h}_i\} \subset \mathbb{H}^{d,K}$  of a graph  $G$ , we can find  $H' \subset \mathbb{H}^{d,K'}$ ,  $H' = \{\mathbf{h}'_i | \mathbf{h}'_i = \sqrt{\frac{K'}{K}} \mathbf{h}_i\}$ , such that the reconstructed graph from  $H'$  via the Fermi-Dirac decoder is the same as the reconstructed graph from  $H$ , with different decoder parameters  $(r, t)$  and  $(r', t')$ .*

Before showing this theorem, we first show a useful result.

**Lemma 3.4.1.** *For any hyperbolic spaces with constant curvatures  $-1/K, -1/K' > 0$ , and any pair of hyperbolic points  $(\mathbf{u}, \mathbf{v})$  embedded in  $\mathbb{H}^{d,K}$ , there exists a mapping  $\phi : \mathbb{H}^{d,K} \rightarrow \mathbb{H}^{d,K'}$  to another pair of corresponding hyperbolic points in  $\mathbb{H}^{d,K'}$ ,  $(\phi(\mathbf{u}), \phi(\mathbf{v}))$  such that the Minkowski inner product is scaled by a constant factor.*

*Proof.* For any hyperbolic embedding  $\mathbf{x} = (x_0, x_1, \dots, x_d) \in \mathbb{H}^{d,K}$  we have the identity:  $\langle \mathbf{x}, \mathbf{x} \rangle_L = -x_0^2 + \sum_{i=1}^d x_i^2 = -K$ . For any hyperbolic curvature  $-1/K < 0$ , consider the mapping  $\phi(\mathbf{x}) = \sqrt{\frac{K'}{K}} \mathbf{x}$ . Then we have the identity  $\langle \phi(\mathbf{x}), \phi(\mathbf{x}) \rangle_L = -K'$  and therefore  $\phi(\mathbf{x}) \in \mathbb{H}^{d,K'}$ . For any pair  $(\mathbf{u}, \mathbf{v})$ ,  $\langle \phi(\mathbf{u}), \phi(\mathbf{v}) \rangle_L = \frac{K'}{K} \left( -\mathbf{u}_0 \mathbf{v}_0 + \sum_{i=1}^d \mathbf{u}_i \mathbf{v}_i \right) = \frac{K'}{K} \langle \mathbf{u}, \mathbf{v} \rangle_L$ . The factor  $\frac{K'}{K}$  only depends on curvature, but not the specific embeddings.  $\square$

Lemma 3.4.1 implies that given a set of embeddings learned in hyperbolic space  $\mathbb{H}^{d,K}$ , we can find embeddings in another hyperbolic space with different curvature,  $\mathbb{H}^{d,K'}$ , such that the Minkowski inner products for all pairs of embeddings are scaled by the same factor  $\frac{K'}{K}$ .

*Proof of Theorem 3.4.1.* The Fermi-Dirac decoder predicts that there exists a link between node  $i$  and  $j$  iff:

$$\left[ e^{(d_L^K(\mathbf{h}_i, \mathbf{h}_j) - r)/t} + 1 \right]^{-1} \geq b,$$

where  $b \in (0, 1)$  is the threshold for determining existence of links. The criterion is equivalent to  $d_L^K(\mathbf{h}_i, \mathbf{h}_j) \leq r + t \log(\frac{1-b}{b})$ .

Given  $H = \{\mathbf{h}_1, \dots, \mathbf{h}_n\}$ , the graph  $G_H$  reconstructed with the Fermi-Dirac decoder has the edge set  $E_H = \{(i, j) | d_L^K(\mathbf{h}_i, \mathbf{h}_j) \leq r + t \log(\frac{1-b}{b})\}$ . Consider the mapping to  $\mathbb{H}^{d,K'}$ ,  $\phi(\mathbf{x}) := \sqrt{\frac{K'}{K}} \mathbf{x}$ . Let  $H' = \{\phi(\mathbf{h}_1), \dots, \phi(\mathbf{h}_n)\}$ . By Lemma 3.4.1,

$$d_L^{K'}(\phi(\mathbf{h}_i), \phi(\mathbf{h}_j)) = \sqrt{K'} \text{arcosh} \left( -\frac{K'}{K} \langle \mathbf{h}_i, \mathbf{h}_j \rangle_L / K' \right) = \sqrt{\frac{K'}{K}} d_L^K(\mathbf{h}_i, \mathbf{h}_j). \quad (3.15)$$

Due to linearity, we can find decoder parameter,  $r'$  and  $t'$  that satisfy  $r' + t' \log(\frac{1-b}{b}) = \sqrt{\frac{K'}{K}} (r + t \log(\frac{1-b}{b}))$ . With such  $r', t'$ , the criterion  $d_L^K(\mathbf{h}_i, \mathbf{h}_j) \leq r + t \log(\frac{1-b}{b})$  is equivalent to  $d_L^{K'}(\phi(\mathbf{h}_i), \phi(\mathbf{h}_j)) \leq r' + t' \log(\frac{1-b}{b})$ . Therefore, the reconstructed graph  $G_{H'}$  based on the set of embeddings  $H'$  is identical to  $G_H$ .  $\square$

For link prediction tasks, Theorem 3.4.1 shows that with infinite precision, the expressive power of hyperbolic spaces with varying curvatures is the same. However, despite the same expressive power, adjusting curvature at every layer is important for good performance in practice due to factors of limited machine precision and normalization. First, with very low or very high curvatures, the scaling factor  $\frac{K'}{K}$  in Theorem 3.4.1 becomes close to 0 or very large, and limited machine precision results in large error due to rounding. This is supported by Figure 3.4 and Table 3.3 where

| Dataset   | Nodes | Edges | Classes | Node features |
|-----------|-------|-------|---------|---------------|
| CORA      | 2708  | 5429  | 7       | 1433          |
| PUBMED    | 19717 | 88651 | 3       | 500           |
| HUMAN PPI | 17598 | 5429  | 4       | 17            |
| AIRPORT   | 3188  | 18631 | 4       | 4             |
| DISEASE   | 1044  | 1043  | 2       | 1000          |
| DISEASE-M | 43193 | 43102 | 2       | 1000          |

Table 3.1: Benchmarks’ statistics

adjusting and training curvature leads to significant performance gain. Second, the norms of hidden layers that achieve the same local minimum in training also vary by a factor of  $\sqrt{K}$ . In practice, however, optimization is much more stable when the values are normalized [93]. In the context of HGCN, trainable curvature provides a natural way to learn embeddings of the right scale at each layer, improving optimization. Figure 3.4 shows the effect of decreasing curvature ( $K = +\infty$  is the Euclidean case) on link prediction performance.

## 3.5 Experiments

We comprehensively evaluate our method on a variety of networks, on both node classification (NC) and link prediction (LP) tasks, in transductive and inductive settings. We compare performance of HGCN against a variety of shallow and GNN-based baselines. We further use visualizations to investigate the expressiveness of HGCN in link prediction tasks, and also demonstrate its ability to learn embeddings that capture the hierarchical structure of many real-world networks.

### 3.5.1 Datasets and Baselines

**Datasets** We use a variety of open transductive and inductive datasets that we detail below (see Table 3.1 for dataset statistics). We compute Gromov’s  $\delta$ -hyperbolicity [3, 96, 130], a notion from group theory that measures how tree-like a graph is. The lower  $\delta$ , the more hyperbolic is the graph dataset, and  $\delta = 0$  for trees. We conjecture that HGCN works better on graphs with small  $\delta$ -hyperbolicity.

1. **Citation networks** CORA [155] and PUBMED [129] are standard benchmarks describing citation networks where nodes represent scientific papers, edges are citations between them, and node labels are academic (sub)areas. CORA contains 2,708 machine learning papers divided into 7 classes while PUBMED has 19,717 publications in the area of medicine grouped in 3 classes.
2. **Disease propagation tree** We simulate the SIR disease spreading model [11], where the label of a node is whether the node was infected or not. Based on the model, we build tree networks, where node features indicate the susceptibility to the disease. We build transductive and inductive variants of this dataset, namely DISEASE and DISEASE-M (which contains multiple tree components).

3. **Protein-protein interactions (PPI) networks** PPI is a dataset of human PPI networks [166]. Each human tissue has a PPI network, and the dataset is a union of PPI networks for human tissues. Each protein has a label indicating the stem cell growth rate after 19 days [179], which we use for the node classification task. The 16-dimensional feature for each node represents the RNA expression levels of the corresponding proteins, and we perform log transform on the features.
4. **Flight networks** AIRPORT is a transductive dataset where nodes represent airports and edges represent the airline routes as from [OpenFlights.org](https://openflights.org). Compared to previous compilations [199], our dataset has larger size (2,236 nodes). We also augment the graph with geographic information (longitude, latitude and altitude), and GDP of the country where the airport belongs to. We use the population of the country where the airport belongs to as the label for node classification.

**Baselines** For shallow methods, we consider Euclidean embeddings (EUC) and Poincaré embeddings (HYP) [136]. We conjecture that HYP will outperform EUC on hierarchical graphs. For a fair comparison with HGCN which leverages node features, we also consider EUC-MIXED and HYP-MIXED baselines, where we concatenate the corresponding shallow embeddings with node features, followed by a MLP to predict node labels or links. For state-of-the-art Euclidean GNN models, we consider GCN [102], GraphSAGE (SAGE) [84], Graph Attention Networks (GAT) [180] and Simplified Graph Convolution (SGC) [189]<sup>2</sup>. We also consider feature-based approaches: MLP and its hyperbolic variant (HNN) [73], which does not utilize the graph structure.

**Evaluation metric** In transductive LP tasks, we randomly split edges into 85/5/10% for training, validation and test sets. For transductive NC, we use 70/15/15% splits for AIRPORT, 30/10/60% splits for DISEASE, and we use standard splits [102, 193] with 20 train examples per class for CORA and PUBMED. One of the main advantages of HGCN over related hyperbolic graph embedding is its inductive capability. For inductive tasks, the split is performed across graphs. All nodes/edges in training graphs are considered the training set, and the model is asked to predict node class or unseen links for test graphs. Following previous works, we evaluate link prediction by measuring area under the ROC curve on the test set and evaluate node classification by measuring F1 score, except for CORA and PUBMED, where we report accuracy as is standard in the literature.

### 3.5.2 Training Details

Here we present details of HGCN’s training pipeline, with optimization and incorporation of DropConnect [182].

For all methods, we perform a hyper-parameter search on a validation set over initial learning rate, weight decay, dropout, number of layers, and activation functions. We measure performance on

---

<sup>2</sup>The equivalent of GCN in link prediction is GAE [101]. We did not compare link prediction GNNs based on shallow embeddings such as [199] since they are not inductive.

| Dataset | DISEASE<br>$\delta = 0$ |                       | DISEASE-M<br>$\delta = 0$ |                       | HUMAN PPI<br>$\delta = 1$ |                       | AIRPORT<br>$\delta = 1$ |                       | PubMed<br>$\delta = 3.5$ |                       | CORA<br>$\delta = 11$ |                       |                       |
|---------|-------------------------|-----------------------|---------------------------|-----------------------|---------------------------|-----------------------|-------------------------|-----------------------|--------------------------|-----------------------|-----------------------|-----------------------|-----------------------|
|         | LP                      | NC                    | LP                        | NC                    | LP                        | NC                    | LP                      | NC                    | LP                       | NC                    | LP                    | NC                    |                       |
| Shallow | EUC                     | 59.8 $\pm$ 2.0        | 32.5 $\pm$ 1.1            | -                     | -                         | -                     | -                       | 92.0 $\pm$ 0.0        | 60.9 $\pm$ 3.4           | 83.3 $\pm$ 0.1        | 48.2 $\pm$ 0.7        | 82.5 $\pm$ 0.3        | 23.8 $\pm$ 0.7        |
|         | HYP [136]               | 63.5 $\pm$ 0.6        | 45.5 $\pm$ 3.3            | -                     | -                         | -                     | -                       | 94.5 $\pm$ 0.0        | 70.2 $\pm$ 0.1           | 87.5 $\pm$ 0.1        | 68.5 $\pm$ 0.3        | 87.6 $\pm$ 0.2        | 22.0 $\pm$ 1.5        |
|         | EUC-MIX                 | 49.6 $\pm$ 1.1        | 35.2 $\pm$ 3.4            | -                     | -                         | -                     | -                       | 91.5 $\pm$ 0.1        | 68.3 $\pm$ 2.3           | 86.0 $\pm$ 1.3        | 63.0 $\pm$ 0.3        | 84.4 $\pm$ 0.2        | 46.1 $\pm$ 0.4        |
|         | HYP-MIX                 | 55.1 $\pm$ 1.3        | 56.9 $\pm$ 1.5            | -                     | -                         | -                     | -                       | 93.3 $\pm$ 0.0        | 69.6 $\pm$ 0.1           | 83.8 $\pm$ 0.3        | 73.9 $\pm$ 0.2        | 85.6 $\pm$ 0.5        | 45.9 $\pm$ 0.3        |
|         |                         |                       |                           |                       |                           |                       |                         |                       |                          |                       |                       |                       |                       |
| NN      | MLP                     | 72.6 $\pm$ 0.6        | 28.8 $\pm$ 2.5            | 55.3 $\pm$ 0.5        | 55.9 $\pm$ 0.3            | 67.8 $\pm$ 0.2        | 55.3 $\pm$ 0.4          | 89.8 $\pm$ 0.5        | 68.6 $\pm$ 0.6           | 84.1 $\pm$ 0.9        | 72.4 $\pm$ 0.2        | 83.1 $\pm$ 0.5        | 51.5 $\pm$ 1.0        |
|         | HNN[73]                 | 75.1 $\pm$ 0.3        | 41.0 $\pm$ 1.8            | 60.9 $\pm$ 0.4        | 56.2 $\pm$ 0.3            | 72.9 $\pm$ 0.3        | 59.3 $\pm$ 0.4          | 90.8 $\pm$ 0.2        | 80.5 $\pm$ 0.5           | 94.9 $\pm$ 0.1        | 69.8 $\pm$ 0.4        | 89.0 $\pm$ 0.1        | 54.6 $\pm$ 0.4        |
| GNN     | GCN[102]                | 64.7 $\pm$ 0.5        | 69.7 $\pm$ 0.4            | 66.0 $\pm$ 0.8        | 59.4 $\pm$ 3.4            | 77.0 $\pm$ 0.5        | 69.7 $\pm$ 0.3          | 89.3 $\pm$ 0.4        | 81.4 $\pm$ 0.6           | 91.1 $\pm$ 0.5        | 78.1 $\pm$ 0.2        | 90.4 $\pm$ 0.2        | 81.3 $\pm$ 0.3        |
|         | GAT [180]               | 69.8 $\pm$ 0.3        | 70.4 $\pm$ 0.4            | 69.5 $\pm$ 0.4        | 62.5 $\pm$ 0.7            | 76.8 $\pm$ 0.4        | 70.5 $\pm$ 0.4          | 90.5 $\pm$ 0.3        | 81.5 $\pm$ 0.3           | 91.2 $\pm$ 0.1        | 79.0 $\pm$ 0.3        | <b>93.7</b> $\pm$ 0.1 | <b>83.0</b> $\pm$ 0.7 |
|         | SAGE [84]               | 65.9 $\pm$ 0.3        | 69.1 $\pm$ 0.6            | 67.4 $\pm$ 0.5        | 61.3 $\pm$ 0.4            | 78.1 $\pm$ 0.6        | 69.1 $\pm$ 0.3          | 90.4 $\pm$ 0.5        | 82.1 $\pm$ 0.5           | 86.2 $\pm$ 1.0        | 77.4 $\pm$ 2.2        | 85.5 $\pm$ 0.6        | 77.9 $\pm$ 2.4        |
|         | SGC [189]               | 65.1 $\pm$ 0.2        | 69.5 $\pm$ 0.2            | 66.2 $\pm$ 0.2        | 60.5 $\pm$ 0.3            | 76.1 $\pm$ 0.2        | 71.3 $\pm$ 0.1          | 89.8 $\pm$ 0.3        | 80.6 $\pm$ 0.1           | 94.1 $\pm$ 0.0        | 78.9 $\pm$ 0.0        | 91.5 $\pm$ 0.1        | 81.0 $\pm$ 0.1        |
|         |                         |                       |                           |                       |                           |                       |                         |                       |                          |                       |                       |                       |                       |
| Ours    | HGCN                    | <b>90.8</b> $\pm$ 0.3 | <b>74.5</b> $\pm$ 0.9     | <b>78.1</b> $\pm$ 0.4 | <b>72.2</b> $\pm$ 0.5     | <b>84.5</b> $\pm$ 0.4 | <b>74.6</b> $\pm$ 0.3   | <b>96.4</b> $\pm$ 0.1 | <b>90.6</b> $\pm$ 0.2    | <b>96.3</b> $\pm$ 0.0 | <b>80.3</b> $\pm$ 0.3 | 92.9 $\pm$ 0.1        | 79.9 $\pm$ 0.2        |
|         | (%) ERR RED             | -63.1%                | -13.8%                    | -28.2%                | -25.9%                    | -29.2%                | -11.5%                  | -60.9%                | -47.5%                   | -27.5%                | -6.2%                 | +12.7%                | +18.2%                |

Table 3.2: ROC AUC for Link Prediction (LP) and F1 score for Node Classification (NC) tasks. For inductive datasets, we only evaluate inductive methods since shallow methods cannot generalize to unseen nodes/graphs. We report graph hyperbolicity values  $\delta$  (lower is more hyperbolic).

the final test set over 10 random parameter initializations. For fairness, we also control the number of dimensions to be the same (16) for all methods. We optimize all models with Adam [100], except Poincaré embeddings which are optimized with RiemannianSGD [26, 197].

We open source our implementation<sup>3</sup> of HGCN and baselines.

**Parameter optimization** Recall that linear transformations and attention are defined on the tangent space of points. Therefore the linear layer and attention parameters are Euclidean. For bias, there are two options: one can either define parameters in hyperbolic space, and use hyperbolic addition operation [73], or define parameters in Euclidean space, and use Euclidean addition after transforming the points into the tangent space. Through experiments we find that Euclidean optimization is much more stable, and gives slightly better test performance compared to Riemannian optimization, if we define parameters such as bias in hyperbolic space. Hence different from shallow hyperbolic embeddings, although our model and embeddings are hyperbolic, the learnable graph convolution parameters can be optimized via Euclidean optimization (Adam Optimizer [100]), thanks to exponential and logarithmic maps. Note that to train shallow Poincaré embeddings, we use Riemannian Stochastic Gradient Descent [26, 197], since its model parameters are hyperbolic. We use early stopping based on validation set performance with a patience of 100 epochs.

**Drop connection** Since rescaling vectors in hyperbolic space requires exponential and logarithmic maps, and is conceptually not tied to the inverse dropout rate in terms of re-normalizing L1 norm, Dropout cannot be directly applied in HGCN. However, as a result of using Euclidean parameters in HGCN, DropConnect [182], the generalization of Dropout, can be used as a regularization. DropConnect randomly zeros out the neural network connections, elements of the Euclidean parameters during training time, improving the generalization of HGCN.

<sup>3</sup>Code available at <http://snap.stanford.edu/hgcn>. We provide HGCN implementations for hyperboloid and Poincaré models. Empirically, both models give similar performance but hyperboloid model offers more stable optimization, because Poincaré distance is numerically unstable [137].

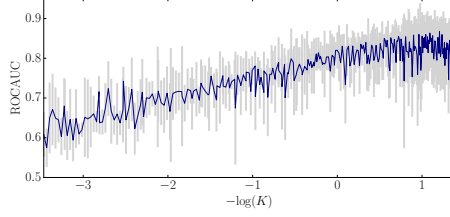


Figure 3.4: Decreasing curvature ( $-1/K$ ) improves link prediction performance on DISEASE.

| Method                | DISEASE                          | AIRPORT                          |
|-----------------------|----------------------------------|----------------------------------|
| HGCN                  | $78.4 \pm 0.3$                   | $91.8 \pm 0.3$                   |
| HGCN-ATT <sub>o</sub> | $80.9 \pm 0.4$                   | $92.3 \pm 0.3$                   |
| HGCN-ATT              | $82.0 \pm 0.2$                   | $92.5 \pm 0.2$                   |
| HGCN-C                | $89.1 \pm 0.2$                   | $94.9 \pm 0.3$                   |
| HGCN-ATT-C            | <b><math>90.8 \pm 0.3</math></b> | <b><math>96.4 \pm 0.1</math></b> |

Table 3.3: ROC AUC for link prediction on AIRPORT and DISEASE datasets.

**Projections** Because of limited machine precision, some numerical errors may cause the embeddings and intermediate hidden representations to not exactly be on the right manifold or tangent space. To solve this, we use projections onto the tangent space and hyperboloid manifold.

Given any vector  $\mathbf{v} \in \mathbb{R}^{d+1}$ , we can project it onto the tangent at a point  $\mathbf{x} \in \mathbb{H}^d$  via:

$$\Pi_{\mathbb{R}^{d+1} \rightarrow T_{\mathbf{x}}\mathbb{H}^d}(\mathbf{v}) := \mathbf{v} + \langle \mathbf{x}, \mathbf{v} \rangle_L \mathbf{x}.$$

It is clear that  $\langle \mathbf{v} + \langle \mathbf{x}, \mathbf{v} \rangle_L \mathbf{x}, \mathbf{x} \rangle_L = \langle \mathbf{v}, \mathbf{x} \rangle_L + \langle \mathbf{x}, \mathbf{v} \rangle_L \langle \mathbf{x}, \mathbf{x} \rangle_L = 0$  and therefore the projection maps to the tangent space. Similarly, a point  $(x_0, \mathbf{x}) \in \mathbb{R}^{d+1}$  can be projected on the hyperboloid manifold  $\mathbb{H}^d$  with:

$$\Pi_{\mathbb{R}^{d+1} \rightarrow \mathbb{H}^d} := (\sqrt{1 + \|\mathbf{x}\|_2^2}, \mathbf{x}).$$

We can easily extend these projections to any negative curvature and we apply these projections after each feature transform and log or exp map, to constrain embeddings and tangent vectors to remain on the manifold and tangent spaces, and avoid numerical errors.

### 3.5.3 Results

Table 3.2 reports the performance of HGCN in comparison to baseline methods. HGCN works best in inductive scenarios where both node features and network topology play an important role. The performance gain of HGCN with respect to Euclidean GNN models is correlated with graph hyperbolicity. HGCN achieves an average of 45.4% (LP) and 12.3% (NC) error reduction compared to the best deep baselines for graphs with high hyperbolicity (low  $\delta$ ), suggesting that GNNs can significantly benefit from hyperbolic geometry, especially in link prediction tasks. Furthermore, the performance gap between HGCN and HNN suggests that neighborhood aggregation has been effective in learning node representations in graphs. For example, in disease spread datasets, both Euclidean attention and hyperbolic geometry lead to significant improvement of HGCN over other baselines. This can be explained by the fact that in disease spread trees, parent nodes contaminate their children. HGCN can successfully model these asymmetric and hierarchical relationships with hyperbolic attention and improves performance over all baselines.

On the CORA dataset with low hyperbolicity, HGCN does not outperform Euclidean GNNs,

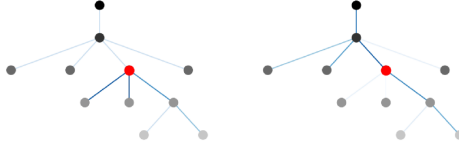


Figure 3.5: Attention: Euclidean GAT (left), HGCN (right). Each graph represents a 2-hop neighborhood of the DISEASE-M dataset.

suggesting that Euclidean geometry is better for its underlying graph structure. However, for small dimensions, HGCN is still significantly more effective than GCN even with CORA. Figure 3.3c shows 2-dimensional HGCN and GCN embeddings trained with LP objective, where colors denote the label class. HGCN achieves much better label class separation.

#### 3.5.4 Analysis

**Ablations** We further analyze the effect of proposed components in HGCN, namely hyperbolic attention (ATT) and trainable curvature (C) on AIRPORT and DISEASE datasets in Table 3.3. We observe that both attention and trainable curvature lead to performance gains over HGCN with fixed curvature and no attention. Furthermore, our attention model ATT outperforms  $ATT_{\mathbf{o}}$  (aggregation in tangent space at  $\mathbf{o}$ ), and we conjecture that this is because the local Euclidean average is a better approximation near the center point rather than near  $\mathbf{o}$ . Finally, the addition of both ATT and C improves performance even further, suggesting that both components are important in HGCN.

**Visualizations** We first visualize the GCN and HGCN embeddings at the first and last layers in Figure 3.3. We train HGCN with 3-dimensional hyperbolic embeddings and map them to the Poincaré disk which is better for visualization. In contrast to GCN, tree structure is preserved in HGCN, where nodes close to the center are higher in the hierarchy of the tree. This way HGCN smoothly transforms Euclidean features to Hyperbolic embeddings that preserve node hierarchy.

Figure 3.5 shows the attention weights in the 2-hop neighborhood of a center node (red) for the DISEASE dataset. The red node is the node where we compute attention. The darkness of the color for other nodes denotes their hierarchy. The attention weights for nodes in the neighborhood are visualized by the intensity of edges. We observe that in HGCN the center node pays more attention to its (grand)parent. In contrast to Euclidean GAT, our aggregation with attention in hyperbolic space allows us to pay more attention to nodes with high hierarchy. Such attention is crucial to good performance in DISEASE, because only sick parents will propagate the disease to their children.

## 3.6 Conclusion

We introduced HGCN, a novel architecture that learns hyperbolic embeddings using graph convolutional networks. In HGCN, the Euclidean input features are successively mapped to embeddings in hyperbolic spaces with trainable curvatures at every layer. HGCN achieves new state-of-the-art in learning embeddings for real-world hierarchical and scale-free graphs.

## Chapter 4

# Knowledge Graph Embeddings

### 4.1 Introduction

In this chapter, we propose a method to learn shallow embeddings of multi-relational graphs which in contrast to standard graphs have multiple edge types. In particular, Knowledge graphs (KGs), consisting of (*head entity*, *relationship*, *tail entity*) triples, are popular multi-relational data structures for representing factual knowledge to be queried and used in downstream applications such as word sense disambiguation, question answering, and information extraction. Real-world KGs such as Yago [164] or Wordnet [122] are usually incomplete, so a common approach to predicting missing links in KGs is via embedding into vector spaces. Embedding methods learn representations of entities and relationships that preserve the information found in the graph, and have achieved promising results for many tasks.

Relations found in KGs have differing properties: for example, (*Michelle Obama*, *married to*, *Barack Obama*) is symmetric, whereas hypernym relations like (*cat*, *specific type of*, *feline*), are not (Figure 4.1). These distinctions present a challenge to embedding methods: preserving each type of behavior requires producing a different geometric pattern in the embedding space. One popular approach is to use extremely high-dimensional embeddings, which offer more flexibility for such patterns. However, given the large number of entities found in KGs, doing so yields very high memory costs.

We saw in Chapter 1 that hyperbolic geometry offers an exciting approach to learn low-dimensional embeddings while preserving latent hierarchies. However, most existing methods [135, 150] focus on embedding simple graphs (e.g., weighted trees) and cannot capture the diverse and complex relationships in KGs.

We propose a new hyperbolic embedding approach that captures such patterns to achieve the best of both worlds. Our proposed approach produces the parsimonious representations offered

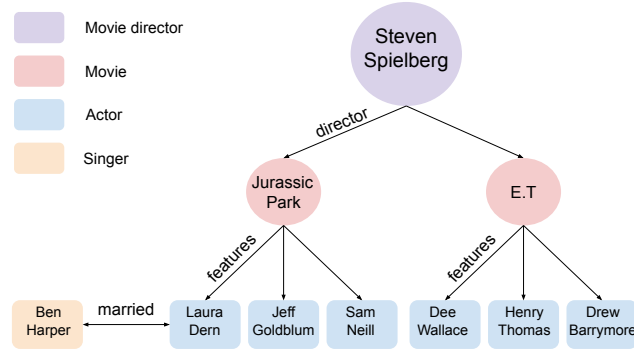


Figure 4.1: A toy example showing how KGs can simultaneously exhibit hierarchies and logical patterns.

by hyperbolic space, especially suitable for hierarchical relations, and is effective even with low-dimensional embeddings. It also uses rich transformations to encode logical patterns in KGs, previously only defined in Euclidean space. To accomplish this, we (1) train hyperbolic embeddings with relation-specific curvatures to preserve multiple hierarchies in KGs; (2) parameterize hyperbolic isometries (distance-preserving operations) and leverage their geometric properties to capture relations' logical patterns, such as symmetry or anti-symmetry; (3) and use a notion of hyperbolic attention to combine geometric operators and capture multiple logical patterns.

We evaluate the performance of our approach, ATTH, on the KG link prediction task using the standard WN18RR [27, 64], FB15k-237 [172] and YAGO3-10 [121] benchmarks. (1) In low (32) dimensions, we improve over Euclidean-based models by up to 6.1% in the mean reciprocal rank (MRR) metric. In particular, we find that hierarchical relationships, such as WordNet's *hypernym* and *member meronym*, significantly benefit from hyperbolic space; we observe a 16% to 24% relative improvement versus Euclidean baselines. (2) We find that geometric properties of hyperbolic isometries directly map to logical properties of relationships. We study symmetric and anti-symmetric patterns and find that reflections capture symmetric relations while rotations capture anti-symmetry. (3) We show that attention based-transformations have the ability to generalize to multiple logical patterns. For instance, we observe that ATTH recovers reflections for symmetric relations and rotations for the anti-symmetric ones.

In high (500) dimensions, we find that both hyperbolic and Euclidean embeddings achieve similar performance, and our approach achieves new state-of-the-art results (SotA), obtaining 49.6% MRR on WN18RR and 57.7% YAGO3-10. Our experiments show that trainable curvature is critical to generalize hyperbolic embedding methods to high-dimensions. Finally, we visualize embeddings learned in hyperbolic spaces and show that hyperbolic geometry effectively preserves hierarchies in KGs.

## 4.2 Related Work

Previous methods for KG embeddings also rely on geometric properties. Improvements have been obtained by exploiting either more sophisticated spaces (e.g., going from Euclidean to complex or hyperbolic space) or more sophisticated operations (e.g., from translations to isometries, or to learning graph neural networks). In contrast, our approach takes a step forward in both directions.

**Euclidean embeddings** In the past decade, there has been a rich literature on Euclidean embeddings for KG representation learning. These include translation approaches [27, 94, 117, 186] or tensor factorization methods such as RESCAL [134] or DistMult [192]. While these methods are fairly simple and have few parameters, they fail to encode important logical properties (e.g., translations can’t encode symmetry).

**Complex embeddings** Recently, there has been interest in learning embeddings in complex space, as in the ComplEx [174] and RotatE [165] models. RotatE learns rotations in complex space, which are very effective in capturing logical properties such as symmetry, anti-symmetry, composition or inversion. The recent QuatE model [201] learns KG embeddings using quaternions. However, a downside is that these embeddings require very high-dimensional spaces, leading to high memory costs.

**Deep neural networks** Another family of methods uses neural networks to produce KG embeddings. For instance, R-GCN [154] extends graph neural networks to the multi-relational setting by adding a relation-specific aggregation step. ConvE and ConvKB [64, 132] leverage the expressiveness of convolutional neural networks to learn entity embeddings and relation embeddings. More recently, the KBGAT [131] and A2N [15] models use graph attention networks for knowledge graph embeddings. A downside of these methods is that they are computationally expensive as they usually require pre-trained KG embeddings as input for the neural network.

**Hyperbolic embeddings** To the best of our knowledge, MuRP [12] is the only method that learns KG embeddings in hyperbolic space in order to target hierarchical data. MuRP minimizes hyperbolic distances between a re-scaled version of the head entity embedding and a translation of the tail entity embedding. It achieves promising results using hyperbolic embeddings with fewer dimensions than its Euclidean analogues. However, MuRP is a translation model and fails to encode some logical properties of relationships. Furthermore, embeddings are learned in a hyperbolic space with fixed curvature, potentially leading to insufficient precision, and training relies on cumbersome Riemannian optimization. Instead, our proposed method leverages expressive hyperbolic isometries to simultaneously capture logical patterns and hierarchies. Furthermore, embeddings are learned using tangent space (i.e., Euclidean) optimization methods and trainable hyperbolic curvatures per

relationship, avoiding precision errors that might arise when using a fixed curvature, and providing flexibility to encode multiple hierarchies.

### 4.3 Methodology

The goal of this work is to learn parsimonious hyperbolic embeddings that can encode complex logical patterns such as symmetry, anti-symmetry, or inversion while preserving latent hierarchies. Our model, ATTH, (1) learns KG embeddings in hyperbolic space in order to preserve hierarchies (Section 4.3.1), (2) uses a class of hyperbolic isometries parameterized by compositions of Givens transformations to encode logical patterns (Section 4.3.2), (3) combines these isometries with hyperbolic attention (Section 4.3.3). We describe the full model in Section 4.3.4 and our optimization strategy in Section 4.3.5. Before we start, we introduce the problem setup and some notation.

**Problem formulation** In the KG embedding problem, we are given a set of triples  $(h, r, t) \in \mathcal{E} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ , where  $\mathcal{V}$  and  $\mathcal{R}$  are entity and relationship sets, respectively. The goal is to map entities  $v \in \mathcal{V}$  to embeddings  $\mathbf{e}_v \in \mathcal{U}^{d_v}$  and relationships  $r \in \mathcal{R}$  to embeddings  $\mathbf{r}_r \in \mathcal{U}^{d_r}$ , for some choice of space  $\mathcal{U}$  (traditionally  $\mathbb{R}$ ), such that the KG structure is preserved.

Concretely, the data is split into  $\mathcal{E}_{Train}$  and  $\mathcal{E}_{Test}$  triples. Embeddings are learned by optimizing a scoring function  $s : \mathcal{V} \times \mathcal{R} \times \mathcal{V} \rightarrow \mathbb{R}$ , which measures triples' likelihoods.  $s(\cdot, \cdot, \cdot)$  is trained using triples in  $\mathcal{E}_{Train}$  and the learned embeddings are then used to predict scores for triples in  $\mathcal{E}_{Test}$ . The goal is to learn embeddings such that the scores of triples in  $\mathcal{E}_{Test}$  are high compared to triples that are not present in  $\mathcal{E}$ .

**Hyperbolic geometry** In this chapter, we work with the Poincaré model of hyperbolic space with negative curvature  $-c$  ( $c > 0$ ):  $\mathbb{B}^{d,c} = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\|^2 < \frac{1}{c}\}$ , where  $\|\cdot\|$  denotes the  $L_2$  norm. In what follows, we will heavily rely on the exponential ( $\exp_o^c$ ) and logarithmic maps ( $\log_o^c$ ) taken at the origin of the ball and refer to Section 1.3.1 in Chapter 1 for a review.

#### 4.3.1 Hierarchies in hyperbolic space

**Hyperbolic Curvature** As described, hyperbolic embeddings enable us to represent hierarchies even when we limit ourselves to low-dimensional spaces. In fact, two-dimensional hyperbolic space can represent any tree with arbitrarily small error [150].

It is important to set the curvature of the hyperbolic space correctly. This parameter provides flexibility to the model, as it determines whether to embed relations into a more curved hyperbolic space (more "tree-like"), or into a flatter, more "Euclidean-like" geometry. For each relation, we learn a relation-specific absolute curvature  $c_r$ , enabling us to represent a variety of hierarchies. As we

show in Section 4.4.5, fixing, rather than learning curvatures can lead to significant performance degradation.

We expect relations that are more tree-like to benefit from more curvature, and to validate this we need a way to measure how close a given graph is to being a tree, which we describe next.

We use two metrics to estimate how hierarchical a relation is: the curvature estimate  $\xi_G$  and the Krackhardt hierarchy score  $\text{Khs}_G$ . While the curvature estimate captures global hierarchical behaviours (how much the graph is tree-like when zooming-out), the Krackhardt score captures a more local behaviour (how many small loops the graph has). See Fig. 4.2 for examples.

**Graph curvature** To estimate the curvature of a relation  $r$ , we restrict to the undirected graph  $G_r$  spanned by the edges labeled as  $r$ . Following [82], let  $\xi_{G_r}(a, b, c)$  be the curvature estimate of a triangle in  $G_r$  with vertices  $\{a, b, c\}$ , which is given by:

$$\begin{aligned} \xi_{G_r}(a, b, c) = & \frac{1}{2d_{G_r}(a, m)} (d_{G_r}(a, m)^2 \\ & + d_{G_r}(b, c)^2/4 \\ & - (d_{G_r}(a, b)^2 + d_{G_r}(a, c)^2)/2), \end{aligned}$$

where  $m$  is the midpoint of the shortest path connecting  $b$  to  $c$ . This estimate is positive for triangles in circles, negative for triangles in trees, and zero for triangles in lines. Moreover, for a triangle in a Riemannian manifold  $M$ ,  $\xi_M(a, b, c)$  estimates the sectional curvature of the plane on which the triangle lies (see [82] for more details). Let  $m_r$  be the total number of connected components in  $G_r$ . We sample 1000  $w_{i,r}$  triangles from each connected component  $c_{i,r}$  of  $G_r$  where  $w_{i,r} = \frac{N_{i,r}^3}{\sum_{i=1}^{m_r} N_{i,r}^3}$ , and  $N_{i,r}$  is the number of nodes in the component  $c_{i,r}$ .  $\xi_{G_r}$  is the mean of the estimated curvatures of the sampled triangles. For the full graph, we take the weighted average of the relation curvatures  $\xi_{G_r}$  with respect to the weights  $\frac{\sum_{i=1}^{m_r} N_{i,r}^3}{\sum_r \sum_{i=1}^{m_r} N_{i,r}^3}$ .

For the directed graph  $G_r$  spanned by the relation  $r$ , we let  $R$  be the adjacency matrix ( $R_{i,j} = 1$  if there is an edge from node  $i$  to node  $j$  and 0 otherwise). Then:

$$\text{Khs}_{G_r} = \frac{\sum_{i,j=1}^n R_{i,j}(1 - R_{j,i})}{\sum_{i,j=1}^n R_{i,j}}.$$

See [107] for more details. We note that for fully observed symmetric relations (each edge is in a two-edge loop),  $\text{Khs}_{G_r} = 0$  while for anti-symmetric relations (no small loops),  $\text{Khs}_{G_r} = 1$ .

### 4.3.2 Hyperbolic Isometries

Relationships often satisfy particular properties, such as symmetry: e.g., if *(Michelle Obama, married to, Barack Obama)* holds, then *(Barack Obama, married to, Michelle Obama)* does as well. These

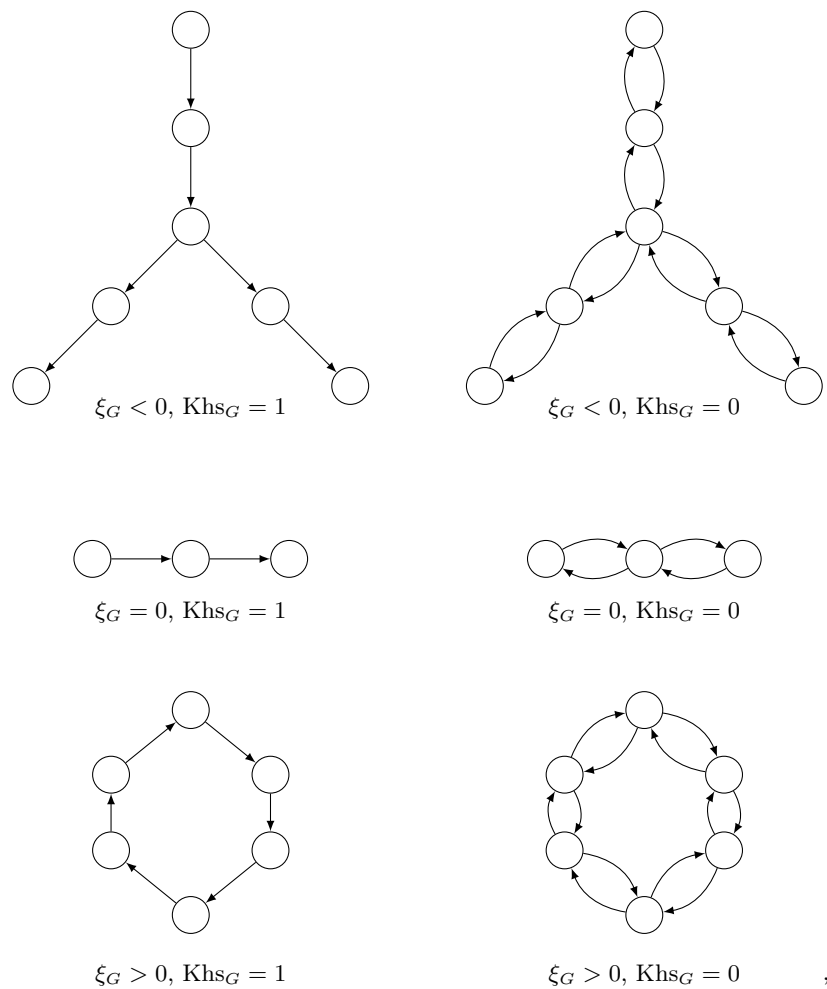


Figure 4.2: The curvature estimate  $\xi_G$  and the Krackhardt hierarchy score  $\text{Khs}_G$  for several simple graphs. The top-left graph is the most hierarchical, while the bottom-right graph is the least hierarchical.

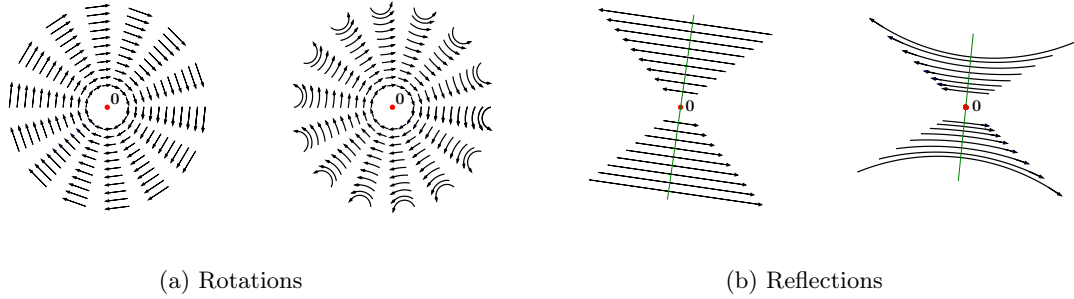


Figure 4.3: Euclidean (left) and hyperbolic (right) isometries. In hyperbolic space, the distance between start and end points after applying rotations or reflections is much larger than the Euclidean distance; it approaches the sum of the distances between the points and the origin, giving more “room” to separate embeddings. This is similar to trees, where the shortest path between two points goes through their nearest common ancestor.

rules are not universal. For instance, (*Barack Obama, born in, Hawaii*) is not symmetric.

Creating and curating a set of deterministic rules is infeasible for large-scale KGs; instead, embedding methods represent relations as parameterized geometric operations that directly map to logical properties. We use two such operations in hyperbolic space: *rotations*, which effectively capture compositions or anti-symmetric patterns, and *reflections*, which naturally encode symmetric patterns.

**Rotations** Rotations have been successfully used to encode compositions in complex space with the RotatE model [165]; we lift these to hyperbolic space. Compared to translations or tensor factorization approaches which can only infer some logical patterns, rotations can simultaneously model and infer *inversion*, *composition*, *symmetric* or *anti-symmetric* patterns.

**Reflections** These isometries reflect along a fixed subspace. While some rotations can represent symmetric relations (more specifically  $\pi$ -rotations), any reflection can naturally represent symmetric relations, since their second power is the identity. They provide a way to fill-in missing entries in symmetric triples, by applying the same operation to both the tail and the head entity. For instance, by modelling *sibling of* with a reflection, we can directly infer (*Bob, sibling of, Alice*) from (*Alice, sibling of, Bob*) and vice versa.

**Parameterization** Unlike RotatE which models rotations via unitary complex numbers, we learn relationship-specific isometries using Givens transformations,  $2 \times 2$  matrices commonly used in numerical linear algebra. Let  $\Theta_r := (\theta_{r,i})_{i \in \{1, \dots, \frac{d}{2}\}}$  and  $\Phi_r := (\phi_{r,i})_{i \in \{1, \dots, \frac{d}{2}\}}$  denote relation-specific parameters. Using an even number of dimensions  $d$ , our model parameterizes rotations and reflections

with block-diagonal matrices of the form:

$$\text{Rot}(\Theta_r) = \text{diag}(G^+(\theta_{r,1}), \dots, G^+(\theta_{r, \frac{d}{2}})), \quad (4.1)$$

$$\text{Ref}(\Phi_r) = \text{diag}(G^-(\phi_{r,1}), \dots, G^-(\phi_{r, \frac{n}{2}})), \quad (4.2)$$

$$\text{where } G^\pm(\theta) := \begin{bmatrix} \cos(\theta) & \mp \sin(\theta) \\ \sin(\theta) & \pm \cos(\theta) \end{bmatrix}. \quad (4.3)$$

Rotations and reflections of this form are hyperbolic isometries (distance-preserving). We can therefore directly apply them to hyperbolic embeddings while preserving the underlying geometry. Additionally, these transformations are computationally efficient and can be computed in linear time in the dimension. We illustrate two-dimensional isometries in both Euclidean and hyperbolic spaces in Figure 4.3.

### 4.3.3 Hyperbolic Attention

Of our two classes of hyperbolic isometries, one or the other may better represent a particular relation. To handle this, we use an attention mechanism to learn the right isometry. Thus we can represent symmetric, anti-symmetric or mixed-behaviour relations (i.e. neither symmetric nor anti-symmetric) as a combination of rotations and reflections.

Let  $\mathbf{x}^H$  and  $\mathbf{y}^H$  be hyperbolic points (e.g., reflection and rotation embeddings), and  $\mathbf{a}$  be an attention vector. Our approach maps hyperbolic representations to tangent space representations,  $\mathbf{x}^E = \log_{\mathbf{o}}^c(\mathbf{x}^H)$  and  $\mathbf{y}^E = \log_{\mathbf{o}}^c(\mathbf{y}^H)$ , and computes attention scores:

$$(\alpha_{\mathbf{x}}, \alpha_{\mathbf{y}}) = \text{Softmax}(\mathbf{a}^T \mathbf{x}^E, \mathbf{a}^T \mathbf{y}^E).$$

We then compute a weighted average using the recently proposed tangent space average [38, 118]:

$$\text{Att}(\mathbf{x}^H, \mathbf{y}^H; \mathbf{a}) := \exp_{\mathbf{o}}^c(\alpha_{\mathbf{x}} \mathbf{x}^E + \alpha_{\mathbf{y}} \mathbf{y}^E). \quad (4.4)$$

### 4.3.4 The ATTH model

We have all of the building blocks for ATTH, and can now describe the model architecture. Let  $(\mathbf{e}_v^H)_{v \in \mathcal{V}}$  and  $(\mathbf{r}_r^H)_{r \in \mathcal{R}}$  denote entity and relationship hyperbolic embeddings respectively. For a triple  $(h, r, t) \in \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ , ATTH applies relation-specific rotations (Equation 4.1) and reflections (Equation 4.2) to the head embedding:

$$\mathbf{q}_{\text{Rot}}^H = \text{Rot}(\Theta_r) \mathbf{e}_h^H, \quad \mathbf{q}_{\text{ref}}^H = \text{Ref}(\Phi_r) \mathbf{e}_h^H. \quad (4.5)$$

ATTH then combines the two representations using hyperbolic attention (Equation 4.4) and applies a hyperbolic translation using a Möbius addition (see Section 1.3.1 in Chapter 1):

$$Q(h, r) = \text{Att}(\mathbf{q}_{\text{Rot}}^H, \mathbf{q}_{\text{Ref}}^H; \mathbf{a}_r) \oplus^{c_r} \mathbf{r}_r^H. \quad (4.6)$$

Intuitively, rotations and reflections encode logical patterns while translations capture tree-like structures by moving between levels of the hierarchy. Finally, query embeddings are compared to target tail embeddings via the hyperbolic distance (Equation 1.3.1). The resulting scoring function is:

$$s(h, r, t) = -d^{c_r}(Q(h, r), \mathbf{e}_t^H)^2 + b_h + b_t, \quad (4.7)$$

where  $(b_v)_{v \in \mathcal{V}}$  are entity biases which act as margins in the scoring function [12, 171], and  $d^{c_r}(\cdot, \cdot)$  is the Poincaré distance function (see Section 1.3.1 in Chapter 1).

The model parameters are then  $\{(\Theta_r, \Phi_r, \mathbf{r}_r^H, \mathbf{a}_r, c_r)_{r \in \mathcal{R}}, (\mathbf{e}_v^H, b_v)_{v \in \mathcal{V}}\}$ . Note that the total number of parameters in ATTH is  $\mathcal{O}(|\mathcal{V}|d)$ , similar to traditional models that do not use attention or geometric operations. The extra cost is proportional to the number of relations, which is usually much smaller than the number of entities.

### 4.3.5 Tangent Space Optimization

Optimization in hyperbolic space normally requires Riemannian Stochastic Gradient Descent (RSGD) [26], which is challenging in practice. Instead, we use tangent space optimization by defining all parameters in the tangent space at the origin (our parameter space), as was done in the previous chapter. We optimize embeddings using standard Euclidean techniques, and use the exponential map to recover the hyperbolic parameters.

Note that tangent space optimization is an exact procedure, which does not incur losses in representational power. This is the case in hyperbolic space specifically because of a completeness property: there is always a global bijection between the tangent space and the manifold.

Concretely, ATTH optimizes the entity and relationship embeddings  $(\mathbf{e}_v^E)_{v \in \mathcal{V}}$  and  $(\mathbf{r}_r^E)_{r \in \mathcal{R}}$ , which are mapped to the Poincaré ball with:

$$\mathbf{e}_v^H = \exp_{\mathbf{o}}^{c_r}(\mathbf{e}_v^E) \quad \text{and} \quad \mathbf{r}_r^H = \exp_{\mathbf{o}}^{c_r}(\mathbf{r}_r^E), \quad (4.8)$$

The trainable model parameters are then  $\{(\Theta_r, \Phi_r, \mathbf{r}_r^E, \mathbf{a}_r, c_r)_{r \in \mathcal{R}}, (\mathbf{e}_v^E, b_v)_{v \in \mathcal{V}}\}$ , which are all Euclidean parameters that can be learned using standard Euclidean optimization techniques.

We train ATTH by minimizing the full cross-entropy loss with uniform negative sampling, where negative examples for a triple  $(h, r, t)$  are sampled uniformly from all possible triples obtained by

| Dataset   | #entities | #relations | #triples | $\xi_G$ |
|-----------|-----------|------------|----------|---------|
| WN18RR    | 41k       | 11         | 93k      | -2.54   |
| FB15k-237 | 15k       | 237        | 310k     | -0.65   |
| YAGO3-10  | 123k      | 37         | 1M       | -0.54   |

Table 4.1: Datasets statistics. The lower the metric  $\xi_G$  is, the more tree-like the knowledge graph is.

perturbing the tail entity:

$$\mathcal{L} = \sum_{t' \sim \mathcal{U}(\mathcal{V})} \log(1 + \exp(y_{t'} s(h, r, t'))), \text{ where } y_{t'} = \begin{cases} -1 & \text{if } t' = t \\ 1 & \text{otherwise.} \end{cases} \quad (4.9)$$

## 4.4 Experiments

In low dimensions, we hypothesize (1) that hyperbolic embedding methods obtain better representations and allow for improved downstream performance for hierarchical data (Section 4.4.2). (2) We expect the performance of relation-specific geometric operations to vary based on the relation’s logical patterns (Section 4.4.3). (3) In cases where the relations are neither purely symmetric nor anti-symmetric, we anticipate that hyperbolic attention outperforms the models which are based on solely reflections or rotations (Section 4.4.4). Finally, in high dimensions, we expect hyperbolic models with trainable curvature to learn the best geometry, and perform similarly to their Euclidean analogues (Section 4.4.5).

### 4.4.1 Experimental Setup

**Datasets** We evaluate our approach on the link prediction task using three standard competition benchmarks, namely WN18RR [27, 64], FB15k-237 [27, 172] and YAGO3-10 [121]. WN18RR is a subset of WordNet containing 11 lexical relationships between 40,943 word senses, and has a natural hierarchical structure, e.g., (*car*, *hypernym of*, *sedan*). FB15k-237 is a subset of Freebase, a collaborative KB of general world knowledge. FB15k-237 has 14,541 entities and 237 relationships, some of which are non-hierarchical, such as *born-in* or *nationality*, while others have natural hierarchies, such as *part-of* (for organizations). YAGO3-10 is a subset of YAGO3, containing 123,182 entities and 37 relations, where most relations provide descriptions of people. Some relationships have a hierarchical structure such as *playsFor* or *actedIn*, while others induce logical patterns, like *isMarriedTo*.

For each KG, we follow the standard data augmentation protocol by adding inverse relations [111] to the datasets. Additionally, we estimate the global graph curvature  $\xi_G$  [82] as was described in Section 4.3.1 and summarize the datasets’ statistics in Table 4.1.

| $\mathcal{U}$      | Model      | WN18RR      |             |             |             | FB15k-237   |             |             |             | YAGO3-10    |             |             |             |
|--------------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                    |            | MRR         | H@1         | H@3         | H@10        | MRR         | H@1         | H@3         | H@10        | MRR         | H@1         | H@3         | H@10        |
| $\mathbb{R}^d$     | RotatE     | .387        | .330        | .417        | .491        | .290        | .208        | .316        | .458        | -           | -           | -           | -           |
|                    | MuRE       | .458        | <u>.421</u> | .471        | .525        | .313        | .226        | .340        | .489        | .283        | .187        | .317        | .478        |
| $\mathbb{C}^d$     | ComplEx-N3 | .420        | .390        | .420        | .460        | .294        | .211        | .322        | .463        | <u>.336</u> | <u>.259</u> | <u>.367</u> | <u>.484</u> |
| $\mathbb{B}^{d,1}$ | MuRP       | <u>.465</u> | .420        | <u>.484</u> | <u>.544</u> | <u>.323</u> | <u>.235</u> | <u>.353</u> | <b>.501</b> | .230        | .150        | .247        | .392        |
| $\mathbb{R}^d$     | REFE       | .455        | .419        | .470        | .521        | .302        | .216        | .330        | .474        | .370        | .289        | .403        | .527        |
|                    | RoTE       | .463        | .426        | .477        | .529        | .307        | .220        | .337        | .482        | .381        | .295        | .417        | .548        |
|                    | ATTE       | .456        | .419        | .471        | .526        | .311        | .223        | .339        | .488        | .374        | .290        | .410        | .537        |
| $\mathbb{B}^{d,c}$ | REFH       | .447        | .408        | .464        | .518        | .312        | .224        | .342        | .489        | .381        | .302        | .415        | .530        |
|                    | ROT H      | <b>.472</b> | <b>.428</b> | <b>.490</b> | <b>.553</b> | .314        | .223        | .346        | .497        | .393        | .307        | .435        | .559        |
|                    | ATTH       | .466        | .419        | .484        | .551        | <b>.324</b> | <b>.236</b> | <b>.354</b> | <b>.501</b> | <b>.397</b> | <b>.310</b> | <b>.437</b> | <b>.566</b> |

Table 4.2: Link prediction results for low-dimensional embeddings ( $d = 32$ ) in the filtered setting. Best score in **bold** and best published underlined. Hyperbolic isometries significantly outperform Euclidean baselines on WN18RR and YAGO3-10, both of which exhibit hierarchical structures.

**Baselines** We compare our method to SotA models, including MuRP [13], MuRE (which is the Euclidean analogue of MuRP), RotatE [165], ComplEx-N3 [111] and TuckER [13]. Baseline numbers in high dimensions (Table 4.5) are taken from the original papers, while baseline numbers in the low-dimensional setting (Table 4.2) are computed using open-source implementations of each model. In particular, we run hyper-parameter searches over the same parameters as the ones in the original papers to compute baseline numbers in the low-dimensional setting.

**Ablations** To analyze the benefits of hyperbolic geometry, we evaluate the performance of ATTE, which is equivalent to ATTH with curvatures set to zero. Additionally, to better understand the role of attention, we report scores for variants of ATTE/H using only rotations (RoTE/H) or reflections (REFE/H).

**Evaluation metrics** At test time, we use the scoring function in Equation 4.7 to rank the correct tail or head entity against all possible entities, and use in use inverse relations for head prediction [111]. Similar to previous work, we compute two ranking-based metrics: (1) mean reciprocal rank (MRR), which measures the mean of inverse ranks assigned to correct entities, and (2) hits at  $K$  ( $H@K$ ,  $K \in \{1, 3, 10\}$ ), which measures the proportion of correct triples among the top  $K$  predicted triples. We follow the standard evaluation protocol in the filtered setting [27]: all true triples in the KG are filtered out during evaluation, since predicting a low rank for these triples should not be penalized.

**Implementation** We conducted a grid search to select the learning rate, optimizer, negative sample size, and batch size, using the validation set to select the best hyperparameters. Our best model hyperparameters are detailed in Table 4.6. We conducted all our experiments on NVIDIA

| Relation                    | Khs <sub>G</sub> | $\xi_G$ | ROTE        | ROTH        | Improvement |
|-----------------------------|------------------|---------|-------------|-------------|-------------|
| member meronym              | 1.00             | -2.90   | .320        | <b>.399</b> | 24.7%       |
| hypernym                    | 1.00             | -2.46   | .237        | <b>.276</b> | 16.5%       |
| has part                    | 1.00             | -1.43   | .291        | <b>.346</b> | 18.9%       |
| instance hypernym           | 1.00             | -0.82   | .488        | <b>.520</b> | 6.56%       |
| member of domain region     | 1.00             | -0.78   | <b>.385</b> | .365        | -5.19%      |
| member of domain usage      | 1.00             | -0.74   | <b>.458</b> | .438        | -4.37%      |
| synset domain topic of      | 0.99             | -0.69   | .425        | <b>.447</b> | 5.17%       |
| also see                    | 0.36             | -2.09   | .634        | <b>.705</b> | 11.2%       |
| derivationally related form | 0.07             | -3.84   | .960        | <b>.968</b> | 0.83%       |
| similar to                  | 0.07             | -1.00   | <b>1.00</b> | <b>1.00</b> | 0.00%       |
| verb group                  | 0.07             | -0.50   | <b>.974</b> | <b>.974</b> | 0.00%       |

Table 4.3: Comparison of H@10 for WN18RR relations. Higher Khs<sub>G</sub> and lower  $\xi_G$  means more hierarchical.

Tesla P100 GPUs and make our implementation publicly available<sup>1</sup>.

For all our Euclidean and hyperbolic models, we conduct a hyperparameter search for the learning rate, optimizer (Adam [100] or Adagrad [66]), negative sample size and batch size. We train each model for 500 epochs and use early stopping after 100 epochs if the validation MRR stops increasing. We report the best hyperparameters for each dataset in Table 4.6.

#### 4.4.2 Results in Low Dimensions

We first evaluate our approach in the low-dimensional setting for  $d = 32$ , which is approximately one order of magnitude smaller than SotA Euclidean methods. Table 4.2 compares the performance of ATTH to that of other baselines, including the recent hyperbolic (but not rotation-based) MuRP model. In low dimensions, hyperbolic embeddings offer much better representations for hierarchical relations, confirming our hypothesis. ATTH improves over previous Euclidean and hyperbolic methods by 0.7% and 6.1% points in MRR on WN18RR and YAGO3-10 respectively. Both datasets have multiple hierarchical relationships, suggesting that the hierarchical structure imposed by hyperbolic geometry leads to better embeddings. On FB15k-237, ATTH and MuRP achieve similar performance, both improving over Euclidean baselines. We conjecture that translations are sufficient to model relational patterns in FB15k-237.

Additionally, we measure performance per relation on WN18RR in Table 4.3 to understand the benefits of hyperbolic geometric on hierarchical relations. We report the Krackhardt hierarchy score (Khs<sub>G</sub>) [12] and estimated curvature per relation. We consider a relation to be hierarchical when its corresponding graph is close to tree-like (low curvature, high Khs<sub>G</sub>). We observe that hyperbolic embeddings offer much better performance on hierarchical relations such as *hypernym* or *has part*, while Euclidean and hyperbolic embeddings have similar performance on non-hierarchical relations such as *verb group*. We also plot the learned curvature per relation versus the embedding dimension

<sup>1</sup>Code available at <https://github.com/HazyResearch/KGEmb>

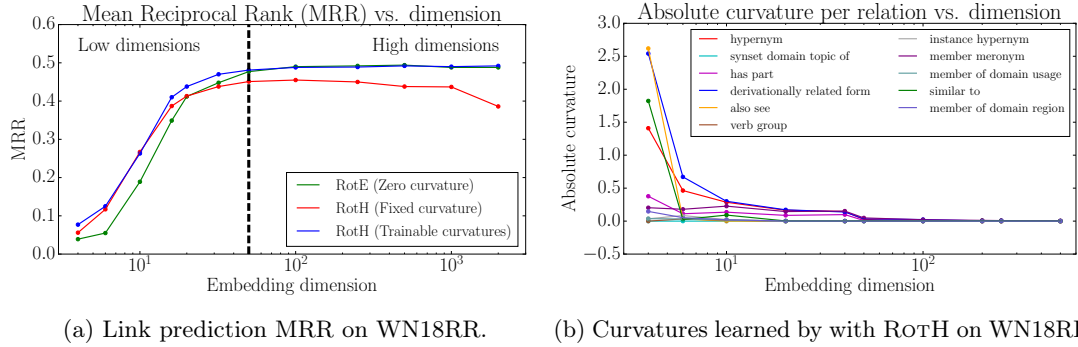


Figure 4.4: (a): ROTH offers improved performance in low dimensions; in high dimensions, fixed curvature degrades performance, while trainable curvature approximately recovers Euclidean space. (b): As the dimension increases, the learned curvature of hierarchical relationships tends to zero.

in Figure 4.4b. We note that the learned curvature in low dimensions directly correlates with the estimated graph curvature  $\xi_G$  in Table 4.3, suggesting that the model with learned curvatures learns more “curved” embedding spaces for tree-like relations.

Finally, we observe that MurP achieves lower performance than MurE on YAGO3-10, while ATTH improves over ATTE by 2.3% in MRR. This suggests that trainable curvature is critical to learn embeddings with the right amount of curvature, while fixed curvature might degrade performance. We elaborate further on this point in Section 4.4.5.

### 4.4.3 Hyperbolic Rotations and Reflections

In our experiments, we find that rotations work well on WN18RR, which contains multiple hierarchical and anti-symmetric relations, while reflections work better for YAGO3-10 (Table 4.5). To better understand the mechanisms behind these observations, we analyze two specific patterns: relation symmetry and anti-symmetry. We report performance per-relation on a subset of YAGO3-10 relations in Table 4.4. We categorize relations into symmetric, anti-symmetric, or neither symmetric nor anti-symmetric categories using data statistics. More concretely, we consider a relation to satisfy a logical pattern when the logical condition is satisfied by most of the triplets (e.g., a relation  $r$  is symmetric if for most KG triples  $(h, r, t)$ ,  $(t, r, h)$  is also in the KG). We observe that reflections encode symmetric relations particularly well, while rotations are well suited for anti-symmetric relations. This confirms our intuition—and the motivation for our approach—that particular geometric properties capture different kinds of logical properties.

### 4.4.4 Attention-based Transformations

One advantage of using relation-specific transformations is that each relation can learn the right geometric operators based on the logical properties it has to satisfy. In particular, we observe that in

| Relation       | Anti-symmetric | Symmetric | ROTH        | REFH        | ATTH        |
|----------------|----------------|-----------|-------------|-------------|-------------|
| hasNeighbor    | <b>X</b>       | ✓         | .750        | <b>1.00</b> | <b>1.00</b> |
| isMarriedTo    | <b>X</b>       | ✓         | .941        | .941        | <b>1.00</b> |
| actedIn        | ✓              | <b>X</b>  | .145        | .110        | <b>.150</b> |
| hasMusicalRole | ✓              | <b>X</b>  | .431        | .375        | <b>.458</b> |
| directed       | ✓              | <b>X</b>  | .500        | .450        | <b>.567</b> |
| graduatedFrom  | ✓              | <b>X</b>  | .262        | .167        | <b>.274</b> |
| playsFor       | ✓              | <b>X</b>  | <b>.671</b> | .642        | .664        |
| wroteMusicFor  | ✓              | <b>X</b>  | <b>.281</b> | .188        | .266        |
| hasCapital     | ✓              | <b>X</b>  | .692        | <b>.731</b> | <b>.731</b> |
| dealsWith      | <b>X</b>       | <b>X</b>  | .286        | .286        | <b>.429</b> |
| isLocatedIn    | <b>X</b>       | <b>X</b>  | .404        | .399        | <b>.420</b> |

Table 4.4: Comparison of geometric transformations on a subset of YAGO3-10 relations.

both low- and high-dimensional settings, attention-based models can recover the performance of the best transformation on all datasets (Tables 4.2 and 4.5). Additionally, per-relationship results on YAGO3-10 in Table 4.4 suggest that ATTH indeed recovers the best geometric operation.

Furthermore, for relations that are neither symmetric nor anti-symmetric, we find that ATTH can outperform rotations and reflections, suggesting that combining multiple operators with attention can learn more expressive operators to model mixed logical patterns. In other words, attention-based transformations alleviate the need to conduct experiments with multiple geometric transformations by simply allowing the model to choose which one is best for a given relation.

#### 4.4.5 Results in High Dimensions

In high dimensions (Table 4.5), we compare against a variety of other models and achieve new SotA results on WN18RR and YAGO3-10, and third-best results on FB15k-237. As we expected, when the embedding dimension is large, Euclidean and hyperbolic embedding methods perform similarly across all datasets. We explain this behavior by noting that when the dimension is sufficiently large, both Euclidean and hyperbolic spaces have enough capacity to represent complex hierarchies in KGs. This is further supported by Figure 4.4b, which shows the learned absolute curvature versus the dimension. We observe that curvatures are close to zero in high dimensions, confirming our expectation that ROTH with trainable curvatures learns a roughly Euclidean geometry in this setting.

In contrast, fixed curvature degrades performance in high dimensions (Figure 4.4a), confirming the importance of trainable curvatures and its impact on precision and capacity (previously studied by [150]). Additionally, we show the embeddings’ norms distribution in Fig. 4.5. Fixed curvature results in embeddings being clustered near the boundary of the ball while trainable curvatures adjusts the embedding space to better distribute points throughout the ball. Precision issues that might arise with fixed curvature could also explain MurP’s low performance in high dimensions. Trainable curvatures allow ROTH to perform as well or better than previous methods in both low and high dimensions.

| $\mathcal{U}$      | Model      | WN18RR      |             |             |             | FB15k-237   |             |             |             | YAGO3-10    |             |             |             |
|--------------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                    |            | MRR         | H@1         | H@3         | H@10        | MRR         | H@1         | H@3         | H@10        | MRR         | H@1         | H@3         | H@10        |
| $\mathbb{R}^d$     | DistMult   | .430        | .390        | .440        | .490        | .241        | .155        | .263        | .419        | .340        | .240        | .380        | .540        |
|                    | ConvE      | .430        | .400        | .440        | .520        | .325        | .237        | .356        | .501        | .440        | .350        | .490        | .620        |
|                    | TuckER     | .470        | .443        | .482        | .526        | <u>.358</u> | <u>.266</u> | <u>.394</u> | .544        | -           | -           | -           | -           |
|                    | MurE       | .475        | .436        | .487        | .554        | .336        | .245        | .370        | .521        | .532        | .444        | .584        | .694        |
| $\mathbb{C}^d$     | ComplEx-N3 | .480        | .435        | .495        | .572        | .357        | .264        | .392        | .547        | <u>.569</u> | <u>.498</u> | <u>.609</u> | <u>.701</u> |
|                    | RotatE     | .476        | .428        | .492        | .571        | .338        | .241        | .375        | .533        | .495        | .402        | .550        | .670        |
| $\mathbb{H}^d$     | Quaternion | <u>.488</u> | <u>.438</u> | <u>.508</u> | <u>.582</u> | .348        | .248        | .382        | <b>.550</b> | -           | -           | -           | -           |
| $\mathbb{B}^{d,1}$ | MurP       | .481        | .440        | .495        | .566        | .335        | .243        | .367        | .518        | .354        | .249        | .400        | .567        |
| $\mathbb{R}^d$     | REFE       | .473        | .430        | .485        | .561        | .351        | .256        | .390        | .541        | <b>.577</b> | <b>.503</b> | <b>.621</b> | <b>.712</b> |
|                    | ROTE       | .494        | .446        | .512        | .585        | .346        | .251        | .381        | .538        | .574        | .498        | <b>.621</b> | .711        |
|                    | ATTE       | .490        | .443        | .508        | .581        | .351        | .255        | .386        | .543        | .575        | .500        | <b>.621</b> | .709        |
| $\mathbb{B}^{d,c}$ | REFH       | .461        | .404        | .485        | .568        | .346        | .252        | .383        | .536        | .576        | .502        | .619        | .711        |
|                    | ROTH       | <b>.496</b> | <b>.449</b> | <b>.514</b> | <b>.586</b> | .344        | .246        | .380        | .535        | .570        | .495        | .612        | .706        |
|                    | ATTH       | .486        | .443        | .499        | .573        | .348        | .252        | .384        | .540        | .568        | .493        | .612        | .702        |

Table 4.5: Link prediction results for high-dimensional embeddings (best for  $d \in \{200, 400, 500\}$ ) in the filtered setting. DistMult, ConvE and ComplEx results are taken from [64]. Best score in **bold** and best published underlined. ATTE and ATTH have similar performance in the high-dimensional setting, performing competitively with or better than state-of-the-art methods on WN18RR, FB15k-237 and YAGO3-10.

#### 4.4.6 Visualizations

In Figure 4.6, we visualize the embeddings learned by ROTE versus ROTH for a sub-tree of the *organism* entity in WN18RR. To better visualize the hierarchy, we apply  $k$  inverse rotations for all nodes at level  $k$  in the tree.

By contrast to ROTE, ROTH preserves the tree structure in the embedding space. Furthermore, we note that ROTE cannot simultaneously preserve the tree structure and make non-neighboring nodes far from each other. For instance, *virus* should be far from *male*, but preserving the tree structure (by going one level down in the tree) while making these two nodes far from each other is difficult in Euclidean space. In hyperbolic space, however, we observe that going one level down in the tree is achieved by translating embeddings towards the left. This pattern essentially illustrates the translation component in ROTH, allowing the model to simultaneously preserve hierarchies while making non-neighbouring nodes far from each other.

## 4.5 Conclusion

We introduce ATTH, a hyperbolic KG embedding model that leverages the expressiveness of hyperbolic space and attention-based geometric transformations to learn improved KG representations in low-dimensions. ATTH learns embeddings with trainable hyperbolic curvatures, allowing it to learn the right geometry for each relationship and generalize across multiple embedding dimensions. ATTH

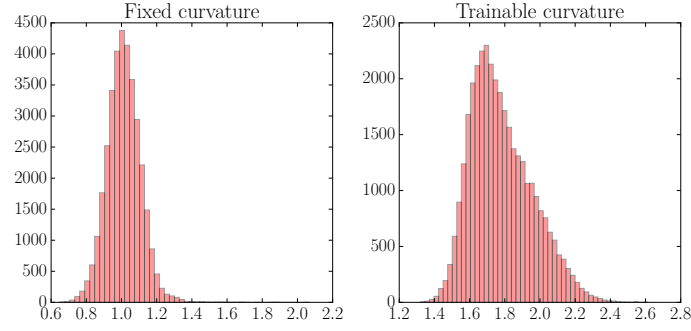
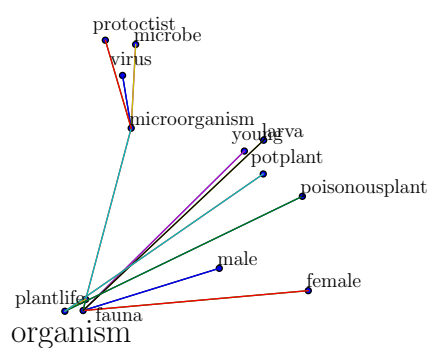
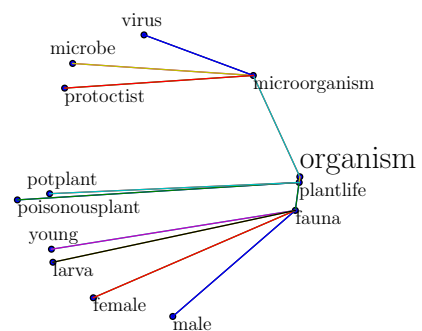


Figure 4.5: Histogram of embeddings norm learned with fixed and trainable curvatures for the hypernym relation in WN18RR.

achieves new SotA on WN18RR and YAGO3-10, real-world KGs which exhibit hierarchical structures. Future directions for this work include exploring other tasks that might benefit from hyperbolic geometry, such as hypernym detection. The proposed attention-based transformations can also be extended to other geometric operations.



(a) ROTE embeddings.



(b) ROTH embeddings.

Figure 4.6: Visualizations of the embeddings learned by ROTE and ROTH on a sub-tree of WN18RR for the *hypernym* relation. In contrast to ROTE, ROTH preserves hierarchies by learning tree-like embeddings.

| Dataset   | embedding dimension | model | learning rate | optimizer | batch size | negative samples |
|-----------|---------------------|-------|---------------|-----------|------------|------------------|
| WN18RR    | 32                  | REFE  | 0.001         | Adam      | 100        | 250              |
|           |                     | ROTE  | 0.001         | Adam      | 100        | 250              |
|           |                     | ATTE  | 0.001         | Adam      | 100        | 250              |
|           |                     | REFH  | 0.0005        | Adam      | 250        | 250              |
|           |                     | ROTH  | 0.0005        | Adam      | 500        | 50               |
|           |                     | ATTH  | 0.0005        | Adam      | 500        | 50               |
|           | 500                 | REFE  | 0.1           | Adagrad   | 500        | 50               |
|           |                     | ROTE  | 0.001         | Adam      | 100        | 500              |
|           |                     | ATTE  | 0.001         | Adam      | 1000       | 50               |
|           |                     | REFH  | 0.05          | Adagrad   | 500        | 50               |
|           |                     | ROTH  | 0.001         | Adam      | 1000       | 50               |
|           |                     | ATTH  | 0.001         | Adam      | 1000       | 50               |
| FB15k-237 | 32                  | REFE  | 0.075         | Adagrad   | 250        | 250              |
|           |                     | ROTE  | 0.05          | Adagrad   | 500        | 50               |
|           |                     | ATTE  | 0.05          | Adagrad   | 500        | 50               |
|           |                     | REFH  | 0.05          | Adagrad   | 500        | 250              |
|           |                     | ROTH  | 0.1           | Adagrad   | 100        | 50               |
|           |                     | ATTH  | 0.05          | Adagrad   | 500        | 100              |
|           | 500                 | REFE  | 0.05          | Adagrad   | 500        | 50               |
|           |                     | ROTE  | 0.05          | Adagrad   | 100        | 50               |
|           |                     | ATTE  | 0.05          | Adagrad   | 500        | 50               |
|           |                     | REFH  | 0.05          | Adagrad   | 500        | 50               |
|           |                     | ROTH  | 0.05          | Adagrad   | 1000       | 50               |
|           |                     | ATTH  | 0.05          | Adagrad   | 500        | 50               |
| YAGO3-10  | 32                  | REFE  | 0.005         | Adam      | 2000       | NA               |
|           |                     | ROTE  | 0.005         | Adam      | 2000       | NA               |
|           |                     | ATTE  | 0.005         | Adam      | 2000       | NA               |
|           |                     | REFH  | 0.005         | Adam      | 1000       | NA               |
|           |                     | ROTH  | 0.001         | Adam      | 1000       | NA               |
|           |                     | ATTH  | 0.001         | Adam      | 1000       | NA               |
|           | 500                 | REFE  | 0.005         | Adam      | 4000       | NA               |
|           |                     | ROTE  | 0.005         | Adam      | 4000       | NA               |
|           |                     | ATTE  | 0.005         | Adam      | 2000       | NA               |
|           |                     | REFH  | 0.001         | Adam      | 1000       | NA               |
|           |                     | ROTH  | 0.0005        | Adam      | 1000       | NA               |
|           |                     | ATTH  | 0.0005        | Adam      | 1000       | NA               |

Table 4.6: Best hyperparameters in low- and high-dimensional settings. NA negative samples indicates that the full cross-entropy loss is used, without negative sampling.

## Part II

# Algorithms in Hyperbolic Spaces

## Chapter 5

# Hierarchical Clustering

### 5.1 Introduction

We now move onto the second part of this dissertation, which is on algorithms that operate in hyperbolic spaces, starting with Hierarchical Clustering (HC). HC is a fundamental problem in data analysis, where given datapoints and their pairwise similarities, the goal is to construct a hierarchy over clusters, in the form of a tree whose leaves correspond to datapoints and internal nodes correspond to clusters. HC naturally arises in standard applications where data exhibits hierarchical structure, ranging from phylogenetics [69] and cancer gene sequencing [159, 160] to text/image analysis [163], community detection [114] and everything in between. A family of easy to implement, yet slow, algorithms includes agglomerative methods (e.g., Average-Linkage) that build the tree in a bottom-up manner by iteratively merging pairs of similar datapoints or clusters together. In contrast to “flat” clustering techniques like  $k$ -means, HC provides fine-grained interpretability and rich cluster information at all levels of granularity and alleviates the requirement of specifying a fixed number of clusters a priori.

Despite the abundance of HC algorithms, the HC theory was underdeveloped, since no “global” objective function was associated with the final tree output. A well-formulated objective allows us to compare different algorithms, measure their quality, and explain their success or failure.<sup>1</sup> To address this issue, Dasgupta [62] recently introduced a discrete cost function over the space of binary trees with  $n$  leaves. A key property of his cost function is that low-cost trees correspond to meaningful hierarchical partitions in the data. He initially proved this for symmetric stochastic block models, and later works provided experimental evidence [147], or showed it for hierarchical stochastic block models, suitable for inputs that contain a ground-truth hierarchical structure [55, 56]. These works led to important steps towards understanding old and building new HC algorithms [10, 43, 45, 127].

---

<sup>1</sup>Contrast this lack of HC objectives with flat clustering, where  $k$ -means objectives and others have been studied intensively from as early as 1960s (e.g., [86]), leading today to a comprehensive theory on clustering.

The goal of this work is to improve the performance of HC algorithms using a differentiable relaxation of Dasgupta’s discrete optimization problem. There have been recent attempts at gradient-based HC via embedding methods, which do not directly relax Dasgupta’s optimization problem. UFit [52] addresses a different “ultrametric fitting” problem using Euclidean embeddings, while gHHC [124] assumes that partial information about the optimal clustering—more specifically leaves’ hyperbolic embeddings—is known. Further, these two approaches lack theoretical guarantees in terms of clustering quality and are outperformed by discrete agglomerative algorithms.

A relaxation of Dasgupta’s discrete optimization combined with the powerful toolbox of gradient-based optimization has the potential to yield improvements in terms of (a) **clustering quality** (both theoretically and empirically), (b) **scalability**, and (c) **flexibility**, since a gradient-based approach can be integrated into machine learning (ML) pipelines with end-to-end training. However, due to the inherent discreteness of the HC optimization problem, several challenges arise:

1. How can we continuously **parameterize the search space** of discrete binary trees? A promising direction is leveraging *hyperbolic embeddings* which are more aligned with the geometry of trees than standard Euclidean embeddings [152]. However, hyperbolic embedding methods typically assume a *fully known* [136, 150] or partially known graph [124] that will be embedded, whereas the challenge here is searching over an exponentially large space of trees with *unknown* structure.
2. How can we derive a **differentiable relaxation** of the HC cost? One of the key challenges is that this cost relies on discrete properties of trees such as the lowest common ancestor (LCA).
3. How can we **decode** a discrete binary tree from continuous representations, while ensuring that the ultimate discrete cost is close to the continuous relaxation?

Here, we introduce HYPHC, an end-to-end differentiable model for HC with provable guarantees in terms of clustering quality, which can be easily incorporated into ML pipelines.

1. Rather than minimizing the cost function by optimizing over discrete trees, we parameterize trees using leaves’ hyperbolic embeddings. In contrast with Euclidean embeddings, hyperbolic embeddings can represent trees with arbitrarily low distortion in just two dimensions [152]. We show that the leaves themselves provide enough information about the underlying tree, avoiding the need to explicitly represent the discrete structure of internal nodes.
2. We derive a continuous LCA analogue, which leverages the analogies between shortest paths in trees and hyperbolic geodesics (Fig. 5.1), and propose a differentiable variant of Dasgupta’s cost.
3. We propose a decoding algorithm for the internal nodes which maps the learned leaf embeddings to a dendrogram (cluster tree) of low distortion.

We show (a) that our approach produces good **clustering quality**, in terms of Dasgupta cost. Theoretically, assuming perfect optimization, the optimal clustering found using HYPHC yields a  $(1 + \varepsilon)$ -approximation to the minimizer of the discrete cost, where  $\varepsilon$  can be made arbitrarily small,

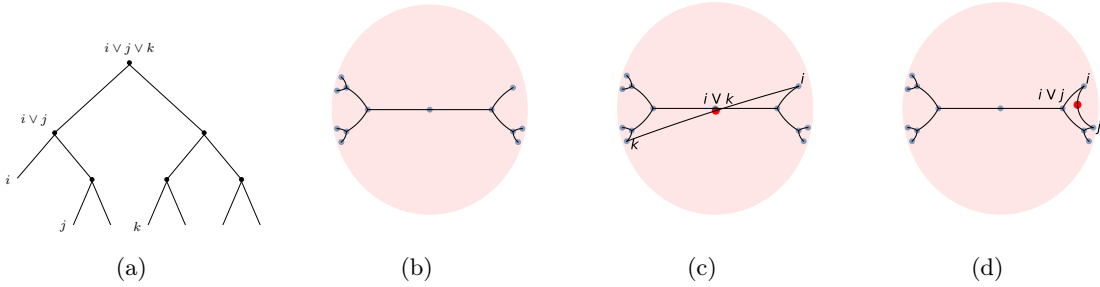


Figure 5.1: The tree shown in (a) is embedded into hyperbolic space ( $\mathbb{B}_2$ ) in (b), (c), and (d). In (c) and (d) we show the hyperbolic LCA (in red) and illustrate the relationship between the discrete LCA, which is central to Dasgupta’s cost, and geodesics (shortest paths) in hyperbolic space.

and controls the tradeoffs between quality guarantees and optimization challenges. Notice that due to our perfect optimization assumption, this does not contradict previously known computational hardness results based on the Small Set Expansion for achieving constant approximations [43], but allows us to leverage the powerful toolbox of nonconvex optimization. Empirically, we find that even approximate HYPHC solutions found with gradient descent outperform or match the performance of the best discrete and continuous methods on a variety of HC benchmarks. Additionally, (b) we propose two extensions of HYPHC that enable us to **scale** to large inputs and we also study the tradeoffs between clustering quality and speed. Finally, (c) we demonstrate the **flexibility** of HYPHC by jointly optimizing embeddings for HC and a downstream classification task, improving accuracy by up to 3.8% compared to the two-stage embed-then-classify approach.

## 5.2 Related Work

We first review related work for similarity HC.

**Agglomerative Hierarchical Clustering** One of the first suite of algorithms developed to solve HC was bottom-up linkage methods. These are simple and easy to implement algorithms that recursively merge similar datapoints to form some small clusters and then gradually larger and larger clusters emerge. Well-known heuristics include Single Linkage, Complete Linkage and Average Linkage, that we briefly describe here. All three heuristics start with  $n$  datapoints forming singleton clusters initially and perform exactly  $n - 1$  merges in total until they produce a binary tree corresponding to the HC output. At any given step, if  $A$  and  $B$  denote two already formed clusters, the criterion for which clusters to merge next is to *minimize* the minimum, maximum and average distance between two clusters  $A$  and  $B$ , for Single, Complete and Average Linkage respectively. If instead of pairwise distances the input was given as a similarity graph, analogous formulas can be used, where the criterion is to maximize the respective quantities. These algorithms can be made to run in time

$O(n^2 \log n)$ . A bottleneck at the core of the computations for such algorithms is the nearest neighbor problem, for which known almost quadratic lower bounds apply [9, 59]. However, recent works [1, 46] try to address this in Euclidean spaces where we are provided features by using locality sensitive hashing techniques and approximate nearest neighbors. Instead, here, we learn these features using similarities only, and then use a fast decoding algorithm on the learned features to output the final tree. Finally, HC has been studied in the model of parallel computation, where a variation of Boruvka’s minimum spanning tree algorithm was used [17].

**Dasgupta’s cost** Having an objective function for HC is crucial not only for guiding the optimization, but also for having theoretically grounded solutions. Unfortunately, there has been a lack of global objectives measuring the quality of a HC, which is in stark contrast with the plethora of flat clustering objectives like  $k$ -means,  $k$ -center, correlation clustering (e.g., [14, 44]) and many more. This lack of optimization objectives for HC was initially addressed indirectly in [61] (by comparing a tree against solutions to  $k$ -clustering objectives for multiple  $k$ ), and has again emerged recently by Dasgupta [62], who introduced a cost to evaluate and compare the performance of HC algorithms. This discrete HC cost has the key property that good clustering trees should yield a low cost. The formulation of this objective favors binary trees and the optimum solution can always be assumed to be binary, as a tree with higher fan-out in any internal node can easily be modified into a binary tree with the same cost (or less). In particular, one interesting aspect of this objective is that running Recursive Sparsest Cut would produce a HC with provable guarantees with respect to his cost function [43]. Subsequent work was able to shed light to Average Linkage performance; specifically, [127] studied the complement to Dasgupta’s cost function and showed that Average Linkage will find a solution within 33% of the optimizer. Further techniques based on semidefinite programming relaxations led to improved approximations [4, 10, 45], however with a significant overhead in runtime (see [47] for a survey).

**Gradient-based Hierarchical Clustering** Chierchia et al. [52] introduce an ultra-metric fitting framework (UFit) to learn embeddings in ultra-metric spaces. These are restrictive metric spaces (and a special case of tree metrics) where the triangle inequality is strengthened to  $d(x, z) \leq \max\{d(x, y), d(y, z)\}$ . UFit can be extended to HC using an agglomerative method to produce a dendrogram.

Monath et al. [124] propose a differentiable HC objective, which yields improvements in scalability and downstream task performance. Their approach, gHHC, is the first attempt at HC with hyperbolic embeddings. This model assumes input leaves’ hyperbolic embeddings are given (by fixing their distance to the origin) and optimizes embeddings of internal nodes. Once learned, the embeddings can be decoded into a discrete tree using heuristics and post-processing rules. gHHC differs from the traditional similarity-based HC setting, assuming additional information about the optimal clustering, in the form of hyperbolic leaves’ embeddings. These are computed using normalized Euclidean

features, which is mismatched to the geometry of the data.

Observing that the internal tree structure can be directly inferred from leaves' hyperbolic embeddings, we propose an approach that is a direct relaxation of Dasgupta's discrete optimization framework and does not assume any information about the optimal clustering. In contrast with previous gradient-based approaches, our approach has theoretical guarantees in terms of clustering quality and empirically outperforms agglomerative heuristics.

## 5.3 Background

We first introduce our notation in Section 5.3.1, and then describe our problem setup in Section 5.3.2.

### 5.3.1 Notation

We consider a dataset  $\mathcal{D}$  with  $n$  datapoints and pairwise similarities  $(w_{ij})_{i,j \in [n]}$ . A HC of  $\mathcal{D}$  is a rooted binary tree  $T$  with exactly  $n$  leaves, such that each leaf corresponds to a datapoint in  $\mathcal{D}$ , and intermediate nodes represent clusters. For two leaves  $(i, j)$  in  $T$ , we denote  $i \vee j$  their LCA, and  $T[i \vee j]$  the subtree rooted at  $i \vee j$ , which represents the smallest cluster containing both  $i$  and  $j$ . In particular, we denote  $\text{leaves}(T[i \vee j])$  the leaves of  $T[i \vee j]$ , which correspond to datapoints in the cluster  $i \vee j$ . Finally, given three leaf nodes  $(i, j, k)$  in  $T$ , we say that the relation  $\{i, j|k\}$  holds if  $i \vee j$  is a descendant of  $i \vee j \vee k$  (Fig. 5.1a). In this scenario, we have  $i \vee k = j \vee k$ .

Throughout this chapter, we use the two-dimensional Poincaré model of hyperbolic space with negative curvature -1 described in Chapter 1:

$$\mathbb{B}_2 = \{x \in \mathbb{R}^2 : \|x\|_2 < 1\}.$$

### 5.3.2 HC Discrete Optimization Framework

The goal of HC is to find a binary tree  $T$  that is mindful of pairwise similarities in the data. Dasgupta [62] introduced a cost function over possible binary trees with the crucial property that good trees should have a low cost:  $C_{\text{Dasgupta}}(T; w) = \sum_{i,j} w_{ij} |\text{leaves}(T[i \vee j])|$ . Intuitively, a good tree for Dasgupta's cost merges similar nodes (high  $w_{ij}$ ) first in the hierarchy (small subtree). In other words, a good tree should cluster the data such that similar datapoints have LCAs further from the root than dissimilar datapoints. As the quantity  $|\text{leaves}(T[i \vee j])|$  may be hard to parse, we note here a simpler way to think of this term based on triplets of datapoints  $i, j, k$ : whenever a tree splits  $i, j, k$  for the first time, then  $k \notin \text{leaves}(T[i \vee j])$  if and only if  $k$  was separated from  $i, j$ . In

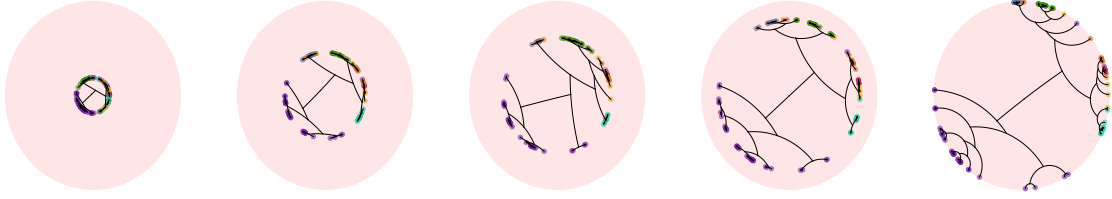


Figure 5.2: Visualization of HYPHC embeddings and decoded trees during optimization.

particular, Wang and Wang [184] observe that:

$$C_{\text{Dasgupta}}(T; w) = \sum_{ijk} [w_{ij} + w_{ik} + w_{jk} - w_{ijk}(T; w)] + 2 \sum_{ij} w_{ij} \quad (5.1)$$

where  $w_{ijk}(T; w) = w_{ij} \mathbb{1}[\{i, j|k\}] + w_{ik} \mathbb{1}[\{i, k|j\}] + w_{jk} \mathbb{1}[\{j, k|i\}]$ ,

which reduces difficult-to-compute quantities about subtree sizes to simpler statements about LCAs. For binary trees, exactly one of  $\mathbb{1}[\{i, j|k\}]$ ,  $\mathbb{1}[\{i, k|j\}]$ ,  $\mathbb{1}[\{j, k|i\}]$  holds, and these are defined through the notion of LCA. As we shall see next, we can relax this objective using a continuous notion of LCA in hyperbolic space. Dasgupta’s cost-based perspective leads to a natural optimization framework for HC, where the goal is to find  $T^*$  such that:

$$T^* = \underset{\text{all binary trees } T}{\operatorname{argmin}} C_{\text{Dasgupta}}(T; w). \quad (5.2)$$

## 5.4 Hyperbolic Hierarchical Clustering

Our goal is to relax the discrete optimization problem in Eq. (5.2). To do so, we represent trees using the hyperbolic embeddings of their leaves in the Poincaré disk (Section 5.4.1).<sup>2</sup> We introduce a differentiable HC cost, using a continuous LCA analogue (Section 5.4.2), and then propose a decoding algorithm,  $\text{dec}(\cdot)$ , which maps embeddings to discrete binary trees (Section 5.4.3). Our main result is that, when embeddings are optimized over a special set of *spread* embeddings (Definition 5.4.1), solving our continuous relaxation yields a  $(1 + \varepsilon)$ -approximation to the minimizer of Dasgupta’s cost (Section 5.4.4), where  $\varepsilon$  can be made arbitrarily small. Next, we present our continuous optimization framework and then detail the different components of HYPHC.

**HYPHC continuous optimization framework** Formally, if  $\mathcal{Z} \subset \mathbb{B}_2^n$  denotes an arbitrary constrained set of embeddings, we propose to optimize the following continuous constrained optimization

<sup>2</sup>We developed our theory in two dimensions. As shown in Sala et al. [150], this can present optimization difficulties due to precision requirements. However, using Prop 3.1 of [150], we can increase the dimension to reduce this effect.

problem as a proxy for the discrete problem in Eq. (5.2):

$$Z^* = \operatorname{argmin}_{Z \in \mathcal{Z}} C_{\text{HYPHC}}(Z; w, \tau) \text{ and } T = \operatorname{dec}(Z^*) \quad (5.3)$$

where  $C_{\text{HYPHC}}(\cdot; w, \tau)$  is our differentiable cost and  $\tau$  is a temperature used to relax the max function.

#### 5.4.1 Continuous Tree Representation via Hyperbolic Embeddings

To perform gradient-based HC, one needs a continuous parameterization of possible binary trees on a fixed number of leaves. We parameterize binary trees using the (learned) hyperbolic embedding of their leaves. Our insight is that because hyperbolic space induces a close correspondence between embeddings and tree metrics [152], the leaf embeddings alone provide enough information to recover the full tree. In fact, as hyperbolic points are pushed towards the boundary, the embedding becomes more tree-like (Fig. 5.2) [150]. Thus, rather than optimize over all possible tree structures, which could lead to combinatorial explosion, we embed only the leaf nodes in the Poincaré disk using an embedding look-up:

$$i \rightarrow z_i \in \mathbb{B}_2. \quad (5.4)$$

These leaves' embeddings are optimized with gradient-descent using a continuous relaxation of Dasgupta's cost (Section 5.4.2), and are then decoded into a discrete binary tree to produce a HC on the leaves (Section 5.4.3).

#### 5.4.2 Differentiable Objective Function

Having a continuous representation of trees is not sufficient for gradient-based HC, since Dasgupta's cost requires computing the discrete LCA. We leverage the similarities between geodesics in hyperbolic space and shortest paths in trees, to derive a continuous analogue of the discrete LCA. We then use this hyperbolic LCA to introduce a differentiable version of Dasgupta's cost,  $C_{\text{HYPHC}}(\cdot; w, \tau)$ .

**Hyperbolic lowest common ancestor** Given two leaf nodes  $i$  and  $j$ , their LCA  $i \vee j$  in  $T$  is the node on the shortest path connecting  $i$  and  $j$  (denoted  $i \rightsquigarrow j$ ) that is closest to the root node  $r$ :

$$i \vee j = \operatorname{argmin}_{k \in i \rightsquigarrow j} d_T(r, k). \quad (5.5)$$

Analogously, we define the hyperbolic LCA between two points  $(z_i, z_j)$  in hyperbolic space as the point on their geodesic (shortest path, denoted  $z_i \rightsquigarrow z_j$ ) that is closest to the origin (the root):

$$z_i \vee z_j := \operatorname{argmin}_{z \in z_i \rightsquigarrow z_j} d(o, z). \quad (5.6)$$

Note that  $z_i \vee z_j$  is also the orthogonal projection of the origin onto the geodesic (Fig. 5.1) and its hyperbolic distance to the origin can be computed exactly.

**Lemma 5.4.1.** *Let  $(x, y) \in (\mathbb{B}_2)^2$  and  $x \vee y$  denote the point on the geodesic connecting  $x$  and  $y$  that minimizes the distance to the origin  $o$ , and let  $d_o(\cdot)$  denote the hyperbolic distance to the origin. Let  $\theta$  be the angle between  $(x, y)$  and  $\alpha$  be the angle between  $(x, x \vee y)$ . We have:*

$$\begin{aligned} \alpha &= \tan^{-1} \left( \frac{1}{\sin(\theta)} \left( \frac{\|x\|_2(\|y\|_2^2 + 1)}{\|y\|_2(\|x\|_2^2 + 1)} - \cos(\theta) \right) \right), \\ \text{and } d_o(x \vee y) &= 2 \tanh^{-1}(\sqrt{R^2 + 1} - R), \\ \text{where } R &= \sqrt{\left( \frac{\|x\|_2^2 + 1}{2\|x\|_2 \cos(\alpha)} \right)^2 - 1}. \end{aligned} \tag{5.7}$$

*Proof.* We use circle inversions to show this result. Circle inversions are Euclidean transformations on the plane that map circles to circles and preserve angles [29], and can represent hyperbolic reflections (isometric transformations) along hyperbolic geodesics. In particular, the circle inversion formula can be used to recover the center of the circle that is orthogonal to the boundary of the disk and that coincides with the geodesic between two given points. Consider the circle defined by the geodesic connecting  $x$ , and  $y$  and let  $R$  denote its radius,  $p$  the orthogonal projection of  $o$  onto this circle and  $\Delta = \overline{oo'}$  denote the distance between the origin and the circle center (Fig. 5.3). By the circle inversion property and using the fact that the Poincaré disk has radius one, we have  $1 = (\Delta - R)(\Delta + R)$ , which yields:

$$R^2 = \Delta^2 - 1. \tag{5.8}$$

Additionally, if  $\theta = \angle xoy$  and  $\alpha = \angle pox$ , we have by the Pythagorean theorem:

$$\begin{cases} R^2 = (\Delta - \|x\|_2 \cos(\alpha))^2 + \|x\|_2^2 \sin^2(\alpha) \\ R^2 = (\Delta - \|y\|_2 \cos(\theta - \alpha))^2 + \|y\|_2^2 \sin^2(\theta - \alpha). \end{cases} \tag{5.9}$$

This leads to the system of equations:

$$\begin{cases} 2\sqrt{R^2 + 1} \cos(\alpha) = \frac{\|x\|_2^2 + 1}{\|x\|_2} \\ 2\sqrt{R^2 + 1} \cos(\theta - \alpha) = \frac{\|y\|_2^2 + 1}{\|y\|_2} \end{cases} \tag{5.10}$$

which is solved for  $R$  and  $\alpha$  defined in Eq. (5.7). Now using Eq. (5.8), we have:  $\|p\|_2 = \Delta - R = \sqrt{R^2 + 1} - R$ . Finally, we get the result using the hyperbolic distance function and noting that the orthogonal projection of a point on a geodesic that does not contain that point is minimizing the distance between the point and the geodesic, that is  $p = x \vee y$ .  $\square$

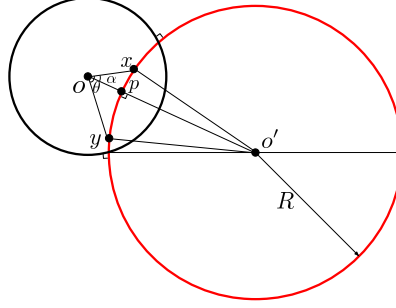


Figure 5.3: Circle inversion used for hyperbolic LCA construction in Lemma 5.4.1.

**HYPHC’s objective function** The non-differentiable term in Eq. (5.1) is  $w_{ijk}(T; w)$ , which is the similarity of the pair that has the deepest LCA in  $T$ , i.e. the LCA that is farthest—in tree distance—from the root. We use this qualitative interpretation to derive a continuous version of Dasgupta’s cost. Consider an embedding of  $T$  with  $n$  leaves,  $Z = \{z_1, \dots, z_n\}$ . The notion of deepest LCA can be extended to continuous embeddings by looking at the continuous LCA that is farthest—in hyperbolic distance—from the origin. Our differentiable HC objective is then:

$$C_{\text{HYPHC}}(Z; w, \tau) = \sum_{ijk} (w_{ij} + w_{ik} + w_{jk} - w_{\text{HYPHC},ijk}(Z; w, \tau)) + 2 \sum_{ij} w_{ij} \quad (5.11)$$

where  $w_{\text{HYPHC},ijk}(Z; w, \tau) = (w_{ij}, w_{ik}, w_{jk}) \cdot \sigma_\tau(d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k))^\top$ ,

and  $\sigma_\tau(\cdot)$  is the scaled softmax function:  $\sigma_\tau(\alpha)_i = e^{\alpha_i/\tau} / \sum_j e^{\alpha_j/\tau}$ .

### 5.4.3 Hyperbolic Decoding

The output of HC needs to be a discrete binary tree, and optimizing the HYPHC loss in Eq. (5.3) only produces leaves’ embeddings. We propose a way to decode a binary tree structure from embeddings by iteratively merging the most similar pairs based on their hyperbolic LCA distance to the origin (Algorithm 1). Intuitively, because  $\text{dec}(\cdot)$  uses LCA distances to the origin (and not pairwise distances), it can recover the underlying tree that is directly being optimized by HYPHC (which is only defined through these LCA depths).

### 5.4.4 Approximation Ratio Result

Assuming one can perfectly solve the constrained optimization in Eq. (5.3), our main result is that the solution of the continuous optimization, once decoded, recovers the optimal tree with a constant approximation factor, which can be made arbitrarily small. We first provide more conditions on the constrained set of embeddings in Eq. (5.3) by introducing a special set of *spread* embeddings  $\mathcal{Z} \subset \mathbb{B}_2^n$ :

**Algorithm 1** Hyperbolic binary tree decoding  $\text{dec}(Z)$ 


---

```

1: Input: Embeddings  $Z = \{z_1, \dots, z_n\}$ ; Output: Rooted binary tree with  $n$  leaves.
2:  $F \leftarrow$  forest  $(\{i\} : i \in [n])$ 
3:  $S \leftarrow \{(i, j) : \text{pairs sorted by deepest hyperbolic LCA } (d_o(z_i \vee z_j))\}$ ;
4: for  $(i, j) \in S$  do
5:   if  $i$  and  $j$  not in same tree in  $F$  then
6:      $r_i, r_j \leftarrow$  roots of trees containing  $i, j$ ;
7:     create new node with children  $r_i, r_j$ ;
8:   end if
9: end for
10: return  $F$  (which is a binary tree at the end of the algorithm)

```

---

**Definition 5.4.1.** An embedding  $Z \in \mathbb{B}_2^n$  is called spread if for every triplet  $(i, j, k)$ :

$$\max\{d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} - \min\{d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} > \delta \cdot O(n), \quad (5.12)$$

where  $\delta$  is a constant of hyperbolic space (Gromov's delta hyperbolicity, see Section 1.3.2 in Chapter 1).

Intuitively, the spread constraints spread points apart and force hyperbolic LCAs to be distinguishable from each other. Theoretically, we show that this induces a direct correspondence between embeddings and binary trees, i.e. the embeddings yield a metric that is close to that of a binary tree metric. More concretely, every spread leaf embedding is compatible with an underlying tree, in the sense that our decoding algorithm is guaranteed to return the underlying tree. This is the essence of the correspondence between spread embeddings and trees, which is the main ingredient behind Theorem 5.4.1.

**Theorem 5.4.1.** Consider a dataset with  $n$  datapoints and pairwise similarities  $(w_{ij})$  and let  $T^*$  be the solution of Eq. (5.2). Let  $\mathcal{Z}$  be the set of spread embeddings and  $Z^* \in \mathcal{Z}$  be the solution of Eq. (5.3) for some  $\tau > 0$ . For any  $\varepsilon > 0$ , if  $\tau \leq \mathcal{O}(1/\log(1/\varepsilon))$ , then:

$$\frac{C_{\text{Dasgupta}}(\text{dec}(Z^*); w)}{C_{\text{Dasgupta}}(T^*; w)} \leq 1 + \varepsilon. \quad (5.13)$$

The proof is deferred to Appendix B. The insight is that when embeddings are sufficiently spread out, there is an equivalence between leaf embeddings and trees. In one direction, we show that any spread embedding  $Z$  has a continuous cost  $C_{\text{HYPER}}(Z; w, \tau)$  close to the discrete cost  $C_{\text{Dasgupta}}(T; w)$  of some underlying tree  $T$ . Conversely, we can leverage classical hyperbolic embedding results [152] to show that Dasgupta's optimum  $T^*$  can be embedded as a spread embedding in  $\mathcal{Z}$ .

**Optimization challenges** Note that smaller  $\tau$  values in Eq. (5.13) lead to better approximation guarantees but make the optimization more difficult. Indeed, hardness in the discrete problem arises from the difficulty in enumerating all possible solutions with discrete exhaustive search; on

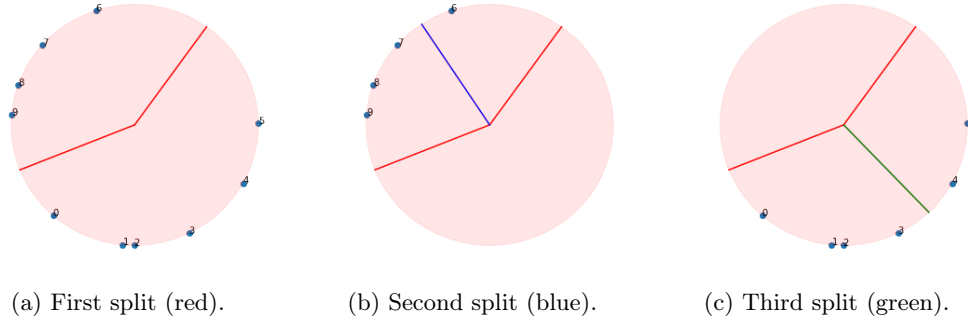


Figure 5.4: An illustration of the greedy decoding algorithm.

the other hand, in this continuous setting it is easier to specify the solution, which trades off for challenges arising from nonconvex optimization—i.e. how do we find the global minimizer of the constrained optimization problem in Eq. (5.3). In our experiments, we use a small constant for  $\tau$  following standard practice. To optimize over the set of spread embeddings, one could add triplet constraints on the embeddings to the objective function. We empirically find that even solutions of the *unconstrained* problem found with stochastic gradient descent have a good approximation ratio for the discrete objective, compared to baseline methods (Section 5.6.2).

## 5.5 HYPHC Practical Considerations

Here, we propose two empirical techniques to reduce the runtime complexity of HYPHC and improve its scalability, namely triplet sampling and a greedy decoding algorithm. Note that while our theoretical guarantees in Theorem 5.4.1 are valid under the assumptions that we use the exact decoding and compute the exact loss, we empirically validate that HYPHC, combined with these two techniques, still produces a clustering of good quality in terms of Dasgupta’s cost (Section 5.6.3).

**Greedy decoding** Algorithm 1 requires solving the closest pair problem, for which an almost-quadratic  $n^{2-o(1)}$  lower bound holds under plausible complexity-theoretic assumptions [9, 59]. Instead, we propose a greedy top-down decoding that runs significantly faster. We normalize leaves’ embeddings so that they lie on a hyperbolic diameter and sort them based on their angles in the Poincaré disk. We then recursively split the data into subsets using the largest angle split (Fig. 5.4). Intuitively, if points are normalized, the LCA distance to the origin is a monotonic function of the angle between points. Therefore, this top-down decoding acts as a proxy for the bottom-up decoding in Algorithm 1. In the worst case, this algorithm runs in quadratic time. However, if the repeated data splits are roughly evenly sized, it can be asymptotically faster, i.e.,  $O(n \log n)$ . In our experiments, we observe that this algorithm is significantly faster than the exact decoding.

To provide more intuition about the greedy decoding, we illustrate different steps of greedy

decoding on a small example in Fig. 5.4. The first split is computed using the two largest angles splits (red lines in Fig. 5.4a). Then, the algorithm recurses on the two created subsets and uses the largest angle in each subset to split the data (blue and green lines in Fig. 5.4b and Fig. 5.4c).

**Triplet sampling** Computing the loss term in Eq. (5.11) requires summing over all triplets which takes  $O(n^3)$  time. Instead, we generate all unique pairs of leaves and sample a third leaf uniformly at random from all the other leaves, which yields roughly  $n^2$  triplets. Note that an important benefit of HYPHC is that no matter how big the input is, HYPHC can always produce a clustering by sampling fewer triplets. We view this as an interesting opportunity to scale to large datasets and discuss the scalability-clustering quality tradeoff of HYPHC in Section 5.6.3. Finally, note that triplet sampling can be made parallel, unlike agglomerative algorithms, even if both take  $\mathcal{O}(n^2)$  time.

## 5.6 Experiments

We first describe our experimental protocol (Section 5.6.1) and evaluate the **clustering quality** of HYPHC in terms of discrete Dasgupta cost (Section 5.6.2). We then analyze the clustering **quality/speed tradeoffs** of HYPHC (Section 5.6.3). Finally, we demonstrate the **flexibility** of HYPHC using end-to-end training for a downstream classification task (Section 5.6.4).

### 5.6.1 Experimental Setup

**Datasets** We measure the clustering quality of HYPHC on six standard datasets from the UCI Machine Learning repository,<sup>3</sup> as well as CIFAR-100 [109], which exhibits a hierarchical structure (each image belongs to a fine-grained class that is itself part of a coarse superclass). Note that the setting studied in this work is similarity-based HC, where the input is only pairwise similarities, rather than features representing the datapoints. For all datasets, we use the cosine similarity to compute a complete input similarity graph.

In our experiments, we compute similarities using the cosine similarity measure on the datapoints' features. For all datasets in the UCI machine learning repository, we use the available features and normalize them so that each attribute has mean zero and standard deviation one. For CIFAR-100, where the raw data comes in the form of images, we used a pretrained BiT [105] convolutional neural network to compute 2048-dimensional image features (one to last layer).

**Baselines** We compare HYPHC to similarity-based HC methods, including competitive agglomerative clustering approaches such as single, average, complete and Ward Linkage (SL, AL, CL and WL respectively). We also compare to Bisecting K-Means (BKM) [127], which is a fast top-down algorithm that splits the data into two clusters at every iteration using local search.<sup>4</sup> Finally, we

<sup>3</sup><https://archive.ics.uci.edu/ml/datasets.php>

<sup>4</sup>BKM is the direct analog of Hierarchical K-Means in the context of similarity-based HC [127].

|            |             | Zoo                 | Iris                | Glass               | Segmentation        | Spambase             | Letter               | CIFAR-100            |
|------------|-------------|---------------------|---------------------|---------------------|---------------------|----------------------|----------------------|----------------------|
|            | # Points    | 101                 | 150                 | 214                 | 2310                | 4601                 | 20K                  | 50K                  |
|            | Cost        | DC.10 <sup>-5</sup> | DC.10 <sup>-5</sup> | DC.10 <sup>-6</sup> | DC.10 <sup>-9</sup> | DC.10 <sup>-10</sup> | DC.10 <sup>-12</sup> | DC.10 <sup>-13</sup> |
|            | Upper Bound | 3.887               | 14.12               | 3.959               | 4.839               | 3.495                | 3.031                | 4.408                |
|            | Lower Bound | 2.750               | 7.709               | 2.750               | 3.258               | 3.025                | 2.244                | 3.990                |
| Discrete   | SL          | 2.897               | 8.120               | 3.018               | 3.705               | 3.250                | 2.625                | 4.149                |
|            | AL          | 2.829               | 7.939               | <u>2.906</u>        | 3.408               | 3.159                | 2.437                | <u>4.056</u>         |
|            | CL          | <b>2.802</b>        | 7.950               | 2.939               | 3.460               | 3.184                | 2.481                | 4.078                |
|            | WL          | <u>2.827</u>        | 7.938               | 2.920               | 3.434               | 3.170                | 2.453                | 4.060                |
|            | BKM         | 2.861               | 8.223               | 2.948               | <u>3.375</u>        | <u>3.127</u>         | <b>2.383</b>         | <u>4.056</u>         |
| Continuous | UFit        | 2.896               | <u>7.916</u>        | 2.925               | 3.560               | 3.204                | -                    | -                    |
|            | HYPHC       | <b>2.802</b>        | <b>7.881</b>        | <b>2.902</b>        | <b>3.341</b>        | <b>3.126</b>         | <u>2.384</u>         | <b>4.053</b>         |

Table 5.1: Clustering quality results measured in discrete Dasgupta’s cost (DC). Best score in bold, second best underlined. Dashes indicate that the method could not scale to large datasets.

compare to the recent gradient-based Ultrametric Fitting (UFit) approach [52]. Note that we do not directly compare to gHHC [124] since this method requires input features. For completeness, we include a comparison in Table 5.2.

To compute numbers for agglomerative clustering methods, we used the corresponding implementation in the scipy Python library.<sup>5</sup> We implemented our own version of BKM following the description in [127] since no open-source version was available. For UFit, we used the open-source implementation and reused the same hyper-parameters as in the original paper [52].<sup>6</sup>

**Evaluation metrics** Our goal in this work is not to show an advantage on different heuristics, but rather to optimize a single well-defined search problem to the best of our abilities. Dasgupta’s cost is a well-studied objective with known guarantees when there is an underlying ground-truth hierarchy [56] (such results have not been established for metrics like dendrogram purity (DP) [88]). We therefore measure the clustering quality by computing the discrete Dasgupta Cost (DC). A lower DC means a better clustering quality. We also report upper and lower bounds for DC defined next. For classification experiments (Section 5.6.4) where the goal is to predict labels, we measure the classification accuracy.

For randomized algorithms and our method (which produce a different solution for every run), we report the best cost over five random seeds. This is standard since all methods can be viewed as *search* algorithms for the latent minimizer of the Dasgupta cost, analogous to how standard combinatorial search algorithms for NP-hard problems rely on random restarts and global perturbations when they reach local optima.

<sup>5</sup><https://docs.scipy.org/doc/scipy/reference/cluster.hierarchy.html>

<sup>6</sup><https://github.com/PerretB/ultrametric-fitting>

|            |            | Zoo         | Iris        | Glass       | Segmentation | Spambase    |
|------------|------------|-------------|-------------|-------------|--------------|-------------|
|            | # Points   | 101         | 150         | 214         | 2310         | 4601        |
|            | # Clusters | 7           | 3           | 6           | 7            | 2           |
| Discrete   | SL         | <u>97.7</u> | <u>76.7</u> | <u>50.3</u> | 51.1         | 61.2        |
|            | AL         | 90.1        | 73.7        | 46.3        | 58.2         | 73.4        |
|            | CL         | 96.6        | 76.1        | 46.9        | 55.1         | 69.1        |
|            | WL         | 90.0        | 74.9        | 48.3        | <u>61.3</u>  | 68.3        |
|            | BKM        | 86.5        | 66.1        | 43.5        | 57.2         | <u>74.9</u> |
| Continuous | UFit       | 97.2        | 76.8        | <b>51.0</b> | <b>61.2</b>  | 61.6        |
|            | gHHC       | -           | -           | 46.3        | -            | 61.4        |
|            | HYPHC      | <b>98.7</b> | <b>77.3</b> | 49.2        | 55.9         | <b>79.2</b> |

Table 5.2: Clustering quality results measured in dendrogram purity (DP). Best score in bold, second best underlined. gHHC scores are directly taken from [123].

**Cost bounds** We also report upper and lower bounds on the discrete Dasgupta cost in Table 5.1, which are computed as:

$$\begin{aligned}
 \text{UB}(w) &:= \sum_{ijk} \max(w_{ij} + w_{ik}, w_{ij} + w_{jk}, w_{ik} + w_{jk}) + 2 \sum_{ij} w_{ij} \geq C_{\text{Dasgupta}}(T; w) \\
 \text{LB}(w) &:= \sum_{ijk} \min(w_{ij} + w_{ik}, w_{ij} + w_{jk}, w_{ik} + w_{jk}) + 2 \sum_{ij} w_{ij} \leq C_{\text{Dasgupta}}(T; w),
 \end{aligned} \tag{5.14}$$

for all rooted binary tree  $T$ . For datasets with more than a thousand of nodes, we sample triplets uniformly at random for both the lower and upper bounds, and report the average over 10 random seeds.

Note that datasets where the relative gap between the upper and lower bound is larger indicate datasets where similarities induce a more hierarchical structure. As noted in [184], an instance for which the lower bound can be achieved by a tree, is a “perfect” HC instance (termed “perfect HC-structure” in [184]), in the sense that for every three points, the tree decomposes them in the most preferred way, i.e., by cutting the highest similarity weight last, towards the bottom of the tree. As the optimum tree gets higher costs, approaching the upper bound above, the instance loses its hierarchical structure; for example, if the given graph is a unit weight clique with no hierarchy to be found, both upper and lower bounds coincide and actually this implies that any tree gets the same cost, as was shown in Dasgupta [62].

In Table 5.1, we note that on dataset where there is a large relative gap between the upper and lower bounds (e.g. Iris or Segmentation), the relative improvement of HYPHC compared to the best baseline is more important compared to datasets with a smaller gap (e.g. CIFAR-100). This suggest that when there is a good underlying HC in the data, HYPHC is able to get closer to it than heuristic algorithms.

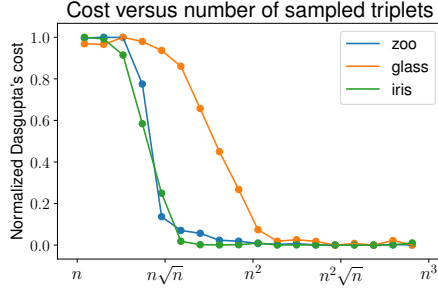


Figure 5.5: Normalized cost vs number of triplets sampled.

**Implementation** We implemented HYPHC in PyTorch and make our implementation publicly available.<sup>7</sup> We conducted our experiments on a single NVIDIA Tesla P100 GPU. We train HYPHC for 50 epochs (of the sampled triples) and optimize embeddings with Riemannian Adam [19] using the open-source Riemannian optimization software geopt [104]. We set the embedding dimension to two in all experiments, and normalize embeddings during optimization as described in the greedy decoding strategy (Section 5.5). We perform a hyper-parameter search over learning rate values  $[1e^{-3}, 5e^{-4}, 1e^{-4}]$  and temperature values  $[1e^{-1}, 5e^{-2}, 1e^{-2}]$ .

### 5.6.2 Clustering Quality

We report the performance of HYPHC—fast version with greedy decoding and triplet sampling—in Table 5.1, and compare to baseline methods. On all datasets, HYPHC outperforms or matches the performance of the best discrete method, and significantly improves over UFit, the only similarity-based continuous method. This confirms our intuition that directly optimizing a continuous relaxation of Dasgupta’s objective can improve clustering quality, compared to baselines that are optimized independently of the objective.

We also include a comparison to gHHC (which uses features) for completeness in Table 5.2. gHHC evaluates the clustering quality using the dendrogram purity (DP) measure [88]. Given ground truth flat clusters, DP measures how well the clusters are preserved in the hierarchy. Note that this metric will only be a good indicator for clustering quality when the ground truth flat clusters correlate with the hierarchy. A proxy to measure such a correlation is analyzing if methods that do well on DC also do well on DP. On Spambase, HYPHC and BKM have the best DC and DP scores, while SL does poorly for both metrics. This suggests that ground truth flat clusters do correlate well with the optimal hierarchy on this dataset, and we note that HYPHC significantly outperforms gHHC for the DP metric. On Glass, the correlation is not so obvious; for instance SL performs poorly on DC but does very well on DP. We conjecture that the ground truth flat clusters in glass are not directly correlated with the optimal clustering on this dataset.

<sup>7</sup><https://github.com/HazyResearch/HypHC>

|           |         | <b>Zoo</b>                      | <b>Iris</b>                     | <b>Glass</b>                    | <b>Segmentation</b>             |
|-----------|---------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|
| Cost      | Exact   | <b>2.8016</b> .10 <sup>-5</sup> | <b>7.8783</b> .10 <sup>-5</sup> | <b>2.9011</b> .10 <sup>-6</sup> | <b>3.3417</b> .10 <sup>-9</sup> |
|           | Greedy  | 2.8021.10 <sup>-5</sup>         | 7.8807.10 <sup>-5</sup>         | 2.9019.10 <sup>-6</sup>         | 3.3419.10 <sup>-9</sup>         |
| Time (ms) | Exact   | 78                              | 323                             | 400                             | 21717                           |
|           | Greedy  | 3.1                             | 5.2                             | 6.2                             | 303                             |
|           | Speedup | 25x                             | 62x                             | 64x                             | 72x                             |

Table 5.3: Speed/quality analysis for the exact and greedy decoding.

Finally, we visualize embeddings during different iterations of HYPHC on the zoo dataset in Fig. 5.2. Colors indicate ground truth flat clusters for the datapoints and we observe that these are better separated in the dendrogram as optimization progresses. We note that embeddings are pushed towards the boundary of the disk, where distances are more “tree-like” and produce a better HC. This illustrates our intuition that the optimal embedding will be close to a tree metric embedding.

### 5.6.3 Analysis

In our experiments, we used the greedy decoding Algorithm and triplet sampling with  $\mathcal{O}(n^2)$  triplets (Section 5.5). Since our theoretical guarantees apply to the full triplet loss and exact decoding, we are interested in understanding how much quality is lost by using these two empirical techniques.

**Decoding** We report HC costs obtained using the exact (Section 5.4.3) and the greedy (Section 5.5) decoding in Table 5.3, as well as the corresponding runtime in milliseconds (ms), averaged over 10 runs. We observe that the greedy decoding is approximately 60 times faster than the exact decoding, while still achieving almost the same cost. This confirms that, when embeddings are normalized in two dimensions, using angles as a proxy for the LCA’s distances to the origin is a valid approach.

**Triplet sampling** We plot the discrete cost (normalized to be in  $[0, 1]$ ) of HYPHC for different number of sampled triplets in Fig. 5.5. As expected, we note that increasing the number of sampled triplets reduces DC, and therefore improves the clustering quality. This tradeoff forms one of the key benefits of using gradient-based methods: HYPHC can always produce a hierarchical clustering, no matter how large the input is, at the cost of reducing clustering quality or increasing runtime. In our experiments, we find that  $\mathcal{O}(n^2)$  is sufficient to achieve good clustering quality, which is the most expensive step of HYPHC in terms of runtime complexity. Future work could explore better triplet sampling strategies, to potentially use less triplets while still achieving a good clustering quality.

|                  | Zoo                              | Iris                             | Glass                            | Segmentation                     |
|------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
| # Classes        | 7                                | 3                                | 6                                | 7                                |
| LP               | 41.4                             | 76.7                             | 46.8                             | 65.3                             |
| HYPHC-Two-Step   | $84.8 \pm 3.5$                   | $84.4 \pm 1.7$                   | $50.6 \pm 2.6$                   | $64.1 \pm 0.9$                   |
| HYPHC-End-to-End | <b><math>87.9 \pm 3.8</math></b> | <b><math>85.6 \pm 0.8</math></b> | <b><math>54.4 \pm 2.9</math></b> | <b><math>67.7 \pm 3.4</math></b> |

Table 5.4: Classification accuracy for different training strategies.

#### 5.6.4 End-to-End Training

Here, we demonstrate the flexibility of our approach and the benefits of joint training on a downstream similarity-based classification task. Since HYPHC is optimized with gradient-descent, it can be used in conjunction with any standard ML pipeline, such as downstream classification.

We consider four of the HC datasets that come with categorical labels for leaf nodes, split into training, testing and validation sets (30/60/10% splits). We follow a standard graph-based semi-supervised learning setting, where all the nodes (training/validation/testing) are available at training time, but only training labels can be used to train the models. We use the embeddings learned by HYPHC as input features for a hyperbolic logistic regression model [74]. Note that none of the other discrete HC methods apply here, since these do not produce representations.

In Table 5.4, we compare jointly optimizing the HYPHC and the classification loss (End-to-End) versus a two-step embed-then-classify approach which trains the classification module using frozen HYPHC embeddings (Two-Step) (average scores and standard deviation computed over 5 runs). We also compare to Label Propagation (LP), which is a simple graph-based semi-supervised algorithm that does not perform any clustering step. We find that LP is outperformed by both the end-to-end and the two-step approaches on most datasets, suggesting that clustering learns meaningful partitions of the input similarity graph. Further, we observe that end-to-end training improves classification accuracy by up to 3.8%, confirming the benefits of a differentiable method for HC.

### 5.7 Conclusion

We introduced HYPHC, a differentiable approach to learn HC with gradient-descent in hyperbolic space. HYPHC uses a novel technical approach to optimize over discrete trees, by showing an equivalence between trees and constrained hyperbolic embeddings. Theoretically, HYPHC has a  $(1+\varepsilon)$ -factor approximation to the minimizer of Dasgupta’s cost, and empirically, HYPHC outperforms existing HC algorithms.

Our theoretical analysis assumes a perfect optimization and interesting future directions include a better characterization of the hardness arising from optimization challenges, as well as providing an approximation for the continuous optimum. While no constant factor approximation of the continuous optimum is possible, achieving better (e.g. polylogarithmic) approximations, is an interesting future

direction for this work. Finally, we note that our continuous optimization framework can be extended beyond the scope of HC, to applications that require searching of discrete tree structures, such as constituency parsing in natural language processing.

## Chapter 6

# Dimensionality Reduction

### 6.1 Introduction

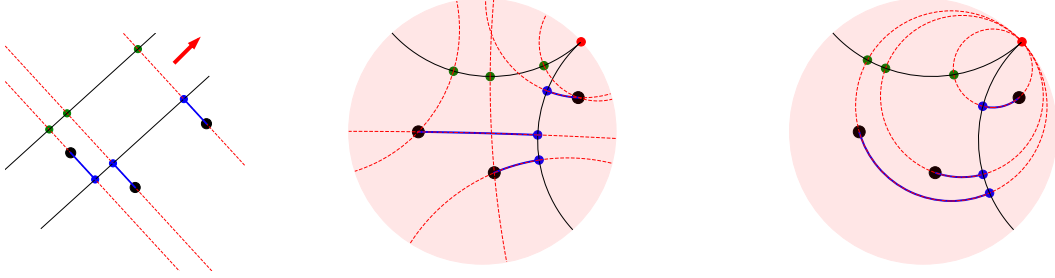
This chapter will look at generalizing another important algorithm to hyperbolic spaces, namely Euclidean Principal Component Analysis (PCA) for the reduction and visualization of hyperbolic data. PCA is a fundamental dimensionality reduction technique which seeks directions that best explain the original data. PCA is an important primitive in data analysis and has many important uses such as (i) dimensionality reduction (e.g. for memory efficiency), (ii) data whitening and pre-processing for downstream tasks, and (iii) data visualization.

Here, we seek a generalization of PCA to hyperbolic geometry. Given a core notion of **directions**, PCA involves the following ingredients:

1. A nested sequence of affine **subspaces** (flags) spanned by a set of directions.
2. A **projection** method which maps points to these subspaces while preserving information (e.g. dot-product) along each direction.
3. A **variance** objective to help choose the best directions.

The PCA algorithm is then defined as combining these primitives: given a dataset, it chooses directions that maximize the *variance of projections* onto a *subspace* spanned by those directions, so that the resulting sequence of directions optimally explains the data. Crucially, the algorithm only depends on the directions of the *affine* subspaces and not their locations in space (Fig. 6.1a). Thus, in practice we can assume that they all go through the origin (and hence become *linear* subspaces), which greatly simplifies computations.

Generalizing PCA to manifolds is a challenging problem that has been studied for decades, starting with Principal Geodesic Analysis (PGA) [70] which parameterizes subspaces using tangent vectors at the mean of the data, and maximizes distances from projections to the mean to find optimal directions. More recently, the Barycentric Subspace Analysis (BSA) method [141] was introduced. It finds a more general parameterization of nested sequences of submanifolds by minimizing the



(a) Euclidean projections. (b) Hyperbolic geodesic projections. (c) Hyperbolic horospherical projections.

Figure 6.1: Given datapoints (black dots), Euclidean and horospherical projections preserve distance information across different subspaces (black lines) pointing towards the same direction or point at infinity, while geodesic projections do not. (a): Distances between points, and therefore the explained variance, are invariant to translations along orthogonal directions (red lines). (b): Geodesic projections do not preserve this property: distances between green projections are not the same as distances between blue projections. (c): Horospherical projections project data by sliding it along the complementary direction defined by horospheres (red circles) and there exist an isometric mapping between the blue and green projections.

unexplained variance. However, both PGA and BSA map points onto submanifolds using closest-point or *geodesic projections*, which do not attempt to preserve information along principal directions; for example, they cannot isometrically preserve hyperbolic distances between *any* points and shrink all path lengths by an exponential factor (Section 6.4.2.2).

Fundamentally, all previous methods only look for *subspaces* rather than directions that explain the data, and can perhaps be better understood as principal *subspace* analysis rather than principal *component* analysis. Like with PCA, they assume that all optimal subspaces go through a chosen *base point*, but unlike in the Euclidean setting, this assumption is now unjustified: “translating” the submanifolds does not preserve distances between projections (Fig. 6.1b). Furthermore, the dependence on the base point is sensitive: as noted above, the shrink factor of the projection depends exponentially on the distances between the subspaces and the data. Thus, having to choose a base point increases the number of necessary parameters and reduces stability.

Here, we propose HOROPCA, a dimensionality reduction method for data defined in hyperbolic spaces which better preserves the properties of Euclidean PCA. We show how to interpret **directions** using *points at infinity* (or *ideal points*), which then allows us to generalize core properties of PCA:

1. To generalize the notion of affine **subspace**, we propose parameterizing geodesic subspaces as the sets spanned by these ideal points. This yields multiple viable nested subspaces (flags) (Section 6.4.1).
2. To maximally preserve information in the original data, we propose a new **projection** method that uses horospheres, a generalization of complementary directions for hyperbolic space. In contrast with geodesic projections, these projections exactly preserve information – specifically,

distances to ideal points – along each direction. Consequently, they preserve distances between points much better than geodesic projections (Section 6.4.2).

3. Finally, we introduce a simple generalization of explained **variance** that is a function of distances only and can be computed in hyperbolic space (Section 6.4.3).

Combining these notions, we propose an algorithm that seeks a sequence of *principal components* that best explain variations in hyperbolic data. We show that this formulation retains the *location-independence* property of PCA: translating target submanifolds along orthogonal directions (horospheres) preserves projected distances (Fig. 6.1c). In particular, the algorithm’s objective depends only on the directions and not locations of the submanifolds (Section 6.5).

We empirically validate HOROPCA on real datasets and for three standard PCA applications. First, (i) we show that it yields much lower distortion and higher explained variance than existing methods, reducing average distortion by up to 77%. Second, (ii) we validate that it can be used for data pre-processing, improving downstream classification by up to 3.8% in Average Precision score compared to methods that don’t use whitening. Finally, (iii) we show that the low-dimensional representations learned by HOROPCA can be visualized to qualitatively interpret hyperbolic data.

## 6.2 Related Work

**PCA methods in Riemannian manifolds** We first review some approaches to extending PCA to general Riemannian geometries, of which hyperbolic geometry is a special case. For a more detailed discussion, see Pennec [141]. The simplest such approach is tangent PCA (tPCA), which maps the data to the tangent space at the Fréchet mean  $\mu$  using the logarithm map, then applies Euclidean PCA. A similar approach, Principal Geodesic Analysis (PGA) [70], seeks geodesic subspaces at  $\mu$  that minimize the sum of squared Riemannian distances to the data. Compared to tPCA, PGA searches through the same subspaces but uses a more natural loss function.

Both PGA and tPCA project on submanifolds that go through the Fréchet mean. When the data is not well-centered, this may be sub-optimal, and Geodesic PCA (GPCA) was proposed to alleviate this issue [91, 92]. GPCA first finds a geodesic  $\gamma$  that best fits the data, then finds other orthogonal geodesics that go through *some* common point  $b$  on  $\gamma$ . In other words, GPCA removes the constraint of PGA that  $b$  is the Fréchet mean. Extensions of GPCA have been proposed such as probabilistic methods [198] and Horizontal Component Analysis [158].

Pennec [141] proposes a more symmetric approach. Instead of using the exponential map at a base point, it parameterizes  $K$ -dimensional subspaces as the *barycenter* loci of  $K + 1$  points. Nested sequences of subspaces (flags) can be formed by simply adding more points. In hyperbolic geometry, this construction coincides with the one based on geodesic hulls that we use in Section 6.4, except that it applies to points inside  $\mathbb{H}^d$  instead of ideal points and thus needs more parameters to parameterize a flag (see Remark C.1.1).

By considering a more general type of submanifolds, Hauberg [87] gives another way to avoid the sensitive dependence on Fréchet mean. However, its bigger search space also makes the method computationally expensive, especially when the target dimension  $K$  is bigger than 1.

In contrast with all methods so far, HYPHC relies on horospherical projections instead of geodesic projections. This yields a generalization of PCA that depends only on the directions and not specific locations of the subspaces.

**Dimension reduction in hyperbolic geometry** We now review some dimension reduction methods proposed specifically for hyperbolic geometry. Cvetkovski and Crovella [60] and Sala et al. [150] are examples of hyperbolic multidimensional scaling methods, which seek configurations of points in lower-dimensional hyperbolic spaces whose pairwise distances best approximate a given dissimilarity matrix. Unlike HYPHC, they do not learn a projection that can be applied to unseen data.

Tran and Vu [173] constructs a map to lower-dimensional hyperbolic spaces whose preimages of compact sets are compact. Unlike most methods, it is data-agnostic and does not optimize any objective. Benjamini and Makarychev [24] adapt the Euclidean Johnson–Lindenstrauss transform to hyperbolic geometry and obtain a distortion bound when the dataset size is not too big compared to the target dimension. They do not seek an analog of Euclidean directions or projections, but nevertheless implicitly use a projection method based on pushing points along *horocycles*, which shares many properties with our *horospherical* projections. In fact, the latter converges to the former as the ideal points get closer to each other.

**From hyperbolic to Euclidean** Liu et al. [118] use “distances to centroids” to compute Euclidean representations of hyperbolic data. The Busemann functions we use bear resemblances to these centroid-based functions but are better analogs of coordinates along given directions, which is a central concept in PCA, and have better regularity properties [32]. Recent works have also used Busemann functions for hyperbolic prototype learning [97, 185]. These works do not define projections to lower-dimensional hyperbolic spaces. In contrast, HYPHC naturally returns both hyperbolic representations (via horospherical projections) and Euclidean representations (via Busemann coordinates). This allows leveraging techniques in both settings.

### 6.3 Background

In this work, we use the  $d$ -dimensional Poincaré model of hyperbolic space for easier visualizations:  $\mathbb{H}^d = \{x \in \mathbb{R}^d : \|x\| < 1\}$ , where  $\|\cdot\|$  is the Euclidean norm. We refer to Section 1.3.1 in Chapter 1 for a review of different concepts in this model, such as distances, geodesics and geodesic submanifolds.

Next, we discuss the generalization of coordinates and directions in hyperbolic space and then review geodesic projections. We finally describe generalizations of the notion of mean and variance

|            | Euclidean               | Hyperbolic              |
|------------|-------------------------|-------------------------|
| Component  | Unit vector $w$         | Ideal point $p$         |
| Coordinate | Dot product $x \cdot w$ | Busemann func. $B_p(x)$ |

Table 6.1: Analogies of components and their corresponding coordinates, in both Euclidean and hyperbolic space.

to non-Euclidean spaces.

### 6.3.1 Directions in Hyperbolic space

The notions of directions, and coordinates in a given direction can be generalized to hyperbolic spaces as follows.

**Ideal points** As with parallel rays in Euclidean spaces, geodesic rays in  $\mathbb{H}^d$  that stay close to each other can be viewed as sharing a common *endpoint at infinity*, also called an *ideal point*. Intuitively, ideal points represent *directions* along which points in  $\mathbb{H}^d$  can move toward infinity. The set of ideal points  $\mathbb{S}_\infty^{d-1}$ , called the *boundary at infinity* of  $\mathbb{H}^d$ , is represented by the unit sphere  $\mathbb{S}_\infty^{d-1} = \{\|x\| = 1\}$  in the Poincaré model. We abuse notations and say that a geodesic submanifold  $M \subset \mathbb{H}^d$  contains an ideal point  $p$  if the boundary of  $M$  in  $\mathbb{S}_\infty^{d-1}$  contains  $p$ .

**Busemann coordinates** In Euclidean spaces, each *direction* can be represented by a unit vector  $w$ . The *coordinate* of a point  $x$  in the direction of  $w$  is simply the dot product  $w \cdot x$ . In hyperbolic geometry, directions can be represented by ideal points but dot products are not well-defined. Instead, we take a ray-based perspective: note that in Euclidean spaces, if we shoot a ray in the direction of  $w$  from the origin, the coordinate  $w \cdot x$  can be viewed as the *normalized distance to infinity in the direction of that ray*. In other words, as a point  $y = tw$ , ( $t > 0$ ) moves toward infinity in the direction of  $w$ :

$$w \cdot x = \lim_{t \rightarrow \infty} (d(0, tw) - d(x, tw)).$$

This approach generalizes to other geometries: given a unit-speed geodesic ray  $\gamma(t)$ , the *Busemann function*  $B_\gamma(x)$  of  $\gamma$  is defined as:<sup>1</sup>

$$B_\gamma(x) = \lim_{t \rightarrow \infty} (d(x, \gamma(t)) - t).$$

Up to an additive constant, this function only depends on the endpoint at infinity of the geodesic ray, and not the starting point  $\gamma(0)$ . Thus, given an ideal point  $p$ , we define the Busemann function  $B_p(x)$  of  $p$  to be the Busemann function of the geodesic ray that starts from the origin of the unit

<sup>1</sup>Note that compared to the above formula, the sign convention is flipped due to historical reasons.

ball model and has endpoint  $p$ . Intuitively, it represents the *coordinates* of  $x$  in the direction of  $p$ . In the Poincaré model, there is a closed formula:

$$B_p(x) = \ln \frac{\|p - x\|^2}{1 - \|x\|^2}.$$

**Horospheres** The level sets of Busemann functions  $B_p(x)$  are called *horospheres centered at  $p$* . In this sense, they resemble spheres, which are level sets of distance functions. However, intrinsically as Riemannian manifolds, horospheres have curvature zero and thus also exhibiting many properties of planes in Euclidean spaces.

Every geodesic with endpoint  $p$  is orthogonal to every horosphere centered at  $p$ . Given two horospheres with the same center, every orthogonal geodesic segment connecting them has the same length. In this sense, *concentric horospheres resemble parallel planes in Euclidean spaces*. In the Poincaré model, horospheres are Euclidean spheres that touch the boundary sphere  $\mathbb{S}_\infty^{d-1}$  at their ideal centers (Fig. 6.2). Given an ideal point  $p$  and a point  $x$  in  $\mathbb{H}^d$ , there is a unique horosphere  $S(p, x)$  passing through  $x$  and centered at  $p$ .

### 6.3.2 Geodesic Projections

PCA uses orthogonal projections to project data onto subspaces. Orthogonal projections are usually generalized to other geometries as *closest-point projections*. Given a target submanifold  $M$ , each point  $x$  in the ambient space is mapped to the closest-point to it in  $M$ :

$$\pi_M^G(x) = \operatorname{argmin}_{y \in M} d_M(x, y).$$

One could view  $\pi_M^G(\cdot)$  as the map that pushes each point  $x$  along an orthogonal geodesic until it hits  $M$ . For this reason, it is also called *geodesic projection*. In the Poincaré model, these can be computed in closed-form (see Section 6.6.1.3).

### 6.3.3 Manifold Statistics

PCA relies on data statistics which do not generalize easily to hyperbolic geometry. One approach to generalize the arithmetic mean is to notice that it is the minimizer of the sum of squared distances to the inputs. Motivated by this, the Fréchet mean [72] of a set of points  $S$  in a Riemannian manifold  $(M, d_M)$  is defined as:

$$\mu_M(S) := \operatorname{argmin}_{y \in M} \sum_{x \in S} d_M(x, y)^2.$$

This definition only depends on the intrinsic distance of the manifold. For hyperbolic spaces, since squared distance functions are convex,  $\mu(S)$  always exists and is unique.<sup>2</sup> Analogously, the Fréchet

<sup>2</sup>For more general geometries, existence and uniqueness hold if the data is well-localized [98].

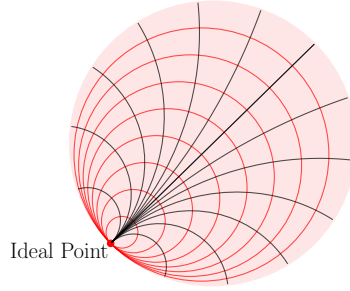


Figure 6.2: Hyperbolic geodesics (black lines) going through an ideal point (in red), and horospheres (red circles) centered at that same ideal point. The hyperbolic lengths of geodesic segments between two horospheres are equal.

variance is defined as:

$$\sigma_M^2(S) := \frac{1}{|S|} \sum_{x \in S}^N d_M(x, \mu(S))^2. \quad (6.1)$$

We refer to [90] for a discussion on different intrinsic statistics in non-Euclidean spaces, and a study of their asymptotic properties.

## 6.4 Generalizing PCA to the Hyperbolic Space

We now describe our approach to generalize PCA to hyperbolic spaces. The starting point of HOROPCA is to pick  $1 \leq K \leq d$  ideal points  $p_1, \dots, p_K \in \mathbb{S}_{\infty}^{d-1}$  to represent  $K$  directions in hyperbolic spaces (Section 6.3.1). Then, we generalize the core concepts of Euclidean PCA. In Section 6.4.1, we show how to generalize *flags*. In Section 6.4.2, we show how to *project* points onto the lower-dimensional submanifold spanned by a given set of directions, while preserving information along each direction. In Section 6.4.3, we introducing a *variance objective* to optimize and show that it is a function of the directions only.

### 6.4.1 Hyperbolic Flags

In Euclidean spaces, one can take the linear spans of more and more components to define a nested sequence of linear subspaces, called a flag. To generalize this to hyperbolic spaces, we first need to adapt the notion of linear/affine spans. Recall that geodesic submanifolds are generalizations of affine subspaces in Euclidean spaces.

**Definition 6.4.1.** *Given a set of points  $S$  (that could be inside  $\mathbb{H}^d$  or on the boundary sphere  $\mathbb{S}_{\infty}^{d-1}$ ), the smallest geodesic submanifold of  $\mathbb{H}^d$  that contains  $S$  is called the geodesic hull of  $S$  and denoted by  $\text{GH}(S)$ .*

Thus, given  $K$  ideal points  $p_1, p_2, \dots, p_K$  and a base point  $b \in \mathbb{H}^d$ , we can define a nested sequence of geodesic submanifolds  $\text{GH}(b, p_1) \subset \text{GH}(b, p_1, p_2) \subset \dots \subset \text{GH}(b, p_1, \dots, p_K)$ . This will be our notion of flags.

**Remark 6.4.1.** The base point  $b$  is only needed here for technical reasons, just like an origin  $\mathbf{o}$  is needed to define linear spans in Euclidean spaces. We will see next that it does not affect the projection results or objectives (Theorem 6.5.1).

**Remark 6.4.2.** We assume that none of  $b, p_1, \dots, p_K$  are in the geodesic hull of the other  $K$  points. This is analogous to being linearly independent in Euclidean spaces.

## 6.4.2 Projections via Horospheres

In Euclidean PCA, points are projected to the subspaces spanned by the given directions in a way that preserves coordinates in those directions. We seek a projection method in hyperbolic spaces with a similar property.

Recall that coordinates are generalized by Busemann functions (Table 6.1), and that horospheres are level sets of Busemann functions. Thus, we propose a projection that preserves coordinates by moving points along horospheres. It turns out that this projection method also preserves distances better than the traditional geodesic projection.

As a toy example, we first show how the projection is defined in the  $K = 1$  case (i.e. projecting onto a geodesic) and why it tends to preserve distances well. We will then show how to use  $K \geq 1$  ideal points simultaneously.

### 6.4.2.1 Projecting onto $K = 1$ direction

For  $K = 1$ , we have one ideal point  $p$  and base point  $b$ , and the geodesic hull  $\text{GH}(b, p)$  is just a geodesic  $\gamma$ . Our goal is to map every  $x \in \mathbb{H}^d$  to a point  $\pi_{b,p}^H(x)$  on  $\gamma$  that has the same Busemann coordinate in the direction of  $p$ :

$$B_p(x) = B_p(\pi_{b,p}^H(x)).$$

Since level sets of  $B_p(x)$  are horospheres centered at  $p$ , the above equation simply says that  $\pi_{b,p}^H(x)$  belongs to the horosphere  $S(p, x)$  centered at  $p$  and passing through  $x$ . Thus, we define:

$$\pi_{b,p}^H(x) := \gamma \cap S(p, x). \quad (6.2)$$

Another important property that  $\pi_{b,p}^H(\cdot)$  shares with orthogonal projections in Euclidean spaces is that it preserves distances along a direction – lengths of geodesic segments that point to  $p$  are preserved after projection (Fig. 6.3):

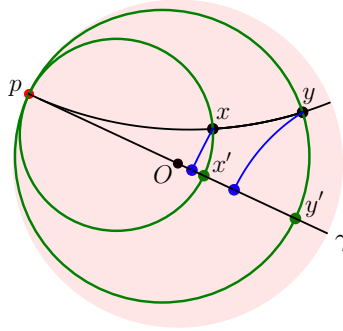


Figure 6.3:  $x', y'$  are horospherical (green) projections of  $x, y$ . Proposition 6.4.1 shows  $d_{\mathbb{H}}(x', y') = d_{\mathbb{H}}(x, y)$ . The distance between the two geodesic (blue) projections is smaller.

**Proposition 6.4.1.** *For any  $x \in \mathbb{H}^d$ , if  $y \in \text{GH}(x, p)$  then:*

$$d_{\mathbb{H}}(\pi_{b,p}^{\mathbb{H}}(x), \pi_{b,p}^{\mathbb{H}}(y)) = d_{\mathbb{H}}(x, y).$$

*Proof.* This follows from the remark in Section 6.3.1 about horospheres: every geodesic going through  $p$  is orthogonal to every horosphere centered at  $p$ , and every orthogonal geodesic segment connecting concentric horospheres has the same length (Fig. 6.2). In this case, the segments from  $x$  to  $y$  and from  $\pi_{b,p}^{\mathbb{H}}(x)$  to  $\pi_{b,p}^{\mathbb{H}}(y)$  are two such segments, connecting  $S(p, x)$  and  $S(p, y)$ .  $\square$

#### 6.4.2.2 Projecting onto $K > 1$ directions

We now generalize the above construction to projections onto higher-dimensional submanifolds. We describe the main ideas here and refer to Appendix C.1 for more details.

Fix a base point  $b \in \mathbb{H}^d$  and  $K > 1$  ideal points  $\{p_1, \dots, p_K\}$ . We want to define a map from  $\mathbb{H}^d$  to  $M := \text{GH}(b, p_1, \dots, p_K)$  that preserves the Busemann coordinates in the directions of  $p_1, \dots, p_K$ , i.e.:

$$B_{p_j}(x) = B_{p_j}(\pi_{b,p_1,\dots,p_K}^{\mathbb{H}}(x)) \text{ for every } j = 1, \dots, K.$$

As before, the idea is to take the intersection with the horospheres centered at  $p_j$ 's and passing through  $x$ :

$$\begin{aligned} \pi_{b,p_1,\dots,p_K}^{\mathbb{H}} : \mathbb{H}^d &\rightarrow M \\ x &\mapsto M \cap S(p_1, x) \cap \dots \cap S(p_K, x). \end{aligned}$$

It turns out that this intersection generally consists of two points instead of one. When that happens, one of them will be strictly closer to the base point  $b$ , and we define  $\pi_{b,p_1,\dots,p_K}^{\mathbb{H}}(x)$  to be that point.

As with Proposition 6.4.1,  $\pi_{b,p_1,\dots,p_K}^H(\cdot)$  preserves distances along  $K$ -dimensional manifolds (Corollary C.1.8). In contrast, geodesic projections in hyperbolic spaces *never* preserve distances (except between points already in the target):

Let  $M \subset \mathbb{H}^d$  be a geodesic submanifold. Then every geodesic segment of distance at least  $r$  from  $M$  gets at least  $\cosh(r)$  times shorter under the geodesic projection  $\pi_M^G(\cdot)$  to  $M$ :

$$\text{length}(\pi_M^G(I)) \leq \frac{1}{\cosh(r)} \text{length}(I).$$

In particular, the shrink factor grows exponentially as the segment  $I$  moves away from  $M$ .  $\square$  The proof is in Appendix C.2.

**Computation** Interestingly, horosphere projections can be computed without actually computing the horospheres. The key idea is that if we let  $P = \text{GH}(p_1, \dots, p_K)$  be the geodesic hull of the horospheres' centers, then the intersection  $S(p_1, x) \cap \dots \cap S(p_K, x)$  is simply the orbit of  $x$  under the rotations around  $P$ . (This is true for the same reason that spheres whose centers lie on the same axis must intersect along a circle around that axis). Thus,  $\pi_{b,p_1,\dots,p_K}^H(\cdot)$  can be viewed as the map that rotates  $x$  around until it hits  $M$ . It follows that it can be computed by:

1. Find the geodesic projection  $c = \pi_P^G(x)$  of  $x$  onto  $P$ .
2. Find the geodesic  $\alpha$  on  $M$  that is orthogonal to  $P$  at  $c$ .
3. Among the two points on  $\alpha$  whose distance to  $c$  equals  $d_{\mathbb{H}}(x, c)$ , returns the one closer to  $b$ .

The detailed computations and proof that this recovers horospherical projections are provided in Appendix C.1.

### 6.4.3 Intrinsic Variance Objective

In Euclidean PCA, directions are chosen to maximally preserve information from the original data. In particular, PCA chooses directions that maximize the Euclidean variance of projected data. To generalize this to hyperbolic geometry, we define an analog of variance that is intrinsic, i.e. dependent only on the distances between data points. As we will see in Section 6.5, having an intrinsic objective helps make our algorithm *location (or base point) independent*.

The usual notion of Euclidean variance is the squared sum of distances to the mean of the projected datapoints. Generalizing this is challenging because non-Euclidean spaces do not have a canonical choice of mean. Previous works have generalized variance either by using the *unexplained variance* or *Fréchet variance*. The former is the squared sum of residual distances to the projections, and thus avoids computing a mean. However, it is not intrinsic. The latter is intrinsic [70] but involves finding the Fréchet mean, which is not necessarily a canonical notion of mean and can only be computed by gradient descent.

Our approach uses the observation that in Euclidean space:

$$\sigma^2(S) = \frac{1}{n} \sum_{x \in S} \|x - \mu(S)\|^2 = \frac{1}{n^2} \sum_{x, y \in S} \|x - y\|^2.$$

Thus, we propose the following generalization of variance:

$$\sigma_{\mathbb{H}}^2(S) = \frac{1}{n^2} \sum_{x, y \in S} d_{\mathbb{H}}(x, y)^2. \quad (6.3)$$

This function agrees with the usual variance in Euclidean space, while being a function of distances only. Thus it is well defined in non-Euclidean space, is easily computed, and, as we will show next, has the desired invariance due to isometry properties of horospherical projections.

## 6.5 HOROPCA

Section 6.4 formulated several simple primitives – including directions, flags, projections, and variance – in a way that is generalizable to hyperbolic geometry. We now revisit standard PCA, showing how it has a simple definition that combines these primitives using optimization. This directly leads to the full HOROPCA algorithm by simply using the hyperbolic analogs of these primitives.

**Euclidean PCA** Given a dataset  $S$  and a target dimension  $K$ , Euclidean PCA greedily finds a sequence of *principal components*  $p_1, \dots, p_K$  that maximizes the *variance* of orthogonal *projections*  $\pi_{\mathbf{o}, p_1, \dots, p_k}^E(\cdot)$  onto the linear<sup>3</sup> *subspaces* spanned by these components:

$$p_1 = \operatorname{argmax}_{\|p\|=1} \sigma^2(\pi_{\mathbf{o}, p}^E(S))$$

$$\text{and } p_{k+1} = \operatorname{argmax}_{\|p\|=1} \sigma^2(\pi_{\mathbf{o}, p_1, \dots, p_k, p}^E(S)).$$

Thus, for each  $1 \leq k \leq K$ ,  $\{p_1, \dots, p_k\}$  is the optimal set of directions of dimension  $k$ .

**HOROPCA** Because we have generalized the notions of flag, projection, and variance to hyperbolic geometry, the HOROPCA algorithm can be defined in the same fashion. Given a dataset  $S$  in  $\mathbb{H}^d$  and a base point  $b \in \mathbb{H}^d$ , we seek a sequence of  $K$  directions that maximizes the variance of

---

<sup>3</sup>Here  $\mathbf{o}$  denotes the origin, and  $\pi_{\mathbf{o}, p_1, \dots, p_k}^E(\cdot)$  denotes the projection onto the *affine* span of  $\{\mathbf{o}, p_1, \dots, p_k\}$ , which is equivalent to the *linear* span of  $\{p_1, \dots, p_k\}$ .

horosphere-projected data:

$$\begin{aligned} p_1 &= \operatorname{argmax}_{p \in \mathbb{S}_\infty^{d-1}} \sigma_{\mathbb{H}}^2(\pi_{b,p}^{\mathbb{H}}(S)) \\ \text{and } p_{k+1} &= \operatorname{argmax}_{p \in \mathbb{S}_\infty^{d-1}} \sigma_{\mathbb{H}}^2(\pi_{b,p_1,\dots,p_k,p}^{\mathbb{H}}(S)). \end{aligned} \tag{6.4}$$

**Base point independence** Finally, we show that algorithm (6.4) always returns the same results regardless of the choice of a base point  $b \in \mathbb{H}^d$ . Since our variance objective only depends on the distances between projected data points, it suffices to show that these distances do not depend on  $b$ .

**Theorem 6.5.1.** *For any  $b, b'$  and any  $x, y \in \mathbb{H}^d$ , the two projected distances  $d_{\mathbb{H}}(\pi_{b,p_1,\dots,p_K}^{\mathbb{H}}(x), \pi_{b,p_1,\dots,p_K}^{\mathbb{H}}(y))$  and  $d_{\mathbb{H}}(\pi_{b',p_1,\dots,p_K}^{\mathbb{H}}(x), \pi_{b',p_1,\dots,p_K}^{\mathbb{H}}(y))$  are equal.*

The proof is included in Appendix C.1. Thus, HOROPCA retains the *location-independence* property of Euclidean PCA: only the *directions* of target subspaces matter; their exact locations do not (Fig. 6.1). Therefore, just like in the Euclidean setting, we can assume without loss of generality that  $b$  is the origin  $\mathbf{o}$  of the Poincaré model. This:

1. alleviates the need to use  $d$  extra parameters to search for an appropriate base point, and
2. simplifies computations, since in the Poincaré model, geodesics submanifolds that go through the origin are simply linear subspaces, which are easier to deal with.

After computing the principal directions which span the target  $M = \text{GH}(\mathbf{o}, p_1, \dots, p_K)$ , the reduced dimensionality data can be found by applying an Euclidean rotation that sends  $M$  to  $\mathbb{H}^K$ , which also preserves hyperbolic distances.

## 6.6 Experiments

We now validate the empirical benefits of HOROPCA on three PCA uses. First, for dimensionality reduction, HOROPCA preserves information (distances and variance) better than previous methods which are sensitive to base point choices and distort distances more (Section 6.6.2). Next, we validate that our notion of hyperbolic coordinates does capture variation in the data and can be used for data whitening in classification tasks (Section 6.6.3). Finally, we visualize the representations learned by HOROPCA in two dimensions (Section 6.6.4). We first start by describing our experimental setup.

### 6.6.1 Experimental Setup

#### 6.6.1.1 Datasets

For dimensionality reduction experiments, we consider standard hierarchical datasets previously used to evaluate the benefits of hyperbolic embeddings. More specifically, we use the datasets in [150] including a fully balanced tree, a phylogenetic tree, a biological graph comprising of diseases'

| dimension            | Sarkar |       |       | PyTorch |       |       |
|----------------------|--------|-------|-------|---------|-------|-------|
|                      | 2      | 10    | 50    | 2       | 10    | 50    |
| <b>Balanced Tree</b> | 0.088  | 0.136 | 0.152 | 0.097   | 0.022 | 0.022 |
| <b>Phylo Tree</b>    | 0.639  | 0.639 | 0.638 | 0.292   | 0.286 | 0.284 |
| <b>Diseases</b>      | 0.247  | 0.212 | 0.210 | 0.126   | 0.054 | 0.055 |
| <b>CS Ph.D.</b>      | 0.388  | 0.384 | 0.380 | 0.189   | 0.140 | 0.139 |

Table 6.2: Average distortion for embeddings learned with different methods. Sarkar refers to combinatorial embeddings learned with Sarkar’s construction [150, 152] while PyTorch refers to embeddings learned using the PyTorch code from [81].

relationships and a graph of Computer Science (CS) Ph.D. advisor-advisee relationships. These datasets have respectively 40, 344, 516 and 1025 nodes, and we use the code from [81] to embed them in the Poincaré ball. For data whitening experiments, we reproduce the experimental setup from [53] and use the Polbooks, Football and Polblogs datasets which have 105, 115 and 1224 nodes each. These real-world networks are embedded in two-dimensions using Chamberlain et al. [37]’s embedding method.

**Embedding** The datasets we use are structured as graphs with nodes and edges. We map these graphs to the hyperbolic space using different embedding methods. The datasets from [150] can be found in the open-source implementation<sup>4</sup> and we compute hyperbolic embeddings for different dimensions using the PyTorch code from [81]. We also consider embeddings computed with Sarkar’s combinatorial construction [152] and report the average distortion measure with respect to the original graph distances in Table 6.2.

For classification experiments, we follow the experimental protocol in [53] and use the embeddings provided in the open-source implementation.<sup>5</sup>

**Centering** For simplicity, we center the input embeddings so that they have a Fréchet mean of zero. This makes computations of projections more efficient. To do so, we compute the Fréchet mean of input hyperbolic points using gradient-descent, and then apply an isometric hyperbolic reflection that maps the mean to the origin.<sup>6</sup>

#### 6.6.1.2 Evaluation metrics

To measure distance-preservation after projection, we use average distortion. If  $\pi(\cdot)$  denotes a mapping from high- to low-dimensional representations, the average distortion of a dataset  $S$  is

<sup>4</sup><https://github.com/HazyResearch/hyperbolics>

<sup>5</sup><https://github.com/hhcho/hyplinear>

<sup>6</sup>This reflection can be computed in closed-form using circle inversions, see for instance Section 2 in [150].

computed as:

$$\frac{1}{\binom{|S|}{2}} \sum_{x,y \in S} \frac{|d_{\mathbb{H}}(\pi(x), \pi(y)) - d_{\mathbb{H}}(x, y)|}{d_{\mathbb{H}}(x, y)}.$$

We also measure the Fréchet variance in Eq. (6.1), which is the analogue of the objective that Euclidean PCA optimizes. Note that the mean in Eq. (6.1) cannot be computed in closed-form and we therefore compute it with gradient-descent.

### 6.6.1.3 Implementation Details

We compare to several dimensionality reduction methods, including Euclidean PCA which we expect to perform poorly on hyperbolic data. We also compare to exact PGA and tangent PCA (tPCA), which approximates PGA by moving the data in the tangent space of the Fréchet mean and then solves Euclidean PCA. We also consider the BSA method and hyperbolic Multi-dimensional Scaling (hMDS) [150], which takes a distance matrix as input and recovers a configuration of points that best approximates these distances. Note that hMDS does not learn projections and cannot work on points that have not been seen before. For instance, hMDS cannot be used for whitening, where the algorithm needs to be run multiple times on different inputs (e.g. train and test). Finally, to demonstrate their dependence on base points, we also include two baselines that perturb the base point in PGA and BSA by adding Gaussian noise.

**Implementation of geodesic projections** Proposition C.1.3 and Algorithm 2 contain detailed descriptions and proofs of implementing geodesic projections and horospherical projections in the hyperboloid model, for which there exist elegant and efficient implementations. However, the Poincaré model is useful for visualizations, and was a popular model for many baselines in the ML literature. Therefore here we give high-level descriptions of alternate methods to compute these projection methods from simple primitives directly in the Poincaré model. We then describe implementations of all methods and make the corresponding PyTorch code available.<sup>7</sup>

Recall that geodesic projections map points to a target submanifold such that the projection of a point is the point on the submanifold that is closest to it. In the Poincaré model, when the projection submanifold goes through the origin then geodesic projections can be computed efficiently.

Consider any geodesic submanifold  $P$  that contains the origin in the Poincaré Ball: this must be a linear space since any geodesic that goes through the origin is a straight-line in this model. The Euclidean reflection with respect to  $P$  is a hyperbolic isometry (i.e. preserves distances). Given any point  $x$ , we can compute its Euclidean (and hyperbolic) reflection with respect to  $P$ ,  $r_P(x)$ . The midpoint between the  $x$  and  $r_P(x)$  belongs to  $P$  and corresponds to the geodesic projection of  $x$  onto  $P$ . In particular, there is a closed-form expression for the midpoint of two hyperbolic points which can be derived using Möbius operations.

<sup>7</sup><https://github.com/HazyResearch/HoroPCA>.

|           | Balanced Tree                     |                                   | Phylo Tree                        |                                    | Diseases                          |                                    | CS Ph.D.                          |                                    |
|-----------|-----------------------------------|-----------------------------------|-----------------------------------|------------------------------------|-----------------------------------|------------------------------------|-----------------------------------|------------------------------------|
|           | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )           | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )            | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )            | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )            |
| PCA       | 0.84                              | 0.34                              | 0.94                              | 0.40                               | 0.90                              | 0.26                               | 0.84                              | 1.68                               |
| tPCA      | 0.70                              | 1.16                              | 0.63                              | 14.34                              | 0.63                              | 3.92                               | 0.56                              | 11.09                              |
| PGA       | $0.63 \pm 0.07$                   | $2.11 \pm 0.47$                   | $0.64 \pm 0.01$                   | $15.29 \pm 0.51$                   | $0.66 \pm 0.02$                   | $3.16 \pm 0.39$                    | $0.73 \pm 0.02$                   | $6.14 \pm 0.60$                    |
| PGA-Noise | $0.87 \pm 0.08$                   | $0.29 \pm 0.30$                   | $0.64 \pm 0.02$                   | $15.08 \pm 0.77$                   | $0.88 \pm 0.04$                   | $0.53 \pm 0.19$                    | $0.79 \pm 0.03$                   | $4.58 \pm 0.64$                    |
| BSA       | $0.50 \pm 0.00$                   | $3.02 \pm 0.01$                   | <u><math>0.61 \pm 0.03</math></u> | $18.60 \pm 1.16$                   | $0.52 \pm 0.02$                   | $5.95 \pm 0.25$                    | <u><math>0.70 \pm 0.01</math></u> | $8.15 \pm 0.96$                    |
| BSA-Noise | $0.74 \pm 0.12$                   | $1.06 \pm 0.67$                   | $0.68 \pm 0.02$                   | $13.71 \pm 0.72$                   | $0.80 \pm 0.11$                   | $1.62 \pm 1.30$                    | $0.79 \pm 0.02$                   | $4.41 \pm 0.59$                    |
| hMDS      | <u>0.22</u>                       | <b>7.54</b>                       | 0.74                              | <u>40.51</u>                       | <u>0.21</u>                       | <u>15.05</u>                       | 0.83                              | <u>19.93</u>                       |
| HoroPCA   | <b><math>0.19 \pm 0.00</math></b> | <u><math>7.15 \pm 0.00</math></u> | <b><math>0.13 \pm 0.01</math></b> | <b><math>69.16 \pm 1.96</math></b> | <b><math>0.15 \pm 0.01</math></b> | <b><math>15.46 \pm 0.19</math></b> | <b><math>0.16 \pm 0.02</math></b> | <b><math>36.79 \pm 0.70</math></b> |

Table 6.3: Dimensionality reduction results on 10-dimensional hyperbolic embeddings reduced to two dimensions. Results are averaged over 5 runs for non-deterministic methods. Best in **bold** and second best underlined.

**Implementation of baselines** Having described computations of geodesic projections, we now detail our baselines implementation. For PCA and tPCA, we used the Singular Value Decomposition (SVD) PyTorch implementation to implement both PCA and tPCA. Before using the SVD algorithm, tPCA maps points to the tangent space at the Fréchet mean using the logarithmic map. Having centered the data, this mapping can be done using the logarithmic map at the origin which is simply:

$$\log_{\mathbf{o}}(x) = \tanh^{-1}(\|x\|) \frac{x}{\|x\|}.$$

Both PGA and BSA rely on geodesic projections. In particular, when the data is centered, the target submanifolds in BSA and PGA are simply linear spaces going through the origin. We can therefore easily compute geodesic projections in these methods using the computations described above. To implement hMDS, we simply implemented the formulas in the original paper (see Algorithm 2 in [150]).

**HoroPCA implementation** The idea of the main horosphere projection algorithm onto  $K > 1$  directions is to simply compute the intersection of the horospheres  $S(p_1, x) \cap \dots \cap S(p_K, x)$  (Section 6.4.2.2). Although Appendix C.1.3 shows how in the hyperboloid model, horospherical projections can be computed efficiently without actually computing these intersections, it is also possible to implement HYPHC by simply computing these intersections. This can be done in a simple iterative fashion.

For example, in the Poincaré ball, horospheres can be represented as Euclidean spheres. Note that the intersection of two  $d$ -dimensional Euclidean hyperspheres  $S(p_0, r_0)$  and  $S(p_1, r_1)$  is a  $d - 1$ -dimensional hypersphere which can be represented by a center, a radius, and  $d - 1$ -dimensional subspace. The radius is uniquely determined by the radii of the original spheres  $r_0, r_1$  and the distance between their centers  $\|p_0 - p_1\|$  and analyzing the 2-dimensional case, for which there is a simple closed formula. The center can similarly be found as a weighted combination of the centers  $p_0, p_1$  based on the formula for the 2D case. Lastly, this  $d - 1$ -dimensional subspace can be represented by

|         | Balanced Tree                     |                                  | Phylo Tree                       |                                    | Diseases                          |                             | CS Ph.D.                          |                               |
|---------|-----------------------------------|----------------------------------|----------------------------------|------------------------------------|-----------------------------------|-----------------------------|-----------------------------------|-------------------------------|
|         | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )          | distortion ( $\downarrow$ )      | variance ( $\uparrow$ )            | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )     | distortion ( $\downarrow$ )       | variance ( $\uparrow$ )       |
| PCA     | 0.91                              | 1.09                             | 0.99                             | 0.28                               | 0.95                              | 0.75                        | 0.96                              | 1.25                          |
| tPCA    | 0.83                              | 4.01                             | 0.27                             | 62.54                              | 0.64                              | 35.45                       | 0.55                              | 86.10                         |
| PGA     | $0.67 \pm 0.01$                   | $11.1 \pm 0.9$                   | $\underline{0.08} \pm 0.00$      | $\underline{1110.3} \pm 0.6$       | $0.55 \pm 0.00$                   | $56.8 \pm 0.5$              | $0.25 \pm 0.00$                   | $270.3 \pm 0.5$               |
| BSA     | $0.58 \pm 0.02$                   | $16.7 \pm 1.3$                   | $\underline{0.08} \pm 0.00$      | $1110.1 \pm 0.1$                   | $0.54 \pm 0.01$                   | $57.7 \pm 0.5$              | $0.25 \pm 0.00$                   | $271.0 \pm 0.4$               |
| hMDS    | $\underline{0.12}$                | $\underline{58.62}$              | $\underline{0.08}$               | 498.05                             | $\underline{0.32}$                | <b>568.22</b>               | $\underline{0.18}$                | <b>1241.34</b>                |
| HoroPCA | <b><math>0.06 \pm 0.01</math></b> | <b><math>62.7 \pm 0.9</math></b> | <b><math>0.01 \pm 0.0</math></b> | <b><math>1190.3 \pm 2.9</math></b> | <b><math>0.09 \pm 0.01</math></b> | $\underline{161.3} \pm 1.2$ | <b><math>0.05 \pm 0.00</math></b> | $\underline{994.7} \pm 321.3$ |

Table 6.4: Dimensionality reduction results on 10-dimensional combinatorial embeddings (Sarkar’s construction) reduced to two dimensions. Results are averaged over 5 runs for non-deterministic methods. Best in **bold** and second best underlined.

noting it is the orthogonal space to the vector  $p_1 - p_0$ . Thus the intersection of these  $d$ -dimensional hyperspheres can be easily computed, and this process can be iterated to find the intersection of  $k$  spheres  $S(p_i, r_i)$ .

### 6.6.2 Dimensionality Reduction

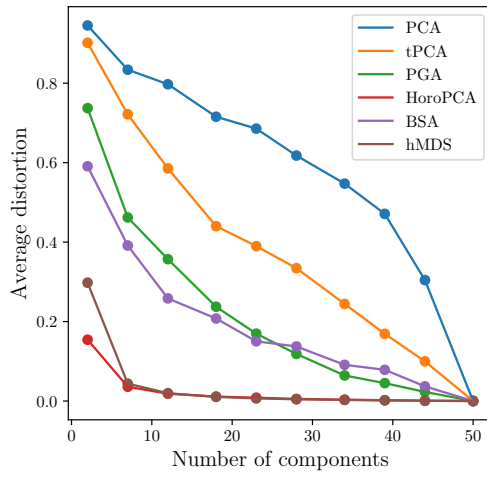
**PyTorch embeddings** We report metrics for the reduction of 10-dimensional embeddings to two dimensions in Table 6.3. All results suggest that HOROPCA better preserves information contained in the high-dimensional representations.

On distance preservation, HOROPCA outperforms all methods with significant improvements on larger datasets. This supports our theoretical result that horospherical projections better preserve distances than geodesic projections. Furthermore, HOROPCA also outperforms existing methods on the explained Fréchet variance metric on all but one dataset. This suggests that our distance-based formulation of the variance (Eq. (6.3)) effectively captures variations in the data.

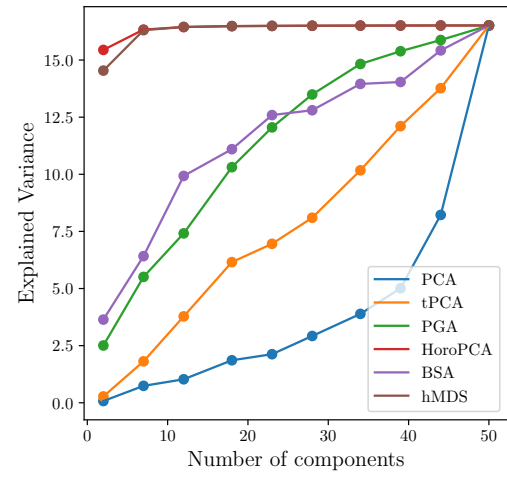
We also note that as expected, both PGA and BSA are sensitive to base point choices: adding noise to the base point leads to significant drops in performance. In contrast, HOROPCA is by construction invariant to base point choices.

**Sarkar embeddings** Table 6.3 showed dimensionality reduction results for embeddings learned with optimization. Here, we consider the same reduction experiment on combinatorial embeddings and report the results in Table 6.4. The results confirm the trends observed in Table 6.3: HYPHC outperforms baseline methods, with significant improvements on distance preservation.

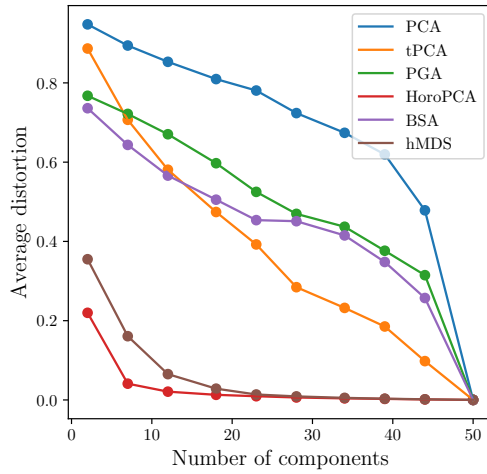
**More dimension/component configurations** We consider the reduction of 50-dimensional PyTorch embeddings of the Diseases and CS Ph.D. datasets. We plot average distortion and explained Fréchet variance for different number of components in Fig. 6.4. HYPHC significantly outperforms all previous generalizations of PCA. HYPHC also outperforms hMDS, which is a competitive baseline, but not a PCA method as discussed before.



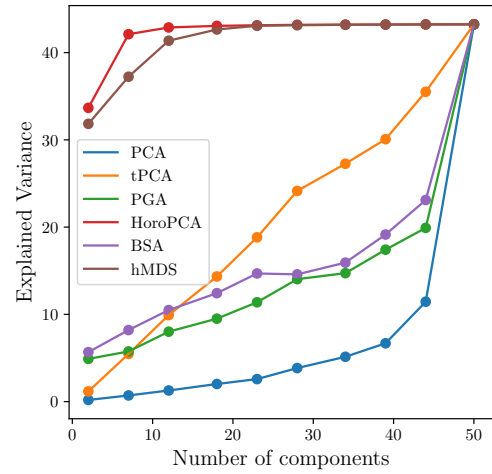
(a) Diseases average distortion.



(b) Diseases explained Fréchet variance.



(c) CS Ph.D. average distortion.



(d) CS Ph.D. explained Fréchet variance.

Figure 6.4: Average distortion and explained Fréchet variance for different numbers of principal components.

|              | Polbooks              | Football              | Polblogs              |
|--------------|-----------------------|-----------------------|-----------------------|
| eSVM         | <u>69.9</u> $\pm$ 1.2 | 20.7 $\pm$ 3.0        | 92.3 $\pm$ 1.5        |
| hSVM         | 68.3 $\pm$ 0.6        | 20.9 $\pm$ 2.5        | 92.2 $\pm$ 1.6        |
| tPCA+eSVM    | 68.5 $\pm$ 0.9        | <u>21.2</u> $\pm$ 2.2 | <u>92.4</u> $\pm$ 1.5 |
| HOROPCA+eSVM | <b>72.2</b> $\pm$ 2.8 | <b>25.0</b> $\pm$ 1.0 | <b>92.8</b> $\pm$ 0.9 |

Table 6.5: Data whitening experiments. We report classification accuracy averaged over 5 embedding configurations. Best in **bold** and second best underlined.

### 6.6.3 Hyperbolic Data Whitening

An important use of PCA is for data whitening, as it allows practitioners to remove noise and decorrelate the data, which can improve downstream tasks such as regression or classification. Recall that standard PCA data whitening consists of (i) finding principal directions that explain the data, (ii) calculating the coordinates of each data point along these directions, and (iii) normalizing the coordinates for each direction (to have zero mean and unit variance).

Because of the tight connection between HOROPCA and Euclidean PCA, these steps can easily map to the hyperbolic case, where we (i) apply HOROPCA and (ii) use the Busemann coordinates (distances along the principal components). In particular, since horospheres, being Riemannian manifolds, are isometric to Euclidean spaces, we can directly use the Busemann coordinates (distances to ideal points) as a new Euclidean representation.

We evaluate the benefit of this whitening step on a simple classification task. We compare to tPCA, which computes representations in the tangent space at the mean, and then performs Euclidean PCA. Note that these whitening steps (tPCA and HOROPCA with Busemann coordinates) yield Euclidean coordinates, which allows leveraging powerful tools developed specifically for learning on Euclidean data (e.g. L2 regularization). We also compare directly classifying the data with a Euclidean Support Vector Machine (eSVM) classifier and its hyperbolic counterpart (hSVM).

We reproduce the experimental setup from [53] who split the datasets in 50% train and 50% test sets, run classification on 2-dimensional embeddings and average results over 5 different embedding configurations as was done in the original paper (Table 6.5).<sup>8</sup> HOROPCA whitening improves downstream classification on all datasets compared to eSVM and hSVM or tPCA whitening. This suggests that HOROPCA can be leveraged for hyperbolic data whitening. Further, this confirms that Busemann coordinates do capture variations in the original data.

### 6.6.4 Visualizations

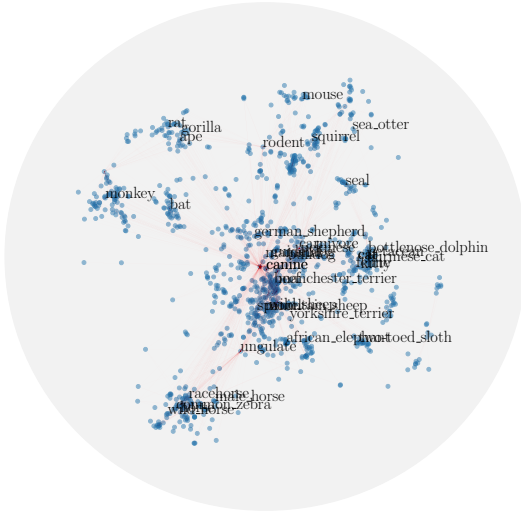
When learning embeddings for ML applications (e.g. classification), increasing the dimensionality can significantly improve the embeddings' quality. To effectively work with these higher-dimensional

<sup>8</sup>Note that the results slightly differ from [53] which could be because of different implementations or data splits.

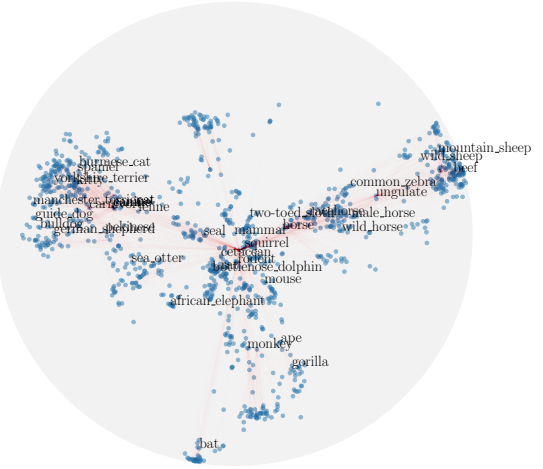
embeddings, it is useful to visualize their structure and organization, which often requires reducing their representations to two or three dimensions. Here, we consider embeddings of the mammals subtree of the Wordnet noun hierarchy learned with the algorithm from [136]. We reduce embeddings to two dimensions using PCA, PGA, BSA and HOROPCA and show the results in Fig. 6.5. As we can see, the reduced representations obtained with HOROPCA yield better visualizations. For instance, we can see some hierarchical patterns such as “feline hypernym of cat” or “cat hypernym of burmese cat”. These patterns are harder to visualize for other methods, since these do not preserve distances as well as HOROPCA, e.g. BSA has 0.532 average distortion on this dataset compared to 0.078 for HOROPCA.

## 6.7 Conclusion

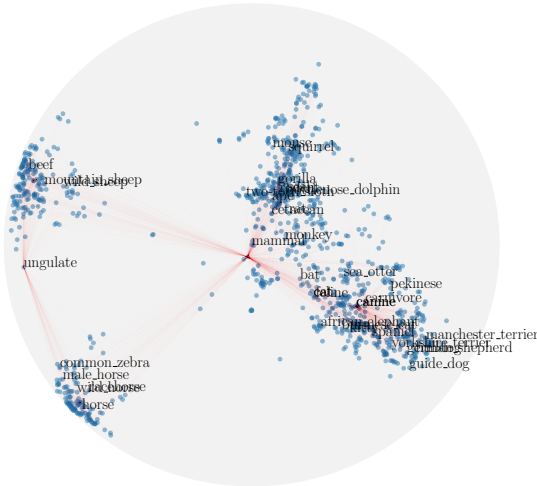
We proposed HOROPCA, a method to generalize PCA to hyperbolic spaces. In contrast with previous PCA generalizations, HOROPCA preserves the core location-independence PCA property. Empirically, HOROPCA significantly outperforms previous methods on the reduction of hyperbolic data. Future extensions of this work include deriving a closed-form solution or analyzing the stability properties of HOROPCA.



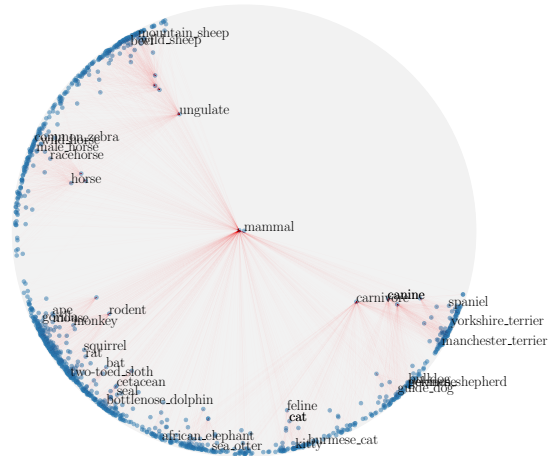
(a) PCA reduction (average distortion: 0.942)



(b) PGA reduction (average distortion: 0.534)



(c) BSA reduction (average distortion: 0.532)



(d) HOROPCA reduction (average distortion: 0.078)

Figure 6.5: Visualization of embeddings of the WordNet mammal subtree computed by reducing 10-dimensional Poincaré embeddings [136].

## Chapter 7

# Conclusion

In this dissertation, we studied representation learning methods and algorithms that operate in hyperbolic spaces. We started by motivating the use of hyperbolic embeddings to represent data that has hierarchical and tree-like structures, and we reviewed some important connections between tree metrics and hyperbolic metrics, such as Sarkar’s theorem which shows that any tree metric can be embedded in the hyperbolic space of two dimensions with arbitrarily low distortion.

Using these motivations, we then developed models to embed data in hyperbolic spaces. We proposed a hyperbolic Graph Convolutional Network (GCN) model, the first hyperbolic GCN to learn graph representations in the presence of node features. We also proposed an attention-based model to learn hyperbolic representations of multi-relational data such as Knowledge Graphs. On a variety of applications such as link prediction and node classification, we demonstrated that these approaches significantly outperformed their Euclidean counterparts and achieved state-of-the-art results on datasets with hierarchical structures.

Motivated by the fact that standard Euclidean algorithms cannot be used to analyze and visualize these hyperbolic representations, we then derived the hyperbolic analogues of two fundamental machine learning algorithms: hierarchical clustering (HC) and dimensionality reduction. For both HC and dimensionality reduction, we developed a theoretical framework and showed strong guarantees for both approaches: for instance, we showed that under perfect optimization assumption, our HC solutions were near-optimal and we showed that our proposed PCA algorithm achieved significantly lower distortion than geodesic-based methods such as PGA [70].

We hope that our work will help improve the quality of representations for machine learning applications, and help develop the analogues of other much needed downstream algorithms. In particular, a highly important problem that remains open is how to approximately search for nearest neighbors in hyperbolic spaces. We think that an interesting future direction could be using the notion of hyperbolic coordinates introduced in this dissertation to develop efficient nearest neighbor search algorithms, or more principled neural network operations.

Another open research direction is in the choice of embeddings' geometry. While we focused on hyperbolic spaces for tree-like graphs, many real-world datasets have mixed geometries. For instance some graphs may not be tree-like (in terms of Gromov's delta hyperbolicity) but can be described using a small family of trees [16, 149]. What is the right geometry in this scenario? Could we use a locally hyperbolic manifold to embed these graphs? Developing representations learning models for these mixed geometries is an important open research direction. Some ongoing efforts are using product spaces of constant curvature [82] more general Riemannian manifold parameterized by their metric tensor [119].

A common challenge that arises with hyperbolic embeddings is precision issues (e.g., near the boundary of the Poincaré ball, distances become very large which can cause numerical issues). While these precision challenges are well understood [150], developing numerically stable hyperbolic embeddings is an open and active research problem [196].

Finally, a future research direction would be using the tools introduced in this dissertation in medical applications, such as RNA sequencing or molecular generation, which are areas that both deal with a great amount of hierarchical structures.

# Appendix A

## Geometry Review

To understand the intuition behind hyperbolic geometry, its connection to tree metric spaces and the tools to build machine learning models that operate in hyperbolic spaces, we first need to highlight some important geometry concepts, which enable calculus on manifolds (e.g. computing distances and volumes in hyperbolic space).

In Appendix [A.1](#), we first start by describing topological manifolds, which are broadly speaking topological spaces that locally resemble the Euclidean space and are fundamental building blocks for the theory of smooth manifolds. In Appendix [A.2](#), we describe how additional structure can be added on top of manifolds to make a sense of the notion of smoothness, leading to important notions such as tangent vectors. Finally, in Appendix [A.3](#), we describe the tools needed to do calculus on smooth manifolds, and in particular how Riemannian metrics can be used to compute lengths of curves and distances between points.

### A.1 Topological Manifolds

In this section, we review topological manifolds, which are spaces that essentially look like the flat Euclidean space locally. We start by describing topological spaces, which are sets with open subsets (Appendix [A.1.1](#)), allowing to generalize the notion of continuity of functions to arbitrary sets (Appendix [A.1.2](#)), and then define the notion of topological manifold (Appendix [A.1.3](#)).

#### A.1.1 Definitions

A topological space is a space endowed with a topological structure (collections of *open* subsets), which can generalize the notion of continuity in Euclidean spaces to more general spaces (e.g. sets of points).

**Definition A.1.1** (Continuity in  $\mathbb{R}$ ). *A function  $f : \mathbb{R} \rightarrow \mathbb{R}$  is continuous at  $x \in \mathbb{R}$  if given any  $\varepsilon > 0$  there exists  $\delta > 0$  such that:  $\forall y \in ]x - \delta, x + \delta[, f(y) \in ]f(x) - \varepsilon, f(x) + \varepsilon[$ .*

This definition of continuity relies on the notions of open intervals, it states that for any small open interval around  $f(x)$ , there exist a small open interval around  $x$  that maps to it. Similarly, a topology defines open subsets in abstract set of points:

**Definition A.1.2** (Topology). *Given a set (collection of elements)  $X$ , a collection of subsets of  $X$ ,  $\tau_X$ , is a topology if it satisfies the axioms:*

1. *The empty set  $\emptyset$  and  $X$  itself belong to  $\tau_X$ .*
2. *The intersection of any finite number of elements in the topology is also in the topology.*
3. *Any arbitrary (finite or infinite) union of elements in the topology is also in the topology.*

The elements of  $\tau_X$  are called *open subsets* of  $X$ , and the collection  $\tau_X$  is called a topology on  $X$ . The pair  $(X, \tau_X)$  is called a *topological space*. As we shall see next (Appendix A.1.2), we can generalize the notion of continuity to arbitrary set of points using this notion of open subsets.

**Example A.1.1.** Given a set  $X$ , the collection  $\tau_X = \{\emptyset, X\}$  forms a topology called the *trivial topology*. The collection formed by the power set (all possible subsets) is also a topology called the *discrete topology*.

**Definition A.1.3** (Basis). *A basis  $\mathcal{B}$  for a topology on a set  $X$  is a collection of open subsets of  $X$  satisfying the following axioms:*

1. *For every  $x \in X$ , then there exists an element  $B \in \mathcal{B}$  such that  $x \in B$ .*
2. *If  $B_1, B_2 \in \mathcal{B}$  and  $x \in B_1 \cap B_2$ , then there exists  $B_3 \in \mathcal{B}$  such that  $x \in B_3 \subset B_1 \cap B_2$ .*

Let  $\mathcal{B}$  be a collection of subsets of  $X$  that satisfies these properties, and let  $\tau_X$  be the set of subsets of  $X$  which are arbitrary unions of sets in  $\mathcal{B}$ . Then  $\tau_X$  is the unique topology on  $X$  for which  $\mathcal{B}$  is a base and is called the *topology generated by  $\mathcal{B}$* . To see this, note that 1. in Definition A.1.3 implies that  $X \in \tau_X$ , and 2. implies that  $\tau_X$  is closed under taking finite intersections.

**Example A.1.2.** Recall the definition of metric spaces.

**Definition A.1.4** (Metric Space). *A metric space is a pair  $(X, d)$  where  $X$  is a set and  $d_X$  is a metric on  $X$ , i.e. a function  $d : X \times X \rightarrow \mathbb{R}_+$  such that for any  $x, y, z \in X$ :*

1.  $d(x, y) = 0 \Leftrightarrow x = y$  (identity of indiscernibles)
2.  $d(x, y) = d(y, x)$  (symmetry)
3.  $d(x, z) \leq d(x, y) + d(y, z)$  (triangle inequality).

We can show that any metric space  $(X, d)$  is a topological space. Let's define balls of radius  $r > 0$  centered at  $x \in X$  as the subsets:  $B_r(x) = \{y \in X : d(x, y) < r\}$  and consider the collection  $\mathcal{B} = \{B_r(x) : x \in X, r \geq 0\}$ . This collection satisfies the axioms in Definition A.1.3:

1. It is clear that for every  $x \in X$ , we have  $x \in B_r(x)$  for any  $r > 0$ .

2. Suppose  $x \in B_{r_1}(x_1) \cap B_{r_2}(x_2)$ . We want to show that there exists  $x_3 \in X, r_3 > 0$  such that  $x \in B_{r_3}(x_3) \subset B_{r_1}(x_1) \cap B_{r_2}(x_2)$ . Let  $\varepsilon_1 := r_1 - d(x, x_1)$  and let  $y \in B_{\varepsilon_1}(x)$ . By triangle inequality, we have  $d(y, x_1) \leq d(y, x) + d(x, x_1) < r_1$ . Therefore  $B_{\varepsilon_1}(x) \subset B_{r_1}(x_1)$  and a similar argument with  $\varepsilon_2 := r_2 - d(x, x_2)$  yields  $B_{\varepsilon_2}(x) \subset B_{r_2}(x_2)$ . Therefore,  $x \in B_{\min\{\varepsilon_1, \varepsilon_2\}}(x) \subset B_{r_1}(x_1) \cap B_{r_2}(x_2)$ .

Therefore, the collection formed by all unions of balls in  $\mathcal{B}$  is a topology on  $X$ , called the *standard topology*. Note that while every metric space is a topological space, not all topological spaces are metric spaces.

Topological spaces can describe many concepts of spaces, from the standard topology on metric spaces, to the discrete topology on arbitrary sets. In the context of manifold theory, we need to restrict ourselves to topological spaces that are “well-behaved” (e.g. in which limits of sequences are unique) and we describe some important properties next.

**Definition A.1.5** (Hausdorff space). *A topological space  $X$  is said to be Hausdorff if for any two distinct elements in the set  $x, y \in X$ , there exists open sets  $U, V \subset X$  such that  $x \in U, y \in V$  and  $U \cap V = \emptyset$ .*

In other words, any two distinct points can be separated by open sets. All metric spaces are Hausdorff but there are many examples of non-Hausdorff topological spaces. For instance, if  $X = \{1, 2\}$  and  $\tau_X = \{\emptyset, X\}$  is the trivial topology, then  $(X, \tau_X)$  is not Hausdorff.

**Definition A.1.6** (Second-countability). *A set is said to be countable if it is finite or it admits a bijection with the set of natural numbers  $\mathbb{N}$ . A topological space is said to be second-countable if it has a countable basis.*

For instance, the real numbers with the standard topology is second countable. While the usual base of open intervals is not countable, one can show that the topology generated by the base  $\mathcal{B} = \{]a, b[ \mid a, b \in \mathbb{Q}\}$  (which is countable) is the same as the standard topology. Therefore, the real line is a second-countable space.

### A.1.2 Continuity

As mentioned in the previous section, a topological structure on a set allows to generalize the notion of continuity to sets of points. Having introduced basic definitions, we now move on to the notions of continuity of functions in topological spaces.

**Definition A.1.7** (Continuity in topological spaces). *Let  $X$  and  $Y$  be topological spaces. A mapping  $f : X \rightarrow Y$  is continuous if for each open set  $U \subset Y$  the set  $f^{-1}(U)$  is open in  $X$ .*

Note that if  $X$  and  $Y$  are the real numbers under the standard metric topology (which is generated by open intervals), then this is equivalent to the Definition A.1.1 of continuity.

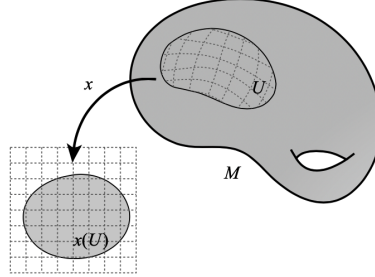


Figure A.1: A coordinate chart  $(U, x)$  on a manifold  $M$ . Source: [71]

**Definition A.1.8** (Homeomorphism). *Let  $X$  and  $Y$  be topological spaces. A mapping  $f : X \rightarrow Y$  is a homeomorphism if it is bijective and both  $f$  and  $f^{-1}$  are continuous. In this case, we say that  $X$  and  $Y$  are homeomorphic.*

“Being homeomorphic” is an equivalence relation on topological spaces and can be used to classify and compare topological spaces. When two topological spaces are homeomorphic, they are indistinguishable to the eyes of topology and any topological property (e.g. dimension, being Hausdorff, compactness) that holds for the first topological space holds for the other as well.

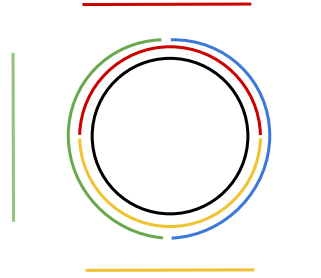
Broadly speaking, topological spaces are geometric objects and saying that two spaces are homeomorphic means that we can continuously deform one object into the other. For instance, a circle and a square are the same to the eyes of topology, while a torus and a sphere are not (we would have to create a hole in the sphere to transform it into a torus, which is not a continuous mapping).

### A.1.3 Topological Manifolds

As mentioned in the previous section, topological spaces that are homeomorphic are the same from a topological viewpoint, even though they might be very different geometrically speaking (e.g. a circle and a square are the same to the eyes of topology but these objects have very different shapes). Our goal here is to add more structure to understand the geometry of topological spaces and we define topological manifolds next.

**Definition A.1.9** (Topological Manifold). *We say that a topological space is a topological manifold of dimension  $n$  if it has the following properties:*

1. *It is a Hausdorff space.*
2. *It is second-countable.*
3. *For each point  $p \in M$ , there exists an open subset  $U \subseteq M$  containing  $p$  and a homeomorphism  $\varphi : U \rightarrow \varphi(U) \subseteq \mathbb{R}^n$ . The pair  $(U, \varphi)$  is called a coordinate chart.*

Figure A.2: Illustration of four charts covering the circle  $\mathbb{S}^1$ .

Thus, we can parameterize points  $q \in U$  using  $n$  local coordinate functions  $(x^1, \dots, x^n)$  such that  $\varphi(q) = (x^1(q), \dots, x^n(q))$ .

At a high level, manifolds are topological spaces that are locally equivalent to the Euclidean space, which generalizes the notion of surfaces to higher dimensions. The simplest example of a manifold is  $\mathbb{R}^n$  since it is homeomorphic to itself everywhere.

**Example A.1.3.** Consider circle  $\mathbb{S}^1 = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1\}$  as a subset of  $\mathbb{R}^2$ . It is clear that  $\mathbb{S}^1$  is a Hausdorff and second-countable because it is a topological subspace of  $\mathbb{R}^2$ . To show that  $\mathbb{S}^1$  is locally Euclidean, consider the upper hemisphere  $U = \{(x, y) \in \mathbb{R}^2 : y > 0\}$ .  $U$  is an open supset of  $\mathbb{R}^2$  and we can define a homeomorphism from  $U$  to  $\mathbb{R}$  via:

$$\varphi : (x, y) \mapsto x.$$

Then  $(U, \varphi)$  is a coordinate chart and we can similarly find other charts to show that  $\mathbb{S}^1$  is locally Euclidean everywhere and it is therefore a topological manifold (Fig. A.2).

**Remark A.1.1.** Note that topological manifolds are only locally Euclidean in the sense that we can't generally map the entire space to the Euclidean space with a single chart. For instance with the circle, issues arise around the poles and there exists no continuous one-to-one mapping from  $\mathbb{S}^1$  to  $\mathbb{R}$ .

**Definition A.1.10 (Atlas).** Given a topological space  $M$ , an atlas is an indexed family of charts  $\mathcal{A} = \{(U_\alpha, \Psi_\alpha) : \alpha \in I\}$  on  $M$  which covers  $M$ , that is:  $\cup_{\alpha \in I} U_\alpha = M$ .

Given an atlas, one can stitch together all charts to reconstruct the original manifold.

**Example A.1.4.** The surface of the Earth is a topological manifold: it locally looks flat on the human scale, but from a global perspective, it is approximately spherical. In particular, atlases in cartography are a collection of maps (charts) covering the surface of the Earth.

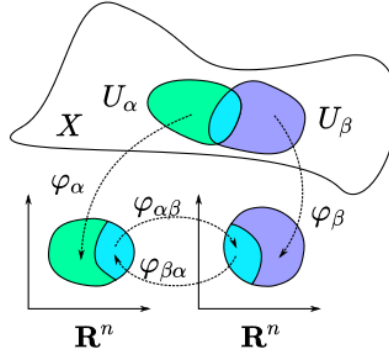


Figure A.3: Transition function between two coordinate charts on a smooth manifold. Source: [https://en.wikipedia.org/wiki/Atlas\\_\(topology\)](https://en.wikipedia.org/wiki/Atlas_(topology))

## A.2 Smooth Manifolds

As mentioned in the introduction, our goal is to introduce the necessary tools to do calculus on manifolds, e.g. characterize which functions on a manifold are smooth and provide a way to compute their derivatives. In this section, we review smooth manifolds which provide a natural setting to compute derivatives of functions.

We saw that topological manifolds have local coordinate charts which suggests the possibility to do differential calculus on  $M$ . For instance, given a function  $f : M \rightarrow \mathbb{R}$  and a chart  $(U, \varphi)$ , one could consider the composition  $f \circ \varphi^{-1} : \mathbb{R}^n \rightarrow \mathbb{R}$ , which is a real-valued function whose domain is an open subset of the Euclidean space. We could say that  $f$  is smooth if its composition with the coordinate function is smooth, and then compute partial derivatives in the local chart representation. However, this definition of smoothness is very dependent on a specific chart and issues may arise especially for the case where charts intersect each other.

In this section, we describe how structure can be added on topological manifolds to construct a family of coordinate charts that agree with each others, namely a smooth atlas (Appendix A.2.1). This definitions allows to unambiguously characterize smooth functions on manifolds (Appendix A.2.2), and compute their derivatives using tangent vectors (Appendix A.2.3).

### A.2.1 Definitions

Before introducing the notion of differentiable structures and smooth manifolds, we need to define maps from one coordinate chart to another. These maps are used to compare charts and are called transition maps (Fig. A.3).

**Definition A.2.1** (Transition Maps). *Given a topological manifold  $M$  and two charts  $(U_\alpha, \varphi_\alpha)$  and  $(U_\beta, \varphi_\beta)$  such that  $U_\alpha \cap U_\beta$  is non-empty, the transition map:  $\tau_{\alpha,\beta} : \varphi_\alpha(U_\alpha \cap U_\beta) \rightarrow \varphi_\beta(U_\alpha \cap U_\beta)$  is*

the map defined by:

$$\tau_{\alpha,\beta} := \varphi_\beta \circ \varphi_\alpha^{-1}.$$

To see where this definition comes from, consider a function  $f : M \rightarrow \mathbb{R}$ . We have:

$$f \circ \varphi_\alpha^{-1} = (f \circ \varphi_\beta^{-1}) \circ (\varphi_\beta \circ \varphi_\alpha^{-1}).$$

Since  $\varphi_\alpha$  and  $\varphi_\beta$  are homeomorphisms, the map  $\varphi_\beta \circ \varphi_\alpha^{-1}$  is also a homeomorphism. However, to do differential calculus, one would need  $\varphi_\beta \circ \varphi_\alpha^{-1}$  to also be differentiable, which is exactly the transition map in Definition A.2.1.

**Definition A.2.2** (Compatible charts). *Two charts  $(U_\alpha, \varphi_\alpha)$  and  $(U_\beta, \varphi_\beta)$  are said to be  $C^k$ -compatible if either  $U_\alpha \cap U_\beta = \emptyset$  or the transition map  $\varphi_\beta \circ \varphi_\alpha^{-1}$  is a  $C^k$  map with  $C^k$  inverse.*

**Definition A.2.3** (Differentiable atlas). *A  $C^k$ -differentiable atlas  $\mathcal{A}$  on a topological manifold  $M$  is an indexed family of charts  $\mathcal{A} = \{(U_\alpha, \varphi_\alpha) : \alpha \in I\}$  on  $M$  which covers  $M$  and such that any two charts in  $\mathcal{A}$  are  $C^k$ -compatible.*

**Remark A.2.1.** If the transition maps are  $C^\infty$  then we say that we have a *smooth atlas*.

This notion of smooth atlas provides a good starting point to compute derivatives of functions on manifolds, by using ordinary calculus in each of the charts (which is possible since these are compatible with each other). As there are many possible smooth atlases for a given manifold, we define next smooth maximal atlases, which allow to define the derivative of functions on manifolds unambiguously.

**Definition A.2.4** (Maximal atlas). *An atlas  $\mathcal{A}$  on a manifold  $M$  is said to be a maximal if any chart  $(V, \psi)$  that is  $C^\infty$ -compatible with every coordinate chart of  $\mathcal{A}$  is also a member of  $\mathcal{A}$ .*

We now have all the tools to define smooth manifolds.

**Definition A.2.5** (Smooth manifold). *A pair  $(M, \mathcal{A})$  where  $M$  is a topological manifold and  $\mathcal{A}$  is a smooth maximal atlas on  $M$  is called a smooth manifold.*

In summary, smooth manifolds are manifolds in which the local charts are compatible with each other, which makes computations of derivatives on the entire manifold possible, as we shall see next.

## A.2.2 Smooth Functions

Having introduced the notion of smooth manifolds, we now describe the differentiation of functions on smooth manifolds. While we can't compute derivatives everywhere on  $M$  using local coordinates, we can use the fact that we have charts with smooth transition functions to compute derivatives. These charts map points on the manifold to standard Euclidean space  $\mathbb{R}^n$ , and we can use the image of the chart to do calculus and define derivatives on manifolds.

**Definition A.2.6** (Smooth function). *A real valued function  $f : M \rightarrow \mathbb{R}^k$  on an  $n$ -dimensional smooth manifold  $M$  is called smooth if for any  $p \in M$ , there exists  $(U, \varphi)$  a coordinate chart ( $p \in U$ ) such that the map:  $f \circ \varphi^{-1} : \mathbb{R}^n \rightarrow \mathbb{R}^k$  is smooth on the open subset  $\varphi(U) \subset \mathbb{R}^n$ . If  $k = 1$ , we denote  $C^\infty(M)$  the set of smooth real-valued functions on  $M$ .*

**Remark A.2.2.** The above definition of smoothness of functions is defined purely in terms of derivatives in the local coordinate chart representation, which can be computed using ordinary Euclidean differentiation rules.

**Remark A.2.3.** Note that smoothness does not depend on the choice of a chart at  $p$ : using the chain rule and the differentiability of transition function on differentiable manifolds, one can show that if  $f$  is differentiable in a particular chart at  $p$  then it is differentiable in all charts at  $p$ .

### A.2.3 Tangent Space

The previous subsection introduced the notion of smooth functions on manifolds. We now review the notion of tangent vectors which make the calculations of derivatives of these smooth functions possible.

Before we describe tangent vectors on manifolds, let's first discuss a simple example in the Euclidean space  $\mathbb{R}^2$ . Supposed we have a curve  $\gamma(t) = (x(t), y(t)) \in \mathbb{R}^2$ , and a function  $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ . Let's say we want to compute the derivative of  $f$  along  $\gamma$  for  $t = 0$  and let's write  $p = \gamma(0)$  and  $v = (\dot{x}(0), \dot{y}(0))$ . We can use the chain rule and have:

$$\left. \frac{df(\gamma(t))}{dt} \right|_0 = \left. \frac{\partial f}{\partial x} \right|_p \dot{x}(0) + \left. \frac{\partial f}{\partial y} \right|_p \dot{y}(0).$$

We note that this is also equal to the directional derivative of  $f$  along  $v$  and we have:

$$\frac{df(\gamma(t))}{dt} = \lim_{h \rightarrow 0} \frac{f(p + hv) - f(p)}{h}.$$

In some sense, the speed vector  $v$ , which is the vector tangent to the curve  $\gamma$  at time  $t = 0$ , provides a local linear approximation of the curve (straight line) and we can use it to compute the derivative of  $f$  along  $\gamma$ . We would like to generalize some of these ideas to general manifolds. However, computing local linear approximations of manifolds requires some work because all of the above calculations relied on the fact that our curve was embedded in  $\mathbb{R}^2$  and we were able to use standard calculus rules (e.g. chain rule).

General smooth manifolds are not necessarily embedded in Euclidean space, and we need to define a local linear approximation of manifolds and a notion of tangent vectors that is independent of the representation (embedding) of the manifold and only depend on its intrinsic properties. This definition is going to be built using the concept of directional derivatives, which provide a coordinate-free way to compute derivatives.

**Definition A.2.7** (Directional derivative). *Given a real valued function  $f : M \rightarrow \mathbb{R}$  on a differentiable manifold  $M$ , and a smooth curve  $\gamma(t)$  such that  $\gamma(0) = p \in M$ , the directional derivative of  $f$  at  $p$  with respect to the curve  $\gamma$  is:*

$$V_{p,\gamma}(f) := \left. \frac{d(f \circ \gamma)(t)}{dt} \right|_{t=0}.$$

$V_{p,\gamma}(\cdot)$  is an operator that acts on functions of  $C^\infty(M)$  which satisfies the Leibniz rule, and it can be shown that the space formed by all possible directional derivatives forms a vector space that is isomorphic to  $\mathbb{R}^n$  called the *tangent space*.

**Definition A.2.8** (Tangent space). *Let  $M$  be a smooth manifold and  $p \in M$ . The collection of all directional derivatives at  $p$  forms a vector space with the same dimension as the manifold  $M$  called the tangent space to  $M$  at  $p$  and denoted  $T_p M$ .*

Intuitively, the tangent space  $T_p M$  is the space that contains all the possible directions of curves on  $M$  passing through a point  $p$ . The elements of  $T_p M$  are called *tangent vectors* and the union of all tangent spaces is called the *tangent bundle*.

**Definition A.2.9** (Tangent Bundle). *Given a  $n$ -dimensional smooth manifold  $M$ , the collection of tangent spaces at all point is itself a smooth manifold of dimension  $2n$  called the tangent bundle.*

Since the tangent space is a vector space, we can add and scale directional derivatives to get a new directional derivative. In particular, this gives rise to a local linear approximation of smooth manifolds, which provides the necessary tools for calculus on manifolds.

**Coordinate basis.** Now let's take a closer look at the definition of tangent vectors. If  $(U, \varphi)$  is a chart such that  $p \in U$  and  $\varphi(q) = (x^1(q), \dots, x^n(q))$  for all  $q \in U$ , we have using the multi-variable chain rule:

$$\begin{aligned} V_{p,\gamma}(f) &= \left. \frac{d(f \circ \varphi^{-1} \circ \varphi \circ \gamma)(t)}{dt} \right|_{t=0} \\ &= \sum_{i=1}^n (x^i \circ \gamma)'|_{t=0} \frac{\partial(f \circ \varphi^{-1})}{\partial x^i} |_{\varphi(p)}. \end{aligned}$$

It happens that this decomposition is the directional derivative expressed in a *coordinate basis* of tangent vectors (directional derivatives) which are defined as follows:

$$\left( \frac{\partial}{\partial x^i} \right)_p (f) := \frac{\partial(f \circ \varphi^{-1})}{\partial x^i} (\varphi(p)).$$

These vectors form a natural basis of the tangent space, i.e. any directional derivative along a curve  $\gamma$  at  $p \in M$  can be expressed as:

$$V_{p,\gamma} = \sum_{i=1}^n v^i \left( \frac{\partial}{\partial x^i} \right)_p.$$

**Example A.2.1.** Let's look at the simple case where  $M$  is the Euclidean space  $\mathbb{R}^n$ . Consider a point  $p \in \mathbb{R}^n$ . The chart in this simple manifold are the identity mappings, and we get that the basis vector in the tangent space are simply the partial derivatives in the canonical basis of  $\mathbb{R}^n$ . Therefore, we recover the directional derivative formula in  $\mathbb{R}^n$ :  $V_{p,v}(f) = v \cdot \nabla f(p)$ .

### A.3 Riemannian Manifolds

In the previous section, we described smooth manifolds, which are topological manifolds in which coordinate charts are compatible with each other and vary smoothly. This allowed us to define smooth functions on manifolds, and compute their derivatives using tangent vectors, which are directional derivative operators and form a vector space that is a local linear approximation of the manifold.

We now move onto Riemannian geometry, which provides a way to measure the length of tangent vectors and define important geometric concepts on manifolds. Historically, Riemannian geometry was developed in the 19<sup>th</sup> century by famous mathematicians including Gauss and Riemann, and served as a fundamental building block for the work of Einstein on general relativity and space-time in the 20<sup>th</sup> century.

Broadly speaking, Riemannian manifolds are smooth manifolds equipped with a smooth metric in the tangent space (positive-definite inner product) which can measure the length of tangent vectors, leading to generalization of distances, lengths of curves, angles and volumes on smooth manifolds. In this section, we first start by defining Riemannian metrics and Riemannian manifolds (Appendix A.3.1). We then review how to compute the length of smooth curves, leading to the notion of shortest path or geodesics (Appendix A.3.2). Finally, we describe an important one-to-one mapping from tangent vectors to the manifold via the exponential map (Appendix A.3.3).

#### A.3.1 Definitions

Recall that in Euclidean space, the length of a curve  $\gamma : [0, 1] \rightarrow \mathbb{R}^n$  is given by:

$$L(\gamma) = \int_a^b \|\dot{\gamma}(t)\|_2 dt,$$

where  $\|\cdot\|_2$  is the standard Euclidean norm and  $\dot{\gamma}(t)$  is the velocity of  $\gamma$  which is tangent to the curve. To generalize this formula to manifolds, we need a way to measure the length of tangent vectors, which can be done using Riemannian metrics.

**Definition A.3.1** (Riemannian Manifold). *Given a smooth manifold  $M$ , a Riemannian metric is a smooth function that smoothly assigns to each  $p \in M$  an inner product on the tangent space:*

$$g_p : T_p M \times T_p M \rightarrow \mathbb{R}.$$

A Riemannian manifold is a pair  $(M, g)$  where  $M$  is a smooth manifold and  $g$  is a Riemannian metric.

As we saw before, given a coordinate chart  $(U, \phi)$  around a point  $p$ , we can define a natural basis of the tangent space  $T_p M$  given by the vectors  $\{(\frac{\partial}{\partial x^i})_p, 1 \leq i \leq n\}$ . We adopt the following notation:

$$g_{ij}(p) := g_p\left(\left(\frac{\partial}{\partial x^i}\right)_p, \left(\frac{\partial}{\partial x^j}\right)_p\right).$$

$g_{ij}$  are called the components of the Riemannian metric which can be expressed in this basis as an  $n \times n$  symmetric positive definite matrix called the *Riemannian metric tensor*,  $G(p)$ . The inner product of two tangent vectors  $u, v \in T_p M$  expressed in this basis can then be computed as:  $g_p(u, v) = \sum_{ij} g_{ij}(p) u_i v_j = u^\top G(p) v$ .

**Remark A.3.1.** Saying that  $g$  varies smoothly with  $p$  means that the functions  $g_{ij}$  are smooth.

**Example A.3.1.** The simplest examples of a Riemannian metric is the Euclidean metric where for all  $p \in \mathbb{R}^n$ ,  $g_{ij}(p) = \delta_{ij}$ . In particular, if  $u, v \in \mathbb{R}^d$  are two tangent vectors we get:  $g_p(u, v) = \sum_{ij} g_{ij}(p) u_i v_j = \sum_i u_i v_i = u \cdot v$  which is the standard Euclidean inner product.

Many geometric constructions can be defined on Riemannian manifolds. For instance, the norm of a tangent vector  $v \in T_p M$  is computed as:  $|v|_p := \sqrt{g_p(v, v)}$  while the angle between two nonzero tangent vectors  $u, v \in T_p M$  is the unique  $\theta \in [0, \pi]$  such that:  $\cos(\theta) = \frac{g_p(u, v)}{|u|_p |v|_p}$ .

Depending on the metric's symmetries and properties, these geometric quantities may be easier or harder to compute. In particular, the next proposition states that there exists many possible Riemannian metrics to choose from for a given manifold.

**Proposition A.3.1.** *Every smooth manifold has a Riemannian metric.*

### A.3.2 Geodesics

An important concept in Euclidean geometry is that of straight lines, which are shortest paths between points. We can generalize this concept to manifolds by seeking curves with minimal length, called geodesics.

**Arc Length.** An important fact about Riemannian metric is that they provide a way to compute the length of smooth curves on manifolds. Given a smooth curve segment  $\gamma : [a, b] \rightarrow M$ , the length of  $\gamma$  can be computed as in the Euclidean case as:

$$L(\gamma) = \int_a^b |\dot{\gamma}(t)|_{\gamma(t)} dt,$$

where  $\dot{\gamma}(t)$  is a tangent vector in  $T_{\gamma(t)} M$  which is the speed of the curve  $\gamma$ .

**Remark A.3.2.** An important fact is that this definition of the length is independent of the parametrization of the curve, i.e. if  $\phi : [c, d] \rightarrow [a, b]$  is a diffeomorphism and  $\gamma \circ \phi$  is a reparametrization of  $\gamma$  then  $L(\gamma) = L(\gamma \circ \phi)$

This definition of the length can be extended to general piecewise smooth curves by breaking them down into smooth curve segments and summing up the integrals over each segment. In particular, it allows to define a distance metric (satisfying the metric axioms in Example A.1.2) on Riemannian manifolds  $(M, g)$ , such that for any  $p, q \in M$ :

$$d_M(p, q) = \inf\{L(\gamma) : \gamma \text{ is a piecewise smooth curve from } p \text{ to } q\}.$$

In other words, the distance between two points on a Riemannian manifold is the length of the curve with minimum length joining the two point.

**Remark A.3.3.** It can be shown that the standard metric topology (generated by open balls) induced by this distance metric coincides with the original topology of the manifold  $M$ .

**Geodesics.** Now we may ask about a characterization of curves that achieve the infimum above. The answer is that there exist a certain type of curves on manifolds, which are locally length-minimizing. These curves are called *geodesics* and are broadly speaking the curves with zero acceleration and constant speed. These geodesics can be determined by solving a system of partial differential equations. More specifically, if  $(U, \varphi)$  is a chart around  $p \in M$ , and  $\gamma(t) = (\gamma^1(t), \dots, \gamma^n(t)) \subset U$  is a curve, then  $\gamma$  is a geodesic if it satisfies the *geodesic equations*:

$$\frac{d^2 \gamma^k}{dt^2} = - \sum_{i,j} \Gamma_{ij}^k \frac{d\gamma^i}{dt} \frac{d\gamma^j}{dt}.$$

The symbols  $\Gamma_{ij}^k$  are called *Christoffel symbols* and are computed as:

$$\Gamma_{ij}^k(\gamma(t)) = \frac{1}{2} \sum_l g^{kl} \left( \frac{\partial g_{jl}}{\partial x^i} + \frac{\partial g_{il}}{\partial x^j} - \frac{\partial g_{ij}}{\partial x^l} \right), \quad (\text{A.1})$$

where  $g^{kl}$  are the entries of the inverse metric tensor, which is simply the inverse of the matrix  $G(p)$ .

Geodesics are locally length-minimizing in the sense that if two points  $p, q \in M$  are sufficiently close and connected by a geodesic curve  $\gamma$ , then  $d(p, q) = L(\gamma)$ . Now we may ask if this results holds for any  $p, q$  that are not necessarily close. It can be shown as a consequence of the Hopf-Rinow theorem that if a manifold is geodesically complete (i.e. we can indefinitely extend geodesics) then any two points can be connected by a distance minimizing geodesic.

**Example A.3.2.** In Euclidean space, we saw that the Riemannian metric tensor is simply the identity matrix for all  $p \in \mathbb{R}^n$ . The Christoffel symbols are then simply equal to zero, and the

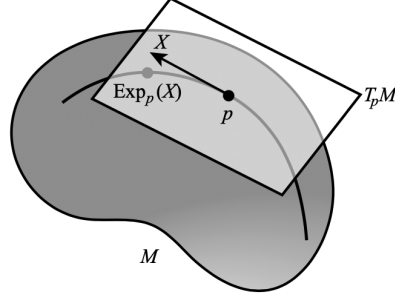


Figure A.4: Illustration of the exponential map on a Riemannian manifold. Source: [71].

geodesic equation becomes:

$$\frac{d^2 \gamma^k}{dt^2} = 0.$$

Therefore, Euclidean geodesics are of the form  $\gamma(t) = t \cdot a + b$ , for  $a, b \in \mathbb{R}^n$ , which are straight lines in  $\mathbb{R}^n$ . Further, the Euclidean space is complete since we can extend straight lines indefinitely in both directions. We can apply the Hopf-Rinow theorem to compute the Euclidean distance between two points which is equal to the length of the unique geodesic (straight line) connecting the points. If  $x, y \in \mathbb{R}^n$ , the straight line connecting the points is  $\gamma(t) = t \cdot (y - x) + x$  and we can use the curve length formula to compute the distance between the points:

$$\begin{aligned} L(\gamma) &= \int_0^1 \sqrt{g_{\gamma(t)}(\dot{\gamma}(t), \dot{\gamma}(t))} dt \\ &= \int_0^1 \sqrt{\sum_{1 \leq i \leq n} (y_i - x_i)^2} dt \\ &= \|x - y\|_2, \end{aligned}$$

which recovers the standard Euclidean distance.

**Example A.3.3.** Let's now take a look at the upper half-space model of hyperbolic geometry  $\mathbb{H}^2 = \{(x, y) \in \mathbb{R}^2 : y > 0\}$ , with the Riemannian metric tensor:

$$G_{(x,y)} = \begin{bmatrix} 1/y^2 & 0 \\ 0 & 1/y^2 \end{bmatrix}.$$

We compute the Christoffel symbols using Eq. (A.1):

$$\begin{aligned}\Gamma_{11}^1 &= \frac{1}{2}g^{11}\left(\frac{\partial g_{11}}{\partial x} + \frac{\partial g_{11}}{\partial x} - \frac{\partial g_{11}}{\partial x}\right) \\ &= \frac{1}{2}g^{11}\frac{\partial(1/y^2)}{\partial x} \\ &= 0.\end{aligned}$$

Similarly, we have:

$$\begin{aligned}\Gamma_{11}^2 &= \frac{1}{2}g^{22}\left(\frac{\partial g_{12}}{\partial x} + \frac{\partial g_{12}}{\partial x} - \frac{\partial g_{11}}{\partial y}\right) \\ &= \frac{1}{2}g^{22}\left(-\frac{\partial(1/y^2)}{\partial y}\right) \\ &= \frac{y^2}{2} \frac{2}{y^3} \\ &= \frac{1}{y}.\end{aligned}$$

Using similar calculations, we can show that  $\Gamma_{22}^1 = \Gamma_{12}^2 = 0$  and  $\Gamma_{22}^2 = \Gamma_{21}^1 = -\frac{1}{y}$ . Therefore we can write the geodesic equation for  $\gamma(t) = (x(t), y(t))$ :

$$\ddot{x} = \frac{2\dot{x}\dot{y}}{y}, \quad \ddot{y} = \frac{\dot{y}^2 - \dot{x}^2}{y}.$$

If  $\dot{x} = 0$ , then  $x(t) = \text{constant}$  and therefore straight lines that are perpendicular to the x-axis are geodesics in this model of hyperbolic space. It can also be shown that half-circles whose origin is on the x-axis are also geodesics.

### A.3.3 The Exponential Map

We end this chapter by reviewing the notion of exponential map, which essentially parameterizes Riemannian manifolds locally in terms of a map from tangent vectors to points on the manifolds (Fig. A.4). This map is called the exponential map and is defined in terms of geodesics.

Recall that a curve is a geodesic if it satisfies the geodesic equations. Given  $p \in M$  and  $v \in T_p M$  sufficiently short, the theory of ordinary differential equations tells us that there exists exactly one geodesic  $\gamma$  defined on the interval  $[0, 1]$  such that  $\gamma(0) = p$  and  $\dot{\gamma}(0) = v$ . This geodesic is used to define the exponential map as follows.

**Definition A.3.2** (Exponential map). *Let  $(M, g)$  be a Riemannian manifold. The exponential map is the map  $\exp_p : T_p M \rightarrow M$  such that  $\exp_p(v) = \gamma(1)$  where  $\gamma$  is the unique geodesic of  $M$  which, at the instant  $t = 0$ , passes through  $p$  with velocity  $v$ .*

**Remark A.3.4.** For general Riemannian manifolds, the exponential map may only be defined in a small neighborhood around zero. In the case of complete manifolds such as the Euclidean or hyperbolic spaces, the exponential map is defined everywhere.

**Theorem A.3.1.** *Given a Riemannian manifold  $(M, g)$  and a point  $p \in M$ , the exponential map  $\exp_p$  is a diffeomorphism from some open ball in  $T_p M$  to some open ball in  $M$ .*

This theorem implies that we can define an inverse mapping which maps points on the manifold to the tangent space, called the logarithmic map. The exponential and logarithmic maps are very important for computational computations on manifolds, for instance they can be used to optimize functions on manifolds using Riemannian gradient descent [26], or generalize dimensionality reduction to Riemannian manifolds [70].

## A.4 Hyperbolic Geometry

Having introduced the basics of Riemannian geometry, we now move on to the main topic of this dissertation, hyperbolic geometry. We describe the hyperboloid model of hyperbolic space which can be embedded in Euclidean space and allows for nice extrinsic computations. One of the main advantages of this model is its similarity with spherical geometry. At a high-level, it can be thought of as an  $n$ -dimensional sphere with unit imaginary radius in Minkowski space. In particular, the formulas for geodesic and distances share a lot in common with formulas in spherical geometry, except that the trigonometric functions are replaced with hyperbolic functions.

We first start by describing hyperbolic spaces as Minkowski spaces to draw analogies with spherical geometry (Appendix A.4.1). We then review the Riemannian geometry of hyperbolic spaces (Appendix A.4.2) and describe hyperbolic geodesics and corresponding exponential and logarithmic maps (Appendix A.4.3). Finally, we discuss the connexions between the hyperboloid model and the Poincaré model of hyperbolic space (Appendix A.4.4).

### A.4.1 Minkowski Spaces

We first describe *Minkowski spaces*, which is the embedding space where the hyperboloid model sits in as hypersurfaces. The  $(n + 1)$ -dimensional Minkowski space  $\mathbb{R}^{1,n}$  is like the flat Euclidean space  $\mathbb{R}^{1+n}$  except with a dot product that has a negative sign in the first coordinate. More precisely,  $\mathbb{R}^{1,n}$  is the vector space  $\mathbb{R}^{1+n}$  equipped with the indefinite, non-degenerate bilinear form:

$$\langle (x_0, x_1, \dots, x_n), (y_0, y_1, \dots, y_n) \rangle_L = -x_0 y_0 + \sum_{i=1}^n x_i y_i,$$

which serves as the “dot product”.

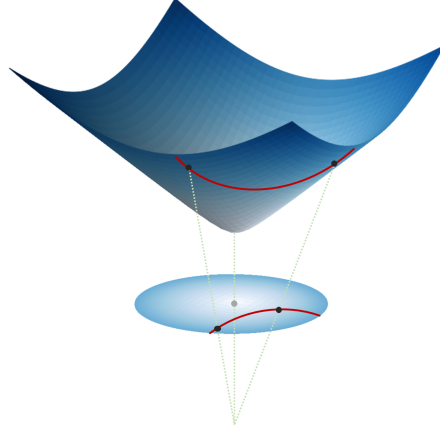


Figure A.5: Illustration of the hyperboloid (top) and its connection to the Poincaré disk (bottom).

Like with Euclidean spaces, the quantity  $\langle \vec{x}, \vec{x} \rangle_L$  is called the (*Minkowski*) *squared norm* of the vector  $\vec{x}$ . Two vectors  $\vec{x}, \vec{y} \in \mathbb{R}^{1,n}$  are called *orthogonal* if  $\langle \vec{x}, \vec{y} \rangle_L = 0$ . The *orthogonal complement* of a linear subspace  $V \subset \mathbb{R}^{1,n}$  is the set  $V^\perp = \{ \vec{z} \in \mathbb{R}^{1,n} : \langle \vec{z}, \vec{x} \rangle_L = 0 \text{ for every } \vec{x} \in V \}$ , which still has dimension  $\dim \mathbb{R}^{1,n} - \dim V$ . However, unlike in Euclidean spaces, vectors in  $\mathbb{R}^{1,n}$  can have negative or zero squared norms, and linear subspaces can intersect their orthogonal complements.

**Types of vectors in  $\mathbb{R}^{1,n}$**  Vectors with negative, zero, and positive (Minkowski) squared norms are called *time-like*, *light-like*, and *space-like*, respectively. Time-like and light-like vectors together form a solid double cone in  $\mathbb{R}^{1,n}$ . If  $\vec{x}$  is a time-like or light-like vector, we call it *future-pointing* if its first coordinate is positive, otherwise we call it *past-pointing*. It follows from Cauchy-Schwarz inequality that.

**Proposition A.4.1.** *If  $\vec{x}, \vec{y}$  are future-pointing time-like or light-like vectors then  $\langle \vec{x}, \vec{y} \rangle_L < 0$ .*

A linear subspace  $V$  of  $\mathbb{R}^{1,n}$  is called *space-like* if every non-zero vector in  $V$  is space-like. In that case, the bilinear form  $\langle \cdot, \cdot \rangle_L$  restricts to a positive-definite bilinear form on  $V$ , thus making it isometric to an Euclidean vector space. It follows from Proposition A.4.1 that:

**Proposition A.4.2.** *The orthogonal complement of a time-like vector is a space-like linear subspace.*

We are now ready to introduce the hyperboloid model  $\mathbb{H}^n$ . It sits inside  $\mathbb{R}^{1,n}$  in a similar way to how the unit sphere  $\mathbb{S}^n$  sits inside the Euclidean space  $\mathbb{R}^{n+1}$ .

**Definition A.4.1.** *The hyperboloid model of  $n$ -dimensional hyperbolic spaces is the set  $\mathbb{H}^n$  of future-pointing vectors in  $\mathbb{R}^{1,n}$  with Minkowski squared norm  $-1$ :*

$$\mathbb{H}^n := \{ y \in \mathbb{R}^{1,n} : \langle y, y \rangle_L = -1, y_0 > 0 \}.$$

In two dimensions, the hyperboloid can be visualized as a surface in  $\mathbb{R}^3$  which corresponds to the upper sheet of a two sheet hyperboloid of revolution (Fig. A.5).

#### A.4.2 The Hyperbolic Space as a Riemannian Manifold

We now describe the geometry of hyperbolic spaces, starting with charts and tangent spaces, and then introducing a Riemannian metric in the tangent space.

**Charts** Recall that manifolds are sets that locally look Euclidean and in which we can find local mappings from the manifold to the Euclidean space, called charts. A point in  $\mathbb{H}^n$  can be parametrized via  $(\sqrt{1 + \|x\|_2^2}, x) \in \mathbb{H}^n$  for  $x \in \mathbb{R}^n$ . In fact, this parameterization yields a very nice global chart  $\varphi$  (diffeomorphism) from  $\mathbb{H}^n$  to  $\mathbb{R}^n$  such that  $\varphi(y_0, y_1, \dots, y_n) = (y_1, \dots, y_n)$  and with inverse:

$$\begin{aligned} \varphi^{-1}: \mathbb{R}^n &\longrightarrow \mathbb{H}^n \\ x &\longmapsto (\sqrt{1 + \|x\|_2^2}, x_1, \dots, x_n). \end{aligned}$$

**Tangent space** Using this global chart representation, let's take a look at tangent vectors. Recall that for  $p = (p_0, p_1, \dots, p_n) \in \mathbb{H}^n$  and a chart  $(U, \varphi)$  such that  $p \in U$ , we can compute a basis  $\{(\frac{\partial}{\partial x_i})_p, 1 \leq i \leq n\}$  of the tangent space  $T_p \mathbb{H}^n$  via:

$$\left(\frac{\partial}{\partial x_i}\right)_p(f) := \frac{\partial(f \circ \varphi^{-1})}{\partial x_i}(\varphi(p)),$$

where  $f$  is any smooth function on  $\mathbb{H}^n$ . Using the global chart on  $\mathbb{H}^n$  from above, we get that basis tangent vectors are:

$$\begin{aligned} \left(\frac{\partial}{\partial x_i}\right)_p(f) &= \frac{\partial f(\sqrt{1 + \|x\|_2^2}, x_1, \dots, x_n)}{\partial x_i} \Big|_{\varphi(p)} \\ &= \left(\frac{\partial \sqrt{1 + \|x\|_2^2}}{\partial x_i}\right) \Big|_{\varphi(p)} \left(\frac{\partial f}{\partial y_0}\right) \Big|_p + \left(\frac{\partial f}{\partial y_i}\right) \Big|_p \quad (\text{Multi-variable chain rule}) \\ &= \frac{p_i}{\sqrt{1 + \|p\|_2^2}} \left(\frac{\partial}{\partial y_0}\right)_p(f) + \left(\frac{\partial}{\partial y_i}\right)_p(f). \end{aligned}$$

Viewing  $f$  as a function on  $\mathbb{R}^{n+1}$ , the partial derivatives  $\frac{\partial}{\partial y_i}$  can be canonically identified with the basis vectors of  $\mathbb{R}^{n+1}$  ( $e_0, e_1, \dots, e_n$ ). Therefore, we can rewrite the basis vectors of  $T_p \mathbb{H}^n$  as:

$$\left(\frac{\partial}{\partial x_i}\right)_p = \frac{p_i}{\sqrt{1 + \|p\|_2^2}} e_0 + e_i \quad \forall 1 \leq i \leq n.$$

In other words, the tangent space  $T_p\mathbb{H}^n$  is the vector space:

$$T_p\mathbb{H}^n = \left\{ (v_0, v) \in \mathbb{R}^{n+1} : v_0 = \frac{v \cdot p}{\sqrt{1 + \|p\|_2^2}} \right\}.$$

This can be equivalently written as:  $T_p\mathbb{H}^n = \{u \in \mathbb{R}^{n+1} : \langle u, p \rangle_L = 0\}$ . Therefore we have the following properties of  $\mathbb{H}^n$ .

**Proposition A.4.3.** *For every  $p \in \mathbb{H}^n$ , the tangent space  $T_p\mathbb{H}^n$  of  $\mathbb{H}^n$  at  $p$  is (parallel to) the orthogonal complement of  $p$  in  $\mathbb{R}^{1,n}$ .*

Another property that further illustrates the analogy with spheres in Euclidean spaces is the following.

**Proposition A.4.4.** *Every  $k$ -dimensional geodesic submanifold of  $\mathbb{H}^n$  is the intersection of  $\mathbb{H}^n$  with a  $(k+1)$ -dimensional linear subspace of  $\mathbb{R}^{1,n}$ .*

$T_p\mathbb{H}^n$  provides a local, first-order linear approximation of the hyperbolic manifold at  $p$ . We can add and scale vectors in  $T_p\mathbb{H}^n$  and this linear approximation which show to be very handy when developing algorithms in hyperbolic spaces.

**Riemannian metric** The Minkowski inner product naturally induces a Riemannian metric  $g$  in the hyperbolic space  $\mathbb{H}^n$  which makes  $(\mathbb{H}^n, g)$  a Riemannian manifold of dimension  $n$  with constant negative curvature  $-1$ .

**Proposition A.4.5.** *When restricted to each tangent space of  $\mathbb{H}^n$ , the bilinear form  $\langle \cdot, \cdot \rangle_L$  is positive definite. This defines a Riemannian metric on  $\mathbb{H}^n$  which has constant curvature  $-1$ .*

Given  $p \in \mathbb{H}^n$  and two tangent vectors  $(u_0, u), (v_0, v) \in T_p\mathbb{H}^n$ , their inner product can be computed as:

$$g_p((u_0, u), (v_0, v)) = -u_0^2 v_0^2 + u \cdot v.$$

We let the reader verify that this is indeed a Riemannian metric, i.e. positive definite on  $T_p\mathbb{H}^n$ , and we denote  $\|v\|_L = \sqrt{\langle v, v \rangle_L}$  the norm of a tangent vector  $v \in T_p\mathbb{H}^n$ .

To compute the Riemannian metric tensor in matrix form, we need to compute its component in the coordinate basis  $\{(\frac{\partial}{\partial x_i})_p, 1 \leq i \leq n\}$  which we derive from the global coordinate chart  $\varphi$ . We have:

$$\begin{aligned} g_{ij} &= g_p \left( g_p \left( \frac{\partial}{\partial x_i} \right)_p, \left( \frac{\partial}{\partial x_j} \right)_p \right) \\ &= g_p \left( \frac{p_i}{\sqrt{1 + \|p\|_2^2}} e_0 + e_i, \frac{p_j}{\sqrt{1 + \|p\|_2^2}} e_0 + e_j \right) \\ &= -\frac{p_i p_j}{1 + \|p\|_2^2} + \delta_{ij}, \end{aligned}$$

where  $\delta_{ij}$  is the Kronecker delta.

Therefore in matrix form we have:  $G(p) = I_n - \frac{pp^\top}{1+\|p\|_2^2}$ . Using the Sherman–Morrison formula, we get that the inverse metric tensor is  $I_n + pp^\top$  and therefore  $g^{kl} = \delta_{kl} + p_k p_l$ .

### A.4.3 Hyperbolic Geodesics

We now move onto the notion of geodesics in hyperbolic spaces, which are length-minimizing curves. In Euclidean space, the shortest path connecting any two points is simply the straight line connecting them. In hyperbolic spaces, this is no longer true; for instance in Fig. A.5, connecting two points with a straight line yields a path that may not even be on the manifold.

To find geodesics in  $\mathbb{H}^n$ , we need to compute Christoffel symbols and solve a system of partial differential equations called geodesic equations (see Appendix A.3.2). In this model of hyperbolic space,  $g_{ij} = -\frac{p_i p_j}{1+\|p\|_2^2} + \delta_{ij}$ . Computing the Christoffel symbols in this model quickly becomes complicated, and we can instead use another model of hyperbolic space, the upper half-space model in which these computations become much more tractable (see Example A.3.3). We can then use the isometric mapping from the upper half-space to the hyperboloid to recover hyperbolic geodesics. In particular, we have the following results.

**Geodesics and distances** Let  $p \in \mathbb{H}^n$  and  $u \in T_p \mathbb{H}^n$ , and assume that  $u$  is unit-speed, *i.e.*  $\langle u, u \rangle_L = 1$ . The unique unit-speed geodesic  $\gamma_{p,u}(\cdot)$  such that  $\gamma_{p,u}(0) = p$ ,  $\dot{\gamma}_{p,u}(0) = u$  is given by:

$$\gamma_{x,u}(t) = \cosh(t)x + \sinh(t)u.$$

The corresponding intrinsic distance between two points  $(x, y) \in \mathbb{H}^n$  can be computed as:

$$d_{\mathbb{H}}(x, y) = \operatorname{arcosh}(-\langle x, y \rangle_L).$$

**Exponential and Logarithmic Maps** Given  $x \in \mathbb{H}^n$  and a tangent vector  $v \in T_x \mathbb{H}^n$ , the exponential map  $\exp_x : T_x \mathbb{H}^n \rightarrow \mathbb{H}^n$  assigns to  $v$  the point  $\exp_x(v) := \gamma(1)$ , where  $\gamma$  is the unique geodesic satisfying  $\gamma(0) = x$  and  $\dot{\gamma}(0) = v$ . The logarithmic map is the reverse map that maps back to the tangent space at  $x$  such that  $\log_x(\exp_x(v)) = v$ .

In general Riemannian manifolds, these operations are only defined locally but in the hyperbolic space, they form a bijection between the entire hyperbolic space and the tangent space at a point. We have the following direct expressions of the exponential and the logarithmic maps:

$$\exp_x(v) = \cosh(\|v\|_L)x + \sinh(\|v\|_L)\frac{v}{\|v\|_L}, \quad \log_x(y) = d_L(x, y)\frac{y + \langle x, y \rangle_L x}{\|y + \langle x, y \rangle_L x\|_L}.$$

These maps are be very useful when designing neural networks and algorithms that operate in hyperbolic spaces, as they can be used to map points to tangent spaces (which are vector space that

are isomorphic to  $\mathbb{R}^n$  and in which standard operations like addition and multiplication are defined) and then map them back to the manifold.

#### A.4.4 Connexions with the Poincaré Ball Model

Another important model of hyperbolic space in the Poincaré ball model described in Section 1.3.1. In two dimensions, this model is extremely useful to visualize data because it sits in the unit disk. Historically, the Poincaré model has been used for artistic realizations. Escher explored the concept of representing infinity in a two-dimensional disk in his circle limit illustration made in woodcut Fig. 1.6c.

This model of hyperbolic space is isomorphic to the hyperboloid model (Fig. A.5), and the *stereographic projection*

$$(x_0, x_1, \dots, x_n) \rightarrow \left( \frac{x_1}{1+x_0}, \dots, \frac{x_n}{1+x_0} \right)$$

is an isometry between  $\mathbb{H}^n$  and the Poincaré model  $\mathbb{B}^n$ . Its inverse map is given by

$$(y_1, \dots, y_n) \rightarrow \frac{(1 + \sum_i y_i^2, 2y_1, \dots, 2y_n)}{1 - \sum_i y_i^2}.$$

This yields geodesics that are either straight lines that go through the origin of the ball, or segments of circles that are perpendicular to the boundary of the ball (Fig. 1.6a).

## Appendix B

# Deferred Proofs of Chapter 5

### B.1 Preliminaries

We first introduce our notational conventions in Appendix B.1.1 and then review some useful results of hyperbolic metrics that will be used throughout our proofs in Appendix B.1.2.

#### B.1.1 Notation

**Binary trees** A *binary tree* is one that has all degrees either 1 (a leaf node) or 3 (internal node).<sup>1</sup> A binary tree with  $n$  leaves has exactly  $n - 1$  internal nodes (of degree 3). A *rooted binary tree* is a binary tree such that one of its internal nodes (i.e. non-leaf node), the root, has degree exactly 2. Note that if we think of one leaf in a binary tree as the root, then removing it (and letting its unique neighbor be the new root) converts this into a *rooted binary tree* in the traditional sense (See Fig. B.2c and Fig. B.2d for examples of rooted and unrooted binary trees). By convention, we let  $T$  denote any binary tree with leaves  $1, \dots, n$  and root 0 if its rooted.

**Tree and hyperbolic metrics** With  $i, j, k$ , we refer to nodes in  $T$ , and  $z_i, z_j, z_k$  correspond to points in a hyperbolic embedding  $Z = \{z_1, \dots, z_n\} \subset \mathbb{B}_2^n$ . With  $o$  or  $z_0$ , we denote the origin of hyperbolic space and  $d_{\mathbb{B}_2}$  is the hyperbolic metric; every tree  $T$  defines a tree metric  $d_T$ . We overload  $d(i, j) := d_T(i, j)$  and  $d(z_i, z_j) := d_{\mathbb{B}_2}(z_i, z_j)$  when the types are clear. There is a close correspondence between hyperbolic and tree metrics, and we define the notion of quasi-isometries, which we will use throughout our proofs.

**Definition B.1.1** (Quasi-isometric embedding). *Let  $T$  be a binary tree on  $(n + 1)$  leaves and*

---

<sup>1</sup>We use undirected trees in our proofs.

$Z = \{z_0, \dots, z_n\}$  an embedding set. The pair  $(Z, T)$  is  $(1 + \varepsilon, \kappa)$ -quasi-isometric if:

$$d(i, j) \leq d(z_i, z_j) \leq (1 + \varepsilon)d(i, j) + \kappa,$$

for all  $0 \leq i, j \leq n$ .

**Remark B.1.1.** Note that Definition B.1.1 is equivalent to saying there is a quasi-isometric embedding from  $\{z_0, \dots, z_n\}$  to the root and leaves of  $T$ , i.e. the two metrics agree up to a linear transform. Compared to the usual definition of quasi-isometry, we consider only a one-sided version for convenience in the proofs.

**Node depth** We overload  $d_0(i) = d_T(0, i)$  to refer to the distance from node  $i$  to the root in  $T$ , and  $d_o(z_i) = d_{\mathbb{B}_2}(z_0, z_i)$  to be the distance to the origin in  $\mathbb{B}_2$ . Intuitively,  $d_0(\cdot)$  and  $d_o(\cdot)$  represent the “depth” of a node or point. Note that the depths are dependent on the choice of a base point as the root of a tree or origin of the space. We always use  $o$  for  $\mathbb{B}_2$  and the node indexed by 0 for trees.

**LCA** We let  $i \vee j$  denote the LCA of two leaf nodes in  $T$ , and  $z_i \vee z_j$  denote the hyperbolic LCA defined in Eq. (5.6). In particular, we say that  $\{i, j|k\}_T$  holds if the LCA of  $(i, j)$  has a larger depth than that of  $(i, k)$  and  $(j, k)$ , i.e.  $d_0(i \vee j) \geq \max\{d_0(i \vee k), d_0(j \vee k)\}$ . Similarly, we say that  $\{z_i, z_j|z_k\}_{\mathbb{B}_2}$  holds if  $d_o(z_i \vee z_j) \geq \max\{d_o(z_i \vee z_k), d_o(z_j \vee z_k)\}$ . We overload  $\{i, j|k\}_T = \{i, j|k\}$  and  $\{z_i, z_j|z_k\}_{\mathbb{B}_2} = \{z_i, z_j|z_k\}$  when the types are clear.

### B.1.2 Hyperbolic LCA and the Gromov product

We review two useful results of  $\delta$ -hyperbolic metrics which we will use throughout our proofs in the next section. First, our notion of LCA depth is closely related to the Gromov product (see Definition 1.3.1 in Chapter 1), which is exactly the tree depth for 0-hyperbolic metrics (Lemma B.1.1). Second, any finite set of points in  $\mathbb{B}_2$  can be embedded in a binary tree (Proposition B.1.1):

**Lemma B.1.1** (LCA depth is close to the Gromov product). *For any  $i, j \in T$ ,*

$$d_0(i \vee j) = \langle i, j \rangle.$$

*There exists  $\delta > 0$  such that for any  $z_i, z_j \in \mathbb{B}_2$ ,*

$$\langle z_i, z_j \rangle \leq d_o(z_i \vee z_j) \leq \langle z_i, z_j \rangle + \delta.$$

*Proof.* This is a direct application of Lemma 6.1 and 6.2 in [28] to tree metrics (which are 0-hyperbolic) and to the hyperbolic space  $\mathbb{B}_2$  (which is  $\delta$ -hyperbolic).<sup>2</sup>  $\square$

<sup>2</sup>Note that the notion of  $\delta$ -hyperbolicity in [28] uses the incenter condition, which is equivalent to the four point

**Proposition B.1.1** (Tree-likeness of hyperbolic space). *There is a constant  $C_n$  such that for any set of points  $\{z_0, z_1, \dots, z_n\} \subset \mathbb{B}_2^{n+1}$ , there is a binary tree  $T$  on leaves  $0, 1, \dots, n$  such that:*

$$\forall 0 \leq i, j \leq n : d_T(i, j) \leq d_{\mathbb{B}_2}(z_i, z_j) \leq d_T(i, j) + C_n, \quad (\text{B.1})$$

with  $C_n = \delta \cdot O(n)$ .

*Proof.* The statement of Proposition B.1.1 without the binary or leaf condition is a standard result which is Proposition 1.3.1 in Chapter 1 (See Proposition 6.7 in [28] for a proof). That is, there exists a tree  $T$  with  $n$  nodes (not necessarily binary) satisfying Eq. (B.1). We modify  $T$  to satisfy the leaf constraint (i.e.  $\{z_0, z_1, \dots, z_n\}$  are leaves' embeddings) and to satisfy the binary condition (i.e.  $T$  is binary in the sense that every node has degree 1 or 3).

**Leaf condition** Let  $k \in [n]$  be a node in  $T$  and  $m$  be the minimum edge length in  $T$ . If  $k$  has degree greater than 1 (i.e.  $k$  is not a leaf node), then we shrink every edge connected to it by some constant  $c < \min\{\delta, \frac{m}{n}\}$ , create a dummy node  $p$  in place of  $k$  and connect  $k$  to it (Fig. B.1a). Now all tree distances involving  $k$  are the same, and all other distances going through  $p$  are shrunk by at most  $2c$ . The resulting tree  $T'$  has  $n$  leaves and is such that:

$$\forall 0 \leq i, j \leq n : d_{T'}(i, j) \leq d_T(i, j) \leq d_{T'}(i, j) + 2c.$$

**Binary condition** Next, we modify  $T'$  to be binary. For every node of degree 2, simply delete it, which does not affect the tree metric restricted to  $[n]$  (we could not have deleted a node in  $[n]$  since they are all leaves now). For every node of degree 4 or more, we replace it by multiple copies connected by edges of length  $c$  and decrease original edges by  $c$  every time we create a copy of the original node (Fig. B.1b), which causes the distances to shrink by at most  $2c(n-1)$  (case of star trees). Therefore, distances in this new binary tree  $T''$  satisfy:

$$\forall 0 \leq i, j \leq n : d_{T''}(i, j) \leq d_{T'}(i, j) \leq d_{T''}(i, j) + 2c(n-1).$$

With this final binary tree on  $(n+1)$  leaves  $T''$ , we have:

$$\forall 0 \leq i, j \leq n : d_{T''}(i, j) \leq d_{\mathbb{B}}(z_i, z_j) \leq d_{T''}(i, j) + C_n + 2c + 2c(n-1).$$

Thus the statement holds for  $C'_n = C_n + 2cn$ , which is still  $\delta \cdot O(n)$  since  $c < \delta$ .  $\square$

---

condition (Definition 1.3.2), up to changing  $\delta$  by a constant multiple. In standard hyperbolic space, Lemma B.1.1 is in fact satisfied for  $\delta = \log 3$ .

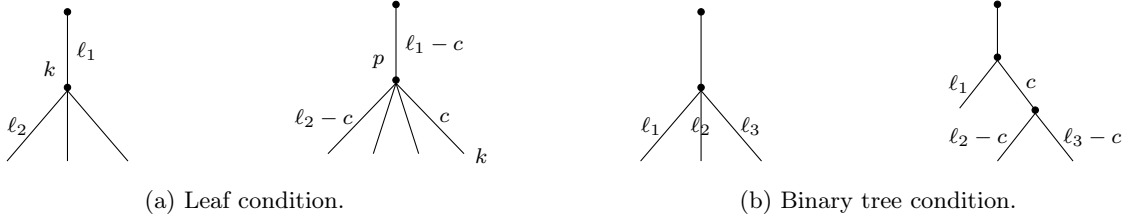


Figure B.1: Tree transformations used to satisfy the leaf and the binary conditions in Proposition B.1.1

**Remark B.1.2.** Using the definition of quasi-isometries (Definition B.1.1), Proposition B.1.1 implies that for any embedding  $Z \in \mathbb{B}_2^n$ , there exists a binary tree  $T$  on  $n$  leaves such that there is a  $(1, C_n)$ -quasi-isometry from the leaves of  $T$  to  $Z$ .

## B.2 Proof of Theorem 5.4.1

Our goal in this section is to show our main result, Theorem 5.4.1, which gives a  $(1+\varepsilon)$ -approximation ratio for Dasgupta's discrete objective. We first give an overview of the strategy used to show Theorem 5.4.1 and state the two results used in the proof (Lemma B.2.1 and Lemma B.2.2) in Appendix B.2.1. We then go over the details of Lemma B.2.1 in Appendix B.2.2 and Lemma B.2.2 in Appendix B.2.3.

### B.2.1 Proof Outline

Our proof of Theorem 5.4.1 relies on two main results; the existence of a constrained embedding, the spread embedding set (Definition 5.4.1), such that the rooted binary tree decoded from any embedding in that set has a discrete cost close to the continuous cost (Lemma B.2.1), and reciprocally, any rooted binary tree has a corresponding embedding in that set (Lemma B.2.2).

First, we define the constrained spread embedding set and give more precise conditions on the spread constants, which we'll use throughout the proof.

**Definition B.2.1** (Spread Embeddings). *An embedding  $Z \in \mathbb{B}_2^n$  is called spread if for every triplet  $(i, j, k)$ :*

$$\max\{d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} - \min\{d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} > 3C_n + 2\delta + 1, \quad (\text{B.2})$$

where  $\delta$  is Gromov's delta hyperbolicity (Lemma B.1.1) and  $C_n = \delta \cdot O(n)$  is defined in Proposition B.1.1.

The spread constraints force LCAs to be distinguishable from each other, which intuitively encourages binary tree metrics. Now using this definition, we show that any spread embedding

decoded using Algorithm 1 returns a tree that has a discrete cost close to the embeddings' continuous cost.

**Lemma B.2.1.** *Let  $Z \in \mathcal{Z} \subset \mathbb{B}_2^n$  be a spread embedding. Then:*

$$|C_{\text{Dasgupta}}(\text{dec}(Z); w) - C_{\text{HYPHC}}(Z; w, \tau)| \leq 4e^{-1/\tau} \sum_{ijk} \max\{|w_{ij}|, |w_{ik}|, |w_{jk}|\}$$

We then show that any rooted binary tree has a corresponding spread embedding that decodes to it.

**Lemma B.2.2.** *For any unit-weight rooted binary tree  $T$  on  $n$  leaves, there exists a spread embedding  $Z \in \mathcal{Z} \subset \mathbb{B}_2^n$  such that  $\text{dec}(Z) = T$ .*

These two Lemmas show the tight equivalence between our continuous  $C_{\text{HYPHC}}$  cost on spread embeddings, and the discrete Dasgupta cost  $C_{\text{Dasgupta}}$  on rooted binary trees. Finally, putting these together, we show our main result which is that the discrete tree returned by HYPHC has a  $(1 + \varepsilon)$ -approximation factor for Dasgupta's minimum (Theorem 5.4.1).

**Theorem 5.4.1.** *Consider a dataset with  $n$  datapoints and pairwise similarities  $(w_{ij})$  and let  $T^*$  be the solution of Eq. (5.2). Let  $\mathcal{Z}$  be the set of spread embeddings and  $Z^* \in \mathcal{Z}$  be the solution of Eq. (5.3) for some  $\tau > 0$ . For any  $\varepsilon > 0$ , if  $\tau \leq \mathcal{O}(1/\log(1/\varepsilon))$ , then:*

$$\frac{C_{\text{Dasgupta}}(\text{dec}(Z^*); w)}{C_{\text{Dasgupta}}(T^*; w)} \leq 1 + \varepsilon. \quad (5.13)$$

*Proof.* Let  $\mathcal{Z} \subset \mathbb{B}_2^n$  be the set of spread leaves' embeddings embeddings (Definition B.2.1),  $\tau > 0$  and:

$$\begin{aligned} T^* &= \text{argmin}_T C_{\text{Dasgupta}}(T; w) \\ Z^* &= \text{argmin}_{Z \in \mathcal{Z}} C_{\text{HYPHC}}(Z; w, \tau). \end{aligned}$$

WLOG, assume that all edges in  $T^*$  have unit weight.<sup>3</sup> Since  $T^*$  is a unit-weight rooted binary, we can apply Lemma B.2.2 to find  $Z \in \mathcal{Z}$  such that  $\text{dec}(Z) = T^*$ .

Next, let  $\Delta := C_{\text{Dasgupta}}(\text{dec}(Z^*); w) - C_{\text{Dasgupta}}(T^*; w)$ . We have:

$$\begin{aligned} 0 \leq \Delta &\leq C_{\text{Dasgupta}}(\text{dec}(Z^*); w) - C_{\text{HYPHC}}(Z^*; w, \tau) + C_{\text{HYPHC}}(Z; w, \tau) - C_{\text{Dasgupta}}(T^*; w) \\ &= C_{\text{Dasgupta}}(\text{dec}(Z^*); w) - C_{\text{HYPHC}}(Z^*; w, \tau) + C_{\text{HYPHC}}(Z; w, \tau) - C_{\text{Dasgupta}}(\text{dec}(Z); w) \\ &\leq 2 \sup_{Z \in \mathcal{Z}} |C_{\text{Dasgupta}}(\text{dec}(Z); w) - C_{\text{HYPHC}}(Z; w, \tau)| \\ &\leq 8 e^{-1/\tau} \sum_{ijk} \max\{|w_{ij}|, |w_{ik}|, |w_{jk}|\}. \end{aligned}$$

---

<sup>3</sup>Any weighted version of  $T^*$  would achieve the same Dasgupta cost, so we consider the unit weight case for simplicity.

The first inequality follow from the fact that  $C_{\text{HYPHC}}(Z^*; w, \tau) \leq C_{\text{HYPHC}}(Z; w, \tau)$  since  $Z^*$  is the minimizer, and the last inequality uses Lemma B.2.1. Then:

$$\frac{C_{\text{Dasgupta}}(\text{dec}(Z^*); w)}{C_{\text{Dasgupta}}(T^*; w)} \leq 1 + 8 e^{-1/\tau} \left( \frac{\sum_{ijk} \max\{w_{ij}, w_{ik}, w_{jk}\}}{C_{\text{Dasgupta}}(T^*; w)} \right). \quad (\text{B.3})$$

Since  $C_{\text{Dasgupta}}(T^*; w) \geq \sum_{ijk} \min\{w_{ij} + w_{ik}, w_{ij} + w_{jk}, w_{ik} + w_{jk}\} + 2 \sum_{ij} w_{ij}$ , we finally have:

$$\frac{C_{\text{Dasgupta}}(\text{dec}(Z^*); w)}{C_{\text{Dasgupta}}(T^*; w)} \leq 1 + \mathcal{O}(e^{-1/\tau}). \quad (\text{B.4})$$

□

**Remark B.2.1.** At a high level, this bound suggests that a better approximation of the argmax function (lower  $\tau$ ) gives a better approximation for Dasgupta's discrete objective.

**Remark B.2.2.** The optimization set  $\mathcal{Z}$  defines a local constraint on every triplet  $z_i, z_j, z_k$  (Definition 5.4.1), and can be enforced on triplets concurrently with sampling them for the main loss (Eq. (5.11)). The max and min operations can be relaxed using softmax, and the separation can be enforced with any auxiliary constraint (e.g. hinge loss) .

## B.2.2 Proof of Lemma B.2.1

The goal of this section is to show Lemma B.2.1. We first introduce a notion of LCA agreement (Section B.2.2.1). We show any spread embedding is in LCA agreement with some unrooted binary tree  $T$ , and gets decoded into the rooted version of  $T$  (Lemma B.2.3 in Section B.2.2.2). This then allows us to show Lemma B.2.1 in Section B.2.2.3.

### B.2.2.1 LCA Agreement

We seek to understand when the continuous HYPHC cost of an embedding is close to the discrete cost of the tree decoded with Algorithm 1. The discrete and continuous costs both depend on the ordering of triplets, in terms of their LCAs' depths (i.e. distances to the origin or root). If this ordering is the same in the continuous embedding space and in the decoded discrete tree, then the costs should be close, up to the continuous approximation error of the softmax function. This motivates us to introduce the notion of LCA agreement.

**Definition B.2.2.** Given a metric space  $(X, d)$ , and binary operation  $\vee : X \times X \rightarrow X$ , we define the set of LCA triplets on any points  $x_1, \dots, x_n \in X$  with respect to the base point  $x_0$  to be

$$\{\{i, j|k\}, i, j, k \in [n] : d(x_0, x_i \vee x_j) > \min\{d(x_0, x_i \vee x_k), d(x_0, x_j \vee x_k)\}\}.$$

**Definition B.2.3** (LCA agreement). *We say two sets of points  $x_1, \dots, x_n \in X^n$  and  $y_1, \dots, y_n \in Y^n$  (possibly in different metric spaces) are in LCA agreement or LCA equivalent with respect to  $x_0$  and  $y_0$ , if their set of LCA triplets are identical.*

Note that LCA agreement is a relative notion and depends on the base point. In our proofs, we always refer to LCA agreement as being with respect to the origin  $z_0 = o \in \mathbb{B}_2$  for embeddings, and with respect to the node labelled 0 for trees.

Intuitively, LCA agreement implies that the LCAs of an embedding are consistent with a ground truth tree. Since our proposed decoding algorithm only relies on LCA distances, we can show that if there exist a unrooted binary tree  $T$  such that  $(Z, T)$  are in LCA agreement, then  $\text{dec}(Z)$  recovers the rooted version of  $T$ .

**Lemma B.2.3.** *Let  $Z$  be an embedding and  $T$  be a binary tree on  $(n+1)$  leaves (not rooted) such that  $(Z \cup \{z_0 = o\}, T)$  are in LCA agreement. Let  $T'$  be the rooted binary tree on  $n$  leaves that is obtained by removing the leaf 0 from  $T$ . Then  $\text{dec}(Z) = T'$ .*

*Proof.* To be a little more formal, define  $\text{dec}(\cdot)$  to depend on three things: a set of points ( $Z$  or  $T$ ), a base point ( $z_0 = o$  or 0), and a LCA construction ( $\vee_{\mathbb{B}_2}$  or  $\vee_T$ ).

Let  $T'$  be the rooted binary tree that is obtained by removing the leaf 0 from  $T$ , and relabelling its neighbor  $r$  in  $T$  as 0, the root of  $T'$  (see Fig. B.2c and Fig. B.2d). Because  $Z$  and  $T$  are in LCA agreement, the tree constructed from  $\text{dec}(Z; o, \vee_{\mathbb{B}_2})$  is exactly the same as the tree constructed from  $\text{dec}(T; 0, \vee_T)$ . Thus we only have to show that  $T' = \text{dec}(T; 0, \vee_T)$ .

We use an induction argument to show that  $T' = \text{dec}(T; 0, \vee_T)$ . Let  $r$  be the unique neighbor of 0 in  $T$ , and let  $T_0$  and  $T_1$  be the two trees on the other edges of  $r$  aside from the one pointing to 0 (Fig. B.2e). Note that any distances  $d_T(0, i \vee j)$  for any  $i, j$  both contained in  $T_0$  or  $T_1$  are strictly smaller than distances  $d_T(0, i \vee j)$  for any  $i \in T_0, j \in T_1$ . Therefore the decoding algorithm merges all pairs  $(i, j) \in T_0$  and  $(i, j) \in T_1$  first. Inductively, this decoding algorithm will exactly return  $T_0$  and  $T_1$ .

Finally, the decoding algorithm will merge any two  $i \in T_0, j \in T_1$ . This creates a new parent and connects them to the roots of  $T_0, T_1$  (Algorithm 1, line 7). The structure of this tree is therefore the same as the tree rooted at  $r$ , in other wise  $T'$ , as desired.  $\square$

### B.2.2.2 Any spread embedding is in LCA agreement with some binary tree

Lemma B.2.3 shows that under the LCA agreement condition, the decoding algorithm will produce a discrete tree that preserves the ordering of triplets (in terms of LCAs' depths), thus the continuous and discrete costs are close. Furthermore, we know by Proposition B.1.1 that any embedding has a corresponding quasi-isometric binary tree. Using this, we seek a condition under which a quasi-isometric pair  $(Z, T)$  is in LCA agreement.

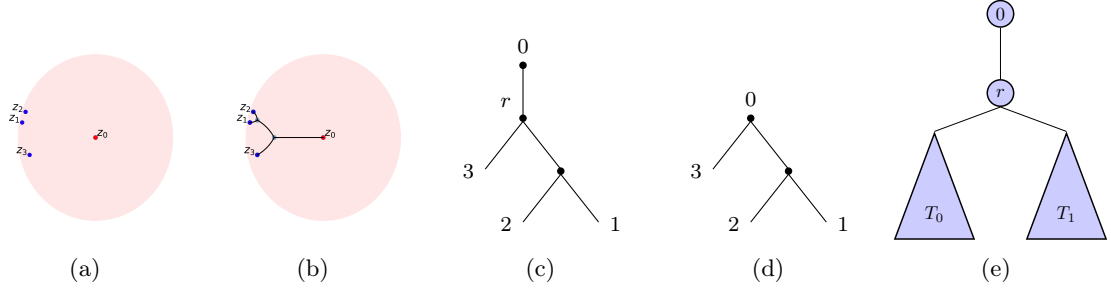


Figure B.2: (a): Leaves embeddings and the origin in  $\mathbb{B}_2$ . (b): Binary tree that is a quasi-isometry of the leaves embeddings and the origin (Proposition B.1.1) and corresponding discrete binary tree (not rooted) in (c). (d): Rooted binary tree obtained by decoding hyperbolic leaves embeddings. Observe that the quasi-isometric tree in (c) and the decoded tree in (d) are LCA equivalent with respect to 0. (e): Figure used in induction proof in Lemma B.2.3.

First, we show that under a quasi-isometric embedding (Definition B.1.1), the hyperbolic LCA depth  $d_o(z_i \vee z_j)$  is close to the tree LCA depth  $d_0(i \vee j)$  (Lemma B.2.4). Next, we derive a condition under which a quasi-isometric pair  $(Z, T)$  is in LCA agreement (Lemma B.2.5). Finally, we show that any spread embedding satisfies this condition, and more specifically, we show that any spread embedding is in LCA agreement with some binary tree, such that the “deepest” LCA is always distinguishable from the embeddings (Lemma B.2.6).

**Lemma B.2.4** (Hyperbolic LCA depth is close to Tree LCA depth). *Let  $Z \subset \mathbb{B}_2^{n+1}$  be an embedding with  $z_0 = o$  and  $T$  a binary tree (not rooted) on  $(n+1)$  leaves such that  $(Z, T)$  is  $(1 + \varepsilon, \kappa)$ -quasi-isometric. If  $M$  is the diameter of  $T$ , then for any leaves  $1 \leq i, j, k, l \leq n$ :*

$$|(d_o(z_i \vee z_j) - d_o(z_k \vee z_l)) - (d_0(i \vee j) - d_0(k \vee l))| \leq \frac{3\varepsilon}{2}M + \frac{3}{2}\kappa + \delta. \quad (\text{B.5})$$

*Proof.* We bound the hyperbolic LCA depth by the corresponding tree depths:

$$\begin{aligned} d_o(z_i \vee z_j) &\geq \langle z_i, z_j \rangle \\ &= \frac{1}{2} (d_o(z_i) + d_o(z_j) - d(z_i, z_j)) \\ &\geq \frac{1}{2} (d_0(i) + d_0(j) - (1 + \varepsilon)d(i, j) - \kappa) \\ &= d_0(i \vee j) - \frac{\varepsilon}{2}d(i, j) - \frac{1}{2}\kappa. \end{aligned}$$

The first line is Lemma B.1.1, the second line is Definition 1.3.1, and the third applies Definition B.1.1.

In the other direction, again using Lemma B.1.1, Definition 1.3.1, and Definition B.1.1:

$$\begin{aligned}
d_o(z_i \vee z_j) &\leq \langle z_i, z_j \rangle + \delta \\
&= \frac{1}{2} (d_o(z_i) + d_o(z_j) - d(z_i, z_j)) + \delta \\
&\leq \frac{(1 + \epsilon)(d_0(i) + d_0(j)) + 2\kappa}{2} - \frac{d(i, j)}{2} + \delta \\
&\leq \frac{1}{2} (d_0(i) + d_0(j) - d(i, j)) + \frac{\epsilon}{2} (d_0(i) + d_0(j)) + \kappa + \delta \\
&= d_0(i \vee j) + \frac{\epsilon}{2} d_0(i) + \frac{\epsilon}{2} d_0(j) + \kappa + \delta.
\end{aligned}$$

Finally, Eq. (B.5) follows by adding these inequalities with  $d(i, j) \leq M$  for any  $0 \leq i, j \leq n$ .  $\square$

Lemma B.2.4 allows us to provide a concrete condition for when  $(Z, T)$  are in LCA agreement, which will be our main tool for showing the consistency of our relaxation.

**Lemma B.2.5.** *Let  $Z \subset \mathbb{B}_2^{n+1}$  be an embedding with  $z_0 = o$  and  $T$  a binary tree (not rooted) on  $(n + 1)$  leaves, such that  $(Z, T)$  is  $(1 + \epsilon, \kappa)$ -quasi-isometric. Define  $M$  to be the diameter of  $T$  and  $m$  to be the length of the smallest edge not including a leaf of  $T$ . If  $m > \frac{3\epsilon}{2}M + \frac{3}{2}\kappa + \delta$ , then  $(Z, T)$  are in LCA agreement.*

*Proof.* Consider an arbitrary triple  $(i, j, k)$ , and suppose  $\{ij|k\}_T$  holds. It suffices to show that  $\{z_i, z_j|z_k\}_{\mathbb{B}_2}$  holds to show that  $(Z, T)$  are in LCA agreement (Definition B.2.3). Applying Lemma B.2.4,

$$\begin{aligned}
d_o(z_i \vee z_j) - d_o(z_i \vee z_k) &\geq d_0(i \vee j) - d_0(i \vee k) - \left( \frac{3\epsilon}{2}M + \frac{3}{2}\kappa + \delta \right) \\
&\geq m - \frac{3\epsilon}{2}M - \frac{3}{2}\kappa - \delta > 0.
\end{aligned}$$

In the second line, we used the fact that since  $i \vee j$  and  $i \vee k$  are both internal nodes in  $T$ , their distance is at least  $m$  by assumption (also,  $i \vee j$  is deeper than  $i \vee k$  by assumption, so the sign of the difference is positive). Similarly  $d_o(z_i \vee z_j) - d_o(z_j \vee z_k) > 0$ , so  $z_i \vee z_j$  is the deepest out of the 3 LCAs, as desired.  $\square$

Lemma B.2.5 gives a technical condition for when an embedding  $Z$  is in LCA agreement with a tree  $T$ . Next, we claim that the constrained set of spread embeddings  $\mathcal{Z} \subseteq \mathbb{B}_2^n$  is such that for every embedding  $Z \in \mathcal{Z}$ , there is always a corresponding tree  $T$  satisfying Lemma B.2.5. Intuitively, Lemma B.2.5 says that  $m$  should be large, i.e. the internal edges of the tree should be spread far apart. Using our notion of hyperbolic LCA, we can codify this by enforcing that different LCAs should be far from each other. By leveraging global properties of hyperbolic space, this is in fact sufficient.

**Lemma B.2.6.** *Suppose that embedding  $Z \in \mathcal{Z} \subset \mathbb{B}_2^n$  is spread. Then there exist a binary tree (not rooted) on  $(n+1)$  leaves  $T$  such that  $(Z \cup \{z_0 = o\}, T)$  are in LCA agreement. In particular, for any  $1 \leq i, j, k \leq n$  such that  $\{i, j|k\}_T$  holds, then:*

$$d_o(z_i \vee z_j) > \max\{d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} + 1.$$

*Proof.* Append the origin  $z_0$  to  $Z$  (so we have a collection of  $n+1$  points). By Proposition B.1.1, there is a binary tree  $T$  (not rooted) on  $(n+1)$  leaves that is  $(1, C_n)$ -quasi-isometric for  $Z \cup \{z_0\}$ , where  $C_n = \delta \cdot \mathcal{O}(n)$ . Consider an internal edge  $e \in T$ , i.e. an edge connecting non-leaf nodes. Since  $T$  is binary,  $e$  has endpoints  $i \vee j, i \vee j \vee k$  for some triplet  $\{ij|k\}_T$  with  $1 \leq i, j, k \leq n$ .<sup>4</sup>

Let  $a, b, c$  be the ordering of  $i, j, k$  such that  $d_o(z_a \vee z_b) > d_o(z_b \vee z_c) > d_o(z_a \vee z_c)$ . Consider:

$$\begin{aligned} d_0(a \vee b) - d_0(a \vee c) &\geq d_o(z_a \vee z_b) - d_o(z_a \vee z_c) - \left(\frac{3}{2}C_n + \delta\right) \\ &> \frac{3}{2}C_n + \delta + 1, \end{aligned}$$

where the first line applies Lemma B.2.4 with  $\varepsilon = 0, \kappa = C_n$ , and the second uses the definition of spread (Definition B.2.1), which holds for any  $1 \leq a, b, c \leq n$ . In particular,  $d_0(a \vee b) - d_0(a \vee c) > 0$ , so clearly  $\{a, b|c\}_T$  holds since  $T$  is binary, and we must have:

$$d_0(a \vee b) - d_0(a \vee c) = d_0(i \vee j) - d_0(i \vee j \vee k) = d(i \vee j, i \vee j \vee k),$$

which is the length of the edge  $e$  we are considering. Since this holds generically for any edge  $e$  among internal nodes of  $T$ , this also holds for the minimum edge length  $m$ :

$$m > \frac{3}{2}C_n + \delta + 1. \tag{B.6}$$

In particular, the conditions of Lemma B.2.5 apply; that is,  $m$  satisfies  $m > \frac{3\varepsilon}{2}M + \frac{3}{2}\kappa + \delta$  for  $\varepsilon = 0, \kappa = C_n$ . Applying Lemma B.2.5, we get that  $(Z \cup \{z_0\}, T)$  are in LCA agreement.

We now turn to the second part of Lemma B.2.6 which bounds the difference in LCA depth. Let  $1 \leq i, j, k \leq n$  such that  $\{i, j|k\}_T$  holds. Since  $(Z, T)$  are in LCA agreement, we can assume WLOG that  $d_o(z_i \vee z_j) > d_o(z_i \vee z_k) > d_o(z_j \vee z_k)$ . Applying Lemma B.2.4 again, we have:

$$\begin{aligned} d_o(z_i \vee z_j) - d_o(z_i \vee z_k) &\geq d_0(i \vee j) - d_0(i \vee k) - \left(\frac{3}{2}C_n + \delta\right) \\ &\geq m - \frac{3}{2}C_n - \delta \\ &> 1, \end{aligned}$$

---

<sup>4</sup>Note that triplets here are defined on leaves  $[n]$ , excluding 0. The reason is that, since the LCA is computed with respect to 0 which is a leaf in  $T$ ,  $0 \vee i = 0 \forall i$ , and therefore there is no internal edge whose endpoint is an LCA on 0.

where we used Eq. (B.6) in the last inequality.  $\square$

### B.2.2.3 The tree decoded from any spread embedding has a cost close to the HYPHC cost

We now have all the tools to show that any spread embedding decodes to a tree such that the discrete and continuous costs are close (Lemma B.2.1).

**Lemma B.2.1.** *Let  $Z \in \mathcal{Z} \subset \mathbb{B}_2^n$  be a spread embedding. Then:*

$$|C_{\text{Dasgupta}}(\text{dec}(Z); w) - C_{\text{HYPHC}}(Z; w, \tau)| \leq 4e^{-1/\tau} \sum_{ijk} \max\{|w_{ij}|, |w_{ik}|, |w_{jk}|\}$$

*Proof.* Let  $Z \in \mathcal{Z} \subset \mathbb{B}_2^n$  be a spread embedding. Using Lemma B.2.6, we know that there exists a binary tree (not rooted) on  $(n+1)$  leaves  $T$ , such that  $(Z \cup \{z_0 = o\}, T)$  are in LCA agreement. Let  $T'$  be the rooted binary tree that is obtained by removing the leaf 0 from  $T$ , and relabelling its neighbor  $r$  in  $T$  as 0, the root of  $T'$ . Using Lemma B.2.3, we know that  $T' = \text{dec}(Z)$  and:

$$|C_{\text{Dasgupta}}(\text{dec}(Z); w) - C_{\text{HYPHC}}(Z; w, \tau)| \leq \sum_{ijk} |w_{ijk}(T'; w) - w_{\text{HYPHC},ijk}(Z; w, \tau)|.$$

Let  $\delta_{ijk} := |w_{ijk}(T'; w) - w_{\text{HYPHC},ijk}(Z; w, \tau)|$ . WLOG, assume that  $\{i, j|k\}_{T'}$  holds for a triplet  $(i, j, k) \in T'$ .  $T$  and  $T'$  are equivalent in the LCA agreement sense with respect to 0, since LCA agreement is defined over leaves in  $[n]$ . That is, the LCA of any pair  $i, j \in [n]$  with respect to 0 in  $T$  is the same as the LCA with respect to 0 in  $T'$ . Therefore using the definition of LCA agreement, we have:

$$d_1 := d_o(z_i \vee z_j) \geq \max\{d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} := d_2.$$

Denote  $w_{ijk}^* := \max\{|w_{ij}|, |w_{ik}|, |w_{jk}|\}$  and  $\Sigma_{ijk} := e^{d_o(z_i \vee z_j)/\tau} + e^{d_o(z_i \vee z_k)/\tau} + e^{d_o(z_j \vee z_k)/\tau}$ . Then:

$$\begin{aligned} \delta_{ijk} &= \left| w_{ij} \left( 1 - \frac{e^{d_o(z_i \vee z_j)/\tau}}{\Sigma_{ijk}} \right) + w_{ik} \left( \frac{e^{d_o(z_i \vee z_k)/\tau}}{\Sigma_{ijk}} \right) + w_{jk} \left( \frac{e^{d_o(z_j \vee z_k)/\tau}}{\Sigma_{ijk}} \right) \right| \\ &\leq 2w_{ijk}^* \left( \frac{e^{d_o(z_i \vee z_k)/\tau} + e^{d_o(z_j \vee z_k)/\tau}}{\Sigma_{ijk}} \right) \\ &\leq 4w_{ijk}^* e^{(d_2 - d_1)/\tau} \\ &\leq 4w_{ijk}^* e^{-1/\tau}. \end{aligned} \tag{B.7}$$

In the last line, we applied the second part of Lemma B.2.6.  $\square$

### B.2.3 Proof of Lemma B.2.2

We have shown in Lemma B.2.6 that any spread embedding gets decoded into a tree such that the continuous and discrete costs are close. We now show the other direction, that any rooted binary tree has a corresponding spread embedding which decodes back to it (Lemma B.2.2), and therefore the discrete and continuous costs are close.

We use Sarkar's result (see Proposition 1.3.2 in Chapter 1) for low-distortion hyperbolic embeddings of trees [152], which in our terminology states that if every edge of  $T$  is weighed with a scalar  $\zeta = \mathcal{O}(1/\varepsilon)$ , then there is an embedding  $Z$  such that  $(Z, T)$  is  $(1 + \varepsilon, 0)$ -quasi-isometric, that is:

$$d_T(i, j) \leq d(z_i, z_j) \leq (1 + \varepsilon)d_T(i, j).$$

We now rely on Sarkar's result to find a spread embedding for a given rooted binary tree.

**Lemma B.2.7.** *Let  $T$  be any unit-weight binary tree (not rooted) on  $(n + 1)$  leaves. Then there is a spread embedding  $Z \in \mathcal{Z} \subset \mathbb{B}_2^{n+1}$  with  $z_0 = 0$ , such that  $(Z, T)$  are in LCA agreement.*

*Proof.* Let  $\varepsilon > 0$ . Put a weight  $\zeta$  ( $\zeta$  to be decided later) on every edge of  $T$ . Using Proposition 1.3.2, embed  $T$  to an embedding  $Z \subset \mathbb{B}_2^{n+1}$ , such that  $(Z, T)$  is  $(1 + \varepsilon, 0)$ -quasi-isometric, and WLOG reflect the embeddings (isometric transformation) so that  $z_0$  is at the hyperbolic origin.

Consider any triplet  $1 \leq i, j, k \leq n$ , and WLOG let  $\{ij|k\}_T$ . Applying Lemma B.2.4 with  $\kappa = 0$ ,

$$\begin{aligned} d_o(z_i \vee z_j) - d_o(z_i \vee z_k) &\geq d_0(i \vee j) - d_0(i \vee k) - \frac{3\varepsilon}{2}\zeta n - \delta \\ &\geq \zeta \left(1 - \frac{3\varepsilon}{2}n\right) - \delta \end{aligned}$$

In the second line we used the fact that the diameter of  $T$  is at most  $\zeta n$  (since there are  $n + 1$  nodes and all edges are equally weighted), and in the third that  $i \vee j \neq i \vee k$  and the minimum tree distance between any distinct nodes is  $\zeta$ .

Finally, we could choose  $\varepsilon \leq \frac{1}{3n}$  and  $\zeta > 6C_n + 6\delta + 2$ . Note that choosing such  $\zeta = \Theta(n)$  works since Proposition 1.3.2 says  $\zeta = \Theta(1/\varepsilon) = \Theta(n)$  is possible for the embedding, and Proposition B.1.1 says it is sufficient since  $C_n = O(n)$ . Then:

$$\begin{aligned} &\max\{d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} - \min\{d_o(z_i \vee z_j), d_o(z_i \vee z_k), d_o(z_j \vee z_k)\} \\ &\geq d_o(z_i \vee z_j) - d_o(z_i \vee z_k) \\ &\geq \zeta \left(1 - \frac{3\varepsilon}{2}n\right) - \delta \\ &> 3C_n + 2\delta + 1. \end{aligned}$$

Since  $i, j, k$  were arbitrary, this shows that  $Z$  is spread (as defined in Definition B.2.1). Finally,

note that the min edge length of  $T$  is  $m = \zeta$ , the maximum path length is at most  $M = n\zeta$ , and  $\zeta > \frac{3}{2}\varepsilon(n\zeta) + \delta$  by choice of  $\varepsilon$  and  $\zeta$ . Since  $(Z, T)$  were a  $(1 + \epsilon, 0)$ -quasi-isometry, Lemma B.2.5 then implies that  $(Z, T)$  are in LCA agreement.  $\square$

Finally, we show Lemma B.2.2 using the previous Lemma and Lemma B.2.3.

**Lemma B.2.2.** *For any unit-weight rooted binary tree  $T$  on  $n$  leaves, there exists a spread embedding  $Z \in \mathcal{Z} \subset \mathbb{B}_2^n$  such that  $\text{dec}(Z) = T$ .*

*Proof.* Attach a leaf 0 to the root of  $T$ , apply Lemma B.2.7, then apply Lemma B.2.3.  $\square$

# Appendix C

## Deferred Proofs of Chapter 6

### C.1 Horospherical Projection: Proofs and Discussions

We show that the horospherical projection  $\pi_{b,p_1,\dots,p_K}^H$  in Section 6.4.2.2 is well-defined, shares many nice properties with Euclidean orthogonal projections, and has a closed-form expression.

More specifically, this section is organized as follows. In Appendix C.1.1, we show that horospherical projections are well-defined (Theorem C.1.3). In Appendix C.1.2, we show that they are base-point independent (Corollary C.1.6), non-expanding (Corollary C.1.10), and distance-preserving along a family of  $K$ -dimensional submanifolds (Corollary C.1.8). Finally, in Appendix C.1.3, we explain how they can be computed efficiently in the hyperboloid model.

#### C.1.1 Well-definedness

Recall that given a base point  $b \in \mathbb{H}^d$  and  $K > 1$  ideal points  $\{p_1, \dots, p_K\}$ , we would like to define  $\pi_{b,p_1,\dots,p_K}^H$  by

$$x \rightarrow M \cap S(p_1, x) \cap S(p_2, x) \cap \dots \cap S(p_K, x),$$

where  $M = \text{GH}(b, p_1, \dots, p_K)$  is the target submanifold and  $S(p_j, x)$  is the horosphere centered at  $p_j$  and passing through  $x$ . For this definition to make sense, the intersection in the right hand side must contain exactly one point for each  $x \in \mathbb{H}^d$ . Unfortunately, this is not the case: In fact, the intersection generally consists of two points. Nevertheless, we will show that there is a consistent way to choose one from these two points, making the function  $\pi_{b,p_1,\dots,p_K}^H$  well-defined. This is the result of Theorem C.1.3.

First, to understand the above intersection, we give a more concrete description of  $\cap_j S(p_j, x)$ .

**Lemma C.1.1.** *Let  $P = \text{GH}(p_1, p_2, \dots, p_K)$ . Then for every  $x \in \mathbb{H}^d$ , the intersection of horospheres*

$$S(x) = S(p_1, x) \cap S(p_2, x) \cap \dots \cap S(p_K, x)$$

is precisely the orbit of  $x$  under the group  $G$  of rotations around  $P$ .

*Proof.* First, note that every rotation around  $P$  preserves the horospheres  $S(p_j, x)$  - just like how every rotation around an axis preserves every sphere whose center is on that axis. It follows that  $S(x)$  is preserved by  $G$ . In particular, the orbit of  $x$  under  $G$  is contained in  $S(x)$ .

It remains to show that  $S(x)$  contains no other points. To this end, consider any  $y \neq x$  in  $S(x)$ . The perpendicular bisector  $B$  of  $x$  and  $y$  is a totally geodesic hyperplane of  $\mathbb{H}^d$  that contains every  $p_j$  (because each  $p_j$  is intuitively the center of a sphere that goes through  $x$  and  $y$ ). Thus, by the definition of geodesic hull,  $B \supset P$ . In particular, the reflection through  $B$  sends  $x$  to  $y$  and fixes every point in  $P$ .

Now take any geodesic hyperplane  $A$  that contains both  $P$  and  $y$ , so that the reflection through  $A$  fixes  $y$  and every point in  $P$ . Then the composition of the reflections through  $B$  and  $A$  is a rotation that sends  $x$  to  $y$  and fixes every point in  $P$ . In other words, it is a rotation around  $P$  that sends  $x$  to  $y$ . Therefore,  $y$  belongs to the orbit of  $x$  under  $G$ .  $\square$

**Corollary C.1.1.** If  $x \in P$  then  $S(x) = \{x\}$ . Otherwise, let  $\pi_P^G(x)$  be the geodesic projection of  $x$  onto  $P$ , and  $Q(x)$  be the geodesic submanifold that orthogonally complements  $P$  at  $\pi_P^G(x)$ . Then  $S(x) \subset Q(x)$  and is precisely the (hyper)sphere in  $Q(x)$  that is centered at  $\pi_P^G(x)$  and passing through  $x$ .

*Proof.* This follows from Lemma C.1.1. If  $x \in P$  then every rotation around  $P$  fixes  $x$ , so the orbit of  $x$  is just itself.

Now consider the case  $x \notin P$ . All rotations around  $P$  must preserve  $\pi_P^G(x)$  and the orthogonal complement  $Q(x)$  of  $P$  at  $\pi_P^G(x)$ . Furthermore, when restricted to the space  $Q(x)$ , these rotations are precisely the rotations in  $Q(x)$  around the point  $\pi_P^G(x)$ . Thus, for every  $y \in Q(x)$ , the orbit of  $y$  under  $G$  is a sphere in  $Q(x)$  centered at  $\pi_P^G(x)$ . In particular,  $S(x)$ , which is the orbit of  $x$ , is the sphere in  $Q(x)$  that is centered at  $\pi_P^G(x)$  and passing through  $x$ .  $\square$

Corollary C.1.1 gives the following characterization of the intersection

$$M \cap S(p_1, x) \cap S(p_2, x) \cap \cdots \cap S(p_K, x) = M \cap S(x) :$$

Note that  $P = \text{GH}(p_1, \dots, p_K)$  is a geodesic submanifold of  $M = \text{GH}(b, p_1, \dots, p_K)$  and that  $\dim P = \dim M - 1$ . Thus, through every point  $y \in P$ , there is a unique geodesic  $\alpha$  on  $M$  that goes through  $y$  and is perpendicular to  $P$ .

**Corollary C.1.2.** If  $x \in P$  then  $M \cap S(x) = \{x\}$ . Otherwise, let  $\alpha$  be the geodesic on  $M$  that goes through  $\pi_P^G(x)$  and is perpendicular to  $P$ . Then  $M \cap S(x)$  consists of the two points on  $\alpha$  whose distance to  $\pi_P^G(x)$  equals  $d_{\mathbb{H}}(x, \pi_P^G(x))$ .

*Proof.* The case  $x \in P$  is clear since  $x \in M$  and  $S(x) = \{x\}$  by Corollary C.1.1.

For the other case, let  $Q(x)$  be the orthogonal complement of  $P$  at  $\pi_P^G(x)$ . Then by Corollary C.1.1,  $S(x)$  is precisely the sphere in  $Q(x)$  that is centered at  $c(x)$  and passing through  $x$ .

Now note that  $M \cap Q(x) = \alpha$ . Since  $S(x) \subset Q(x)$ , this gives  $M \cap S(x) = M \cap Q(x) \cap S(x) = \alpha \cap S(x)$ . We know that every sphere intersects every line through the center at two points.  $\square$

Therefore, to define  $\pi_{b,p_1,\dots,p_K}^H(x)$ , we just need to choose one of the two points in  $M \cap S(x)$  in a consistent way (so that the map is differentiable). To this end, note that  $P$  cuts  $M$  into two half-spaces, and exactly one of them contains the base point  $b$ . (Recall that  $b \in M$  and  $b \notin P$  by the “independence” condition). We denote this half by  $P_b$ . Then, while  $S(x)$  contains two points in  $M$ , it only contains one point in  $P_b$ :

**Theorem C.1.3.** *Let  $\alpha$  be the geodesic on  $M$  that goes through  $\pi_P^G(x)$  and is perpendicular to  $P$ . Let  $\alpha^+$  be the half of  $\alpha$  contained in  $P_b$ . Then  $\alpha^+$  intersects the sphere  $S(x)$  at a unique point  $x'$ , which is also the unique intersection point between  $P_b$  and  $S(x)$ . Thus, we can define  $\pi_{b,p_1,\dots,p_K}^H$  by*

$$\pi_{b,p_1,\dots,p_K}^H(x) = \alpha^+ \cap S(x) = P_b \cap S(x).$$

*Equivalently,  $\pi_{b,p_1,\dots,p_K}^H(x)$  is the point in  $M \cap S(x)$  that is strictly closer to  $b$ .*

*Proof.* By Corollary C.1.1,  $S(x)$  is a sphere centered at  $\pi_P^G(x)$ . Then, since  $\alpha^+$  is a geodesic ray starting at  $\pi_P^G(x)$ , it must intersect  $S(x)$  at a unique point  $x'$ .

Next, we have

$$P_b \cap S(x) = P_b \cap M \cap S(x) = P_b \cap \alpha \cap S(x) = \alpha^+ \cap S(x),$$

where the first equality holds because  $P_b \subset M$ , the second because  $M \cap S(x) = \alpha \cap S(x)$  by the proof of Corollary C.1.2, and the third because  $P_b \cap \alpha = \alpha^+$ . Thus,  $P_b \cap S(x)$  is precisely  $x'$ .

Finally, let  $x''$  be the other point of  $M \cap S(x) = \alpha \cap S(x)$ . Then  $P$  is the perpendicular bisector of  $x'$  and  $x''$  in  $M$ . Thus, every point on the same side of  $P$  in  $M$  as  $x'$  (but not on the boundary  $P$ ) is strictly closer to  $x'$  than to  $x''$ . By definition,  $b$  is one of such point. Thus,  $\pi_{b,p_1,\dots,p_K}^H(x)$  is the point in  $M \cap S(x)$  that is closer to  $b$ .  $\square$

### C.1.2 Geometric Properties

From Lemma C.1.1 and Theorem C.1.3, we obtain another interpretation of  $\pi_{b,p_1,\dots,p_K}^H$ : It maps  $\mathbb{H}^d$  to  $P_b \subset M$  by rotating every point  $x \in \mathbb{H}^d$  around  $P$  until it hits  $P_b$ . In other words, we have

**Theorem C.1.4** (The “open book” interpretation). *For any  $x \notin P$ , let  $M_x = \text{GH}(P \cup \{x\})$ . Then  $P$  cuts  $M_x$  into two half-spaces; we denote the half that contains  $x$  by  $P_x$ . Then, when restricted to  $P_x$ , the map*

$$\pi_{b,p_1,\dots,p_K}^H : P_x \rightarrow P_b$$

is simply a rotation around  $P$ .  $\square$

Following this, we call  $P$  the *spine* of the horosphere projection. The identity  $\mathbb{H}^d = \cup_{x \notin P} P_x$  can be thought of as an *open book decomposition* of  $\mathbb{H}^d$  into *pages*  $P_x$  that are bounded by the spine  $P$ . The horosphere projection  $\pi_{b,p_1,\dots,p_K}^H$  then simply acts by collapsing every page onto a specified page  $P_b$ .

Here are some consequences of this interpretation:

**Corollary C.1.5.**  $\pi_{b,p_1,\dots,p_K}^H$  only depends on the spine  $P$  and not specifically on  $p_1, \dots, p_K$ . Thus, when we are not interested in the specific ideal points, we simply write  $\pi_{b,P}^H$ .

*Proof.* As noted above,  $\pi_{b,p_1,\dots,p_K}^H(x)$  can be obtained by rotating  $x$  around  $P$  until it hits  $P_b$ . This operation does not use the exact positions of  $p_j$  at all.  $\square$

**Corollary C.1.6.** The choice of  $b$  does not affect the geometry of the projection  $\pi_{b,P}^H$ . More precisely, for any two base points  $b, b' \notin P$ , the horosphere projections

$$\pi_{b,P}^H : \mathbb{H}^d \rightarrow P_b \quad \text{and} \quad \pi_{b',P}^H : \mathbb{H}^d \rightarrow P_{b'}$$

only differ by a rotation  $P_b \rightarrow P_{b'}$  around  $P$ .

*Proof.* By Theorem C.1.4, when restricted to any page  $P_x$ , the maps  $\pi_{b,P}^H : \mathbb{H}^d \rightarrow P_b$  and  $\pi_{b',P}^H : \mathbb{H}^d \rightarrow P_{b'}$  are just rotations around  $P$ . The difference between any two rotations around  $P$  is another rotation around  $P$ .  $\square$

In particular, Corollary C.1.6 implies Theorem 6.5.1:

**Theorem 6.5.1.** For any  $b, b'$  and any  $x, y \in \mathbb{H}^d$ , the two projected distances  $d_{\mathbb{H}}(\pi_{b,p_1,\dots,p_K}^H(x), \pi_{b,p_1,\dots,p_K}^H(y))$  and  $d_{\mathbb{H}}(\pi_{b',p_1,\dots,p_K}^H(x), \pi_{b',p_1,\dots,p_K}^H(y))$  are equal.

As discussed in Section 6.5, this theorem helps reduce parameters and simplifies the computation of  $\pi_{b,P}^H$ .

**Remark C.1.1.** Corollary C.1.5 and Corollary C.1.6 together imply that the HYPHC algorithm (6.4) only depends on the geodesic hulls  $\text{GH}(p_1), \text{GH}(p_1, p_2), \dots, \text{GH}(p_1, \dots, p_K)$  of the ideal points and not the specific ideal points themselves. It follows that, theoretically, the search space of (6.4) has dimension  $dK - \frac{1}{2}K(K+1)$  – the same as the dimension of the space of flags in Euclidean spaces.

In our implementation, for simplicity we parametrize the  $K$  ideal points independently, which results in a suboptimal search space dimension  $(d-1)K$ . Nevertheless, this is still slightly more efficient than the parametrizations used in PGA and BSA, which require have  $(d+1)K$ -dimensions.

The following corollaries say that horospherical projections share a nice property with Euclidean orthogonal projections: When projecting to a  $K$ -dimensional submanifold, they preserve the distances along  $K$  dimensions and collapse the distances along the other  $d-K$  orthogonal dimensions:

**Corollary C.1.7.**  $\pi_{b,P}^H$  is invariant under rotations around  $P$ . In other words, if a rotation around  $P$  takes  $x$  to  $y$  then  $\pi_{b,P}^H(x) = \pi_{b,P}^H(y)$ .

Consequently, every  $x \notin P$  belongs to a  $(d - K)$ -dimensional submanifold that is collapsed to a point by  $\pi_{b,P}^H$ .

*Proof.* The open book interpretation tells us that  $\pi_{b,P}^H(x)$  and  $\pi_{b,P}^H(y)$  are precisely the intersections of  $P_b$  with  $S(x)$  and  $S(y)$ , respectively. If  $y$  belongs to the rotation orbit  $S(x)$  of  $x$  then the rotation orbit  $S(y)$  of  $y$  is the same as  $S(x)$ . Thus  $\pi_{b,P}^H(x) = \pi_{b,P}^H(y)$ .

Hence, for every  $x \in \mathbb{H}^d$ ,  $S(x)$  is collapsed to a point by  $\pi_{b,P}^H$ . To see that  $\dim S(x) = d - K$  when  $x \notin P$ , recall that by Corollary C.1.1, if  $Q(x)$  is the orthogonal complement of  $P$  at  $\pi_P^G(x)$  then  $S(x)$  is a hypersphere inside  $Q(x)$ . Since the ideal points  $p_j$  are assumed to be “affinely independent,” we have  $\dim P = K - 1$ , so  $\dim Q(x) = d - (K - 1)$ , and  $\dim S(x) = \dim Q(x) - 1 = d - K$ .  $\square$

**Corollary C.1.8.** For every  $x \in \mathbb{H}^d$ , there exists a  $K$ -dimensional totally geodesic submanifold (with boundary) that contains  $x$  and is mapped isometrically to  $P_b$  by  $\pi_{b,P}^H$ . If  $x \notin P$  then such a manifold is unique.

*Proof.* If  $x \notin P$  then the submanifold  $P_x$  in Theorem C.1.4 is a geodesic submanifold that contains  $x$  and is mapped isometrically to  $P_b$  by  $\pi_{b,P}^H$ . As in the proof of Corollary C.1.7, we have  $\dim P = K - 1$  and  $\dim P_x = \dim P + 1 = K$ .

Since Corollary C.1.7 implies that the other  $d - K$  dimensions are collapsed by  $\pi_{b,P}^H$ , no other distances from  $x$  can be preserved. Thus,  $P_x$  is the unique submanifold with the desired properties.

If  $x \in P$  then  $x \in P_y$  for every  $y \notin P$ . Note that this means *every* distance from  $x$  is preserved by  $\pi_{b,P}^H$ .  $\square$

The following corollaries say that, like Euclidean orthogonal projections, horospherical projections never increase distances. Thus, minimizing distortion is roughly equivalent to maximizing projected distances. This is another motivation for Eq. (6.3).

**Corollary C.1.9.** For every  $x \in \mathbb{H}^d$  and every tangent vector  $\vec{v}$  at  $x$ ,

$$\|\pi_{b,P}^H(\vec{v})\|_{\mathbb{H}} \leq \|\vec{v}\|_{\mathbb{H}}.$$

*Proof.* This follows from Corollary C.1.8 and Corollary C.1.7:

If  $x \in P$  then the proof of Corollary C.1.8 implies that  $\pi_{b,P}^H$  preserves *every* distance from  $x$ . Thus, the desired inequality is actually an equality.

If  $x \notin P$  then by  $\vec{v}$  has an orthogonal decomposition  $\vec{v} = \vec{u} + \vec{u}^\perp$ , where  $\vec{u}$  and  $\vec{u}^\perp$  are tangent and perpendicular to  $P_x$ , respectively. By Corollary C.1.8 and Corollary C.1.7,  $\pi_{b,P}^H$  preserves the length of  $\vec{u}$  while collapsing  $\vec{u}^\perp$  to 0. It follows that  $\|\pi_{b,P}^H(\vec{v})\|_{\mathbb{H}} = \|\vec{u}\|_{\mathbb{H}} \leq \|\vec{v}\|_{\mathbb{H}}$ .  $\square$

**Corollary C.1.10.**  $\pi_{b,P}^H$  is non-expanding. In other words, for every  $x, y \in \mathbb{H}^d$ ,

$$d_{\mathbb{H}}(\pi_{b,P}^H(x), \pi_{b,P}^H(y)) \leq d_{\mathbb{H}}(x, y).$$

*Proof.* We first show that for any path  $\gamma(t)$  in  $\mathbb{H}^d$ ,

$$\text{length}(\pi_{b,P}^H(\gamma)) \leq \text{length}(\gamma).$$

Indeed, note that the velocity vector of  $\pi_{b,P}^H(\gamma)$  is precisely  $\pi_{b,P}^H(\dot{\gamma})$ . Then, by Corollary C.1.9,

$$\begin{aligned} \text{length}(\pi_{b,P}^H(\gamma)) &= \int \|\pi_{b,P}^H(\dot{\gamma}(t))\|_{\mathbb{H}} dt \\ &\leq \int \|\dot{\gamma}(t)\|_{\mathbb{H}} dt = \text{length}(\gamma). \end{aligned}$$

Now for any  $x, y \in \mathbb{H}^d$ , let  $\gamma(t)$  be the geodesic segment from  $x$  to  $y$ . Then  $\pi_{b,P}^H(\gamma)$  is a path connecting  $\pi_{b,P}^H(x)$  and  $\pi_{b,P}^H(y)$ . Thus, the projected distance is at most the length of  $\pi_{b,P}^H(\gamma)$ , which by the above argument is at most  $\text{length}(\gamma) = d_{\mathbb{H}}(x, y)$ .  $\square$

### C.1.3 Computation of Horospherical Projections

Theorem C.1.3 suggests that  $\pi_{b,P}^H(x)$  can be computed in three steps:

1. Find the geodesic projection  $\pi_P^G(x)$  of  $x$  onto  $P$ .
2. Find a geodesic ray  $\alpha^+$  on  $P_b$  that starts at  $\pi_P^G(x)$  and is orthogonal to  $P$ .
3. Return the unique point on  $\alpha^+$  that is of distance  $d_{\mathbb{H}}(x, \pi_P^G(x))$  from  $\pi_P^G(x)$ .

**Remark C.1.2.** The above steps are slightly different from the ones mentioned in Section 6.4.2.2. However, by Theorem C.1.3, these two descriptions are equivalent. We decided to use the “closer-to- $b$ ” description in Section 6.4.2.2 because it is slightly more self-contained, but the actual implementation will be based on the above three steps.

It turns out that these subroutines are easier to implement in the *hyperboloid model* instead of the Poincaré model of hyperbolic spaces (see Appendix A.4.4 for a review and see [170] for a more detailed treatment). In the hyperboloid model, the ideal points of hyperbolic spaces are represented by light-like *directions* (instead of individual vectors):

**Proposition C.1.1.** *Each ideal point of  $\mathbb{H}^d$  is represented by a 1-dimensional linear subspace spanned by some  $\vec{v} \in \mathbb{R}^{1,d}$  with  $\langle \vec{v}, \vec{v} \rangle_L = 0$ . The map*

$$(t, x_1, \dots, x_d) \rightarrow \left( \frac{x_1}{t}, \dots, \frac{x_d}{t} \right)$$

*gives a correspondence between a light-like vector  $\vec{v} \in \mathbb{R}^{1,d}$  and an ideal point  $p \in \mathbb{S}_{\infty}^{d-1}$  in the Poincaré*

**Algorithm 2** Horospherical Projection

---

```

1: Input: point  $x$ , ideal points  $\{p_1, \dots, p_K\}$ , base point  $b$ ;
2:  $z \leftarrow \pi_{\text{span}(p_1, \dots, p_K)}^{\text{Mink}}(x)$  ▷ orthogonal projection in ambient space
3:  $w \leftarrow z / \|z\|$  ▷ rescale to a vector on the hyperboloid
4:  $u \leftarrow b - w$  ▷ subtraction in ambient space
5:  $u \leftarrow u - \pi_{\text{span}(p_1, \dots, p_K)}^{\text{Mink}}(u)$  ▷ orthogonal projection in ambient space
6:  $u \leftarrow u / \|u\|$  ▷ make  $u$  the unit tangent vector orthogonal to the spine  $P$ 
7:  $y \leftarrow \exp_w(d_{\mathbb{H}}(x, w) \cdot u)$  ▷ exponential map in  $H^d$ 
8: return  $y$ 

```

---

model that is represented by  $\text{span}(\vec{v})$ . This correspondence is compatible with the stereographic projection in Proposition A.4.5. Its inverse map is given by

$$(y_1, \dots, y_d) \rightarrow (1, y_1, \dots, y_d).$$

Now we describe how these three steps can be implemented in the hyperboloid model. The results of this section are summarized in Algorithm 2. To transfer back and forth between the hyperboloid and Poincaré models, we use the formulas in Proposition A.4.5 and Proposition C.1.1.

**Step 1: Computing geodesic projections** We first describe how to compute the geodesic (or closest-point) projection from  $\mathbb{H}^d$  to a geodesic submanifold  $P$  in  $\mathbb{H}^d$ . This process is very similar to how geodesic projections work in Euclidean spheres: We first perform an orthogonal projection onto the linear subspace  $V$  that cuts out  $P$ , then rescale the result to get a vector on  $\mathbb{H}^d$ .

Generally, orthogonal projections in Minkowski spaces are very similar to those in Euclidean spaces. However, since vectors in  $\mathbb{R}^{1,d}$  can have norm zero, the orthogonal projection  $\pi_V^{\text{Mink}}$  is not well-defined for every subspace  $V$ . Thus, a little extra argument is needed.

**Proposition C.1.2** (Orthogonal projections onto time-containing linear subspaces). *Let  $V$  be a linear subspace of  $\mathbb{R}^{1,d}$  that contains some time-like vectors. Then*

1.  $V \cap V^\perp = \{0\}$ . Consequently, we can define a linear orthogonal projection  $\pi_V^{\text{Mink}} : \mathbb{R}^{1,d} \rightarrow V$  as follows:  
*Since  $\mathbb{R}^{1,d} = V \oplus V^\perp$ , every vector  $\vec{x} \in \mathbb{R}^{1,d}$  can be uniquely written as  $\vec{x} = \vec{z} + \vec{n}$  for some  $\vec{z} \in V$  and  $\vec{n} \in V^\perp$ . Then, we let  $\pi_V^{\text{Mink}}(\vec{x}) := \vec{z}$ .*
2. Let  $A$  be a matrix whose column vectors form a linear basis of  $V$ . Let  $B$  be the  $(1+d) \times (1+d)$  symmetric matrix associated to the bilinear form  $B$ , i.e. the diagonal matrix with entries  $(-1, 1, 1, \dots, 1)$ . Then  $A^\top B A$  is non-singular, and the linear projection  $\pi_V^{\text{Mink}}$  is given by  $\vec{x} \rightarrow A(A^\top B A)^{-1} A^\top B \vec{x}$
3. If  $\vec{x}$  is a future-pointing time-like vector then so is  $\pi_V^{\text{Mink}}(\vec{x})$ .

*Proof.*

1. Let  $\vec{v}$  be a time-like vector in  $V$ . Then  $\vec{v}^\perp$  is space-like by Proposition A.4.2. Since  $V \cap V^\perp \subset V^\perp \subset \vec{v}^\perp$ , the intersection  $V \cap V^\perp$  must be space-like. On the other hand, for every  $\vec{w} \in V \cap V^\perp$ , we have  $\langle \vec{w}, \vec{w} \rangle_L = 0$ , so  $\vec{w}$  must be light-like. It follows that  $V \cap V^\perp = \{0\}$ . The  $\mathbb{R}^{1,d} = V \oplus V^\perp$  part of the claim is a standard linear algebra fact.
2. This follows from the same argument that deduces the formula for Euclidean orthogonal projections.
3. To avoid cumbersome notations, let  $\vec{z} = \pi_V^{\text{Mink}}(\vec{x})$ . Then since  $\vec{x} - \vec{z}$  is orthogonal to  $V$  and in particular to  $\vec{z}$ , we have the Pythagorean formula

$$\langle \vec{z}, \vec{z} \rangle_L + \langle \vec{x} - \vec{z}, \vec{x} - \vec{z} \rangle_L = \langle \vec{x}, \vec{x} \rangle_L.$$

On the other hand, since  $\vec{x} - \vec{z}$  is orthogonal to  $V$  and in particular orthogonal to some time-like vectors in  $V$ , by Proposition A.4.2, it must be space-like. Thus, if  $\langle \vec{x}, \vec{x} \rangle_L < 0$  then the above equation implies  $\langle \vec{z}, \vec{z} \rangle_L < 0$ , which means  $\vec{z}$  is time-like.

Now either  $\vec{z}$  or  $-\vec{z}$  is future-pointing. In the latter case, since  $\vec{x}$  is future-pointing, Proposition A.4.1 implies  $\langle -\vec{z}, \vec{x} \rangle_L < 0$ . On the other hand, we have  $\langle \vec{z}, \vec{x} \rangle_L = \langle \vec{z}, \vec{z} \rangle_L < 0$ , contradicting the above inequality. Thus,  $\vec{z}$  is future-pointing.  $\square$

**Proposition C.1.3** (Geodesic projections in  $\mathbb{H}^d$ ). *Let  $P$  be a geodesic submanifold of  $\mathbb{H}^d$ . Recall that by Proposition A.4.4,  $P = \mathbb{H}^d \cap V$  for some linear subspace  $V$  of  $\mathbb{R}^{1,d}$ . Then for every  $x \in \mathbb{H}^d$ , the geodesic projection  $\pi_P^G(x)$  of  $x$  onto  $P$  in  $\mathbb{H}^d$  is given by*

$$\pi_P^G(x) = \frac{\vec{z}}{\sqrt{-\langle \vec{z}, \vec{z} \rangle_L}},$$

where  $\vec{z} = \pi_V^{\text{Mink}}(\vec{x})$  is the linear orthogonal projection of  $\vec{x}$  onto  $V$ .

*Proof.* Again, to avoid cumbersome notations, let

$$\vec{w} = \frac{\vec{z}}{\sqrt{-\langle \vec{z}, \vec{z} \rangle_L}}.$$

We will show that  $\vec{w} = \pi_P^G(x)$ . First, note that since  $\vec{z}$  is a future-pointing time-like vector by Proposition C.1.2,  $\vec{w} \in \mathbb{H}^d$ . Since  $\vec{w} \in V$ , it follows that  $\vec{w} \in P$ .

Let  $W$  be the linear span of  $\vec{w}$  and  $\vec{x}$ , so that  $W \cap \mathbb{H}^d$  is the geodesic  $\gamma$  in  $\mathbb{H}^d$  that connects  $w$  and  $x$ . Note that  $\vec{w}$  and  $\vec{x} - \vec{z}$  form a basis of  $W$ . They are both orthogonal to the tangent space  $T_w P$  of  $P$  at  $\vec{w}$  because:

- By Proposition A.4.3,  $\vec{w}$  is orthogonal to every tangent vector of  $\mathbb{H}^d$  at  $\vec{w}$ .
- $T_w P$  is contained in  $V$ , which is orthogonal to  $\vec{x} - \vec{z}$ .

Thus,  $W$  is orthogonal to  $T_w P$ . It follows that the geodesic  $\gamma$  is orthogonal to  $P$ , which means  $\vec{v} = \pi_P^G(x)$ .  $\square$

**Step 2: Finding orthogonal geodesic ray** Recall that if  $P$  is a geodesic submanifold of  $\mathbb{H}^d$  and  $b \in \mathbb{H}^d$  is a point not in  $P$ , then the geodesic hull  $M$  of  $P \cup \{b\}$  in  $\mathbb{H}^d$  is a geodesic submanifold of  $\mathbb{H}^d$  with dimension  $\dim P + 1$ . Thus,  $P$  cuts  $M$  into two halves, and we denote the half that contains  $b$  by  $P_b$ .

Given a point  $w \in P$ , there exists a unique geodesic ray  $\alpha^+$  that starts at  $w$ , stays on  $P_b$ , and is orthogonal to  $P$ . In this section, we describe how to compute  $\alpha^+$ .

Recall that by Proposition A.4.4,  $P = V \cap \mathbb{H}^d$  for some linear subspace  $V$ .

**Proposition C.1.4.** *The vector*

$$\vec{u} = (\vec{b} - \vec{w}) - \pi_V^{\text{Mink}}(\vec{b} - \vec{w})$$

*is tangent to  $M$  and orthogonal to  $P$  at  $w$ . Furthermore, it points toward the side of  $P_b$ .*

*Proof.* By construction,  $\vec{u}$  is orthogonal to  $V$ . Note that  $V$  contains both  $\vec{w}$  and the tangent space  $T_w P$  of  $P$  at  $\vec{w}$ . Thus,  $\vec{u}$  is orthogonal to both of them. Together with Proposition A.4.3, this implies that  $\vec{u}$  is a tangent vector of  $\mathbb{H}^d$  at  $\vec{w}$  that is orthogonal to  $P$ .

Next, note that  $M$  is the intersection of  $\mathbb{H}^d$  with the linear span of  $V \cup \{b\}$ , and that  $\vec{u}$  belongs to this linear span. Thus, since  $\vec{u}$  is a tangent vector of  $\mathbb{H}^d$  at  $\vec{w}$ , it must be tangent to  $M$ .

By construction,  $\vec{u}$  points from  $\vec{w}$  toward  $\vec{b}$  instead of away from it. More rigorously, note that by essentially the same argument as above,

$$\vec{a} = (\vec{b} - \vec{w}) - \pi_{\vec{w}}^{\text{Mink}}(\vec{b} - \vec{w})$$

is a tangent vector at  $\vec{w}$  of the geodesic on  $\mathbb{H}^d$  that goes from  $\vec{w}$  to  $\vec{b}$ . Furthermore, since  $V = T_w P \oplus \text{span}(\vec{w})$ , the definition of  $\vec{u}$  means that it is the projection of  $\vec{a}$  onto the orthogonal complement of  $T_w P$  in  $T_w \mathbb{H}^d$ .

In an Euclidean vector space, the dot product of any vector with its projection onto any direction is positive unless the projection is zero. Since  $T_p \mathbb{H}^d$  is a space-like subspace, this statement applies to  $\vec{a}$  and  $\vec{u}$ . Note that  $\vec{u}$  cannot be zero because otherwise  $\vec{b}$  would belong to  $V$  and hence  $P$ . Thus, we conclude that  $\vec{a} \cdot \vec{u} > 0$ , which means that  $\vec{u}$  points toward the side of  $P_b$ .  $\square$

Thus,  $\vec{u}$  is the tangent vector at  $w$  of the desired ray  $\alpha^+$ . To compute points on this ray, we can use the exponential map at  $w$ .

**Step 3: The exponential map** Finally, given a distance  $d$  and a tangent vector  $\vec{u}$  at  $w$  of a geodesic ray  $\alpha^+$ , we need to compute the point on  $\alpha^+$  that is of distance  $d$  from  $w$ . This is based on the following lemma:

**Lemma C.1.2.** *Suppose that  $\vec{u}$  is a unit tangent vector of  $\mathbb{H}^d$  at  $w$ . Then*

$$\alpha(t) = (\cosh t)\vec{w} + (\sinh t)\vec{u}$$

*is a unit-speed geodesic with  $\alpha(0) = \vec{w}$  and  $\dot{\alpha}(0) = \vec{u}$ .*

*Proof.* This follows from a simple direct computation. □

It follows that

**Proposition C.1.5.** *If  $\vec{u}$  is the tangent vector at the starting point  $w$  of a geodesic ray  $\alpha^+$  then*

$$(\cosh d)\vec{w} + (\sinh d)\frac{\vec{u}}{\langle \vec{u}, \vec{u} \rangle_L}$$

*is the point on  $\alpha^+$  that is of distance  $d$  from  $w$ .* □

From the results in this section, we conclude that Algorithm 2 computes the horospherical projection  $\pi_{b,P}^H(x)$ .

## C.2 Geodesic Projection: Distortion Analysis

We show that in hyperbolic geometry, the geodesic projection  $\pi_M^G$  (also called closest-point projection) to a geodesic submanifold  $M$  always strictly decreases distances unless the inputs are already in  $M$ ; furthermore, all paths of distance at least  $r$  from  $M$  get shorter by at least  $\cosh r$  times under the projection. Note that most ideas and results in this section already exist in the literature. For a comprehensive treatment, see [170].

This section is organized as follows. First, we use basic hyperbolic trigonometry to prove a bound on projected distances onto geodesics in dimension two (Proposition C.2.3). Then we generalize this bound to higher dimensions (Theorem C.2.1) and obtain an infinitesimal version of it (Corollary C.2.2). The main theorem (Theorem C.2.3) then follows immediately.

**Hyperbolic trigonometry in two dimensions** We first recall the laws of sine and cosine for triangle in hyperbolic planes:

**Proposition C.2.1.** *Consider any triangle in  $\mathbb{H}^2$  with edge lengths  $A, B, C$ . Let  $\alpha, \beta, \gamma$  be the angles at the vertices opposite to edges  $A, B, C$ . Then*

$$\frac{\sinh A}{\sin \alpha} = \frac{\sinh B}{\sin \beta} = \frac{\sinh C}{\sin \gamma},$$

$$\cos \gamma = \frac{\cosh A \cosh B - \cosh C}{\sinh A \sinh B}.$$

*Proof.* See [170], section 2.6. □

**Proposition C.2.2.** *Consider any quadrilateral in  $\mathbb{H}^2$  with vertices  $x, y, z, w$  (in that order). Suppose that the angles at  $z$  and  $w$  are 90 degrees. Then*

$$\begin{aligned} \cosh(xy) &= \cosh(zw) \cosh(xw) \cosh(yz) \\ &\quad - \sinh(xw) \sinh(yz). \end{aligned}$$

Here  $\cosh(xy)$  is a shorthand for  $\cosh(d_{\mathbb{H}}(x, y))$ .

*Proof.* Applying the laws of sine to triangle  $wxz$  gives

$$\frac{\sinh(xw)}{\sin \angle(zx, zw)} = \frac{\sinh(xz)}{\sin \angle(wx, wz)} = \sinh(xz),$$

so

$$\sin \angle(zx, zw) = \frac{\sinh(xw)}{\sinh(xz)}.$$

On the other hand, applying the law of cosine to triangle  $xyz$  gives

$$\cos \angle(zx, zy) = \frac{\cosh(zx) \cosh(zy) - \cosh(xy)}{\sinh(zx) \sinh(zy)}.$$

Now note that  $\sin \angle(zx, zw) = \cos \angle(zx, zy)$ . Thus,

$$\frac{\sinh(xw)}{\sinh(xz)} = \frac{\cosh(zx) \cosh(zy) - \cosh(xy)}{\sinh(zx) \sinh(zy)},$$

which can be simplified to

$$\cosh(zx) \cosh(zy) - \cosh(xy) = \sinh(xw) \sinh(zy).$$

Finally, note that the law of cosine for triangle  $wxz$  gives

$$\frac{\cosh(wx) \cosh(wz) - \cosh(xz)}{\sinh(wx) \sinh(wz)} = \cos \angle(wx, wz) = 0,$$

so  $\cosh(xz) = \cosh(wx) \cosh(wz)$ . Plugging this into the above equation gives

$$\begin{aligned} &\cosh(wx) \cosh(wz) \cosh(zy) - \cosh(xy) \\ &= \sinh(xw) \sinh(zy), \end{aligned}$$

which can be arranged to get the desired identity. □

Now we are ready to prove a bound on the projected distances onto a geodesic in  $\mathbb{H}^2$ :

**Proposition C.2.3.** *Let  $L$  be a geodesic in  $\mathbb{H}^2$ . Consider any  $x, y \in \mathbb{H}^2$  that are on the same side of  $L$  and are of distances at least  $r$  from  $L$ . Let  $d = d_{\mathbb{H}}(x, y)$  and  $d' = d_{\mathbb{H}}(\pi_L^G(x), \pi_L^G(y))$  denote the original and projected distances of  $x$  and  $y$ . Then,*

$$\sinh(d'/2) \leq \frac{\sinh(d/2)}{\cosh r}.$$

*Proof.* Let  $r_x = d_{\mathbb{H}}(x, \pi_L^G(x))$  and  $r_y = d_{\mathbb{H}}(y, \pi_L^G(y))$ . Then by Proposition C.2.2,

$$\cosh d = \cosh d' \cosh r_x \cosh r_y - \sinh r_x \sinh r_y.$$

Since  $\cosh a \cosh b - \sinh a \sinh b = \cosh(a - b)$ , this gives

$$\cosh d = (\cosh d' - 1) \cosh r_x \cosh r_y + \cosh(r_x - r_y).$$

Subtracting 1 from both sides and using  $\cosh(a) - 1 = 2 \sinh^2(a/2)$  gives

$$\begin{aligned} & 2 \sinh^2(d/2) \\ &= 2 \sinh^2(d'/2) \cosh r_x \cosh r_y + 2 \sinh^2((r_x - r_y)/2). \end{aligned}$$

Thus,

$$\sinh^2(d/2) \geq \sinh^2(d'/2) \cosh r_x \cosh r_y.$$

Since  $r_x, r_y \geq r$ , this implies

$$\sinh^2(d/2) \geq \sinh^2(d'/2) \cosh^2 r,$$

which is equivalent to the desired inequality.  $\square$

**General dimensions** Now we show that Proposition C.2.3 generalizes to higher dimensions.

**Theorem C.2.1.** *Let  $M \subset \mathbb{H}^d$  be a totally geodesic submanifold. Consider any  $x, y \in \mathbb{H}^d$  that are of distances at least  $r$  from  $M$ . Let  $d = d_{\mathbb{H}}(x, y)$  and  $d' = d_{\mathbb{H}}(\pi_M^G(x), \pi_M^G(y))$  denote the original and projected distances of  $x$  and  $y$ . Then,*

$$\sinh(d'/2) \leq \frac{\sinh(d/2)}{\cosh r}.$$

*Proof.* This is clearly true if  $\pi_M^G(x) = \pi_M^G(y)$  or if both  $x$  and  $y$  belong to  $M$ . Thus, without loss of generality, we can assume that  $\pi_M^G(x) \neq \pi_M^G(y)$  and  $\pi_M^G(x) \neq x$ .

Let  $L$  be the geodesic joining  $\pi_M^G(x)$  and  $\pi_M^G(y)$  and  $L_x$  be the half-plane of  $\text{GH}(L \cup \{x\})$  that is bounded by  $L$  and contains  $x$ . Note that if we rotate  $y$  around  $L$  until it hits  $L_x$  at a point  $\bar{y}$  then

$$(i) \quad \pi_L^G(\bar{y}) = \pi_L^G(y).$$

$$(ii) \quad d_{\mathbb{H}}(\bar{y}, L) = d_{\mathbb{H}}(y, L).$$

(iii)  $d_{\mathbb{H}}(x, \bar{y}) \leq d_{\mathbb{H}}(x, y)$  by Corollary C.1.10.

By (i), (ii) and Proposition C.2.3,

$$\sinh(d'/2) \leq \frac{\sinh(d_{\mathbb{H}}(x, \bar{y}))}{2}.$$

Combining this with (iii) gives the desired inequality.  $\square$

Since  $\sinh(t) \approx t$  as  $t \rightarrow 0$ , applying Theorem C.2.1 as  $y \rightarrow x$  gives

**Corollary C.2.2.** Under the geodesic projection  $\pi_M^G(\cdot)$ , every tangent vector  $\vec{v}$  at  $x$  gets at least  $\cosh(d_{\mathbb{H}}(x, M))$  times shorter.  $\square$

This implies the main theorem of this section:

**Theorem C.2.3.** Let  $\gamma(t)$  be any smooth path in  $\mathbb{H}^d$  that stays a distance at least  $r$  away from  $M$ . Then

$$\text{length}(\pi_M^G(\gamma)) \leq \frac{\text{length}(\gamma)}{\cosh r}.$$

In particular,  $\text{length}(\pi_M^G(\gamma)) < \text{length}(\gamma)$  unless  $\gamma$  is entirely in  $M$ .

*Proof.* Note that the velocity vector of  $\pi_M^G(\gamma)$  is precisely  $\pi_M^G(\dot{\gamma})$ . Then, by Corollary C.2.2,

$$\begin{aligned} \text{length}(\pi_M^G(\gamma)) &= \int \|\pi_M^G(\dot{\gamma}(t))\|_{\mathbb{H}} dt \\ &\leq \int \frac{\|\dot{\gamma}(t)\|_{\mathbb{H}}}{\cosh r} dt = \frac{\text{length}(\gamma)}{\cosh r}. \end{aligned}$$

Since  $\cosh r \geq 1$ , this implies

$$\text{length}(\pi_M^G(\gamma)) \leq \text{length}(\gamma).$$

This can only be an equality if the  $\cosh r$  factor is constantly 1, which means  $\gamma$  stays in  $M$ .  $\square$

When  $\gamma$  is a geodesic, this gives a proof of Section 6.4.2.2: Let  $M \subset \mathbb{H}^d$  be a geodesic submanifold. Then every geodesic segment of distance at least  $r$  from  $M$  gets at least  $\cosh(r)$  times shorter under the geodesic projection  $\pi_M^G(\cdot)$  to  $M$ :

$$\text{length}(\pi_M^G(I)) \leq \frac{1}{\cosh(r)} \text{length}(I).$$

In particular, the shrink factor grows exponentially as the segment  $I$  moves away from  $M$ .  $\square$

**Remark C.2.1.** The above proposition does *not* imply that if  $x$  and  $y$  are of distances at least  $r$  from  $M$  then  $d_{\mathbb{H}}(x, y)$  gets at least  $\cosh r$  times shorter under the geodesic projection. This is because the geodesic segment between  $x$  and  $y$  can get closer to  $M$  in the middle. This is inevitable when  $x$

and  $y$  are extremely far apart: For example, if  $d_{\mathbb{H}}(x, M) = d_{\mathbb{H}}(y, M) = r$ , then the triangle inequality implies

$$d_{\mathbb{H}}(\pi_M^G(x), \pi_M^G(y)) \geq d_{\mathbb{H}}(x, y) - 2r.$$

Hence, if  $d_{\mathbb{H}}(x, y) \gg r$  then the shrink factor cannot be much smaller than 1. Generally, the rule of thumb is that geodesic projections shrink distances by a big factor if the input points are not very far from each other but quite far from the target submanifold.

# Bibliography

- [1] Amir Abboud, Vincent Cohen-Addad, and Hussein Houdrougé. Subquadratic high-dimensional hierarchical clustering. In *Advances in Neural Information Processing Systems*, pages 11576–11586, 2019.
- [2] Sami Abu-El-Haija, Bryan Perozzi, Rami Al-Rfou, and Alexander A Alemi. Watch your step: Learning node embeddings via graph attention. In *Advances in Neural Information Processing Systems*, pages 9180–9190, 2018.
- [3] Aaron B Adcock, Blair D Sullivan, and Michael W Mahoney. Tree-like structure in large social and information networks. In *2013 IEEE 13th International Conference on Data Mining*, pages 1–10. IEEE, 2013.
- [4] Sara Ahmadian, Vaggos Chatziafratis, Alessandro Epasto, Euiwoong Lee, Mohammad Mahdian, Konstantin Makarychev, and Grigory Yaroslavtsev. Bisect and conquer: Hierarchical clustering via max-uncut bisection. *arXiv preprint arXiv:1912.06983*, 2019.
- [5] Amr Ahmed, Nino Shervashidze, Shravan Narayanamurthy, Vanja Josifovski, and Alexander J Smola. Distributed large-scale natural graph factorization. In *Proceedings of the 22nd international conference on World Wide Web*, pages 37–48. ACM, 2013.
- [6] Rami Al-Rfou, Dustin Zelle, and Bryan Perozzi. Ddgc: Learning graph representations for deep divergence graph kernels. *Proceedings of the 2019 World Wide Web Conference on World Wide Web*, 2019.
- [7] Gregorio Alanis-Lobato, Pablo Mier, and Miguel A Andrade-Navarro. Efficient embedding of complex networks to hyperbolic space via their laplacian. *Scientific reports*, 6:30108, 2016.
- [8] Gregorio Alanis-Lobato, Pablo Mier, and Miguel Andrade-Navarro. The latent geometry of the human protein interaction network. *Bioinformatics*, 34(16):2826–2834, 2018.
- [9] Josh Alman and Ryan Williams. Probabilistic polynomials and hamming nearest neighbors. In *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*, pages 136–150. IEEE, 2015.

- [10] Noga Alon, Yossi Azar, and Danny Vainstein. Hierarchical clustering: a 0.585 revenue approximation. In *Conference on Learning Theory*, pages 153–162. PMLR, 2020.
- [11] Roy M Anderson and Robert M May. *Infectious diseases of humans: dynamics and control*. Oxford university press, 1992.
- [12] Ivana Balažević, Carl Allen, and Timothy Hospedales. Multi-relational poincaré graph embeddings. In *Advances in Neural Information Processing Systems*, pages 4465–4475, 2019.
- [13] Ivana Balazevic, Carl Allen, and Timothy Hospedales. Tucker: Tensor factorization for knowledge graph completion. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5188–5197, 2019.
- [14] Nikhil Bansal, Avrim Blum, and Shuchi Chawla. Correlation clustering. *Machine Learning*, 56(1-3):89–113, 2004.
- [15] Trapit Bansal, Da-Cheng Juan, Sujith Ravi, and Andrew McCallum. A2n: Attending to neighbors for knowledge graph inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4387–4392, 2019.
- [16] Yair Bartal, Nova Fandina, and Ofer Neiman. Covering metric spaces by few trees. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132, page 20. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019.
- [17] MohammadHossein Bateni, Soheil Behnezhad, Mahsa Derakhshan, MohammadTaghi Hajiaghayi, Raimondas Kiveris, Silvio Lattanzi, and Vahab Mirrokni. Affinity clustering: Hierarchical clustering at scale. In *Advances in Neural Information Processing Systems*, pages 6864–6874, 2017.
- [18] Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, et al. Interaction networks for learning about objects, relations and physics. In *Advances in Neural Information Processing Systems*, pages 4502–4510, 2016.
- [19] Gary Bécigneul and Octavian-Eugen Ganea. Riemannian adaptive optimization methods. *arXiv preprint arXiv:1810.00760*, 2018.
- [20] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. In *Advances in neural information processing systems*, pages 585–591, 2002.
- [21] Mikhail Belkin and Partha Niyogi. Semi-supervised learning on riemannian manifolds. *Machine learning*, 56(1-3):209–239, 2004.

- [22] Mikhail Belkin, Partha Niyogi, and Vikas Sindhwani. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of machine learning research*, 7(Nov):2399–2434, 2006.
- [23] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.
- [24] Itai Benjamini and Yury Makarychev. Dimension reduction for hyperbolic space. *Proceedings of the American Mathematical Society*, pages 695–698, 2009.
- [25] Louis J Billera, Susan P Holmes, and Karen Vogtmann. Geometry of the space of phylogenetic trees. *Advances in Applied Mathematics*, 27(4):733–767, 2001.
- [26] Silvere Bonnabel. Stochastic gradient descent on riemannian manifolds. *IEEE Transactions on Automatic Control*, 2013.
- [27] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in neural information processing systems*, pages 2787–2795, 2013.
- [28] Brian H Bowditch. A course on geometric group theory, msj memoirs, vol. 16, mathematical society of japan, tokyo, 2006. *MR2243589 (2007e: 20085)*, 2007.
- [29] DA Brannan, F Esplen, and JJ Gray. *Geometry*. sl cambridge, 2011.
- [30] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann Lecun. Spectral networks and locally connected networks on graphs international conference on learning representations (iclr2014). *CBLIS, April*, 2014.
- [31] Thang D Bui, Sujith Ravi, and Vivek Ramavajjala. Neural graph learning: Training neural networks using graphs. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, pages 64–71, 2018.
- [32] Herbert Busemann. *The geometry of geodesics*. Academic Press Inc., New York, N. Y., 1955.
- [33] James W Cannon, William J Floyd, Richard Kenyon, Walter R Parry, et al. Hyperbolic geometry. *Flavors of geometry*, 31:59–115, 1997.
- [34] Shaosheng Cao, Wei Lu, and Qiongkai Xu. Grarep: Learning graph representations with global structural information. In *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*, pages 891–900. ACM, 2015.
- [35] Shaosheng Cao, Wei Lu, and Qiongkai Xu. Deep neural networks for learning graph representations. In *AAAI*, pages 1145–1152, 2016.

- [36] Manfredo Perdigao do Carmo. *Riemannian geometry*. Birkhäuser, 1992.
- [37] Benjamin Paul Chamberlain, James Clough, and Marc Peter Deisenroth. Neural embeddings of graphs in hyperbolic space. *arXiv preprint arXiv:1705.10359*, 2017.
- [38] Ines Chami, Zhitao Ying, Christopher Ré, and Jure Leskovec. Hyperbolic graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, pages 4869–4880, 2019.
- [39] Ines Chami, Sami Abu-El-Haija, Bryan Perozzi, Christopher Ré, and Kevin Murphy. Machine learning on graphs: A model and comprehensive taxonomy. *arXiv preprint arXiv:2005.03675*, 2020.
- [40] Ines Chami, Albert Gu, Vaggos Chatziafratis, and Christopher Ré. From trees to continuous embeddings and back: Hyperbolic hierarchical clustering. *Advances in Neural Information Processing Systems*, 33, 2020.
- [41] Ines Chami, Adva Wolf, Da-Cheng Juan, Frederic Sala, Sujith Ravi, and Christopher Ré. Low-dimensional hyperbolic knowledge graph embeddings. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [42] Ines Chami, Albert Gu, Dat Nguyen, and Christopher Ré. Hyperbolic dimensionality reduction via horospherical projections. In *International Conference on Machine Learning*. PMLR, 2021.
- [43] Moses Charikar and Vaggos Chatziafratis. Approximate hierarchical clustering via sparsest cut and spreading metrics. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 841–854. SIAM, 2017.
- [44] Moses Charikar, Venkatesan Guruswami, and Anthony Wirth. Clustering with qualitative information. *Journal of Computer and System Sciences*, 71(3):360–383, 2005.
- [45] Moses Charikar, Vaggos Chatziafratis, and Rad Niazadeh. Hierarchical clustering better than average-linkage. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 2291–2304. SIAM, 2019.
- [46] Moses Charikar, Vaggos Chatziafratis, Rad Niazadeh, and Grigory Yaroslavltssev. Hierarchical clustering for euclidean data. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pages 2721–2730, 2019.
- [47] Evangelos Chatziafratis. Hierarchical clustering with global objectives: Approximation algorithms and hardness results. [https://cs.stanford.edu/~vaggos/vaggos\\_thesis.pdf](https://cs.stanford.edu/~vaggos/vaggos_thesis.pdf), 2020. PhD Thesis, Stanford University.

- [48] Haochen Chen, Bryan Perozzi, Yifan Hu, and Steven Skiena. Harp: Hierarchical representation learning for networks. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [49] Wei Chen, Wenjie Fang, Guangda Hu, and Michael W Mahoney. On the hyperbolicity of small-world and treelike random graphs. *Internet Mathematics*, 9(4):434–491, 2013.
- [50] Zhengdao Chen, Joan Bruna Estrach, and Lisha Li. Supervised community detection with line graph neural networks. In *7th International Conference on Learning Representations, ICLR 2019*, 2019.
- [51] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2019. URL <https://arxiv.org/pdf/1905.07953.pdf>.
- [52] Giovanni Chierchia and Benjamin Perret. Ultrametric fitting by gradient descent. In *Advances in neural information processing systems*, pages 3175–3186, 2019.
- [53] Hyunghoon Cho, Benjamin DeMeo, Jian Peng, and Bonnie Berger. Large-margin classification in hyperbolic space. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pages 1832–1840. PMLR, 2019.
- [54] Aaron Clauset, Cristopher Moore, and Mark EJ Newman. Hierarchical structure and the prediction of missing links in networks. *Nature*, 453(7191):98, 2008.
- [55] Vincent Cohen-Addad, Varun Kanade, and Frederik Mallmann-Trenn. Hierarchical clustering beyond the worst-case. In *Advances in Neural Information Processing Systems*, pages 6201–6209, 2017.
- [56] Vincent Cohen-Addad, Varun Kanade, Frederik Mallmann-Trenn, and Claire Mathieu. Hierarchical clustering: Objective functions and algorithms. *Journal of the ACM (JACM)*, 66(4): 1–42, 2019.
- [57] Michael AA Cox and Trevor F Cox. Multidimensional scaling. In *Handbook of data visualization*, pages 315–347. Springer, 2008.
- [58] Daniel Fernando Daza Cruz, Thomas Kipf, and Max Welling. A modular framework for unsupervised graph representation learning, 2019.
- [59] Karthik CS and Pasin Manurangsi. On closest pair in euclidean metric: Monochromatic is as hard as bichromatic. In *10th Innovations in Theoretical Computer Science Conference (ITCS 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2018.

- [60] Andrej Cvetkovski and Mark Crovella. Multidimensional scaling in the poincaré disk. *arXiv preprint arXiv:1105.5332*, 2011.
- [61] Sanjoy Dasgupta. Performance guarantees for hierarchical clustering. In *International Conference on Computational Learning Theory*, pages 351–363. Springer, 2002.
- [62] Sanjoy Dasgupta. A cost function for similarity-based hierarchical clustering. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 118–127, 2016.
- [63] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems*, pages 3844–3852, 2016.
- [64] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2D knowledge graph embeddings. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [65] Bhuwan Dhingra, Christopher J Shallue, Mohammad Norouzi, Andrew M Dai, and George E Dahl. Embedding text in hyperbolic spaces. *NAACL HLT*, 2018.
- [66] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.
- [67] David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alán Aspuru-Guzik, and Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints. In *Advances in neural information processing systems*, pages 2224–2232, 2015.
- [68] Andre Esteva, Brett Kuprel, Roberto A Novoa, Justin Ko, Susan M Swetter, Helen M Blau, and Sebastian Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *nature*, 542(7639):115–118, 2017.
- [69] Joseph Felsenstein. *Inferring phylogenies*, volume 2. Sinauer associates Sunderland, MA, 2004.
- [70] P. T. Fletcher, Conglin Lu, S. M. Pizer, and Sarang Joshi. Principal geodesic analysis for the study of nonlinear statistics of shape. *IEEE Transactions on Medical Imaging*, 23(8):995–1005, 2004.
- [71] Tom Fletcher. Terse notes on riemannian geometry, 2010.
- [72] Maurice Fréchet. Les éléments aléatoires de nature quelconque dans un espace distancié. In *Annales de l’institut Henri Poincaré*, 1948.
- [73] Octavian Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In *Advances in neural information processing systems*, pages 5345–5355, 2018.

- [74] Octavian-Eugen Ganea, Gary Becigneul, and Thomas Hofmann. Hyperbolic entailment cones for learning hierarchical embeddings. In *International Conference on Machine Learning*, pages 1632–1641, 2018.
- [75] Victor Garcia and Joan Bruna. Few-shot learning with graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [76] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 1263–1272. JMLR. org, 2017.
- [77] Marco Gori, Gabriele Monfardini, and Franco Scarselli. A new model for learning in graph domains. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, volume 2, pages 729–734. IEEE, 2005.
- [78] Mikhael Gromov. Hyperbolic groups. In *Essays in group theory*, pages 75–263. Springer, 1987.
- [79] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864. ACM, 2016.
- [80] Aditya Grover, Aaron Zweig, and Stefano Ermon. Graphite: Iterative generative modeling of graphs. In *International Conference on Machine Learning*, pages 2434–2444, 2019.
- [81] Albert Gu, Frederic Sala, Beliz Gunel, and Christopher Ré. Learning mixed-curvature representations in product spaces. In *International Conference on Learning Representations*, 2018.
- [82] Albert Gu, Fred Sala, Beliz Gunel, and Christopher Ré. Learning mixed-curvature representations in product spaces. In *International Conference on Learning Representations*, 2019.
- [83] Caglar Gulcehre, Misha Denil, Mateusz Malinowski, Ali Razavi, Razvan Pascanu, Karl Moritz Hermann, Peter Battaglia, Victor Bapst, David Raposo, Adam Santoro, et al. Hyperbolic attention networks. In *International Conference on Learning Representations*, 2019.
- [84] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, pages 1024–1034, 2017.
- [85] William L Hamilton, Rex Ying, and Jure Leskovec. Representation learning on graphs: Methods and applications. *arXiv preprint arXiv:1709.05584*, 2017.
- [86] John A Hartigan. *Clustering algorithms*. John Wiley & Sons, Inc., 1975.
- [87] Søren Hauberg. Principal curves on riemannian manifolds. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38(9):1915–1921, 2016. doi: 10.1109/TPAMI.2015.2496166.

- [88] Katherine A Heller and Zoubin Ghahramani. Bayesian hierarchical clustering. In *Proceedings of the 22nd international conference on Machine learning*, pages 297–304, 2005.
- [89] Mikael Henaff, Joan Bruna, and Yann LeCun. Deep convolutional networks on graph-structured data. *arXiv preprint arXiv:1506.05163*, 2015.
- [90] Stephan Huckemann and Benjamin Eltzner. Statistical methods generalizing principal component analysis to non-euclidean spaces. In *Handbook of Variational Methods for Nonlinear Geometric Data*, pages 317–338. Springer, 2020.
- [91] Stephan Huckemann and Herbert Ziezold. Principal component analysis for riemannian manifolds, with an application to triangular shape spaces. *Advances in Applied Probability*, 38(2):299–319, 2006.
- [92] Stephan Huckemann, Thomas Hotz, and Axel Munk. Intrinsic shape analysis: Geodesic pca for riemannian manifolds modulo isometric lie group actions. *Statistica Sinica*, pages 1–58, 2010.
- [93] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456, 2015.
- [94] Guoliang Ji, Shizhu He, Liheng Xu, Kang Liu, and Jun Zhao. Knowledge graph embedding via dynamic mapping matrix. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 687–696, 2015.
- [95] Ian Jolliffe. Principal component analysis. In *International encyclopedia of statistical science*, pages 1094–1096. Springer, 2011.
- [96] Edmond Jonckheere, Poonsuk Lohsoonthorn, and Francis Bonahon. Scaled gromov hyperbolic graphs. *Journal of Graph Theory*, 2008.
- [97] Martin Keller-Ressel. A theory of hyperbolic prototype learning. *arXiv preprint arXiv:2010.07744*, 2020.
- [98] Wilfrid S Kendall. Probability, convexity, and harmonic maps with small image i: uniqueness and fine existence. *Proceedings of the London Mathematical Society*, 3(2):371–406, 1990.
- [99] Valentin Khrulkov, Leyla Mirvakhabova, Evgeniya Ustinova, Ivan Oseledets, and Victor Lempitsky. Hyperbolic image embeddings. *arXiv preprint arXiv:1904.02239*, 2019.
- [100] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

- [101] Thomas N Kipf and Max Welling. Variational graph auto-encoders. *NeurIPS Workshop on Bayesian Deep Learning*, 2016.
- [102] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=SJU4ayYgl>.
- [103] Robert Kleinberg. Geographic routing using hyperbolic space. In *IEEE International Conference on Computer Communications*, 2007.
- [104] Max Kochurov, Rasul Karimov, and Sergei Kozlukov. Geoopt: Riemannian optimization in pytorch. *arXiv preprint arXiv:2005.02819*, 2020.
- [105] Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Joan Puigcerver, Jessica Yung, Sylvain Gelly, and Neil Houlsby. Large scale learning of general visual representations for transfer. *arXiv preprint arXiv:1912.11370*, 2019.
- [106] Ioannis Konstas, Vassilios Stathopoulos, and Joemon M. Jose. On social networks and collaborative recommendation. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 195–202, 2009.
- [107] David Krackhardt. Graph theoretical dimensions of informal organizations. In *Computational organization theory*, pages 107–130. Psychology Press, 1994.
- [108] Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguná. Hyperbolic geometry of complex networks. *Physical Review E*, 2010.
- [109] Alex Krizhevsky et al. Learning multiple layers of features from tiny images. 2009.
- [110] Joseph B Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 1964.
- [111] Timothée Lacroix, Nicolas Usunier, and Guillaume Obozinski. Canonical tensor decomposition for knowledge base completion. *International Conference on Machine Learning*, 2018.
- [112] Matthew Le, Stephen Roller, Laetitia Papaxanthos, Douwe Kiela, and Maximilian Nickel. Inferring concept hierarchies from text corpora via hyperbolic embeddings. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3231–3241, 2019.
- [113] John M Lee. Smooth manifolds. In *Introduction to Smooth Manifolds*, pages 1–31. Springer, 2013.
- [114] Jure Leskovec, Anand Rajaraman, and Jeffrey David Ullman. *Mining of massive data sets*. Cambridge university press, 2019.

- [115] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *International Conference on Learning Representations*, 2016.
- [116] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American society for information science and technology*, 58(7):1019–1031, 2007.
- [117] Yankai Lin, Zhiyuan Liu, Maosong Sun, Yang Liu, and Xuan Zhu. Learning entity and relation embeddings for knowledge graph completion. In *Twenty-ninth AAAI Conference on Artificial Intelligence*, 2015.
- [118] Qi Liu, Maximilian Nickel, and Douwe Kiela. Hyperbolic graph neural networks. In *Advances in Neural Information Processing Systems*, pages 8228–8239, 2019.
- [119] Aaron Lou, Maximilian Nickel, and Brandon Amos. Deep riemannian manifold learning. *Differential Geometry for Machine Learning Workshop at NeurIPS*, 2020.
- [120] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(Nov):2579–2605, 2008.
- [121] Farzaneh Mahdisoltani, Joanna Biega, and Fabian M Suchanek. Yago3: A knowledge base from multilingual wikipedias. 2013.
- [122] George A Miller. Wordnet: a lexical database for english. *Communications of the ACM*, 38(11):39–41, 1995.
- [123] Nicholas Monath, Ari Kobren, Akshay Krishnamurthy, and Andrew McCallum. Gradient-based hierarchical clustering. In *NIPS Workshop on Discrete Structures in Machine Learning*, 2017.
- [124] Nicholas Monath, Manzil Zaheer, Daniel Silva, Andrew McCallum, and Amr Ahmed. Gradient-based hierarchical clustering using continuous representations of trees in hyperbolic space. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 714–722, 2019.
- [125] Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodola, Jan Svoboda, and Michael M Bronstein. Geometric deep learning on graphs and manifolds using mixture model cnns. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5115–5124, 2017.
- [126] Kevin R Moon, Jay S Stanley III, Daniel Burkhardt, David van Dijk, Guy Wolf, and Smita Krishnaswamy. Manifold learning-based methods for analyzing single-cell rna-sequencing data. *Current Opinion in Systems Biology*, 7:36–46, 2018.

- [127] Benjamin Moseley and Joshua Wang. Approximation bounds for hierarchical clustering: Average linkage, bisecting k-means, and local search. In *Advances in Neural Information Processing Systems*, pages 3094–3103, 2017.
- [128] Alessandro Muscoloni, Josephine Maria Thomas, Sara Ciucci, Ginestra Bianconi, and Carlo Vittorio Cannistraci. Machine learning meets complex networks via coalescent embedding in the hyperbolic space. *Nature communications*, 8(1):1–19, 2017.
- [129] Galileo Namata, Ben London, Lise Getoor, Bert Huang, and UMD EDU. Query-driven active surveying for collective classification. 2012.
- [130] Onuttom Narayan and Iraj Saniee. Large-scale curvature of networks. *Physical Review E*, 84(6):066108, 2011.
- [131] Deepak Nathani, Jatin Chauhan, Charu Sharma, and Manohar Kaul. Learning attention-based embeddings for relation prediction in knowledge graphs. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019.
- [132] Dai Quoc Nguyen, Tu Dinh Nguyen, Dat Quoc Nguyen, and Dinh Phung. A Novel Embedding Model for Knowledge Base Completion Based on Convolutional Neural Network. In *Proceedings of the 16th Annual Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 327–333, 2018.
- [133] Chien-Chun Ni, Yu-Yao Lin, Jie Gao, Xianfeng David Gu, and Emil Saucan. Ricci curvature of the internet topology. In *2015 IEEE conference on computer communications (INFOCOM)*, pages 2758–2766. IEEE, 2015.
- [134] Maximilian Nickel, Volker Tresp, and Hans-Peter Kriegel. A three-way model for collective learning on multi-relational data. In *International Conference on Machine Learning*, pages 809–816. Omnipress, 2011.
- [135] Maximillian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems*, pages 6338–6347, 2017.
- [136] Maximillian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems*, pages 6338–6347, 2017.
- [137] Maximillian Nickel and Douwe Kiela. Learning continuous hierarchies in the lorentz model of hyperbolic geometry. In *International Conference on Machine Learning*, pages 3779–3788, 2018.

- [138] Mingdong Ou, Peng Cui, Jian Pei, Ziwei Zhang, and Wenwu Zhu. Asymmetric transitivity preserving graph embedding. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1105–1114. ACM, 2016.
- [139] Fragkiskos Papadopoulos, Maksim Kitsak, M Ángeles Serrano, Marián Boguná, and Dmitri Krioukov. Popularity versus similarity in growing networks. *Nature*, 489(7417):537–540, 2012.
- [140] Fragkiskos Papadopoulos, Constantinos Psomas, and Dmitri Krioukov. Network mapping by replaying hyperbolic growth. *IEEE/ACM Transactions on Networking*, 23(1):198–211, 2014.
- [141] Xavier Pennec. Barycentric subspace analysis on manifolds. *Annals of Statistics*, 46(6A): 2711–2746, 2018.
- [142] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710. ACM, 2014.
- [143] Erzsébet Ravasz and Albert-László Barabási. Hierarchical organization in complex networks. *Physical review E*, 2003.
- [144] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *arXiv preprint arXiv:1506.01497*, 2015.
- [145] Joel W Robbin and Dietmar A Salamon. Introduction to differential geometry. *ETH, Lecture Notes, preliminary version*, 2011.
- [146] Sam T Roweis and Lawrence K Saul. Nonlinear dimensionality reduction by locally linear embedding. *science*, 290(5500):2323–2326, 2000.
- [147] Aurko Roy and Sebastian Pokutta. Hierarchical clustering via spreading metrics. *The Journal of Machine Learning Research*, 18(1):3077–3111, 2017.
- [148] Benedek Rozemberczki, Ryan Davies, Rik Sarkar, and Charles Sutton. Gemsec: Graph embedding with self clustering. In *Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, ASONAM '19*, page 65–72, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450368681. doi: 10.1145/3341161.3342890. URL <https://doi.org/10.1145/3341161.3342890>.
- [149] Roshni Sahoo, Ines Chami, and Christopher Ré. Tree covers: An alternative to metric embeddings. *Differential Geometry for Machine Learning Workshop at NeurIPS*, 2020.
- [150] Frederic Sala, Chris De Sa, Albert Gu, and Christopher Ré. Representation tradeoffs for hyperbolic embeddings. In *International Conference on Machine Learning*, pages 4460–4469, 2018.

- [151] Romeil Sandhu, Tryphon Georgiou, Ed Reznik, Liangjia Zhu, Ivan Kolesov, Yasin Senbabaoglu, and Allen Tannenbaum. Graph curvature for differentiating cancer networks. *Scientific reports*, 5(1):1–13, 2015.
- [152] Rik Sarkar. Low distortion delaunay embedding of trees in hyperbolic plane. In *International Symposium on Graph Drawing*, 2011.
- [153] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.
- [154] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, pages 593–607. Springer, 2018.
- [155] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI magazine*, 2008.
- [156] Andrew W Senior, Richard Evans, John Jumper, James Kirkpatrick, Laurent Sifre, Tim Green, Chongli Qin, Augustin Židek, Alexander WR Nelson, Alex Bridgland, et al. Improved protein structure prediction using potentials from deep learning. *Nature*, 577(7792):706–710, 2020.
- [157] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- [158] Stefan Sommer. Horizontal dimensionality reduction and iterated frame bundle development. In *International Conference on Geometric Science of Information*, pages 76–83. Springer, 2013.
- [159] Therese Sørlie, Charles M Perou, Robert Tibshirani, Turid Aas, Stephanie Geisler, Hilde Johnsen, Trevor Hastie, Michael B Eisen, Matt Van De Rijn, Stefanie S Jeffrey, et al. Gene expression patterns of breast carcinomas distinguish tumor subclasses with clinical implications. *Proceedings of the National Academy of Sciences*, 98(19):10869–10874, 2001.
- [160] Christos Sotiriou, Soek-Ying Neo, Lisa M McShane, Edward L Korn, Philip M Long, Amir Jazaeri, Philippe Martiat, Steve B Fox, Adrian L Harris, and Edison T Liu. Breast cancer classification and prognosis based on gene expression profiles from a population-based study. *Proceedings of the National Academy of Sciences*, 100(18):10393–10398, 2003.
- [161] Balasubramaniam Srinivasan and Bruno Ribeiro. On the equivalence between positional node embeddings and structural graph representations. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SJxzFySKwH>.

- [162] Chris Stark, Bobby-Joe Breitkreutz, Teresa Reguly, Lorrie Boucher, Ashton Breitkreutz, and Mike Tyers. Biogrid: a general repository for interaction datasets. *Nucleic acids research*, 34 (suppl\_1):D535–D539, 2006.
- [163] Michael Steinbach, George Karypis, and Vipin Kumar. A comparison of document clustering techniques. In *KDD workshop on text mining*, volume 400, pages 525–526. Boston, 2000.
- [164] Fabian M Suchanek, Gjergji Kasneci, and Gerhard Weikum. Yago: a core of semantic knowledge. In *Proceedings of the 16th international conference on World Wide Web*, pages 697–706. ACM, 2007.
- [165] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. Rotate: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*, 2019.
- [166] Damian Szklarczyk, John H Morris, Helen Cook, Michael Kuhn, Stefan Wyder, Milan Simonovic, Alberto Santos, Nadezhda T Doncheva, Alexander Roth, Peer Bork, et al. The string database in 2017: quality-controlled protein–protein association networks, made broadly accessible. *Nucleic acids research*, 2016.
- [167] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. Line: Large-scale information network embedding. In *Proceedings of the 24th International Conference on World Wide Web*, pages 1067–1077. International World Wide Web Conferences Steering Committee, 2015.
- [168] Yi Tay, Luu Anh Tuan, and Siu Cheung Hui. Hyperbolic representation learning for fast and efficient neural question answering. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, pages 583–591, 2018.
- [169] Joshua B Tenenbaum, Vin De Silva, and John C Langford. A global geometric framework for nonlinear dimensionality reduction. *science*, 290(5500):2319–2323, 2000.
- [170] William P. Thurston. Geometry and topology of three-manifolds, 1978. Lecture notes. Available at <http://library.msri.org/books/gt3m/>.
- [171] Alexandru Tifrea, Gary Becigneul, and Octavian-Eugen Ganea. Poincaré glove: Hyperbolic word embeddings. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Ske5r3AqK7>.
- [172] Kristina Toutanova and Danqi Chen. Observed versus latent features for knowledge base and text inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, pages 57–66, 2015.

- [173] Duc A Tran and Khanh Vu. Dimensionality reduction in hyperbolic data spaces: Bounding reconstructed-information loss. In *Seventh IEEE/ACIS International Conference on Computer and Information Science (icis 2008)*, pages 133–139. IEEE, 2008.
- [174] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction. In *International Conference on Machine Learning*, pages 2071–2080, 2016.
- [175] Anton Tsitsulin, Davide Mottin, Panagiotis Karras, Alexander Bronstein, and Emmanuel Müller. Netlsd: Hearing the shape of a graph. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD ’18, page 2347–2356, 2018.
- [176] Anton Tsitsulin, Marina Munkhoeva, and Bryan Perozzi. Just slaq when you approximate: Accurate spectral distances for web-scale graphs. In *Proceedings of The Web Conference 2020*, WWW ’20, page 2697–2703, 2020.
- [177] Abraham A Ungar. The hyperbolic pythagorean theorem in the poincaré disc model of hyperbolic geometry. *The American mathematical monthly*, 106(8):759–763, 1999.
- [178] Abraham Albert Ungar. A gyrovector space approach to hyperbolic geometry. *Synthesis Lectures on Mathematics and Statistics*, 1(1):1–194, 2008.
- [179] Joyce van de Leemput, Nathan C Boles, Thomas R Kiehl, Barbara Corneo, Patty Lederman, Vilas Menon, Changkyu Lee, Refugio A Martinez, Boaz P Levi, Carol L Thompson, et al. Cortecon: a temporal transcriptome analysis of in vitro human cerebral cortex development from human embryonic stem cells. *Neuron*, 2014.
- [180] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [181] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. Deep graph infomax. In *International Conference on Learning Representations*, 2019.
- [182] Li Wan, Matthew Zeiler, Sixin Zhang, Yann Le Cun, and Rob Fergus. Regularization of neural networks using dropconnect. In *International conference on machine learning*, pages 1058–1066, 2013.
- [183] Daixin Wang, Peng Cui, and Wenwu Zhu. Structural deep network embedding. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1225–1234. ACM, 2016.

- [184] Dingkan Wang and Yusu Wang. An improved cost function for hierarchical cluster trees. *arXiv preprint arXiv:1812.02715*, 2018.
- [185] Ming-Xi Wang. Laplacian eigenspaces, horocycles and neuron models on hyperbolic spaces, 2021. URL <https://openreview.net/forum?id=ZglaBL5inu>.
- [186] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. Knowledge graph embedding by translating on hyperplanes. In *Twenty-Eighth AAAI Conference on Artificial Intelligence*, 2014.
- [187] Jason Weston, Frédéric Ratle, and Ronan Collobert. Deep learning via semi-supervised embedding. In *Proceedings of the 25th international conference on Machine learning*, pages 1168–1175. ACM, 2008.
- [188] Richard C Wilson, Edwin R Hancock, Elżbieta Pekalska, and Robert PW Duin. Spherical and hyperbolic embeddings of data. *IEEE transactions on pattern analysis and machine intelligence*, 36(11):2255–2269, 2014.
- [189] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *International Conference on Machine Learning*, pages 6861–6871, 2019.
- [190] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. A comprehensive survey on graph neural networks. *arXiv preprint arXiv:1901.00596*, 2019.
- [191] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.
- [192] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations*, 2015.
- [193] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. Revisiting semi-supervised learning with graph embeddings. In *International Conference on Machine Learning*, pages 40–48, 2016.
- [194] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 974–983. ACM, 2018.
- [195] Zhitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, Will Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in Neural Information Processing Systems*, pages 4800–4810, 2018.

- [196] Tao Yu and Christopher M De Sa. Numerically accurate hyperbolic embeddings using tiling-based models. In *Advances in Neural Information Processing Systems*, pages 2023–2033, 2019.
- [197] Hongyi Zhang, Sashank J Reddi, and Suvrit Sra. Riemannian svrg: Fast stochastic optimization on riemannian manifolds. In *Advances in Neural Information Processing Systems*, pages 4592–4600, 2016.
- [198] Miaomiao Zhang and Tom Fletcher. Probabilistic principal geodesic analysis. In *Advances in Neural Information Processing Systems*, pages 1178–1186, 2013.
- [199] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. In *Advances in Neural Information Processing Systems*, pages 5165–5175, 2018.
- [200] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. An end-to-end deep learning architecture for graph classification. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [201] Shuai Zhang, Yi Tay, Lina Yao, and Qi Liu. Quaternion knowledge graph embeddings. In *Advances in Neural Information Processing Systems*, pages 2731–2741, 2019.
- [202] Dengyong Zhou, Olivier Bousquet, Thomas N Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in neural information processing systems*, pages 321–328, 2004.
- [203] X Zhu and Z Ghahramani. Learning from labels and unlabeled data with label propagation. 2002.
- [204] Marinka Zitnik, Marcus W Feldman, Jure Leskovec, et al. Evolution of resilience in protein interactomes across the tree of life. *Proceedings of the National Academy of Sciences*, 116(10): 4426–4433, 2019.

ProQuest Number: 28688401

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by ProQuest LLC (2021).

Copyright of the Dissertation is held by the Author unless otherwise noted.

This work may be used in accordance with the terms of the Creative Commons license or other rights statement, as indicated in the copyright statement or in the metadata associated with this work. Unless otherwise specified in the copyright statement or the metadata, all rights are reserved by the copyright holder.

This work is protected against unauthorized copying under Title 17,  
United States Code and other applicable copyright laws.

Microform Edition where available © ProQuest LLC. No reproduction or digitization of the Microform Edition is authorized without permission of ProQuest LLC.

ProQuest LLC  
789 East Eisenhower Parkway  
P.O. Box 1346  
Ann Arbor, MI 48106 - 1346 USA